Combining Oblivious Pseudorandom Functions

Sebastian Faller[1,2][0009-0005-4126-3098], Marc Fischlin[3][0000-0003-0597-8297], Julius Hardt[3][0009-0006-0499-8594], and Julia Hesse[1][0000-0002-2875-6198]

[1] IBM Research Europe - Zurich
sebastian.faller@ibm.com,JHS@zurich.ibm.com
[2] ETH Zurich
[3] TU Darmstadt
julius.hardt1@tu-darmstadt.de,marc.fischlin@tu-darmstadt.de

Abstract. An oblivious pseudorandom function (OPRF) is an interactive protocol between a client and server, where the client aims to evaluate a keyed pseudorandom function for a key held by the server, without revealing its input. OPRFs are a versatile tool for enhancing privacy, inciting extensive research and standardization efforts in this area. The round-efficient 2Hash-Diffie-Hellman OPRF is widely deployed, but unfortunately, it is prone to quantum attacks. The search for post-quantum alternatives started in 2019 and is currently more disputed, with several candidates on the table.

Given the lack of test of time of post-quantum OPRFs and their underlying cryptographic assumptions, hybridization can help mitigate risks in this interim period. Most preferably, we want to *combine* existing classical deployments like 2HashDH with post-quantum candidates. However, analogous two-party settings like oblivious transfer, where both parties have security requirements, indicate that combining OPRFs might be trickier than it is for hashing or encryption.

In this paper, we give combiners for OPRFs with minimal overhead over the combined schemes. We also formally prove that "ideal" combiners, i.e., ones that do not make additional assumptions on how an underlying OPRF may break, cannot exist. Our constructions avoid this theoretical result by assuming the underlying OPRFs to satisfy certain statistical guarantees. Crucially, these extra conditions are satisfied by both the 2Hash-Diffie-Hellman OPRF and the currently most efficient post-quantum candidates, rendering our results suitable for upgrading existing deployments with quantum-safe guarantees already today.

Keywords: Robust Combiners · Oblivious Pseudorandom Functions · Universal Composability · Hybrid Schemes · Impossibility Result

1 Introduction

An oblivious pseudorandom function (OPRF) [FIPR05] is an interactive protocol that allows a client to evaluate a pseudorandom function (PRF) on secret inputs by interacting with a server that holds a PRF key. Secure OPRFs need to ensure

that (1) the server remains oblivious to the client's input, i.e., it must not learn at which points the client evaluated the OPRF, and (2) that the client cannot evaluate the function at more points than the number of times it has interacted with the server.

OPRFs have recently been used in several large-scale privacy-enhancing technology deployments, serving millions of users. WhatsApp uses them to encrypt user chat histories with a key derived from a password, ensuring that users can recover encryption keys from just passwords should they lose access to their device [DLS21, Wha21]. Signal deploys OPRFs to password-encrypt user metadata [CFS+24]. The Microsoft Edge browser deploys OPRFs to test for password breaches [LKLCM21]. Cloudflare deploys OPRFs in their PrivacyPass system that enhances the usability of CAPTCHAs [Clo25]. In the cryptographic literature, OPRFs are used as building blocks for strong asymmetric PAKE (saPAKE) [JKX18a], password-protected secret sharing (PPSS) [JKKX16], private-information retrieval (PIR) [FIPR05], and private set intersection (PSI) [HL10]. See Casacuberta et al. [CHL22] for an overview of applications and construction methods of OPRFs.

Security assumptions for OPRFs. It is impossible to construct OPRFs with statistical security for both the client and the server. This follows from the fact that OPRFs imply oblivious transfer (OT) [CHL22, App. B.2] and the well-known impossibility results for OT [CK91, BGW88]. Many OPRFs in the literature are based on classical assumptions that are susceptible to quantum attacks [CHL22]. For instance, all real-world applications mentioned in the beginning use the Diffie-Hellman-based 2HashDH OPRF [JKKX16]. Therefore, the looming advent of quantum computing sparked a line of research on OPRFs based on post-quantum assumptions. We give a brief tour of these results here. Albrecht et al. [ADDS21] constructed a first OPRF from lattice assumptions. Albrecht and Gur [AG24] improved this construction to achieve better efficiency. Esgin et al. [ESTX24] improved the efficiency further at the price of using a new interactive lattice assumption. An alternative approach using the new "dark-matter weak PRF" assumption [BIP+18] and fully homomorphic encryption (FHE) was taken in [ADDG24]. Boneh et al. [BKW20] proposed two OPRFs from Isogenies. The first construction was broken by Basso et al. [BKM+21] and is susceptible to the attack on SIDH [CD23, MMP+23, Rob23]. Basso [Bas24] later proposed countermeasures against both attacks. The second construction of [BKW20] is based on CSIDH. It was subsequently broken and fixed by Heimberger et al. [HHM+24]. However, their construction requires using a 5280-bit prime, and the largest CSIDH prime that currently can be used is 1024 bits [DFK+23]. It is an open problem if this can be improved using higher-dimensional Isogenies [CLP24, DEF+25]. Other works only consider semi-honest adversaries [HKL+25, HHM+24, FOO23, DGH+21]. Grassi et al. [GRR+16], Seres et al. [SHB23], Beullens et al. [BDFH25], and Yang et al. [YBH+25] construct OPRFs using the pseudorandomness of the Legendre symbol [Dam90, GRR+16]. To the best of our knowledge, Basso [Bas24], Beullens et al. [BDFH25], and Yang et al. [YBH+25] are the only post-quantum OPRFs that satisfy the strong secu-

rity notions required by many applications like OPAQUE [JKX18a] and WhatsApp's backup protocol [DFG+23], password-protected secret sharing (PPSS) [JKKX16], and Signal [CFS+24].

One can observe that all of these three OPRF candidates rely on assumptions that did not receive the same amount of scrutiny as the assumptions from the RSA and discrete logarithm families. The SIDH-based OPRF construction [BKW20], which turned out to be insecure because the underlying hardness assumption was broken [CD23, MMP+23, Rob23], might be regarded as a cautionary tale.

Transitioning with combiners. A possible mitigation for the risk of weak hardness assumptions is the use of so-called *robust combiners*. A 1-out-of-2 robust combiner is an algorithm which takes two cryptographic schemes (called *candidate schemes*) and outputs the description of a single cryptographic scheme, which is called *combined scheme*. A secure, robust combiner ensures that if at least one of the candidate schemes is secure, then so is the combined scheme. Hence, when instantiated with independent candidate schemes (e.g., cryptographic schemes whose security relies on unrelated hardness assumptions), the combined scheme enjoys security through dissimilar redundancy: Since one of the candidate schemes may be broken without affecting the security of the combined scheme and the breakage of one candidate is unlikely to help break the other candidate, one obtains increased confidence in the combined scheme. Instantiated with one post-quantum and one classical candidate, the combiner is called *hybrid*. Hybrid combiners provide security guardrails when transitioning to post-quantum tools, making the transition as risk-free as possible on the algorithmic level. This becomes particularly important for primitives where post-quantum candidates are premature (e.g., rely on a relatively new hardness assumption, or simply lack the test of time regarding their security analysis). For OPRFs, where research on post-quantum candidates was kicked off only 4 years ago, this is certainly the case.

The challenge of building combiners for OPRFs. Combiners have been built for many cryptographic primitives, including one-way functions, PRGs, PRFs, bit commitments, encryption schemes, MACs, signature schemes [Her02], obfuscation [FHNS16, AJN+16], and interactive protocols such as oblivious transfer [HKN+05, MPW07], PIR [MP06] and PAKEs [HR25, LL25]. The latter are challenging because the security of *both* parties must be ensured — even if one of the candidate schemes is faulty. For example, a faulty candidate may leak the private input of one party to the other. Constructing OPRF combiners comes with additional challenges:

- OPRFs have unstructured, deterministic, and consistent output, so strategies used by previous combiners like secret sharing of the input (OT combiner [HKN+05]) or hashing protocol transcripts (PAKE combiner [HR25, LL25]) do not work.

– OPRFs typically have a codomain with super-polynomial size in the security parameter, which rules out strategies where all possible outputs of one candidate scheme are computed (as in the PIR combiner [MP06]).
– OPRFs are closely related to OT, for which there are lower bounds on specific classes of combiners [HKN+05, Som06, MPW07].

1.1 Our Contributions

In this paper, we analyze cryptographic solutions to securely transition to post-quantum OPRFs. Namely, we give combiners for OPRFs which transform candidate OPRF protocols into a single one with a "best-of-both" security guarantee. That is, the combined OPRF should be secure as long as one of the underlying OPRFs is. A natural candidate for combining pseudorandom functions is the so-called parallel approach, which concatenates the result of two PRFs F^1 and F^2, i.e., computes $H(F^1_K(x), F^2_K(x))$. Another option is a sequential combiner that feeds the result of the first PRF into the second, i.e., computes $H(F^1_K(x), F^2_K(F^1_K(x)))$. An obvious avenue is hence to investigate if these techniques can yield secure combiners for *oblivious* PRFs. Unfortunately, but perhaps not surprisingly, we cannot answer this question with a simple "yes". We find that combining OPRFs is inherently hard and requires a lot of care and awareness.

Impossibility of general black-box combiners. We provide formal evidence that no black-box combiner for OPRFs *without further assumptions on the candidate schemes* exists. Black-box combiners are the ideal version of a combiner, which do not exploit the structure of the underlying OPRFs, and where it does not play any role in the security of the combiner in which way an underlying component breaks. Black-box combiners provide ideal guarantees for transitioning from an established scheme to a new one, in the sense that any flaw in the new scheme does not harm the security of the existing one, and vice versa. This means deploying the combiner can only increase the security of an existing deployment, but never decrease it.

Our impossibility result uses techniques from the impossibility result for OT by Harnik et al. [HKN+05]. However, despite the close relation between OT and OPRFs [CHL22, App. B.1], the proof of [HKN+05] does not directly carry over to the OPRF case. This is because apart from the interactive evaluation of the primitive, which is common among OPRFs and OT, OPRFs can also be evaluated *non-interactively*, which, as we show, leads to complications in Harnik et al.'s original proof. Hence, we have to carefully adapt it to our setting. Along the way, we also point out and fix a potential gap in the proof of [HKN+05]: We argue that their so-called transparency condition, which restricts the kinds of combiners captured by their impossibility result, is not strong enough. By strengthening this condition, we can mend the proof of [HKN+05] for the case of OT, and prove our impossibility result for OPRFs. With this theoretical result, it is clear that practitioners need to make compromises on security when augmenting their 2HashDH implementation with a post-quantum OPRF.

Our combiners. The constructive contribution of our work is a black-box combiner for OPRFs in the Universal Composability (UC) framework by Canetti [Can01] that, *if certain additional conditions are satisfied by the underlying OPRFs*, yields the maximum security of both. Our combiner follows the parallel approach and adds only minimal overhead over the two OPRF executions. We show that our combiner can be instantiated from 2HashDH and the most practical composable post-quantum OPRFs by Beullens et al. [BDFH25] and Yang et al. [YBH+24], both of which are based on the decisional Legendre symbol assumption [Dam90, GRR+16]. As these constructions do not achieve "standard" OPRF security but only a slightly different version [BDFH25] allowing for different attacks, the security of the combiner naturally degrades to a weaker notion that allows the union of attacks possible on each of the candidates. This is, however, sufficient for all known practical purposes, in particular for using our combiner as an OPRF in the OPAQUE protocol or for other key derivation purposes. As a result, our work provides evidence for the feasibility of hybrid *strong* asymmetric password-authenticated key exchange (saPAKE), which was left as a major open question in the recent work on PAKE combiners [HR25, LL25].

Unconditional client security. Let us intuitively explain the extra condition that OPRFs need to satisfy for a combiner to be secure. OPRFs achieve two security guarantees: protecting the client's input, and ensuring the pseudorandomness of the function, i.e., protecting the server's key. If a combiner wants to use its OPRF candidates in a meaningful way, it must feed the PRF input x or some function of it into both OPRFs. Hence, if any of the underlying OPRFs break, information about x can leak. Intuitively, this means we cannot use the underlying OPRFs in a meaningful (x-dependent) way. Our additional condition captures exactly that: we require a strong form of unconditional security for the client. This is inspired by the works of [MPW07, HR25, LL25] that also analyze combiners for primitives where some of the security guarantees hold unconditionally.

An obvious question is whether we can relax the requirements on the underlying OPRFs by accepting more rounds of communication, e.g., combining both OPRFs sequentially. A known way to combine two PRFs F^1 and F^2 sequentially is the PRF $H(F^1_K(\mathsf{pw}), F^2_K(F^1_K(\mathsf{pw})))$. Note that here, to ensure that the entropy of F^1 still contributes to the final result should the pseudorandomness guarantee of F^2 break, we need to concatenate it with the F^2 evaluation. When considering this combined PRF in the OPRF setting, the hope is that we can relax the requirements on the oblivious evaluation of F^2 because it does not get the clear-text pw as input anymore. Indeed, should F^2 break, the combiner paradigm allows us to assume F^1 to be secure, and hence $F^1_K(\mathsf{pw})$ looks uniform and should not reveal pw. But perhaps surprisingly, this strategy does not work. Consider the case where F^2 leaks its input, e.g., due to its reliance on a broken computational assumption. Then the combiner reveals which executions ran.

Game-based combiner. As a side contribution, we also investigate a parallel combiner that XORs the individual PRF values instead of hashing them together, and can, hence, be proven in the plain model. We show the security of our

XOR combiner with respect to a game-based notion of security due to Tyagi et. al. [TCR+22] and Lehmann [Leh19], which we generalize to be applicable to protocols with an arbitrary number of rounds instead of only protocols with two rounds (solving a problem hinted at in [CHL22, Sec. 4.2]). While the XOR combiner does not rely on idealized objects such as random oracles or ideal ciphers, we prove that it cannot realize the standard functionality $\mathcal{F}_{\text{OPRF}}$ for OPRFs in the UC framework. This result also provides a natural separation between one of the game-based notions of security for OPRFs in the literature and $\mathcal{F}_{\text{OPRF}}$ in the UC framework, which was left as an open question in [FLÖ25]. For space constraints, our results on the XOR combiner are deferred to Sec. H.

Conslusion and deployment considerations. Many real-world applications relying on OPRFs, such as WhatsApp's end-to-end-encrypted backup protocol [DFG+23], use the 2HashDH construction (see Sec. 5.1). It can be expected that the developers of these applications want to transition to hybrid OPRFs that provide post-quantum security. We show that augmenting an existing 2HashDH protocol with a post-quantum OPRF through a concatenated hash introduces a certain risk that developers need to be aware of. While this combiner offers the desirable protection against failures on the level of the post-quantum computational assumption, comes with low computational overhead (only one additional hash evaluation in addition to running the candidates) and does not increase the number of protocol rounds, it requires certain unconditional properties from the post-quantum OPRF. Due to the complexity of practical post-quantum constructions, this can fail either at the theoretical level (i.e., a wrong formal proof of unconditional client security) or at the implementation level (e.g., the usage of a computational building block where an unconditional one has to be used, or simply a flaw in the implementation that leaks the OPRF input). We hope that our work helps developers and protocol designers assess the risks that are introduced into their systems.

Still, we advocate for transitioning to post-quantum OPRF by using a concatenated hash of two OPRFs, because of three simple reasons: (1) the combiner protects against failures in the post-quantum computational assumption. (2) The combiner offers better security guarantees than the post-quantum OPRF alone. And (3), our impossibility result and further analysis strongly indicate that there is no better option to combine oblivious pseudorandom functions.

1.2 Related Work

Herzberg [Her02] provided the first formal treatment of robust combiners. However, even before that, there have been various implicit appearances of robust combiners in the literature (see [Her02] for an overview). These combiners were designed for non-interactive primitives such as encryption schemes and signatures. Harnik et al. [HKN+05] and Sommer [Som06] initiated the study of robust combiners for interactive protocols. They investigated the possibility of combiners for key exchange and OT, which is closely related to OPRFs [CHL22,

App. B.1]. After that, there was follow-up work on combiners for OT [MPW07] and on combiners for private information retrieval [MP06].

Hesse, Rosenberg [HR25] and Lyu, Liu [LL25] concurrently introduced the first combiners explicitly[1] proven secure in the UC framework. These combiners combine two PAKE protocols. One of their combiners, the parallel combiner [HR25, Sec. 3.1], requires both candidate schemes to satisfy a specific property (password hiding) unconditionally. This is similar to our combiners for OPRFs. However, due to the differences between the primitives, our security proof is naturally different.

2 Preliminaries

Notation. We define $\mathbb{N}$ as the set of natural numbers (including zero), $\mathbb{N}_+$ as the set of positive integers, and $\mathbb{N}_{\geq j}$ as the set $\{n \in \mathbb{N}_+ \mid n \geq j\}$. For $i, j \in \mathbb{Z}$, we write $[i..j]$ to represent the set $\{n \in \mathbb{Z} \mid i \leq n \leq j\}$ and $[j] = [1..j]$ to represent the set $\{n \in \mathbb{N}_+ \mid n \leq j\}$. We let $\lambda \in \mathbb{N}_+$ denote the security parameter. A randomized algorithm runs in probabilistic polynomial-time (PPT) if it takes at most polynomially many steps in λ. $PPTM$ stands for probabilistic polynomial-time Turing machine. A function $\epsilon(\lambda)$ in the security parameter is *negligible*, written $\epsilon(\lambda) \leq \mathsf{negl}(\lambda)$, if it vanishes faster than the inverse of any positive polynomial. The support of a random variable X, i.e., the values which are output with positive probability, is given by $\mathrm{supp}(X)$. We write $X \overset{c}{\approx} Y$ if two random variables are *computationally indistinguishable*.

Pseudorandom functions. A *pseudorandom function* (PRF) is an efficiently computable function $\mathsf{F}\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ that maps a key $k \in \mathcal{K}$ and an input $x \in \mathcal{X}$ to an output $y \in \mathcal{Y}$, such that no efficient adversary can distinguish between evaluations of a random function and F under a uniformly random key.

Protocols. When two parties interact in a protocol, a *communication round* consists of all messages sent in parallel. In interactive protocols, this usually means that sending a message and receiving an answer for that message counts as two communication rounds.

Robust combiners. A *robust combiner* Comb takes as input some candidate schemes $(\mathcal{C}_i)_{i\in[n]}$ and realizes a desired primitive if a certain threshold of the candidate schemes is secure. More formally:

Definition 1 (Black-box (k, n)-Robust Combiner [HKN+05]). *Let* P *be a cryptographic primitive. A* black-box (k, n)-robust combiner for P *is a protocol*

[1] For example, the contatenation combiner (called "copy combiner" in [Her02]) for signature schemes that are existentially unforgeable under chosen-message attacks (EUF-CMA, [GMR88]) is implicitly a combiner in the UC framework because EUF-CMA and the security definition [Can03] for signature schemes in the UC framework [Can03] have been shown to be equivalent [Can03, Thm. 2].

Comb *that gets* n *candidate protocols* $\mathcal{C}_1, \ldots, \mathcal{C}_n$ *(all running in polynomial time) as input, and implements* P *while satisfying four properties:*

1. (k, n)-Robustness: *If at least* k *of the candidate protocols securely implement* P*, then the combiner* Comb *also securely implements* P.
2. Efficiency: *The running time of the combiner* Comb *is polynomial in the security parameter* λ*, in* n*, and in the lengths of the inputs to* P.
3. Fully black-box proof: *For every candidate protocol* $\mathcal{C}_i$*, there exists an oracle PPTM* $\mathcal{R}_i$ *such that for every adversary* $\mathcal{A}$*, if* $\mathcal{A}$ *breaks* Comb*, then there is a set* $\mathcal{I} \subseteq [n]$ *with* $|\mathcal{I}| > n - k$ *such that for all* $i \in \mathcal{I}$*,* $\mathcal{R}_i^{\mathcal{A}}$ *breaks the candidate.*
4. Black-box implementation: Comb *is a (tuple of) oracle PPTM given access to the candidate protocols via oracle calls to their implementation functions.*

If clear from the context, we use the term *robust combiner* for a black-box $(1, 2)$-robust combiner. There are also combiners that require different security from the individual candidate schemes, e.g., the first candidate can be required to have stronger security properties than the other candidates [HR25].

3 Oblivious Pseudorandom Functions

An oblivious pseudorandom function is a two-party protocol for securely evaluating a PRF $\mathsf{F}\colon \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$. Unless stated otherwise, we implicitly assume that $|\mathcal{K}| \geq 2^\lambda \leq |\mathcal{Y}|$. The protocol is run between a server (or sender) that holds the key $\mathsf{k} \in \mathcal{K}$ for the PRF, and a client (or receiver) that provides an input $x \in \mathcal{X}$ and learns the corresponding output $y = \mathsf{F}(\mathsf{k}, x)$. A secure OPRF ensures that the server does not learn anything about the client's input (obliviousness) and that a client interacting n times with the server does not learn anything about F apart from n random function values (pseudorandomness).

To formalize OPRFs in the UC framework, we build upon the well-established ideal functionalities from the literature [JKKX16, JKX18a, JKX18b, BDFH25]. All of these functionalities, including ours, have in common that the server's key is not modeled as an explicit object. Instead, the ideal functionality provides an interface for non-interactive evaluations of the function, and the honest server's picking a fresh key corresponds to it initiating a new functionality instance with a fresh identifier sid, as we discuss below. Throughout this work, and based on variants in the literature, we will use slight variations of the OPRF functionality. Figure 1 details all these variations. We discuss how they work and the differences between them below, i.e., the standard functionality $\mathcal{F}_{\text{OPRF}}$ in Sec. 3.1, the lazy-extraction 2Hash variant $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$ in Sec. 3.2, and their respective ∞-counterparts that allow for unlimited evaluations in Sec. 4.

3.1 Standard Functionality

We start by explaining the standard ideal OPRF functionality $\mathcal{F}_{\text{OPRF}}$ [JKKX16], which is displayed in Fig. 1, reading the normal text plus the dotted parts.

Ideal OPRF functionalities $\mathcal{F}_{\text{OPRF}}$, $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$, $\mathcal{F}^{\infty}_{\text{OPRF}}$ and $\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}$

Public parameters: Domain $\mathcal{X}$ and codomain $\mathcal{Y} = \{0,1\}^\lambda$ of the PRF, function $f : \mathcal{I} \times \mathcal{K} \to \mathcal{V}$

Global variables:
Mappings $\mathsf{H}_{1,\text{sid}}(\cdot)$, $\mathcal{T}_{\text{preview,sid}}(\cdot,\cdot)$, $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, a set $\text{T}_{\text{programmed,sid}} \leftarrow \emptyset$, an initially empty sequence K_{sid} of adversarial keys and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x,k)$ or $\mathcal{T}_{\text{preview,sid}}(x,v)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) := r$, $\mathcal{F}_{\text{malicious,sid}}(x,k) := r$ or $\mathcal{T}_{\text{preview,sid}}(x,v) := r$. Similarly, if $\mathsf{H}_{1,\text{sid}}(x)$ is referenced for the first time, the functionality chooses $v \leftarrow\!\!\$\ \mathcal{V}$ and sets $\mathsf{H}_{1,\text{sid}}(x) := v$.

Initialization:
On message $(\text{Init}, \text{sid})$ from party S, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity that sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Adaptive server compromise (requires permission from the environment):
On $(\text{Compromise}, \text{sid})$ from $\mathcal{A}$, declare server S as compromised.

Offline evaluation of the honest function:
On $(\text{OfflineEval}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathsf{S}, \mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is compromised, send $(\text{OfflineEval}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\text{OfflineEval}, \text{sid}, k^*, x)$ from $\mathcal{A}$, run $\text{Correlate}(k^*)$, and send $(\text{OfflineEval}, \text{sid}, k^*, x, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to $\mathcal{A}$.

Online evaluation:
- On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathsf{U}, \mathcal{A}\}$, record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$, send $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\text{SndrComplete}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid})$ to $\mathcal{A}$.
- On $(\text{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{Eval}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- On $(\text{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, run $\text{Correlate}(k^*)$, send $(\text{Eval}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to P.

Random oracles:
On $(\mathsf{H}_1, \text{sid}, x)$ from $\mathcal{A}$, reply with $(\mathsf{H}_1, \text{sid}, x, \mathsf{H}_{1,\text{sid}}(x))$.
On $(\mathsf{H}_2, \text{sid}, (x, y^*))$ from $\mathcal{A}$:
- For the first element k^* in K_{sid} which satisfies $f(\mathsf{H}_{1,\text{sid}}(x), k^*) = y^*$, set $t \leftarrow \mathcal{F}_{\text{malicious,sid}}(x, k^*)$. If there is no such element, set $t \leftarrow \mathcal{T}_{\text{preview,sid}}(x, y^*)$.
- Send $(\mathsf{H}_2, \text{sid}, (x, y^*), t)$ to $\mathcal{A}$.

Procedure $\text{Correlate}(k^*)$:
Append k^* to K_{sid}. For all $(x, y) \in \text{dom}(\mathcal{T}_{\text{preview,sid}}) \setminus \text{T}_{\text{programmed,sid}}$ with $f(\mathsf{H}_{1,\text{sid}}(x), k^*) = y$:
Set $\mathcal{F}_{\text{malicious,sid}}(x, k^*) := \mathcal{T}_{\text{preview,sid}}(x, y)$ and add (x, y) to $\text{T}_{\text{programmed,sid}}$.

Fig. 1. Ideal OPRF functionalities $\mathcal{F}_{\text{OPRF}}$ [JKKX16, JKX18a, JKX18b] (with the dotted parts but without the gray parts) and $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$ (full text, based on [BDFH25]) as well as their respective one-sided counterparts $\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}$ and $\mathcal{F}^{\infty}_{\text{OPRF}}$ (without the dotted text).

Initialization and corruption. An OPRF session accommodates a single server and multiple clients. The server, whose identity can be fixed in the session identifier sid, sends an INIT message to $\mathcal{F}_{\text{OPRF}}$ to register itself as the server for a specific functionality instance. In addition to arbitrary corruption[2], $\mathcal{F}_{\text{OPRF}}$ allows the adversary to adaptively compromise the server using the COMPROMISE command. Adaptive compromise is a type of corruption and hence requires permission from the environment (see [JKX18a]). Following the terminology from [JKX18a], we call (statically) corrupted clients *corrupted* and a dishonest server *compromised*, regardless of whether it was statically corrupted or has been adaptively compromised.

Random output tables. $\mathcal{F}_{\text{OPRF}}$ represents an OPRF as an indexed set of lazily sampled function tables, each of which maps each input $x \in \mathcal{X}$ to a uniformly sampled bitstring y of length λ. Lazy sampling means that the tables are initially empty, but as soon as a lookup of the output of an undefined input x occurs, the corresponding y is sampled and returned on this and on all subsequent queries for x. This is essential because an inefficient $\mathcal{F}_{\text{OPRF}}$ would not be useful for analyzing efficient protocols. Each function table is indexed by a label, which we interpret as an identifier with a one-to-one relationship to an OPRF key. Since there is at most one honest server per session, the associated function table is referred to as $\mathcal{F}_{\text{honest,sid}}(\cdot)$. The other ("adversarial") function tables are denoted by $\mathcal{F}_{\text{malicious,sid}}(k, \cdot)$ for $k \in \mathcal{K}$. Due to the fact that there is a superpolynomial amount of possible OPRF keys, function tables are created only on demand.

Offline evaluation. The remaining commands, besides the interfaces for initialization and corruption, are used for evaluating the OPRF. The OFFLINEEVAL command enables non-interactive evaluations. The variant without k^* returns the function value of the specified x under the honest function. It can be called by the honest server or by the adversary if an honest server exists and has been compromised. The variant which additionally accepts a key label k^* is for the adversary only and models evaluations with adversarially chosen keys.

Online evaluation. To begin an online evaluation with server S' and input x, the client (or the adversary) sends an EVAL message with these parameters to $\mathcal{F}_{\text{OPRF}}$. The functionality then notifies the ideal-world adversary about the interaction, but does not reveal the client's input x. An honest server's decision to engage in the evaluation is modeled by it sending SNDRCOMPLETE to the functionality. To complete an interaction, the adversary issues a RCVCOMPLETEHONEST or RCVCOMPLETEMALICIOUS command, which causes the functionality to send the resulting function value to the client (or the adversary, if it sent the original EVAL message). The former command models an untampered interaction between an honest client and the honest server, while the latter embodies an interaction in which the adversary acts as the server or interferes with the

[2] Due to the fast turnaround of an evaluation and stateless client code, most OPRF protocols, including ours, are analyzed restricting to static client corruption.

communication between two honest parties. In the RcvCompleteMalicious case, the adversary needs to specify the label of the function table from which the client receives its output. We stress that even though the adversary can pick a function table, it cannot control the contents of the function tables, as they are randomly sampled by the functionality. This ensures *pseudorandom* function values even when honest clients interact with a malicious server or a network adversary. This stands in contrast to the game-based security definition, which does not guarantee (pseudo)randomness in the case of a malicious server. In any case, be it the evaluation of a malicious function or the honest one, $\mathcal{F}_{\text{OPRF}}$ keeps both the honest client's input and the function value hidden from the server and adversary, ensuring the PRF evaluation to be *oblivious*.

Ticket counter. An important security property for OPRFs is that a malicious client should not be able to evaluate the honest server's PRF at more points than it has had interactions with the server. This is captured in $\mathcal{F}_{\text{OPRF}}$ by maintaining a ticket counter tx_{sid} which is incremented upon the reception of every SndrComplete message from the honest server and decremented when the adversary calls RcvCompleteHonest. While the ticket counter is equal to zero, RcvCompleteHonest messages are ignored. Consequently, every client's successful evaluation of an OPRF realizing $\mathcal{F}_{\text{OPRF}}$ with the honest server's key must be preceded by an interaction with the server. Compared to [JKX18a], we drop transcript prefixes[3] and keep a ticket counter as in [JKKX16]. Furthermore, for improved readability, we adopt the presentation in [BDFH25] and split the RcvComplete interfaces into separate commands for the honest and the malicious function tables.

3.2 The 2Hash Framework

In [BDFH25], inspired by the 2HashDH OPRF, Beullens et al. construct OPRFs evaluating functions of the form $\mathsf{PRF}(k, x) \coloneqq \mathsf{H}_2(x, f(\mathsf{H}_1(x), k))$, where H_1 and H_2 are hash functions that are modeled as (independent) random oracles and f is a weak-key-collision-resistant and unpredictable function. To evaluate such an OPRF interactively, $f(\mathsf{H}_1(x), k)$ is computed using a (generic) secure two-party computation protocol where the client inputs $\mathsf{H}_1(x)$ and server inputs its key k.

OPRFs derived from this framework do not satisfy "full" OPRF security, i.e., they do not UC-emulate $\mathcal{F}_{\text{OPRF}}$, but a different functionality $\mathcal{F}^f_{\text{2H-OPRF}}$ [4]. In a nutshell, $\mathcal{F}^f_{\text{2H-OPRF}}$ allows an adversary to "preview" PRF values for adversarial

[3] The prefixes are necessary for the security proof of the saPAKE compiler OPAQUE [JKX18a], but as noted in [BDFH25], one can generically use the whole transcript of the OPRF to get the needed prefix.

[4] The ideal functionalities $\mathcal{F}_{\text{OPRF}}$ and $\mathcal{F}^f_{\text{2H-OPRF}}$ are uncomparable: $\mathcal{F}^f_{\text{2H-OPRF}}$ allows the adversary to obtain (previewed) OPRF outputs before the function table for these outputs is known to the functionality, and is hence weaker than $\mathcal{F}_{\text{OPRF}}$ in this aspect. On the other hand, $\mathcal{F}^f_{\text{2H-OPRF}}$ requires active adversaries to be aware of plaintext PRF keys, while $\mathcal{F}_{\text{OPRF}}$ admits active attacks from already a unique identifier of the PRF key. So in this aspect, $\mathcal{F}^f_{\text{2H-OPRF}}$ enforces stronger guarantees

keys. Still, since the honest server's PRF remains protected from such previews and the uniformity of even adversarial evaluations is still guaranteed, this weaker OPRF definition is strong enough to be used within, e.g., OPAQUE [BDFH25].

In contrast to $\mathcal{F}_{\text{OPRF}}$, the $\mathcal{F}^{f}_{\text{2H-OPRF}}$ functionality from [BDFH25] does not use a ticket counter. Instead, the honest server approves particular evaluations ssid using the SndrComplete message instead of advancing a counter of permitted evaluations. This results in a stronger functionality, but requires simulators in proofs to extract the (malicious) client's input directly and match it to a particular evaluation, which precludes security proofs for some more efficient protocols, such as 2HashDH that rely on lazy extraction techniques. Since our combiner deals with the plain $\mathcal{F}_{\text{OPRF}}$ functionality as well as the 2Hash extension, in order to maintain consistency, we consider a slightly modified variant of $\mathcal{F}^{f}_{\text{2H-OPRF}}$ that also uses ticket counters and call it $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$ (lazy extraction 2Hash OPRF). It is displayed in Fig. 1 (full text including the dotted and the gray parts). We note that the arguments in [BDFH25] for the applicability of this functionality to higher-level protocols carry over to $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$ as well.

4 OPRF Combiners

In this chapter we investigate how to adopt *parallel* PRF combiner to the oblivious setting. Parallel combiners for PRFs simply mix two PRF values by, e.g., XORing or hashing. It is well known that this preserves the pseudorandomness of the function. However, when the PRF is evaluated *interactively* through an OPRF, we face the additional challenge of protecting the client's input from leaking to the server through a potentially broken OPRF component. In fact, we formally show later that this potential leakage rules out "ideal" combiners for OPRF. We now give a constructive way to still build OPRF combiners relying on components that protect the client input unconditionally. That is, the security for the client must be guaranteed even if the computational hardness assumptions underlying the candidate's security are broken.

For game-based OPRF security, achieving unconditional security for the receiver would amount to requiring that no information about the client's input is leaked, which is already captured by the obliviousness experiment. In Sec. F and H, we make this observation rigorous by providing game-based security definitions for OPRFs and show that the parallel XOR combiner is a robust combiner for OPRFs with unconditional client security. However, in the UC setting, which we are mainly interested in, requiring obliviousness is not sufficient because the simulator additionally needs to be able to extract a unique label for the effective OPRF key used in the evaluation. Consequently, we require the existence of such a simulator for every candidate scheme as well - even in the case of a breakdown of a hardness assumption. To capture this formally, we require all candidate schemes to UC-realize the ideal functionality $\mathcal{F}^{\infty}_{\text{OPRF}}$ (Fig. 1) unconditionally.

and hence there is no direct relation between both functionalities in terms of UC emulation.

$\mathcal{F}_{\text{OPRF}}^{\infty}$ is the same as $\mathcal{F}_{\text{OPRF}}$, except that the ticket counter is set to ∞. Hence, no checking of ticket availability is required as there are always enough tickets remaining. This allows the adversary to evaluate the OPRF of the honest server on any input without any interaction, which obviously voids one of the main security guarantees of $\mathcal{F}_{\text{OPRF}}$. Yet, $\mathcal{F}_{\text{OPRF}}^{\infty}$ still ensures that a corrupted server neither learns the inputs nor the outputs of honest clients, and it ensures that for every evaluation with honest and/or malicious parties, the label of the key used in the interaction can be extracted and that function outputs are uniformly random. This is precisely the property we need for our combiner. We stress that we do not recommend using $\mathcal{F}_{\text{OPRF}}^{\infty}$ as an OPRF security notion; it is simply a handy technical tool for the analysis of our combiner.

One might wonder whether unconditional security for the receiver is an overly strict requirement. The recent works on PAKE combiners [HR25, LL25], where a similar obstacle arises, alleviate this situation to some extent by proposing a combiner that runs the two candidate schemes in sequence, feeding a hash of the transcript and the output of the first scheme into the second one. This way, only the first candidate scheme needs to hide its input perfectly (which, in the case of PAKEs, is a password), while the second scheme merely needs to satisfy a weaker hiding property. Unfortunately, such a strategy does not seem to work for OPRFs: Since OPRFs exhibit deterministic input-output behavior (at least in the case of honest parties), randomization techniques such as including the hash of the (randomized) transcript in the input to the second scheme would disturb the output and thus break the correctness of the combined OPRF.

Moreover, simple *cascade constructions*, in which only the first scheme enjoys unconditional security for the receiver and the output (function value) of the first scheme is passed as the sole input to the second scheme, do not work for OPRFs either. The reason is that, if the second scheme leaks its input, the environment can easily distinguish between the real world and the ideal world: It picks two possible inputs x_0, x_1 for the combiner, forwards them to the corrupted server, and runs an interactive evaluation with an honest client using one of those inputs. Following that, the adversary learns (parts of) the intermediate output of the first scheme through the leakage, and then compares this leakage against the value it would obtain for x_0 and x_1 and the key for this scheme, allowing it to decide whether x_0 or x_1 was used.

Due to the fact that OPRFs output random-looking, unstructured values, other black-box constructions appear to be beyond reach, too. This seems to suggest that a requirement such as unconditional security is actually needed. To back up this intuition, we provide an impossibility result in Sec. 6 showing that there can be no transparent black-box 1-out-of-2 combiner for OPRFs without further assumptions on the candidate schemes. Jumping ahead, while requiring unconditional security for the receiver rules out some OPRFs, in Sec. 5 we show that both the currently most widely used classical OPRF and the currently most efficient composable and post-quantum OPRF candidate satisfy unconditional security. This renders the parallel combiner of high interest for practical post-quantum transition purposes.

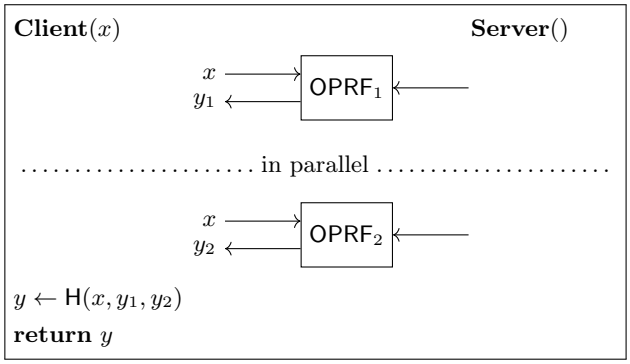


Fig. 2. Hash combiner Comb_H for two OPRFs

4.1 XOR versus Hash

There are two natural ways to build a parallel combiner: XORing the intermediate outputs of the candidate schemes to obtain the final output or, alternatively, hashing them. While for a standard game-based security notion, the XOR combiner is sufficient (and perhaps preferable, because it does not rely on idealized models), it does not work in the UC setting. This is because the XOR induces a structure on OPRF outputs with *related malicious keys*, which is not allowed in $\mathcal{F}_{\text{OPRF}}$. The argument for this is standard and thus deferred to Sec. H. In the following, we direct our attention to the construction of the parallel hash combiner.

4.2 Hash Combiner

The parallel hash combiner, called Comb_H in the following, uses a hash function H to combine the outputs of its candidate schemes and the input x. This destroys any structure among function values computed using different keys and thus solves this shortcoming of the XOR combiner. For simplicity, we present the hash combiner as a 1-out-of-2 combiner, even though it naturally generalizes to a 1-out-of-n combiner. Figure 2 displays a graphical depiction of the combiner.

Definition 2 (OPRF Hash Combiner). *For each pair $(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}})$ of OPRFs, we define the hash combiner Comb_H as the algorithm which outputs the combined scheme displayed in Fig. 2 (and, as a formal specification, in Fig. 7 in the appendix).*

We have the following theorem:

Theorem 1 (Robustness of the Hash Combiner ($\mathcal{F}_{\text{OPRF}} + \mathcal{F}_{\text{OPRF}}$)). *Let Π^1_{OPRF} and Π^2_{OPRF} be two OPRF protocols both UC-realizing $\mathcal{F}^\infty_{\text{OPRF}}$. If there is at least one $i \in \{1, 2\}$ such that Π^i_{OPRF} additionally UC-realizes $\mathcal{F}_{\text{OPRF}}$, then $\mathsf{Comb}_\mathsf{H}(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}})$ UC-realizes $\mathcal{F}_{\text{OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model.*

The main idea of the proof is the fact that even if the underlying computational hardness assumption of one candidate scheme is broken, *both* candidate schemes still implement $\mathcal{F}_{\text{OPRF}}^{\infty}$, which means that their outputs are (almost) uniformly random, even for maliciously chosen keys. This implies that collisions, i.e. two keys and inputs which map to the same output, are unlikely to be found, which enables the simulator — given an input-output pair (x, y_i) (for an $i \in \{1, 2\}$) — to identify the key used for obtaining y_i from OPRF_i on input x. Consequently, whenever the real-world adversary queries the random oracle H on an input $(x, (y_1, y_2))$, the simulator can check whether both pairs (x, y_1) and (x, y_2) are valid evaluations of OPRF_1 resp. OPRF_2 and, if so, determine the key labels lbl_1 and lbl_2 which belong to these evaluations. It can then ask the ideal functionality $\mathcal{F}_{\text{OPRF}}$ for the corresponding OPRF output for key label $(\text{lbl}_1, \text{lbl}_2)$ and program H accordingly. What remains to show is that if H is queried on evaluations of the candidate schemes acquired using the honest keys, i.e. on (x, y_1, y_2) such that $y_1 = \mathcal{F}^1_{\text{honest,sid}}(x)$ and $y_2 = \mathcal{F}^2_{\text{honest,sid}}(x)$, which causes the simulator to ask $\mathcal{F}_{\text{OPRF}}$ for an evaluation on the honest key, it actually gets an answer and does not exhaust the ticket counter. Here we use that one candidate scheme OPRF_j for $j \in \{1, 2\}$ is secure (i.e., it UC-realizes $\mathcal{F}_{\text{OPRF}}$), and, thus, limits the amount of (candidate scheme) OPRF evaluations the real-world adversary can obtain. Therefore, the real-world adversary cannot compute more input-output pairs (x, y_j), and thus neither more triples (x, y_1, y_2) on the honest key for the secure candidate scheme OPRF_j than it has had interactions with the honest server. This means that with overwhelming probability, the simulator always gets an answer from $\mathcal{F}_{\text{OPRF}}$ and the simulation works as expected. For reference, we provide a detailed proof in appendix D.1.

An issue with Thm. 1 is that, to the best of our knowledge, no OPRF that UC-realizes $\mathcal{F}_{\text{OPRF}}$ under post-quantum assumptions is currently known. For this reason, we reinterpret our combiner as a combiner for a "standard" OPRF and a slightly weaker OPRF obtained from the 2Hash framework by Beullens et al. [BDFH25]. The construction is the same as before, but we relax the requirements of the building blocks to work for the post-quantum candidates, i.e., protocols realizing $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$.

Theorem 2 (Robustness of the Hash Combiner ($\mathcal{F}_{\text{OPRF}} + \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$)). *Let Π^1_{OPRF} and Π^2_{OPRF} be two OPRF protocols such that Π^1_{OPRF} UC-realizes $\mathcal{F}^{\infty}_{\text{OPRF}}$ and Π^2_{OPRF} UC-realizes $\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}$. If at least one of the following conditions is fulfilled, then $\mathsf{Comb}_{\mathsf{H}}\big(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}}\big)$ UC-realizes $\mathcal{F}_{\text{CorH-OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model:*

- *Π^1_{OPRF} UC-realizes $\mathcal{F}_{\text{OPRF}}$.*
- *Π^2_{OPRF} UC-realizes $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$.*

Similarly to $\mathcal{F}^{\infty}_{\text{OPRF}}$ for $\mathcal{F}_{\text{OPRF}}$, we provide a weaker functionality $\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}$ that is the same as $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$ except for the absence of the ticket counter (see Fig. 1). The combiner provides security, i.e., the resulting combined scheme UC-realizes $\mathcal{F}_{\text{CorH-OPRF}}$, if both OPRF_1 and OPRF_2 realize the required minimum functionalities $\mathcal{F}^{\infty}_{\text{OPRF}}$, resp. $\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}$, and in addition, at least one of the can-

didate schemes provides full security, i.e. OPRF_1 UC-realizes $\mathcal{F}_{\text{OPRF}}$ or OPRF_2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{f,\infty}$.

The consequence of the weaker building block is that this combiner does not achieve "full" OPRF security but a weakened variant specified by $\mathcal{F}_{\text{CorH-OPRF}}$ in Fig. 3. Recall that 2Hash OPRFs allow the adversary to preview PRF values for adversarial keys just by computing hashes. When used as a building block, this preview capability also affects an adversary against the hash combiner to preview PRF values of the combined PRF. Such previews are exactly what differentiate $\mathcal{F}_{\text{CorH-OPRF}}$ from $\mathcal{F}_{\text{OPRF}}$.

The intuition behind the relaxation is as follows: In any OPRF, a malicious server can choose some malicious key k^* and evaluate the PRF offline, i.e., without any interaction with the client, on this k^* just by virtue of owning the key. The server can later use the same key k^* in an online evaluation with some honest client. An OPRF must ensure that the offline evaluations and the online evaluations of the PRF are coherent or else, the OPRF would not be correct.

Functionality $\mathcal{F}_{\text{OPRF}}$ demands that an adversary provides a unique identifier of the malicious key *immediately*, i.e., already when offline evaluating the PRF. In 2HashDH this identifier is g^{k^*}, which the simulator computes from $H(x)^k$ with the help of a discrete logarithm trapdoor $H(x) = g^r$. Unfortunately, in the Legendre-based constructions, no such trapdoor exists and hence adversarial offline evaluations cannot be "labeled" with a unique key identifier.

To remedy this issue, $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ allows the adversary to get a "preview" output for an offline evaluation. Once an online execution of the protocol happens for the same adversarial key with the adversary providing the unique key, $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ "correlates" the output to the honest client with a preview in case it was given out to the adversary for the same input and adversarial key before.

It is not surprising that a combiner using an OPRF following the 2Hash blueprint inherits this preview necessity from $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$. If the simulator for the 2Hash-OPRF cannot extract the malicious key from an offline evaluation but must defer the extraction to the online evaluation, then a simulator for the combined protocol must also defer its extraction until that point.

The preview interface of $\mathcal{F}_{\text{CorH-OPRF}}$ is simpler than the one in the 2Hash framework, since the hash combiner has only one hash function (i.e., random oracle). It follows the same principle but exposes the correlate interface directly to the adversary. The latter lets the adversary assign previewed values to adversarial functions, with the restriction that once a function identifier lbl has been used in an (online or offline) evaluation, it can no longer be (re-)programmed. Furthermore, each previewed value can only be written once. This ensures pseudorandomness even for adversarial functions. Additionally, the correlation interface does not affect the function table of the honest server. Overall, the functionality provides at least the security guarantees of the correlated OPRF functionality $\mathcal{F}_{\text{corOPRF}}$ from [JKX21][5] and thus allows protocols realizing $\mathcal{F}_{\text{CorH-OPRF}}$, such as our combiner, to be plugged into OPAQUE [JKX18a].

[5] We show this claim in Sec. E.

Ideal OPRF functionality $\mathcal{F}_{\text{CorH-OPRF}}$

Public parameters: Domain $\mathcal{X}$ and codomain $\mathcal{Y} = \{0,1\}^\lambda$ of the OPRF

Global variables:
Mappings $\mathsf{H}_c(\cdot,\cdot)$, $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot, \cdot)$ which are initially undefined everywhere, and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl})$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) := r$, or $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}) := r$.

Similarly, if $\mathsf{H}_c(x,z)$ is referenced for the first time, the functionality samples $y \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathsf{H}_c(x,z) := y$.

Initialization:
On message $(\textsc{Init}, \text{sid})$ from party S, if this is the first $\textsc{Init}$ message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\textsc{Init}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the $\textsc{Init}$ message for the session identifier sid. Ignore all subsequent $\textsc{Init}$ messages for sid.

Adaptive server compromise:
On $(\textsc{Compromise}, \text{sid})$ from $\mathcal{A}$, declare server S as $\textsc{compromised}$. *Note: Message* $(\textsc{Compromise}, \text{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function:
On $(\textsc{OfflineEval}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathsf{S}, \mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is $\textsc{compromised}$, send $(\textsc{OfflineEval}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\textsc{OfflineEval}, \text{sid}, \text{lbl}, x)$ from $\mathcal{A}$, send $(\textsc{OfflineEval}, \text{sid}, \text{lbl}, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}))$ to $\mathcal{A}$.
Online evaluation:
- On $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathsf{U}, \mathcal{A}\}$, record $\langle \text{ssid}, \mathsf{P}, x\rangle$ and send $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\textsc{SndrComplete}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\textsc{SndrComplete}, \text{sid})$ to $\mathcal{A}$.
- On $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{ssid}, \mathsf{P}, x\rangle$ or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not $\textsc{compromised}$, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- On $(\textsc{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}))$ to P.

Random oracles:
On $(\mathsf{H}_c, \text{sid}, x, z)$ from $\mathcal{A}$, if this is the first query for x, z and sid, store record $\langle \mathsf{H}_c, \text{sid}, x, z, \mathsf{H}_c(z)\rangle$ marked $\textsc{fresh}$. In any case, reply with $\mathsf{H}_c(x,z)$.

On $(\textsc{Correlate}, \text{sid}, x, z, \text{lbl})$ from $\mathcal{A}$, if lbl has not been used in an $\textsc{OfflineEval}$ or $\textsc{RcvCompleteMalicious}$ command before, $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl})$ has not been programmed so far and there exists a record $\langle \mathsf{H}_c, \text{sid}, z, y\rangle$ marked $\textsc{fresh}$, mark it $\textsc{programmed}$ and set $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}) := y$.

Fig. 3. Ideal OPRF functionality $\mathcal{F}_{\text{CorH-OPRF}}$. Differences between $\mathcal{F}_{\text{CorH-OPRF}}$ and $\mathcal{F}_{\text{OPRF}}$ are marked in grey.

The proof of Thm. 2 proceeds exactly like the one of Thm. 1, with the following exception: The adversary may now ask for random oracle evaluations for the "outer" hash H of the combiner on inputs $(x, (z_1, z_2))$ such that the simulator cannot (yet) extract the key (label) for (x, z_2) from OPRF_2. However, the simulator can now deal with this situation by asking the ideal functionality $\mathcal{F}_{\text{CorH-OPRF}}$ for the required hash value (interface H_c). The functionality makes sure that once the key for (x, z_2) in OPRF_2 is extracted, the respective function table is made consistent with this already released hash output (interface CORRELATE).

A proof for Thm. 2 can be found in Sec. D.2.

Discussion The hash combiner is very efficient: It does not increase the round complexity and only requires one hash evaluation in addition to the parallel execution of a single instance of each candidate scheme. Moreover, in contrast to the XOR combiner in the appendix, it does not require the candidate schemes to output bitstrings, and it achieves security in the UC framework. The downside of the hash combiner is that its robustness proof relies on observable adaptively (re)programmable random oracles and, consequently, we state it in the classical random oracle model (ROM). Since the feasibility of carrying these properties over to the quantum random oracle model (QROM) remains an open question, the hash combiner is not known to be robust against quantum adversaries.

5 Instantiations

In this section, we discuss how candidate schemes suitable for our combiners, i.e., OPRFs with unconditional client security, can be instantiated. As our main motivation is to construct a hybrid OPRF, we show both classical and post-quantum constructions.

5.1 Classical Construction: 2HashDH

Figure 4 displays the popular 2HashDH OPRF [JKKX16, JKX18a]. It is essentially a Diffie-Hellman key exchange where the generator is derived as $\mathsf{H}(x)$ for PRF input x. This generator is then exponentiated with a "secret blinding key" r of the client and subsequently exponentiated with the secret key k of the server (the PRF key). The client then exponentiates with r^{-1} to obtain $\mathsf{H}(x)^{rkr^{-1}} = \mathsf{H}(x)^k$ and the final PRF value as $\mathsf{H}'(x, \mathsf{H}(x)^k)$.

Unconditional client security demands strong protection of the client's input, but also of its output. For the input, the blinded hash statistically hides x, so this part is relatively straightforward. For the output, though, it may look like 2HashDH can only achieve statistical client security under some computational assumption. Concretely, the om-GapDH assumption in the 2HashDH proof [JKKX16] is used exactly to deal with offline evaluations of $F(k, x) = \mathsf{H}'(x, \mathsf{H}(x)^k)$. The important observation is that the om-GapDH assumption is *only used in the case where the server is honest*. In other words, $\mathcal{F}_{\text{OPRF}}$ only maintains a counter for the honest function table $\mathcal{F}_{\text{honest,sid}}(\cdot)$, and not for

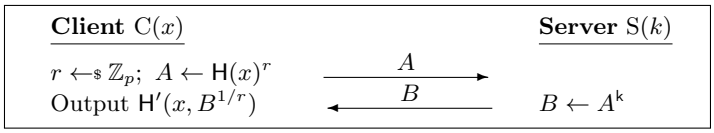


Fig. 4. 2HashDH OPRF [JKKX16].

the adversarial tables. This allows us to prove unconditional client security for 2HashDH. Consequently, 2HashDH can be used to instantiate Π^1_{OPRF} in Thm. 2 or any of the two OPRF candidates in Thm. 1.

Claim. The 2HashDH OPRF [JKKX16,JKX18a] UC-realizes $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$ in the $\mathcal{F}_{\mathrm{RO}}$-hybrid model unconditionally.

Proof sketch. This claim immediately follows from the proofs in [JKKX16] and [JKX18a]. In these proofs, the one-more gap DH assumption is only used to upper-bound the probability that the adversary is able to deplete the ticket counter. However, since $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$ imposes no limits on the number of OPRF evaluations for the honest key, the 2HashDH OPRF UC-realizes $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$ even if the one-more gap DH assumption is broken.

5.2 Post-Quantum Constructions

We are not aware of any direct post-quantum OPRF construction with unconditional security for the client. However, there are OT-based OPRF constructions which can be used in conjunction with appropriate OT protocols that achieve unconditional security for the receiver. In the following, we discuss this method for the Legendre OPRF [BDFH25] and the UC-Gold OPRF [YBH+25, Sec. 6.2]. Both of these constructions instantiate the 2Hash Framework by Beullens et al. [BDFH25] (see Sec. 3.2) with the Legendre PRF resp. power residue PRF [Dam90] and a VOLE-based secure two-party computation (2PC) protocol, the latter of which can be built from OT alone [BDFH25, Sec. 1.2], [YBH+25, Lem. 1]. In its original form, the 2HashDH framework requires the underlying PRF F to satisfy two properties:

- One-more unpredictability (omu): After seeing n evaluations of the function $F(k,\cdot)$ on random inputs under a secret key k, it should be hard for an adversary to predict the output of $F(k,\cdot)$ on the $(n+1)$-th input.
- Weak key-collision resistance (wkcr): Given n random inputs $x_1,\ldots,x_n$ from the domain of F, it should be hard for an adversary to find two different keys $k \neq k'$ such at least one of the n inputs x_i maps to the same function value $y = F(k,x) = F(k',x)$ under both keys.

The Legendre PRF and the power residue PRF are both unconditionally weak-key-collision-resistant [BDFH25, Lem. 7], [YBH+25, Lem. 2, Sec. 6.2].

Similarly to the 2HashDH OPRF, a closer look at the security proof of the 2Hash framework [BDFH25] reveals that it relies on the one-more unpredictability of the underlying PRF only for limiting the number of evaluations with the

honest key. However, $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ does not impose such a limit. In particular, one-more unpredictability is not required to hide the client's input or extract the server's key. This means that an OPRF constructed in the 2Hash framework UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ already if the underlying PRF f is weak-key collision resistant and the 2PC protocol is UC-secure. Since both constructions mentioned above satisfy the former requirement, we will focus on the underlying 2PC protocols. We can ignore the case that the 2PC protocol is insecure for the server because then the simulation is mostly straightforward[6]. Thus, we can narrow our focus on 2PC protocols with unconditional security for the client.

Unconditionally client-secure OT Because these 2PC protocols of the Legendre OPRF and the power residue OPRF ultimately rely on OT, we define a notion of unconditional security for one of the OT parties.

Definition 3 (Unconditional security for OT). *Let $\mathcal{F}_{\text{OT}}$ be the standard ideal functionality for oblivious transfer (see e.g. [CSW20, Fig. 2]). We define an unconditional security notion for the sender and receiver separately.*

- *We say that an OT protocol has* unconditional receiver security (URS) *if it UC-realizes $\mathcal{F}_{\text{OT}}$ in the case of an honest receiver and a statically corrupt sender such that the view of the distinguishing environment $\mathcal{Z}$ in the simulated world is statistically close to the real protocol execution.*
- *We say that an OT protocol has* unconditional sender security (USS) *if it UC-realizes $\mathcal{F}_{\text{OT}}$ in the case of an honest sender and a statically corrupt receiver such that the view of the distinguishing environment $\mathcal{Z}$ in the simulated world is statistically close to the real protocol execution.*

The Legendre OPRF. Beullens et al. [BDFH25] go through a sequence of transformations to construct their OPRF from oblivious transfer. The sequence is depicted in Fig. 5, where $\binom{2^k}{2^k-1}$-OT means an OT where the receiver can get 2^k-1 random messages while the sender gets all 2^k messages, for some $k \in \mathbb{N}_+$, sVOLE means small-set VOLE, and VOLE+ is defined by Beullens et al. [BDFH25] to output additional values to the parties that are needed for their ZK proofs. Note, however, that all transformations are *information-theoretic*, i.e., there already exist simulators for the respective protocol that work without any further computational assumption except that the underlying protocol is secure.

This means that establishing that the Legendre OPRF UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ unconditionally boils down to showing that the underlying OT has unconditional security for the receiver.

Theorem 3. *The Legendre Symbol OPRF UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ if it is instantiated either (1) With OT extensions that use a statistical zero-knowledge proof and with an unconditionally sender-secure base OT (2) Or using an unconditionally receiver-secure OT directly.*

[6] The simulation works almost exactly like the one in [BDFH24, Fig. 10], except that the simulator internally runs the real code of the 2PC protocol to answer 2PC messages from the corrupted client, and E_{excess} can never occur.

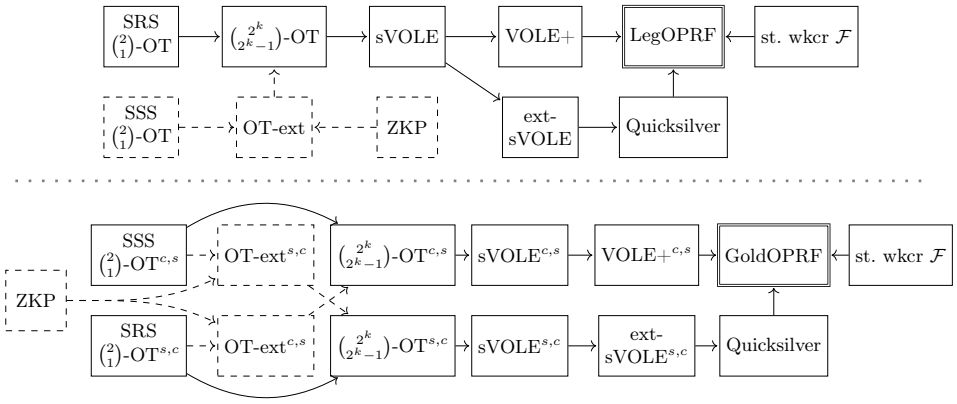


Fig. 5. The security of the Legendre OPRF from Beullens et al. [BDFH25] (top) and the Gold OPRF from Yang et al. [YBH+25] (bottom) relies on the security of the underlying OT via a sequence of information-theoretic transformations. The base-OT sender and receiver can be switched by either using OT extensions with an unconditionally sender-secure (USS) OT (dashed) or a unconditionally receiver-secure (URS) base OT directly. We write ^{s,c} if the OPRF server is the OT receiver and ^{c,s} if the OPRF server is the OT sender. "st. wkcr $\mathcal{F}$" means that the OPRF requires a function family $\mathcal{F}$ with statistical weak key-collision resistance.

Proof. First, recall that assuming an ideal 2PC protocol, the security of the Legendre OPRF depends only on the weak key-collision resistance and one-more unpredictability of the PRF (see Sec. 5.2) As discussed above, the unpredictability of the Legendre PRF and the server security of the 2PC protocol are only used to argue the security of the honest server, so we can focus on weak key-collision resistance and the receiver-security of the 2PC protocol. By [BDFH25, Lem. 7], the former holds statistically for the Legendre OPRF. The 2PC protocol is based on $\mathcal{F}_{\text{VOLE+}}$ and $\mathcal{F}_{\text{ZK}}$. Observe that the VOLE+ simulator given in [BDFH25, Thm. 2] is statistically secure in the sVOLE-hybrid model with simulation error $2^{-s} + \binom{n}{2}p^{-1}$, where s is a statistical security parameter, n is the number of sVOLE correlations and p is the size of the prime order field $\mathbb{F}_p$.

Similarly, the simulator of [YSWW21, Thm. 3] for QuicksilverZK, the protocol employed in the Legendre OPRF to instantiate $\mathcal{F}_{\text{ZK}}$, is also statistically secure in the extended sVOLE-hybrid model. Extended sVOLE is, in turn, statistically secure in the sVOLE-hybrid model [YSWW21, Thm. 1] with simulation error $(d-1)/p^r$, where p^r is the size of the field $\mathbb{F}_{p^r}$ over which the correlations are computed, and d is the degree of the polynomials in the statement. In the OPRF, one has $d = \mathcal{O}(\ell)$, where ℓ is the length of the sequence of Legendre symbols.

This means that for simulating both protocols, the VOLE+ and the ZK proof, the security relies on sVOLE. By [BBD+23, Thm. 1] this is perfectly secure in the $\binom{2^k}{2^k-1}$-OT-hybrid model. A maliciously secure $\binom{2^k}{2^k-1}$-OT can be constructed

in the $\binom{2}{1}$-OT-hybrid model [Roy22, Thm. 6.1] and [Roy22, Thm. 6.3]. The proof uses the security of a length-doubling PRG (for constructing a GGM tree) and collision-resistance of a hash function. As we are already working in the ROM, both can be replaced by a random oracle to obtain unconditional security in the $\binom{2}{1}$-OT-hybrid model. Also note that [Roy22, Thm, 6.1] and [Roy22, Thm. 6.3] hold with respect to *endemic OT*, i.e., an OT where a malicious party can influence the OT correlation it receives. This is weaker than the usual "uniform" OT, where the output has to be uniform for both parties [MR19]. At this point, there are two options: Directly using a $\binom{2}{1}$-OT to get the necessary OT correlations or using OT extensions to boost a small number of base OTs to the required number of OT correlations. Using OT extensions is likely going to be more efficient, but it will reverse the roles of OT sender and receiver. Therefore, the requirements on the OT for making the OPRF unconditionally client secure change. If no OT extensions are used, the OPRF client is executing the OT receiver for the base OTs. Thus, we require an OT with unconditional receiver security. We discuss possible instantiations of the base OT below in Sec. 5.2.

Let us now consider the OT extension case: Concretely, we will rely on the enhanced analysis of OT extensions by Masny and Rindal [MR19]. If the underlying base OT has endemic OT security and the employed zero-knowledge proof is secure, then the OT extension protocol has endemic OT security in the ROM [MR19, Lem. 5.8]. Observe that the OT extension receiver proves a statement to the OT extension sender in zero-knowledge. That means in particular that the security analysis of [MR19, Lem. 5.8] uses the zero-knowledge property of the zero-knowledge proof (implicitly in their Hybrid 4), and we need this to hold statistically. Finally, because the roles of sender and receiver were swapped, the base OT now has to have unconditional sender security (USS).

In summary, an unconditionally secure simulator for an honest LegOPRF client will proceed as follows:

- Say the whole protocol execution requires n_{OT} OT correlations. In the no-extension case, the OPRF client is the OT receiver. Thus, the simulator uses the URS-OT simulator to generate the n_{OT} correlations directly. From the URS of the OT, this simulation produces OT correlations with unconditional security for the OT receiver.
  In the OT extension case, the (potentially malicious) OPRF server will play the role of the OT receiver in the base OTs, and the OPRF client plays the OT sender. The necessary base OT can be generated using the USS-OT simulator. Because these are unconditionally secure OT correlations for the OT sender, the OT extensions will transform them to n_{OT} OT correlations with unconditional security for the OPRF client (if the zero-knowledge proof is statistically zk).
- Next, the simulator can use these OT correlations to one-by-one run the simulators for the all-but-one-OT, the sVOLE, the VOLE+, the extended s-VOLE, QuicksilverZK, and finally the LegOPRF simulator. All these sim-

ulators have unconditional security, as argued above, and thus the whole simulation strategy offers unconditional security.

The GoldOPRF. The GoldOPRF [YBH+25] also uses the 2Hash framework. That means the security of the client relies on wkcr (see Sec. 5.2) and the underlying 2PC protocol. By [YBH+25, Lem. 2, Sec. 6.2], the weak key-collision resistance holds statistically. The 2PC protocol described by Yang et al. [YBH+25, Thm. 2] is statistically secure in the VOLE-hybrid model, where two VOLE protocols are executed (potentially in a preprocessing phase). In one of the VOLEs, the OPRF client is the VOLE sender and the OPRF server is the VOLE receiver, and in the other VOLE, the roles are swapped, i.e., the OPRF client is the VOLE receiver and the OPRF server is the VOLE sender. We will write $\text{VOLE}^{s,c}$ for the first case (OPRF.Server = VOLE.Receiver) and $\text{VOLE}^{c,s}$ for the second case (OPRF.Server = VOLE.Sender). In this terminology, the Legendre OPRF discussed in Sec. 5.2, relied on $\text{VOLE}+^{s,c}$.

Gold relies on similar building blocks as the Legendre OPRF. The construction also relies on Quicksilver as zero-knowledge (ZK) proof, and for non-batched executions, Yang et al. recommend to instantiate the VOLE protocol with the VOLE+ protocol from [BDFH25]. The difference is that the semi-honestly secure core of their protocol requires $\text{VOLE}^{c,s}$, and the Quicksilver proof that is needed for malicious security requires $\text{VOLE}^{s,c}$. We depict the sequence of transformations in Fig. 5. The proof of the next theorem is analogous to the one of Thm. 3.

Theorem 4. *The GoldOPRF UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ if it is instantiated either (1) with OT extensions that use a statistical ZK proof and with an URS base OT for the "core" protocol and a USS-based OT for Quicksilver, (2) with OT extensions that use a statistical ZK proof and with an URS base OT for the "core" protocol and using an URS OT for Quicksilver directly, (3) with OT extensions that use a statistical ZK proof and with an USS base OT for Quicksilver and using an USS OT for the "core" protocol directly, (4) or using an USS OT for the "core" protocol directly and using an URS OT for Quicksilver directly.*

Instantiating the oblivious transfer. We discuss several avenues of constructing UC-secure OT with unconditional security for one party.

Masny and Rindal [MR19] show a generic framework to transform a two-round key exchange protocol into a UC-secure endemic OT. As it turns out, instantiating their framework with Diffie-Hellman yields an unconditionally receiver-secure OT [MR19, Cor. D.3]. However, their Kyber-based instantiation only leads to a computationally receiver-secure OT. This is because a Diffie-Hellman message $A = g^a$ readily is a uniformly random element in the respective group, whereas a Kyber public key is only computationally close to a uniformly random value because it contains an MLWE sample. More concretely, the Masny-Rindal OT yields an unconditionally receiver-secure OT if the underlying two-round key exchange has statistical uniformity, i.e., the following property is satisfied:

Definition 4 (Statistical uniformity). *We say that a two-round key exchange protocol has statistical uniformity iff for all* unbounded *distinguishers* D *and all poly. size hints* $z \in \{0,1\}^*$*, we have* $|\Pr[\mathsf{D}(z, \mathit{msg}_1) = 1] - \Pr[\mathsf{D}(z, u) = 1]| \leq \mathsf{negl}(\lambda)$*, where* $\mathit{msg}_1 \in \mathbb{G}$ *is the first message in the key exchange and* $u \leftarrow\!\!\$\ \mathbb{G}$*. (The messages have to have a group structure for the Masny-Rindal framework.)*

Unfortunately, this does not hold for most post-quantum-secure KEMs, e.g., not for Kyber [SAB+22], HQC [GMA+25], or NTRU [CDH+22]. Also, using the Masny-Rindal OT with Isogenies is currently not an option because this would require hashing into the family of supersingular elliptic curves, which to date remains an open problem.

Peikert, Vaikuntanathan, and Waters [PVW08] also show a generic framework for achieving UC-secure OT in the static corruption model. The framework builds upon an abstraction called *dual-mode encryption*. The appealing feature is that the framework can be instantiated to either provide USS or URS security, depending on the *mode* in which the dual-mode encryption is set up. The paper presents instantiations from DDH, the quadratic residuosity and the decisional composite residuosity assumption, and LWE.

Furthermore, Alamati, De Feo, Montgomery, and Patranabis [ADMP20] constructed a dual-mode encryption scheme from group actions such as CSIDH. Combining this with [PVW08] yields both a URS and a USS OT from Isogenies.

Direct construction. There is also a direct construction of post-quantum UC-secure URS OT: The CSIDH-based construction of Lai, Galbraith, and Delpech de Saint Guilhem [LGD21] does not go through a generic framework but still achieves the required properties.

Summary. For both URS and USS OTs, there are post-quantum-secure instantiations from LWE and from Isogenies. The LWE constructions and the Isogeny-based USS OT use the framework of Peikert, Vaikuntanathan, and Waters [PVW08]. For Isogeny-based URS OT, one can either use [LGD21] or combine [PVW08] with [ADMP20]. The implementations of the Legendre OPRF [BF] and GoldOPRF [Yan] in their current form both do **not** provide unconditional client security under post-quantum assumptions, as they both rely on the Masny-Rindal OT [MR19] with Kyber as key exchange.

Table 1 provides an overview of how further OPRFs from the literature satisfy unconditional client security. However, we note that the ones discussed here in detail are the only suitable candidates for instantiating the combiner, as the other constructions do not achieve composable security guarantees.

6 On the Hardness of Constructing OPRF Combiners

In this section, we present formal evidence that constructing black-box combiners for OPRFs is hard.

6.1 Reduction to the Impossibility of OT Combiners?

In general, when showing impossibility results for robust combiners, one must bypass the trivial solution of *non-black-box* combiners: Assuming that a secure

OPRF	$\mathcal{F}_{\text{OPRF}}$	$\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$	$\mathcal{F}_{\text{OPRF}}^{\infty}$	$\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$	Comment
2HashDH [JKKX16]	✓	×	✓	×	Section 5.1
NR-OT [FIPR05]	×	×	×	×	Depending on OT
NR-Hom [JKK14]	×	×	×	×	IND-CPA of H. Enc.
FHE-based [ADDG24]	×	×	×	×	IND-CPA of FHE
Lattices [AG24]	×	×	×	×	dRLWE
LeOPaRd [ESTX24]	×	×	×	×	knMLWE
Legendre OPRF [BDFH25]	×	✓	×	✓	Section 5.2
GoldOPRF [YBH+25]	×	✓	×	✓	Section 5.2

Table 1. Columns $\mathcal{F}_{\text{OPRF}}$ and $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$ indicate whether the protocol realizes the respective ideal functionality computationally, and the columns with their ∞ counterparts indicate whether the protocol realizes those unconditionally. The comments explain why or why not unconditional client security is achieved.

implementation of the primitive in question exists, in theory, there is a trivial combiner that simply ignores the candidate schemes and instead outputs a hardwired secure implementation of the primitive [HKN+05]. Of course, such a combiner is not very useful, as we are unable to reliably realize it in practice. Instead, we are interested in *black-box* combiners that actually use the candidate schemes at hand in order to build a combined scheme. If we can rule out the existence of such combiners, we can conclude that there is no hope of coming up with a useful combiner. Unfortunately, in general, formalizing what it means for a combiner to "actually use" a candidate scheme is delicate.

To deal with this, we use a technique due to Harnik, Kilian, Naor, Reingold and Rosen [HKN+05], who showed that there is no transparent black-box 1-out-of-2 combiner for OT. In their work, they resorted to black-box combiners, i.e., combiners that treat their candidate schemes as black boxes and whose security proof consists of a fully black-box reduction (see Def. 1). An advantage of these reductions is that they are *relativizing*, i.e., they hold relative to *every* oracle. This gives rise to a setting in which one can disprove the existence of a combiner that "actually uses" its candidate schemes. To this end, Harnik et al. consider a set of oracle worlds that we discuss in more detail below. All of these oracle worlds have in common that they contain a PSPACE-complete oracle that all parties (including the adversary) can access. This oracle ensures that all "hardwired" cryptographic schemes in these worlds are insecure, including potential ones "brought" by the combiner itself. The candidate schemes are then provided to the combiner by additional oracles. Since the implementations of these candidate schemes are external to the oracle worlds, the PSPACE-complete oracle does not affect their security. Consequently, the only way to obtain a secure implementation of a primitive in such an oracle world is to actually use the provided candidate schemes through the oracles.

Harnik et al. put forward two oracle worlds, $World_1$ and $World_2$, both of which contain a PSPACE-complete oracle, as well as two oracles P_A and P_B that provide *secure* implementations of the primitive P in question (which in their case

is OT). Moreover, $World_1$ contains an oracle Invert_A which completely breaks[7] the security of P_A, whereas $World_2$ contains an oracle Invert_B which completely breaks the security of P_B. See Fig. 6 for a depiction of the oracle worlds. Since in each world, only one (oracle-provided) candidate scheme is broken, while the other one remains secure, a secure combiner with black-box access to the candidate schemes via P_A and P_B is expected to output a combined scheme which is secure in $World_1$ *and* in $World_2$.

Harnik et al. next introduce a third world, called the "Bare World" (see Fig. 6, middle part). In the bare world, there is only a PSPACE-complete oracle (and, in our proof later, an additional random oracle). The oracles for the candidate schemes are replaced with a *concrete* and *efficient* implementation simulating these oracles, which works as follows: The server is responsible for simulating P_A, and the client is responsible for simulating P_B. If a party in the combined protocol $\mathrm{P}_{\mathrm{cmb}}$, the client or the server, wants to query an oracle for which it is responsible itself, it simply runs the implementation of the oracle locally to obtain the result. If a party wants to query an oracle for which the other party is responsible, it sends its query in plain to the other party, which then locally computes the answer and sends it back to the requester, also in plain (see Fig. 6). In the bare world, the combined protocol $\mathrm{P}_{\mathrm{cmb}}$ thus operates with concrete implementations only, not relying on any ideal objects. The PSPACE-complete oracle in the bare world, however, renders any such scheme insecure, implying the existence of a successful adversary $\mathcal{A}$ in this world. Harnik et al. show that the view of $\mathcal{A}$ can be simulated in either $World_1$ or $World_2$ with the help of Invert_A resp. Invert_B, causing the attack to work in one of these worlds as well. This yields a contradiction to the assumption about the security of the combiner.

To make the above argument work, Harnik et al. introduced the notion of *transparent* combiners. Intuitively, these are combiners that use the candidate schemes only in the intended, interactive way. This means that as soon as a candidate scheme has generated a message to be sent to the other party (via the primitive's next-message oracle), the combiner must actually do so. This is necessary to simulate, in $World_1$ or $World_2$, the view $\mathcal{A}$ expects.

Definition 5 (Transparent black-box combiner [HKN+05]). *A transparent black-box combiner is a black-box combiner for an interactive primitive, where every call to a candidate's next-message oracle is followed by this message being sent to the other party.*

Theorem ([HKN+05, Thm. 4.1]). *There exists no construction of a transparent black-box 1-out-of-2 OT combiner.*

A false proof attempt for OPRFs. It might now appear that we can prove our impossibility result for OPRFs via a reduction to the impossibility of OT combiners using the following argument: Assuming that there exists a transparent

[7] In the case of OT, this oracle takes the transcript of a protocol execution and returns the receiver's choice bit as well as both of the sender's inputs.

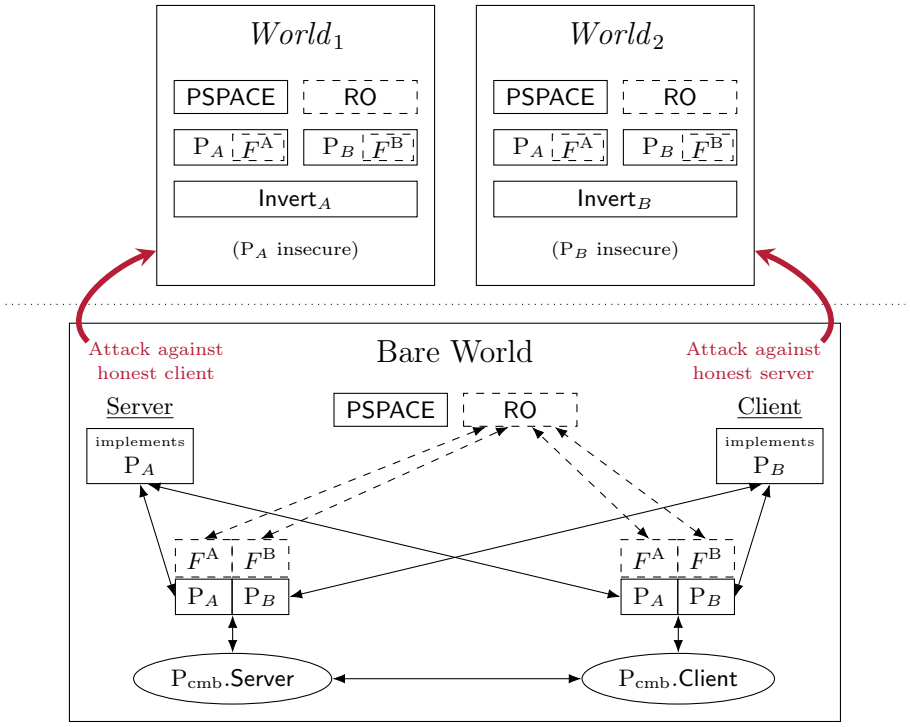


Fig. 6. Oracle worlds in the impossibility proof for combiners for primitive $\mathrm{P} \in \{\mathrm{OT}, \mathrm{OPRF}\}$ by Harnik et al. (OT) and in our proof (OPRF). The combined scheme $\mathrm{P}_{\mathrm{cmb}}$ is constructed from the candidate schemes P_A and P_B. Dashed parts only appear in our proof, which includes the oracles F^A and F^B for non-interactive evaluations of the primitive that only apply to OPRFs.

black-box 1-out-of-2 combiner for OPRFs, we build a transparent black-box 1-out-of-2 combiner for OT, which contradicts the theorem by Harnik et al. To build the OT combiner, we use the well-known fact that OT is 2PC-complete, i.e., that given a secure OT protocol, one can compute any function in a secure way. This allows us to construct OPRF_A from the first candidate scheme OT_A via a secure two-party computation of some PRF, and, similarly, a second OPRF OPRF_B from the second candidate scheme OT_B. We then use the assumed OPRF combiner to obtain a combined OPRF protocol $\mathsf{OPRF}_{\mathrm{cmb}}$. Finally, we use the well-known fact that OPRF implies OT [CHL22, Sec. B.1] (given an authenticated channel) to construct a combined OT scheme $\mathsf{OT}_{\mathrm{cmb}}$.

Perhaps counter-intuitively, the above reasoning does not allow us to preclude the existence of a black-box OPRF combiner using [HKN+05, Thm. 4.1] because the described reduction is not *black-box*. Recall that the construction idea is to use the circuit description of a regular PRF and run a 2PC protocol for the interactive evaluation. In particular, the combiner makes use of the implementation details of the underlying PRF, so its security proof is not relativizing

(and, hence, not fully black-box). Specifically, relative to the PSPACE-complete oracle used by Harnik et al., the PRFs inside both candidate schemes become insecure. This means that we do not have any security guarantees at all for the combined OPRF, and thus, in turn, no security for the resulting OT protocol.

6.2 Impossibility of Transparent Black-Box OPRF Combiners

Since we cannot use the result by Harnik et al. as is, we uncoil their proof technique to prove our impossibility result for OPRFs (Thm. 9). For technical reasons[8], we state our result (Sec. G) with respect to a game-based security notion (Sec. F). Yet, in order to discuss the main technical challenges and how we addressed them, it is sufficient to consider an informal version of our result.

Theorem 9 (informal). *There is no strongly*[9] *transparent black-box 1-out-of-2 combiner for OPRFs that preserves correctness and security in the ROM.*

An overview of our approach. Our high-level argument follows the original result by Harnik et al. [HKN+05]. However, it turns out that the adaptation to OPRFs requires notable changes beyond the syntactic differences between OT and OPRFs. Furthermore, we believe that the transparency condition by Harnik et al. is not strong enough to prove their impossibility result for OT. By requiring a slightly stronger, albeit still natural form of transparency, we obtain an impossibility result for OPRFs. Our fix applies to the original proof for OT, too. Before we elaborate on our changes, we provide an overview of the parts that are similar.

Like Harnik et al., we start by assuming that a transparent black-box combiner for the primitive P at hand (in our case, P = OPRF) exists and define two oracle worlds $World_1$ and $World_2$ with a PSPACE-complete oracle, candidate scheme oracles P_A and P_B, as well as break oracles Invert_A (in $World_1$ only) and Invert_B (in $World_2$ only), as discussed above. Naturally, in our proof, P_A and P_B implement OPRFs instead of OT. Consequently, we also refer to them as OPRF_A and OPRF_B. The Invert oracles in our setting break the pseudorandomness and the obliviousness of the OPRFs, analogously to the OT scenario where they break the security of both the sender and the receiver. We note that this models the general case that the starting primitives achieve both properties in a computational sense only, and thus there is no contradiction to our constructive solutions with unconditional security for one of the two properties from Sec. 4.2. In addition to this setup, which is the same as for OT, we add a random oracle RO to each world in order to obtain a result in the random oracle model. While the random oracle is necessary for our proof, we believe that, since our combiner in Sec. 4 as well as most OPRF constructions in the literature are stated in the ROM, it is reasonable to phrase our impossibility result in this model, too.

[8] It is unclear how to design a (global) OPRF oracle which allows a simulator to extract the key from the server.

[9] We discuss the difference between transparency and strong transparency below.

We next show that OPRFs imply weakly secure, unauthenticated key exchange (against passive adversaries) via a fully black-box reduction by adapting a result due to Gertner, Kannan, Malkin, Reingold and Viswanathan [GKM+00, Thm. 5] to OPRFs. We then use the seminal result by Impagliazzo and Rudich [IR89, Cor. 6.1] on the impossibility of key exchange from a random oracle to deduce that there exists an efficient passive adversary against the key exchange protocol. Combined with the fact that fully black-box reductions are relativizing [RTV04], we can, hence, conclude that there exists an efficient passive adversary against the combined OPRF protocol in the bare world. Given the transcript of the network messages in the bare world, this adversary can either tell a corrupted server which of two inputs the honest client used, or predict a previously unqueried OPRF output for a corrupted client.

The next step is to translate the attack from the bare world into a successful attack against the combined protocol in World_1 or World_2. In these worlds, calls to P_A resp. P_B are answered using the actual oracles in these worlds instead of local implementations of these oracles that are queried over the network (see Fig. 6). Nevertheless, Harnik et al. observed that the missing network messages can be simulated. In particular, the view of a corrupted server in the bare world can be simulated in World_1, and the view of a corrupted client in the bare world can be simulated in World_2. Let us consider the first case: In the bare world, a corrupted server queries P_B over the network, whereas in World_1, the queries are local. Since it is the corrupted server itself that asks these queries, the associated network messages are easy to simulate. However, in the bare world, the corrupted server also receives the queries to P_A from the honest party, whereas in World_1, they are local. Here, the transparency condition of the combiner comes into play: Remember that transparency mandates that whenever the "next-message" oracle for the implementation of a candidate returns a message, this message must immediately be sent to the other party. Consequently, the server immediately learns the answer to this query, and can call its own Invert_A oracle to recover the corresponding query, which allows it to simulate the associated network messages as well. The simulation of the corrupted client's view from the bare world in World_2 works analogously.

Revisiting transparency. The presentation by Harnik et al. does not precisely define the "next-message oracle" in their context. They define their OT oracle $\mathsf{OT} \coloneqq (f_1, f_2, R)$ as a collection of three oracles f_1, f_2 and R which implement a two-message[10] OT protocol. The first two oracles f_1 and f_2 return the first resp. second protocol message, while the third oracle R is used by the client to determine its output after receiving the last message. The loose interpretation of Harnik et al.'s transparency definition lets the "next-message" portion of their OT oracle consist of f_1 and f_2 only.

The above understanding of transparency excludes the oracle R. This sheds light on a gap in this argument: In Harnik et al.'s proof, as well as ours, P_A

[10] In a two-message OT protocol, which is round-optimal, the first message is sent from the client to the server and the second message from the server back to the client.

and P_B are designed such that, given all queries to and answers from f_1 and f_2, it is possible to compute the answers of R for any query with overwhelming probability. While the simulator, hence, learns the answers to all possible queries to R that the non-corrupted party *may* perform, this is not sufficient to simulate the network messages from the bare world. For that, the simulator additionally needs to know exactly *which* queries the honest party asks to R and *at which point in the execution* they occur. Moreover, the fact that the honest party follows the protocol genuinely does not help here. For example, the combiner might run a candidate's final processing of the last protocol message m_2 at the client (which involves a call to R) only with a certain probability, or perhaps even only if other messages and oracle queries fulfill certain conditions, which may be unknown to the simulator. Because of this, we require a stronger notion of transparency:

Definition 6 (Strongly Transparent Black-Box Combiner [HKN$^+$05]). *Let* Comb *be a transparent black-box combiner for an interactive primitive whose interface provides an oracle R which is (exclusively) used to derive the protocol output from the last protocol message and some internal state. We say that* Comb *is strongly transparent iff, in addition, the following holds: (1) For each execution of any candidate scheme, R is called exactly once upon receipt of the last protocol message $\bar{m}$ with $\bar{m}$ and the associated internal state for this interaction. (2) There are no other calls to R.*

Intuitively, this restriction means that the combiner always needs to compute the output of a protocol execution of any candidate scheme as soon as it has acquired the required data to do so; it is nonetheless free to ignore this output in any further computation. Besides, it must not attempt to compute a protocol output on artificial messages not received over the network. Strong transparency is, thus, closer to our intuition that the combiner must "use the candidate schemes only in the intended, interactive way" from above. We note that the additional limitation imposed by strong transparency is quite mild for black-box combiners: Since the combiner cannot assume anything about the structure of the protocol messages and the state produced by the candidates, and the candidates can validate their inputs, it cannot do much with these values apart from invoking the candidates in the intended way.

A technical challenge: non-interactive evaluations. With the strengthened transparency condition, Harnik et al.'s argument for OT directly extends to the sub-oracles for the *interactive* evaluation of OPRFs[11]. As in the OT case, we define three oracles f_1, f_2, and R, where the former two compute protocol messages and the latter computes the output of the OPRF. The simulation works exactly as discussed above, with marginal syntactic changes. However, we now run into a new issue: In contrast to OT, OPRFs also allow for a *non-interactive* evaluation of the underlying function by the server. Thus, our OPRF candidate oracles need to provide an interface for such non-interactive evaluations

[11] Strong transparency can also be used to complete Harnik et al.'s proof for OT.

in the form of a (sub)oracle, and we need to designate these suboracles either as "next-message-oracles" or "non-next-message oracles". But both options seem inappropriate: Classifying the suboracles for non-interactive evaluations as next-message oracles is formally possible, but leads to an unfaithful model because this would require the candidate schemes to send the function values resulting from non-interactive evaluations directly to the other party. However, OPRFs are not expected to do this, so we do not want to take this approach. On the other hand, making these suboracles *non*-next-message suboracles does not work either: Like in the case of the R suboracle above, non-next-message suboracles need not be transparent, so the honest party in the bare world might, as part of executing the combined scheme, perform queries to the oracle for the insecure candidate scheme that the adversary in $World_1$ resp. $World_2$ cannot learn, causing the simulation in these worlds and, consequently, our proof to fail. We also do not want to restrict non-interactive OPRF evaluations (like we did for calls to R) because this would impose a severe and unnatural restriction on the combiners captured by our result. After all, non-interactive evaluations are an essential part of the OPRF interface and have clearly defined semantics even if there is no associated interactive evaluation.

To solve the problem with non-interactive evaluations, we exploit the fact that our impossibility result is stated in the ROM, and implement the non-interactive OPRF evaluations in the bare world as calls to the random oracle RO (see Fig. 6). We can show that it is possible to simulate OPRF_A and OPRF_B in the bare world even if the underlying PRF is provided as a fixed, external random oracle. Thanks to the seminal result by Impagliazzo and Rudich that relative to a random oracle and a PSPACE oracle, no weakly-secure key exchange protocol exists, together with our black-box reduction from weakly-secure key exchange to OPRFs (see Lem. 5), we can still deduce the existence of an attack against the combined OPRF in the bare world. Yet, since non-interactive evaluations of the candidate OPRFs in the bare world no longer cause network messages to the other party, there is no need to simulate these queries in $World_1$ resp. $World_2$ anymore. Consequently, we can construct a successful simulator for the adversary in these oracle worlds as above to complete the proof of our impossibility result.

The full proof and the formal details of the impossibility result are deferred to Sec. G.

6.3 Unfolding the Possibilities for OPRF Combiners

Our impossibility result rules out transparent black-box 1-out-of-2 combiners for OPRFs, implicitly pointing at options to resurrect secure combiners by avoiding the technical restrictions. First note that we only refer to *1-out-of-2* combiners and make no claim about m-out-of-n combiners with $1 < m < n$. For example, for other primitives, the provably lowest values for $m, n \in \mathbb{N}_+$ such that an m-out-of-n combiner exists are $m = 2$, $n = 3$ for transparent black-box OT combiners [HKN+05] and $m = 3$, $n = 4$ for structural indistinguishability obfuscation (iO) combiners [FHNS16]. Moreover, our impossibility result may

be bypassed by combiners that use the candidate OPRFs in an irregular, non-interactive manner, or even in a non-black-box way.

Lastly, and most importantly in light of our constructive solutions, the proof of our result leverages the fact that broken schemes may be *completely* insecure, i.e., their obliviousness *and* pseudorandomness can *both* be successfully attacked. Our combiner from Sec. 4.2 circumvents the impossibility result by requiring the candidate schemes to always retain security for the client, even if hardness assumptions break. In the proof above, this would cause the Invert oracles to return only partial information (instead of the whole original queries), implying that the view from the bare world cannot be accurately simulated in $World_1$ and $World_2$, so the attack from the bare world may no longer work in those worlds. In a sense, our impossibility result thus shows that assuming unconditional security of one of the two security properties is necessary.

Acknowledgements

We would like to thank Ward Beullens for the insightful discussion on this project. Furthermore, we express our gratitude to Stanislaw Jarecki, who identified an error in an earlier revision of this work that caused $\mathcal{F}_{\text{CorH-OPRF}}$ to be weaker than intended. We would also like to thank Lena Heimberger for pointing out some inaccuracies regarding the literature on Isogeny-based OPRFs, and Jacqueline Koch for pointing out an error in the game-based obliviousness definition in an earlier draft of this paper. A first version of the TikZ code for Fig. 5 was generated using AI, see Sec. I.

References

ADDG24. Martin R. Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham. Crypto dark matter on the torus - oblivious PRFs from shallow PRFs and TFHE. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 447–476. Springer, Cham, May 2024.

ADDS21. Martin R. Albrecht, Alex Davidson, Amit Deo, and Nigel P. Smart. Round-optimal verifiable oblivious pseudorandom functions from ideal lattices. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 261–289. Springer, Cham, May 2021.

ADMP20. Navid Alamati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 411–439. Springer, Cham, December 2020.

AG24. Martin R. Albrecht and Kamil Doruk Gür. Verifiable oblivious pseudorandom functions from lattices: Practical-ish and thresholdisable. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 205–237. Springer, Singapore, December 2024.

AJN+16. Prabhanjan Ananth, Aayush Jain, Moni Naor, Amit Sahai, and Eylon Yogev. Universal obfuscation and witness encryption: Boosting correctness

and combining security. Cryptology ePrint Archive, Report 2016/281, 2016.

Bas24. Andrea Basso. A post-quantum round-optimal oblivious PRF from isogenies. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *SAC 2023*, volume 14201 of *LNCS*, pages 147–168. Springer, Cham, August 2024.

BBD+23. Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 581–615. Springer, Cham, August 2023.

BDFH24. Ward Beullens, Lucas Dodgson, Sebastian Faller, and Julia Hesse. The 2Hash OPRF framework and efficient post-quantum instantiations. Cryptology ePrint Archive, Report 2024/450, 2024.

BDFH25. Ward Beullens, Lucas Dodgson, Sebastian H. Faller, and Julia Hesse. The 2Hash OPRF framework and efficient post-quantum instantiations. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 332–362. Springer, Cham, May 2025.

BF. Ward Beullens and Sebastian Faller. GitHub - 2hashframework/legendreoprf — github.com. `https://github.com/2HashFramework/LegendreOPRF`. Last accessed 29-Sep-2025.

BGW88. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.

BIP+18. Dan Boneh, Yuval Ishai, Alain Passelègue, Amit Sahai, and David J. Wu. Exploring crypto dark matter: New simple PRF candidates and their applications. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part II*, volume 11240 of *LNCS*, pages 699–729. Springer, Cham, November 2018.

BKM+21. Andrea Basso, Péter Kutas, Simon-Philipp Merz, Christophe Petit, and Antonio Sanso. Cryptanalysis of an oblivious PRF from supersingular isogenies. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 160–184. Springer, Cham, December 2021.

BKW20. Dan Boneh, Dmitry Kogan, and Katharine Woo. Oblivious pseudorandom functions from isogenies. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 520–550. Springer, Cham, December 2020.

Can00. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. Cryptology ePrint Archive, Report 2000/067, 2000.

Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.

Can03. Ran Canetti. Universally composable signatures, certification and authentication. Cryptology ePrint Archive, Report 2003/239, 2003.

CD23. Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023,*

Part V, volume 14008 of *LNCS*, pages 423–447. Springer, Cham, April 2023.

CDH+22. Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hulsing, Joost Rijneveld, John M. Schanck, Peter Schwabe, William Whyte, Zhenfei Zhang, Tsunekazu Saito, Takashi Yamakawa, and Keita Xagawa. Ntru. `https://csrc.nist.gov/CSRC/media/Projects/post-quantum-cryptography/documents/round-3/submissions/NTRU-Round3.zip`, September 2022.

CFS+24. Graeme Connell, Vivian Fang, Rolfe Schmidt, Emma Dauterman, and Raluca Ada Popa. Secret key recovery in a Global-Scale End-to-End encryption system. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.

CHL22. Sílvia Casacuberta, Julia Hesse, and Anja Lehmann. SoK: Oblivious pseudorandom functions. In *2022 IEEE European Symposium on Security and Privacy*, pages 625–646. IEEE Computer Society Press, June 2022.

CK91. Benny Chor and Eyal Kushilevitz. A zero-one law for Boolean privacy. *SIAM Journal on Discrete Math.*, 4:36–47, 1991.

Clo25. Cloudflare. Privacy Pass. `https://developers.cloudflare.com/waf/tools/privacy-pass/`, February 2025.

CLP24. Mingjie Chen, Antonin Leroux, and Lorenz Panny. SCALLOP-HD: Group action from 2-dimensional isogenies. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14603 of *LNCS*, pages 190–216. Springer, Cham, April 2024.

CSW20. Ran Canetti, Pratik Sarkar, and Xiao Wang. Efficient and round-optimal oblivious transfer and commitment with adaptive security. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 277–308. Springer, Cham, December 2020.

Dam90. Ivan Damgård. On the randomness of Legendre and Jacobi sequences. In Shafi Goldwasser, editor, *CRYPTO'88*, volume 403 of *LNCS*, pages 163–172. Springer, New York, August 1990.

DEF+25. Pierrick Dartois, Jonathan Komada Eriksen, Tako Boris Fouotsa, Arthur Herlédan Le Merdy, Riccardo Invernizzi, Damien Robert, Ryan Rueger, Frederik Vercauteren, and Benjamin Wesolowski. PEGASIS: Practical effective class group action using 4-dimensional isogenies. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part I*, volume 16000 of *LNCS*, pages 67–99. Springer, Cham, August 2025.

DFG+23. Gareth T. Davies, Sebastian H. Faller, Kai Gellert, Tobias Handirk, Julia Hesse, Máté Horváth, and Tibor Jager. Security analysis of the WhatsApp end-to-end encrypted backup protocol. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 330–361. Springer, Cham, August 2023.

DFK+23. Luca De Feo, Tako Boris Fouotsa, Péter Kutas, Antonin Leroux, Simon-Philipp Merz, Lorenz Panny, and Benjamin Wesolowski. SCALLOP: Scaling the CSI-FiSh. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 345–375. Springer, Cham, May 2023.

DGH+21. Itai Dinur, Steven Goldfeder, Tzipora Halevi, Yuval Ishai, Mahimna Kelkar, Vivek Sharma, and Greg Zaverucha. MPC-friendly symmetric cryptography from alternating moduli: Candidates, protocols, and applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 517–547, Virtual Event, August 2021. Springer, Cham.

DLS21. Gérald Doussot, Marie-Sarah Lacharité, and Eric Schorn. End-to-End Encrypted Backups Security Assessment. https://research.nccgroup.com/wp-content/uploads/2021/10/NCC_Group_WhatsApp_E001000M_Report_2021-10-27_v1.2.pdf, October 2021.

ESTX24. Muhammed F. Esgin, Ron Steinfeld, Erkan Tairi, and Jie Xu. LeOPaRd: Towards practical post-quantum oblivious PRFs via interactive lattice problems. Cryptology ePrint Archive, Report 2024/1615, 2024.

FHNS16. Marc Fischlin, Amir Herzberg, Hod Bin Noon, and Haya Shulman. Obfuscation combiners. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 521–550. Springer, Berlin, Heidelberg, August 2016.

FIPR05. Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 303–324. Springer, Berlin, Heidelberg, February 2005.

FLÖ25. Karla Friedrichs, Anja Lehmann, and Cavit Özbay. Game changer: A modular framework for oprf security. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025*. Springer-Verlag, 2025.

FOO23. Sebastian H. Faller, Astrid Ottenhues, and Johannes Ottenhues. Composable oblivious pseudo-random functions via garbled circuits. In Abdelrahaman Aly and Mehdi Tibouchi, editors, *LATINCRYPT 2023*, volume 14168 of *LNCS*, pages 249–270. Springer, Cham, October 2023.

GKM+00. Yael Gertner, Sampath Kannan, Tal Malkin, Omer Reingold, and Mahesh Viswanathan. The relationship between public key encryption and oblivious transfer. In *41st FOCS*, pages 325–335. IEEE Computer Society Press, November 2000.

GMA+25. Philippe Gaborit, Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, Pascal Veron, Paulo Barreto, Shay Gueron, Tim Guneysu, Rafael Misoczki, Nicolas Sendrier, Jean-Pierre Tillich, Valentin Vasseur, Santosh Ghosh, and Jan Richter-Brokmann. Hqc (2025). https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/round-4/submissions/HQC-Round4.zip, March 2025.

GMR88. Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, April 1988.

GRR+16. Lorenzo Grassi, Christian Rechberger, Dragos Rotaru, Peter Scholl, and Nigel P. Smart. MPC-friendly symmetric key primitives. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 430–443. ACM Press, October 2016.

Her02. Amir Herzberg. Folklore, practice and theory of robust combiners. Cryptology ePrint Archive, Report 2002/135, 2002.

HHM+24. Lena Heimberger, Tobias Hennerbichler, Fredrik Meisingseth, Sebastian Ramacher, and Christian Rechberger. OPRFs from isogenies: Designs and analysis. In Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. Cárdenas, editors, *ASIACCS 24*. ACM Press, July 2024.

HKL+25. Lena Heimberger, Daniel Kales, Riccardo Lolato, Omid Mir, Sebastian Ramacher, and Christian Rechberger. Leap: A fast, lattice-based OPRF

with application to private set intersection. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 254–283. Springer, Cham, May 2025.

HKN+05. Danny Harnik, Joe Kilian, Moni Naor, Omer Reingold, and Alon Rosen. On robust combiners for oblivious transfer and other primitives. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 96–113. Springer, Berlin, Heidelberg, May 2005.

HL10. Carmit Hazay and Yehuda Lindell. Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries. *Journal of Cryptology*, 23(3):422–456, July 2010.

HR25. Julia Hesse and Michael Rosenberg. PAKE combiners and efficient post-quantum instantiations. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 395–420. Springer, Cham, May 2025.

IR89. Russell Impagliazzo and Steven Rudich. Limits on the provable consequences of one-way permutations. In *21st ACM STOC*, pages 44–61. ACM Press, May 1989.

JKK14. Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014.

JKKX16. Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online). In *2016 IEEE European Symposium on Security and Privacy*, pages 276–291. IEEE Computer Society Press, March 2016.

JKX18a. Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018.

JKX18b. Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. Cryptology ePrint Archive, Report 2018/163, 2018.

JKX21. Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. On the (in)security of the Diffie-Hellman oblivious PRF with multiplicative blinding. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 380–409. Springer, Cham, May 2021.

Leh19. Anja Lehmann. ScrambleDB: Oblivious (chameleon) pseudonymization-as-a-service. *PoPETs*, 2019(3):289–309, July 2019.

LGD21. Yi-Fu Lai, Steven D. Galbraith, and Cyprien Delpech de Saint Guilhem. Compact, efficient and UC-secure isogeny-based oblivious transfer. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 213–241. Springer, Cham, October 2021.

LKLCM21. Kristin Lauter, Sreekanth Kannepalli, Kim Laine, and Radames Cruz Moreno. Password monitor: Safeguarding passwords in microsoft edge, 2021. `https://www.microsoft.com/en-us/research/blog/password-monitor-safeguarding-passwords-in-microsoft-edge/`.

LL25. You Lyu and Shengli Liu. Hybrid password authentication key exchange in the UC framework. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 421–450. Springer, Cham, May 2025.

MMP+23. Luciano Maino, Chloe Martindale, Lorenz Panny, Giacomo Pope, and Benjamin Wesolowski. A direct key recovery attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 448–471. Springer, Cham, April 2023.

MP06. Remo Meier and Bartosz Przydatek. On robust combiners for private information retrieval and other primitives. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 555–569. Springer, Berlin, Heidelberg, August 2006.

MPW07. Remo Meier, Bartosz Przydatek, and Jürg Wullschleger. Robuster combiners for oblivious transfer. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 404–418. Springer, Berlin, Heidelberg, February 2007.

MR19. Daniel Masny and Peter Rindal. Endemic oblivious transfer. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 309–326. ACM Press, November 2019.

PVW08. Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 554–571. Springer, Berlin, Heidelberg, August 2008.

Rob23. Damien Robert. Breaking SIDH in polynomial time. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 472–503. Springer, Cham, April 2023.

Roy22. Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022.

RTV04. Omer Reingold, Luca Trevisan, and Salil P. Vadhan. Notions of reducibility between cryptographic primitives. In Moni Naor, editor, *TCC 2004*, volume 2951 of *LNCS*, pages 1–20. Springer, Berlin, Heidelberg, February 2004.

SAB+22. Peter Schwabe, Roberto Avanzi, Joppe Bos, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. Crystals-kyber (2022). `https://csrc.nist.gov/CSRC/media/Projects/post-quantum-cryptography/documents/round-3/submissions/Kyber-Round3.zip`, July 2022.

SHB23. István András Seres, Máté Horváth, and Péter Burcsi. The legendre pseudorandom function as a multivariate quadratic cryptosystem: security and applications. *Applicable Algebra in Engineering, Communication and Computing*, pages 1–31, 2023.

Som06. Christian Sommer. Robust combiners for cryptographic primitives. Master's thesis, ETH Zürich, September 2006. Available at `http://www.chsommer.com/combiner.pdf`.

TCR+22. Nirvan Tyagi, Sofía Celi, Thomas Ristenpart, Nick Sullivan, Stefano Tessaro, and Christopher A. Wood. A fast and simple partially oblivious PRF, with applications. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 674–705. Springer, Cham, May / June 2022.

Wha21. WhatsApp. Security of End-to-End Encrypted Backups. https://www.whatsapp.com/security/WhatsApp_Security_Encrypted_Backups_Whitepaper.pdf, September 2021.

Yan. Yibin Yang. GitHub - gconeice/pr-oprf: Gold OPRF in the paper: "Gold OPRF: Post-quantum oblivious power-residue prf" — github.com. https://github.com/gconeice/PR-OPRF. Last accessed 29-Sep-2025.

YBH+24. Yibin Yang, Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, and Tal Rabin. Gold OPRF: Post-quantum oblivious power residue PRF. Cryptology ePrint Archive, Report 2024/1955, 2024.

YBH+25. Yibin Yang, Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, and Tal Rabin. Gold OPRF: Post-quantum oblivious power-residue PRF. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 259–278. IEEE Computer Society Press, May 2025.

YSWW21. Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2986–3001. ACM Press, November 2021.

Table of Contents

A Universal Composability

The universal composability (UC) framework of Canetti [Can00, Can01] is a definitional framework for the modular modelling and security analysis of cryptographic protocols and primitives. The UC model allows to deduce security of composed protocols immediately, if the underlying building blocks are secure realizations of the required primitives.

The UC framework follows the *real-ideal paradigm*, i.e., security is defined as a relation between (the model of) *real protocols* and *idealized protocols*. In the *real world*, the parties send each other messages according to the code/instructions of the real world protocol. Ideal protocols work differently. Here, parties are instances of *dummy machines* that relay all inputs and outputs to an imaginary, incorruptible and trusted third party called *ideal functionality* via imaginary, perfectly secure channels. The ideal functionality will then compute the results of the desired distributed computation locally and send the results back to the dummy parties. The purpose of the ideal world is to define the (security of the) process which the real protocol is supposed to realize. Naturally, ideal protocols reside in the *ideal world*.

Both real and ideal protocols are modeled as machines in a system of interactive Turing machines (ITMs). In addition, the set-up also contains two distinguished machines, the *environment* $\mathcal{Z}$ and the *adversary* $\mathcal{A}$. The environment plays the role of an interactive distinguisher. It provides inputs to all parties and reads back their outputs. Furthermore, it can freely communicate with the adversary. In the real world, the adversary can exercise some control over the parties such as corrupt them or tamper with the network communication. The concrete power of the adversary depends on the specification of the protocol. In the ideal world, the adversary is called simulator. This simulator can interact directly with the functionality to cause side effects and must ensure that the environment can not tell the difference between the real and the ideal world. More formally, a (real) protocol π is said to *UC-emulate* an (ideal) protocol ϕ if and only if for every efficient adversary $\mathcal{A}$ there exists an efficient adversary $\mathcal{S}$ such that no efficient environment $\mathcal{Z}$ can distinguish between getting access to π and $\mathcal{A}$ and getting access to ϕ and $\mathcal{S}$. That is,

$$\mathrm{EXEC}_{\phi,\mathcal{S},\mathcal{Z}}(\lambda) \stackrel{\mathrm{c}}{\approx} \mathrm{EXEC}_{\pi,\mathcal{A},\mathcal{Z}}(\lambda)\,,$$

wherein $\mathrm{EXEC}_{\psi,\mathcal{B},\mathcal{Z}}(\lambda)$ denotes the probability ensemble for the (single-bit) output of the environment $\mathcal{Z}$ in the execution with environment $\mathcal{Z}$, adversary $\mathcal{B} \in \{\mathcal{A}, \mathcal{S}\}$, and protocol $\psi \in \{\pi, \phi\}$, and where $\stackrel{\mathrm{c}}{\approx}$ denotes computational indistinguishability.

The ideal protocol for an ideal functionality $\mathcal{F}$ is denoted by $\mathrm{IDEAL}_{\mathcal{F}}$. We say that a protocol π *UC-realizes* an ideal functionality $\mathcal{F}$ if and only if it UC-emulates $\mathrm{IDEAL}_{\mathcal{F}}$ and satisfies a technical condition called "subroutine respecting". For more details, we refer the reader to the latest version of [Can00]. Finally, we remark that the UC framework defines a generic composition operation and a composition theorem which can be used to analyze protocols consisting of one or more subprotocols in a modular fashion. This composition theorem also implies security under sequential and parallel composition.

B Definition of the Hash Combiner

We recap the combiner in Figure 7, formatted as a UC protocol with interfaces matching the ones of $\mathcal{F}_{\mathrm{OPRF}}$.

Hash combiner $\mathsf{Comb}_{\mathsf{H}}(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}})$ for OPRFs

Given two candidate schemes Π^1_{OPRF} and Π^2_{OPRF}, the hash combiner $\mathsf{Comb}_{\mathsf{H}}$ outputs the following OPRF protocol (combined scheme):

Initialization: On $(\text{Init}, \text{sid})$, the server sends $(\text{Init}, \text{sid} \parallel i)$ to Π^i_{OPRF} for all $i \in \{1, 2\}$.
Offline evaluation: On $(\text{OfflineEval}, \text{sid}, x)$, the server sends $(\text{OfflineEval}, \text{sid} \parallel i, x)$ to Π^i_{OPRF} for $i \in \{1, 2\}$. Upon receiving $(\text{OfflineEval}, \text{sid} \parallel i, x, y_i)$ from Π^i_{OPRF} for all $i \in \{1, 2\}$, the server outputs $(\text{OfflineEval}, \text{sid}, x, \mathsf{H}(\text{sid}, x, y_1, y_2))$.

Online evaluation:
- On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{S}', x)$, client U sends $(\text{Eval}, \text{sid} \parallel i, \text{ssid}, \mathsf{S}', x)$ to Π^i_{OPRF} for all $i \in \{1, 2\}$.
- On $(\text{SndrComplete}, \text{sid})$, the server sends $(\text{SndrComplete}, \text{sid} \parallel i)$ to Π^i_{OPRF} for all $i \in \{1, 2\}$.
- Upon receiving $(\text{Eval}, \text{sid} \parallel i, \text{ssid}, y_i)$ from Π^i_{OPRF} for all $i \in \{1, 2\}$, client U outputs $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{H}(\text{sid}, x, y_1, y_2))$.

Fig. 7. UC execution of the 1-out-of-2 hash combiner for OPRFs. The combined scheme uses a hash function H modeled as a random oracle in the $\mathcal{F}_{\text{RO}}$-hybrid model.

C Robustness of the Hash Combiner

As discussed in Sec. 4.2, we prove two robustness theorems for our hash combiner: One with respect to the standard functionality $\mathcal{F}_{\text{OPRF}}$ only, and one in which one of the two candidate schemes may only realize the relaxed $\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}$ functionality from the 2Hash framework.

C.1 Robustness with Respect to the Standard Functionality $\mathcal{F}_{\text{OPRF}}$

In the first setting, we assume that both of the two candidate schemes (unconditionally) realize $\mathcal{F}^{\infty}_{\text{OPRF}}$. Our theorem then states that if at least one of the candidate schemes (computationally) UC-realizes $\mathcal{F}_{\text{OPRF}}$, then so does the combined scheme.

Theorem 1 (Robustness of the Hash Combiner ($\mathcal{F}_{\text{OPRF}} + \mathcal{F}_{\text{OPRF}}$)). *Let Π^1_{OPRF} and Π^2_{OPRF} be two OPRF protocols both UC-realizing $\mathcal{F}^{\infty}_{\text{OPRF}}$. If there is at least one $i \in \{1, 2\}$ such that Π^i_{OPRF} additionally UC-realizes $\mathcal{F}_{\text{OPRF}}$, then $\mathsf{Comb}_{\mathsf{H}}(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}})$ UC-realizes $\mathcal{F}_{\text{OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model.*

Figures 8 to 10 display the simulator for the combined scheme in the ideal world for the case that the first candidate scheme is secure, i.e., it UC-realizes $\mathcal{F}_{\text{OPRF}}$, while the second candidate scheme is potentially insecure, i.e., it might not UC-realize $\mathcal{F}_{\text{OPRF}}$. Since the other case is symmetric, we omit it here. The full proof is provided in Sec. D.1.

Simulator $\mathsf{Sim}_1^{\text{OPRF}}$ for the OPRF hash combiner (Π_{OPRF}^1 secure)

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, two initially empty sets L^1_{sid} and L^2_{sid} and a ticket counter tx_{sid}.

On $(\text{Init}, \text{sid}, \mathsf{S})$ from $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and let S denote the unique entity whose identity was included in the Init message.
On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{U}, \mathsf{S}')$ from $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{U}, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathsf{U}, \mathsf{S}')$ as well as $(\text{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$.
On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \parallel 1)$ as well as $(\text{SndrComplete}, \text{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\text{Compromise}, \text{sid} \parallel 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, send $(\text{Compromise}, \text{sid})$ to $\mathcal{F}_{\text{OPRF}}$ and declare server S as compromised.
// Compromise if any candidate scheme is compromised.
On $(\text{OfflineEval}, \text{sid} \parallel 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, if S is compromised, send $(\text{OfflineEval}, \text{sid} \parallel 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$.
On $(\text{OfflineEval}, \text{sid} \parallel 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, add lbl_1 to L^1_{sid} and send $(\text{OfflineEval}, \text{sid} \parallel 1, \text{lbl}_1, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.
On $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.
On $(\text{SndrComplete}, \text{sid} \parallel 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \parallel 1)$ to $\mathcal{A}$.
On $(\text{RcvCompleteHonest}, \text{sid} \parallel 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R \coloneqq \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{Nil}, \text{lbl}_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \top, \text{lbl}_2\rangle$, and if $\text{lbl}_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \top, \text{lbl}_2)$. Finally, if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.

On $(\text{RcvCompleteMalicious}, \text{sid} \parallel 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$: Add lbl_1 to L^1_{sid}. Then:
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R \coloneqq \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{Nil}, \text{lbl}_2\rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2\rangle$, and if $\text{lbl}_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2)$.

Procedure $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2)$:
- If $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$, then send $(\text{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ to $\mathcal{F}_{\text{OPRF}}$.
- Else, send $(\text{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, (\text{lbl}_1, \text{lbl}_2))$ to $\mathcal{F}_{\text{OPRF}}$.

Fig. 8. Simulator $\mathsf{Sim}_1^{\text{OPRF}}$ showing that $\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\infty}_{\text{OPRF}})$ UC-emulates $\mathcal{F}_{\text{OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model. Continued in Figure 9.

Simulator $\mathsf{Sim}_1^{\mathrm{OPRF}}$ for the OPRF hash combiner (Π^1_{OPRF} secure)

(part 2 of 3)

On $(\textsc{Compromise}, \mathrm{sid}\|2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$, send $(\textsc{Compromise}, \mathrm{sid})$ to $\mathcal{F}_{\mathrm{OPRF}}$ and declare server S as COMPROMISED.
// Compromise if any candidate scheme is compromised.
On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$, send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x, \mathcal{F}^{2}_{\mathrm{honest,sid}}(x))$ to $\mathcal{A}$.
On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$, add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x, \mathcal{F}^{2}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to $\mathcal{A}$.
On $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$, record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.
On $(\textsc{SndrComplete}, \mathrm{sid} \parallel 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$, send $(\textsc{SndrComplete}, \mathrm{sid} \parallel 2)$ to $\mathcal{A}$.
On $(\textsc{RcvCompleteHonest}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$:
- If P is corrupted ($\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x\rangle$. Otherwise, and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^{2}_{\mathrm{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest ($\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R \coloneqq \langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \top\rangle$, and if $\mathrm{lbl}_1 \neq \textsc{Nil}$, and run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \top)$

On $(\textsc{RcvCompleteMalicious}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$: Add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$. Then:
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^{2}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R \coloneqq \langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \mathrm{lbl}_2\rangle$, and if $\mathrm{lbl}_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \mathrm{lbl}_2)$.

Fig. 9. Simulator $\mathsf{Sim}_1^{\mathrm{OPRF}}$ showing that $\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\mathrm{OPRF}}, \mathcal{F}^{\infty}_{\mathrm{OPRF}})$ UC-emulates $\mathcal{F}_{\mathrm{OPRF}}$ in the $\mathcal{F}_{\mathrm{RO}}$-hybrid model. Continuation of Figure 8 and continued in Figure 10.

Simulator $\mathsf{Sim}_1^{\text{OPRF}}$ for the OPRF hash combiner (Π^1_{OPRF} secure)
(part 3 of 3)

On $(\mathsf{H}, \text{sid}, (x, z_1, z_2))$ from $\mathcal{A}$ to $\mathcal{F}_{\text{RO}}$:
1 : **if** there exists a record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$:
2 : Send $(\mathsf{H}, \text{sid}, (z_1, z_2), y)$ to $\mathcal{A}$; **return**
3 : $\text{lbl}_1 \leftarrow \text{NIL}$; $\text{lbl}_2 \leftarrow \text{NIL}$
4 : // Obtain lbl_1 (key label for Π^1_{OPRF})
5 : **if** $(\exists \text{lbl}'_1 \in \text{L}^1_{\text{sid}})(\mathcal{F}^1_{\text{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1)$: **abort** "collision"
6 : **elseif** $\mathcal{F}^1_{\text{honest,sid}}(x) = z_1$: $\text{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
7 : **elseif** $(\exists \text{lbl}'_1, \text{lbl}''_1 \in \text{L}^1_{\text{sid}})(\text{lbl}'_1 \neq \text{lbl}''_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}''_1)$
8 : $\wedge\, z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: **abort** "collision"
9 : **elseif** $(\exists! \text{lbl}'_1 \in \text{L}^1_{\text{sid}})(z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: $\text{lbl}_1 \leftarrow \text{lbl}'_1$ // Key label found
10 : **else** $\text{lbl}_1 \leftarrow \perp$ // Not a valid evaluation of Π^1_{OPRF}
11 : // Obtain lbl_2 (key label for Π^2_{OPRF})
12 : **if** $(\exists \text{lbl}'_2 \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: **abort** "collision"
13 : **elseif** $\mathcal{F}^2_{\text{honest,sid}}(x) = z_2$: $\text{lbl}_2 \leftarrow \top$ // honest key for Π^2_{OPRF}
14 : **elseif** $(\exists \text{lbl}'_2, \text{lbl}''_2 \in \text{L}^2_{\text{sid}})(\text{lbl}'_2 \neq \text{lbl}''_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}''_2)$
15 : $\wedge\, z_2 = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2))$: **abort** "collision"
16 : **elseif** $(\exists! \text{lbl}'_2 \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: $\text{lbl}_2 \leftarrow \text{lbl}'_2$ // Key label found
17 : **else** $\text{lbl}_2 \leftarrow \perp$ // Not a valid evaluation of Π^2_{OPRF}
18 : // Process
19 : **if** $\perp \in \{\text{lbl}_1, \text{lbl}_2\}$: // No OPRF evaluation
20 : $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$
21 : Send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), y)$ to $\mathcal{A}$; **return**
22 : **elseif** $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$: // Honest keys
23 : **if** S is COMPROMISED :
24 : Send $(\text{OFFLINEEVAL}, \text{sid}, x)$ to $\mathcal{F}_{\text{OPRF}}$; On reply $(\text{OFFLINEEVAL}, \text{sid}, x, y)$:
25 : Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$; send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), y)$ to $\mathcal{A}$; **return**
26 : **else**
27 : Pick a fresh ssid^*; send $(\text{EVAL}, \text{sid}, \text{ssid}^*, \perp, x)$ and
28 : $(\text{RCVCOMPLETEHONEST}, \text{sid}, \text{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}_{\text{OPRF}}$.
29 : **if** there is no response from $\mathcal{F}_{\text{OPRF}}$: **abort** "exhausted"
30 : **else** on response $(\text{EVAL}, \text{sid}, \text{ssid}^*, y)$:
31 : Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$; send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), y)$ to $\mathcal{A}$; **return**
32 : **else** // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
33 : Send $(\text{OFFLINEEVAL}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x)$ to $\mathcal{F}_{\text{OPRF}}$.
34 : On reply $(\text{OFFLINEEVAL}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x, y)$:
35 : Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$; send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), y)$ to $\mathcal{A}$; **return**

Fig. 10. Simulator $\mathsf{Sim}_1^{\text{OPRF}}$ showing that $\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\infty}_{\text{OPRF}})$ UC-emulates $\mathcal{F}_{\text{OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model. Continuation of Figure 9.

C.2 Robustness with Respect to the Standard Functionality $\mathcal{F}_{\text{OPRF}}$ and the 2Hash Framework Functionality $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$

In the second setting, we assume that the first candidate scheme (unconditionally) realizes $\mathcal{F}_{\text{OPRF}}$ and the second candidate scheme (unconditionally) realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$. Our theorem then states that if the first candidate scheme (computationally) UC-realizes $\mathcal{F}_{\text{OPRF}}$ **or** the second candidate scheme (computationally) UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$, then the combined scheme UC-realizes a new correlated hash OPRF functionality, which we introduce in Fig. 3.

Theorem 2 (Robustness of the Hash Combiner ($\mathcal{F}_{\text{OPRF}} + \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$)). *Let Π_{OPRF}^1 and Π_{OPRF}^2 be two OPRF protocols such that Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}^{\infty}$ and Π_{OPRF}^2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$. If at least one of the following conditions is fulfilled, then $\mathsf{Comb}_{\mathsf{H}}\big(\Pi_{\text{OPRF}}^1, \Pi_{\text{OPRF}}^2\big)$ UC-realizes $\mathcal{F}_{\text{CorH-OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model:*

- *Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}$.*
- *Π_{OPRF}^2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$.*

Proof sketch. Figures 11 to 13 show the simulator for $\mathsf{Comb}_{\mathsf{H}}\Big(\mathcal{F}_{\text{OPRF}}, \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}\Big)$ or of $\boxed{\mathsf{Comb}_{\mathsf{H}}\Big(\mathcal{F}_{\text{OPRF}}^{\infty}, \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}\Big)}$ respectively. Note that in contrast to the proof of Theorem 1, we now need to state the simulators for the two cases (Π_{OPRF}^1 secure, Π_{OPRF}^2 insecure) and (Π_{OPRF}^1 insecure, Π_{OPRF}^2 secure) separately since the combiner deals with candidate schemes realizing different functionalities and, hence, the second case is no longer symmetric to the first one. However, as the two cases are still reasonably close, we show the differences using $\boxed{\text{boxes}}$ and $\boxed{\text{dashed boxes}}$, where normal text is the same in both versions, $\boxed{\text{dashed boxed text}}$ is for the case that Π_{OPRF}^1 is secure and $\boxed{\text{boxed}}$ text is for the case that Π_{OPRF}^2 is secure. Note that the simulator has to make use of the additional (Correlate, sid) interface provided by $\mathcal{F}_{\text{CorH-OPRF}}$. The simulator extracts key labels whenever possible from (OfflineEval, sid) and (RcvCompleteMalicious, sid) calls. However, because Π_{OPRF}^2 only realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$, the simulator might end up in a situation where there was a $\mathsf{H}(x, (z_1, z_2))$ query, such that no matching label lbl_2 was extracted so far — but a later (OfflineEval, sid) or (RcvCompleteMalicious, sid) call reveals a label that explains z_2. To that end, the simulator uses $\mathcal{F}_{\text{CorH-OPRF}}$'s H_c interface to get a preview hash output. Once the label lbl_2 is extracted, the (Correlate, sid) interface allows the simulator to ensure that the hash preview and the honest client output coincide on that output.

We provide the full proof in Sec. D.2.

Simulator for the OPRF hash combiner (Π^1_{OPRF} / $\boxed{\Pi^2_{\text{OPRF}}}$ secure)

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}})$ | $\boxed{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}^{\infty}_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}})}$

Initialization:

On message $(\textsc{Init}, \text{sid}, \mathsf{S})$ from $\mathcal{F}_{\text{CorH-OPRF}}$, if this is the first $\textsc{Init}$ message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\textsc{Init}, \text{sid} \parallel 1, \mathsf{S})$ as well as $(\textsc{Init}, \text{sid} \parallel 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the $\textsc{Init}$ message for the session identifier sid. Ignore all subsequent $\textsc{Init}$ messages for sid.

Online evaluation:

- On $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathsf{U}, \mathsf{S}')$ from $\mathcal{F}_{\text{CorH-OPRF}}$, store $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{U}, \textsc{Nil}, \textsc{Nil}\rangle$ and send $(\textsc{Eval}, \text{sid} \parallel i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\textsc{SndrComplete}, \text{sid}, \mathsf{S})$ from $\mathcal{F}_{\text{CorH-OPRF}}$, set tx_{sid}++ and send $(\textsc{SndrComplete}, \text{sid} \parallel i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ (corresponding to Π^1_{OPRF})

On $(\textsc{Compromise}, \text{sid} \parallel 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, send $(\textsc{Compromise}, \text{sid})$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ and declare server S as $\textsc{compromised}$. *Note: Message* $(\textsc{Compromise}, \text{sid} \parallel 1)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \text{sid} \| 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, if S is $\textsc{compromised}$, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\textsc{OfflineEval}, \text{sid} \parallel 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\textsc{OfflineEval}, \text{sid} \parallel 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\textsc{OfflineEval}, \text{sid} \parallel 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \text{sid} \parallel 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, | set tx_{sid}++ | send $(\textsc{SndrComplete}, \text{sid} \parallel 1)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \text{sid} \parallel 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ | or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\text{sid}} = 0$ |. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, | and if S is not $\textsc{compromised}$, set tx_{sid}−− |.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \textsc{Nil}, \text{lbl}_2\rangle$ | or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\text{sid}} = 0$ |. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \top, \text{lbl}_2\rangle$, and if $\text{lbl}_2 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \top, \text{lbl}_2)$. | Finally, if S is not $\textsc{compromised}$, set tx_{sid}−− |.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \parallel 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \textsc{Nil}, \text{lbl}_2\rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2\rangle$, and if $\text{lbl}_2 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2)$.

(part 1 of 3)

Fig. 11. Simulator showing that $\mathsf{Comb}_{\mathsf{H}}\left(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\right)$ | $\boxed{\mathsf{Comb}_{\mathsf{H}}\left(\mathcal{F}^{\infty}_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}\right)}$ UC-emulates $\mathcal{F}_{\text{CorH-OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model. Continued in Figure 12. Normal text is the same in both versions, | dashed boxed | text is for the case that Π^1_{OPRF} is secure and $\boxed{\text{boxed text}}$ is for the case that Π^2_{OPRF} is secure.

Simulator for the OPRF hash combiner ($\boxed{\Pi^1_{\text{OPRF}}}$/$\boxed{\Pi^2_{\text{OPRF}}}$ secure)

(part 2 of 3)

$\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$ (corresponding to Π^2_{OPRF})

On (COMPROMISE, sid‖2) from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, send (COMPROMISE, sid) to $\boxed{\mathcal{F}_{\text{OPRF}}}$/$\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ and declare server S as COMPROMISED. *Note: Message* (COMPROMISE, sid ‖ 2) *requires permission from the environment.*

On (OFFLINEEVAL, sid ‖ 2, x) from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, send (OFFLINEEVAL, sid ‖ 2, x, $\mathcal{F}^2_{\text{honest,sid}}(x)$) to $\mathcal{A}$.

On (OFFLINEEVAL, sid ‖ 2, lbl_2, x) from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, for all records $\langle \mathsf{H}_c, \text{sid}, x', z_1, z_2, \text{lbl}_1\rangle$ such that $f(\mathsf{H}_{1,\text{sid}}(x'), \text{lbl}_2) = z_2$, give input ((CORRELATE, sid, x', (z_1, z_2), $(\text{lbl}_1, \text{lbl}_2)$)) to $\mathcal{F}_{\text{CorH-OPRF}}$. add lbl_2 to L^2_{sid} and send (OFFLINEEVAL, sid ‖ 2, lbl_2, x, $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2)$) to $\mathcal{A}$.

On (EVAL, sid ‖ 2, ssid, S', x) from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send (EVAL, sid ‖ 2, ssid, $\mathcal{A}$, S') to $\mathcal{A}$.

On (SNDRCOMPLETE, sid ‖ 2) from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}\text{++}}$ send (SNDRCOMPLETE, sid ‖ 2) to $\mathcal{A}$.

On (RCVCOMPLETEHONEST, sid ‖ 2, ssid, P) from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ $\boxed{\text{or if S is not (COMPROMISED, sid) and } \mathsf{tx}_{\text{sid}} = 0}$. Otherwise, send (EVAL, sid ‖ 2, ssid, $\mathcal{F}^2_{\text{honest,sid}}(x)$) to P ($= \mathcal{A}$) $\boxed{\text{and if S is not (COMPROMISED, sid), set } \mathsf{tx}_{\text{sid}}\text{−−}}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{NIL}\rangle$ $\boxed{\text{or if S is not COMPROMISED and } \mathsf{tx}_{\text{sid}} = 0}$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \top\rangle$, and if $\text{lbl}_1 \neq \text{NIL}$, run COMPLETEHONCLTSUBSESSION(sid, ssid, P, lbl_1, $\top$). $\boxed{\text{Finally, if S is not COMPROMISED, set } \mathsf{tx}_{\text{sid}}\text{−−}}$.

On (RCVCOMPLETEMALICIOUS, sid ‖ 2, ssid, P, lbl_2) from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}}$/$\boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$:

- For all records $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2, \text{lbl}_1\rangle$ such that $f(\mathsf{H}_{1,\text{sid}}(x), \text{lbl}_2) = z_2$, send (CORRELATE, sid, x, (z_1, z_2), $(\text{lbl}_1, \text{lbl}_2)$) to $\mathcal{F}_{\text{CorH-OPRF}}$.
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send (EVAL, sid ‖ 2, ssid, $\mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}_2)$) to P ($= \mathcal{A}$).
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{NIL}\rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2\rangle$, and if $\text{lbl}_1 \neq \text{NIL}$, run COMPLETEHONCLTSUBSESSION(sid, ssid, P, lbl_1, lbl_2).

Fig. 12. Simulator showing that $\boxed{\mathsf{Comb_H}\left(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\right)}$ $\boxed{\mathsf{Comb_H}\left(\mathcal{F}^{\infty}_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}\right)}$ UC-emulates $\mathcal{F}_{\text{CorH-OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model. Continuation of Figure 11 and continued in Figure 13. Normal text is the same in both versions, dashed boxed text is for the case that Π^1_{OPRF} is secure and boxed text is for the case that Π^2_{OPRF} is secure.

Simulator for the OPRF hash combiner ($\boxed{\Pi^1_{\text{OPRF}}}$ / $\boxed{\Pi^2_{\text{OPRF}}}$ secure)

(part 3 of 3)

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (x, (z_1, z_2)))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (x, (z_1, z_2)), \textsc{Hash}(x, (z_1, z_2)))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2)$:
- If $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$, then send $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
- Else, send $(\textsc{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, (\text{lbl}_1, \text{lbl}_2))$ to $\mathcal{F}_{\text{CorH-OPRF}}$.

Procedure $\textsc{Hash}(x, (z_1, z_2))$:

41 : **if** there exists a record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$:
42 : **return** y
43 : $\text{lbl}_1 \leftarrow \textsc{Nil}$; $\text{lbl}_2 \leftarrow \textsc{Nil}$
44 : // Obtain lbl_1 (key label for Π^1_{OPRF})
45 : **if** $(\exists \text{lbl}'_1, \top \in \text{L}^1_{\text{sid}})(\mathcal{F}^1_{\text{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1)$: **abort** "collision"
46 : **elseif** $(\exists \top \in \text{L}^1_{\text{sid}})(\mathcal{F}^1_{\text{honest,sid}}(x) = z_1)$: $\text{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
47 : **elseif** $(\exists \text{lbl}'_1, \text{lbl}''_1 \in \text{L}^1_{\text{sid}})(\text{lbl}'_1 \neq \text{lbl}''_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}''_1)$
48 : $\wedge\, z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: **abort** "collision"
49 : **elseif** $(\exists! \text{lbl}'_1 \in \text{L}^1_{\text{sid}})(z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: $\text{lbl}_1 \leftarrow \text{lbl}'_1$ // Key label found
50 : **else** $\text{lbl}_1 \leftarrow \perp$ // Not a valid evaluation of Π^1_{OPRF}
51 : // obtain key for Π^2_{OPRF}
52 : **if** $(\exists \text{lbl}'_2, \top \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: **abort** "collision"
53 : **elseif** $(\exists \top \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{honest,sid}}(x) = z_2)$: $\text{lbl}_2 \leftarrow \top$
54 : **elseif** $(\exists \text{lbl}'_2, \text{lbl}''_2 \in \text{L}^2_{\text{sid}})(\text{lbl}'_2 \neq \text{lbl}''_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}''_2)$
55 : $\wedge\, z_2 = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2))$: **abort** "collision"
56 : **elseif** $(\exists! \text{lbl}'_2 \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: $\text{lbl}_2 \leftarrow \text{lbl}'_2$ // key label found
57 : **else** $\text{lbl}_2 \leftarrow \perp$ // Not a valid evaluation of Π^2_{OPRF}
58 : // Process
59 : **if** $\text{lbl}_2 = \perp \wedge \text{lbl}_1 \neq \perp$: // No key for Π^2_{OPRF} but one for Π^1_{OPRF}. Need preview
60 : Give input $(\mathsf{H}_c, \text{sid}, x, (z_1, z_2))$ to $\mathcal{F}_{\text{CorH-OPRF}}$
61 : On a response y:
62 : Record $\langle \mathsf{H}_c, \text{sid}, x, (z_1, z_2), \text{lbl}_1 \rangle$
63 : Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
64 : **if** $\text{lbl}_1 = \perp$: // No OPRF evaluation
65 : $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
66 : **elseif** $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$: // Honest keys
67 : **if** S is COMPROMISED:
68 : Send $(\textsc{OfflineEval}, \text{sid}, x)$ to $\mathcal{F}_{\text{CorH-OPRF}}$; On reply $(\textsc{OfflineEval}, \text{sid}, x, y)$:
69 : Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
70 : **else**
71 : Pick a fresh ssid^*; send $(\textsc{Eval}, \text{sid}, \text{ssid}^*, \perp, x)$ and
72 : $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
73 : **if** there is no response from $\mathcal{F}_{\text{CorH-OPRF}}$: **abort** "exhausted"
74 : **else** on response $(\textsc{Eval}, \text{sid}, \text{ssid}^*, y)$:
75 : Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
76 : **else** // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
77 : Send $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x)$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
78 : On reply $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x, y)$:
79 : Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y

Fig. 13. Simulator showing that $\boxed{\mathsf{Comb}_{\mathsf{H}}\left(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\right)}$ $\boxed{\mathsf{Comb}_{\mathsf{H}}\left(\mathcal{F}^{\infty}_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}\right)}$ UC-emulates $\mathcal{F}_{\text{CorH-OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model. Continuation of Figures 11 to 12. Normal text is the same in both versions, dashed boxed text is for the case that Π^1_{OPRF} is secure and boxed text is for the case that Π^2_{OPRF} is secure.

D Full Proofs of the Robustness of the Hash Combiner

D.1 Proof of Theorem 1 (Robustness of the OPRF hash combiner: $\mathcal{F}_{\text{OPRF}} + \mathcal{F}_{\text{OPRF}} \to \mathcal{F}_{\text{OPRF}}$)

Theorem 1 (Robustness of the Hash Combiner ($\mathcal{F}_{\text{OPRF}} + \mathcal{F}_{\text{OPRF}}$)). *Let Π^1_{OPRF} and Π^2_{OPRF} be two OPRF protocols both UC-realizing $\mathcal{F}^{\infty}_{\text{OPRF}}$. If there is at least one $i \in \{1,2\}$ such that Π^i_{OPRF} additionally UC-realizes $\mathcal{F}_{\text{OPRF}}$, then $\mathsf{Comb}_{\mathsf{H}}(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}})$ UC-realizes $\mathcal{F}_{\text{OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model.*

Proof. We only consider the case that Π^1_{OPRF} is secure and Π^2_{OPRF} may be insecure, i.e., Π^1_{OPRF} UC-realizes $\mathcal{F}_{\text{OPRF}}$, whereas Π^2_{OPRF} potentially merely UC-realizes $\mathcal{F}^{\infty}_{\text{OPRF}}$. This is without loss of generality as the other case is symmetric. Following standard conventions for UC proofs, we further assume w.l.o.g. that the adversary $\mathcal{A}$ is the dummy adversary [Can01] which simply forwards all messages between the environment $\mathcal{Z}$ and instances of protocol ITMs. Moreover, we present the proof as a sequence of games which starts in the real world and progressively leads to the ideal world via incremental changes to the machines in the games. For each game $\mathbf{G}_i$, we let the term $\Pr[\mathbf{G}_i(\lambda)]$ denote the probability that the environment $\mathcal{Z}$ outputs 1 in the game $\mathbf{G}_i$. Table 2 provides an overview of all games in the proof.

Game	Simulator	Functionality	Description
$\mathbf{G}_0$	n/a	n/a	Real world
$\mathbf{G}_1$	n/a	n/a	$\{\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\infty}_{\text{OPRF}}\}$-hybrid world
$\mathbf{G}_2$	Fig. 15 and 16	Fig. 14	Move protocol into simulator
$\mathbf{G}_3$	Fig. 15 and 16	Fig. 17	Implement backdoor interfaces of $\mathcal{F}_{\text{OPRF}}$
$\mathbf{G}_4$	Fig. 18 and 19	Fig. 17	Abort on key collisions
$\mathbf{G}_5$	Fig. 20, 21 and 22	Fig. 17	Program RO for malicious keys
$\mathbf{G}_6$	Fig. 24, 25 and 26	Fig. 23	Program RO for honest keys
$\mathbf{G}_7$	Fig. 27, 28 and 29	Fig. 23	Remove hash eval. for honest clients
$\mathbf{G}_8$	Fig. 31, 32 and 33	Fig. 30	Evaluations by honest clients $\to \mathcal{F}$
$\mathbf{G}_9$	Fig. 8, 9 and 10	Fig. 1	Remove pass-through interfaces

Table 2. Overview of the games in the proof of Theorem 1.

Game $\mathbf{G}_0$: The original game (real world). The initial game is the real execution of the combined OPRF.

$$\mathbf{G}_0 = \text{EXEC}_{\mathsf{Comb}_{\mathsf{H}}(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}}), \mathcal{A}, \mathcal{Z}}$$

Game $\mathbf{G}_1$: Moving to the hybrid world. Since Π^1_{OPRF} UC-realizes $\mathcal{F}_{\text{OPRF}}$ and Π^2_{OPRF} UC-realizes $\mathcal{F}^{\infty}_{\text{OPRF}}$, we can replace the real protocols with the respective ideal protocols they implement. By the composition theorem of the UC framework, this does not affect the view of the environment.

$$\begin{aligned}\mathbf{G}_0 &= \text{EXEC}_{\mathsf{Comb}_{\mathsf{H}}(\Pi^1_{\text{OPRF}}, \Pi^2_{\text{OPRF}}), \mathcal{A}, \mathcal{Z}} \\ &\overset{c}{\approx} \text{EXEC}_{\mathsf{Comb}_{\mathsf{H}}(\text{IDEAL}_{\mathcal{F}_{\text{OPRF}}}, \text{IDEAL}_{\mathcal{F}^{\infty}_{\text{OPRF}}}), \mathcal{A}, \mathcal{Z}} =: \mathbf{G}_1\end{aligned}$$

This change results in a protocol in the $\{\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\infty}_{\text{OPRF}}\}$-hybrid world.

Game $\mathbf{G}_2$: Adding a simulator and a pass-through functionality. Next, we replace the hybrid protocol with the ideal protocol for the trivial ideal functionality $\mathcal{F} := \mathcal{F}_{\text{triv}}$ (Figure 14). The trivial ideal functionality forwards any input message it receives (from any dummy party) as a backdoor message to the adversary. Moreover, the functionality offers a backdoor interface to the adversary for sending arbitrary output messages to any dummy party in the ideal protocol $\text{IDEAL}_{\mathcal{F}_{\text{triv}}}$. Note that any protocol Π UC-realizes $\mathcal{F}_{\text{triv}}$ via a simulator that internally runs the protocol Π because the simulation just adds internal "pass-through" machines which do not change the behaviour of the protocol.

Instantiating the idea above, we add a simulator $\mathcal{S}$ between the real-world adversary and $\mathcal{F}_{\text{triv}}$ that internally runs the protocol $\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\infty}_{\text{OPRF}})$ (see Figure 15 and Figure 16). Hence, we have:

$$\Pr[\mathbf{G}_1(\lambda)] = \Pr[\mathbf{G}_2(\lambda)]$$

In the next game hops, we will gradually change $\mathcal{S}$ and $\mathcal{F}$ to eventually arrive at the simulator $\mathsf{Sim}_1^{\text{OPRF}}$ and the ideal functionality $\mathcal{F}_{\text{OPRF}}$.

Trivial ideal functionality $\mathcal{F}_{\text{triv}}$

On any message $(\mathsf{msg}, \mathsf{sid}, \mathsf{args}\ldots)$ from P, send $(\textsc{Input}, \mathsf{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ to $\mathcal{A}$.
On $(\textsc{Output}, \mathsf{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \mathsf{sid}, \mathsf{args}\ldots)$ to P.

Fig. 14. Trivial ideal functionality $\mathcal{F}_{\text{triv}}$

Simulator $\mathcal{S}$ in game $\mathbf{G}_2$ in the proof of Theorem 1
(part 1 of 2)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere and a ticket counter $\mathsf{tx}_{\mathsf{sid}}$. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^\infty_{\text{OPRF}})$
Initialization:
On message $(\textsc{Input}, \mathsf{sid}, \mathsf{S}, \textsc{Init})$ from $\mathcal{F}_{\text{triv}}$, if this is the first $\textsc{Init}$ message for sid, set $\mathsf{tx}_{\mathsf{sid}} \leftarrow 0$ and send $(\textsc{Init}, \mathsf{sid} \,\|\, 1, \mathsf{S})$ as well as $(\textsc{Init}, \mathsf{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the $\textsc{Init}$ message for the session identifier sid. Ignore all subsequent $\textsc{Init}$ messages for sid.

Offline evaluation: On $(\textsc{Input}, \mathsf{sid}, \mathsf{S}, \textsc{OfflineEval}, x)$ from $\mathcal{F}_{\text{triv}}$, send $(\textsc{Output}, \mathsf{sid}, \mathsf{S}, \textsc{OfflineEval}, x, \textsc{Hash}(x, \mathcal{F}^1_{\text{honest,sid}}(x), \mathcal{F}^2_{\text{honest,sid}}(x)))$ to $\mathcal{F}_{\text{triv}}$.
Online evaluation:
- On $(\textsc{Input}, \mathsf{sid}, \mathsf{U}, \textsc{Eval}, \mathsf{ssid}, \mathsf{S}', x)$ from $\mathcal{F}_{\text{triv}}$, store $\langle \mathsf{sid}, \mathsf{ssid}, \mathsf{U}, x, \textsc{Nil}, \textsc{Nil}\rangle$ and send $(\textsc{Eval}, \mathsf{sid} \,\|\, i, \mathsf{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\textsc{Input}, \mathsf{sid}, \mathsf{S}, \textsc{SndrComplete})$ from $\mathcal{F}_{\text{triv}}$, set $\mathsf{tx}_{\mathsf{sid}}{+}{+}$ and send $(\textsc{SndrComplete}, \mathsf{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ (corresponding to Π^1_{OPRF})
On $(\textsc{Compromise}, \mathsf{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, declare server S as $\textsc{compromised}$. *Note: Message* $(\textsc{Compromise}, \mathsf{sid} \,\|\, 1)$ *requires permission from the environment.*
On $(\textsc{OfflineEval}, \mathsf{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, if S is $\textsc{compromised}$, send $(\textsc{OfflineEval}, \mathsf{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.
On $(\textsc{OfflineEval}, \mathsf{sid} \,\|\, 1, \mathrm{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, send $(\textsc{OfflineEval}, \mathsf{sid} \,\|\, 1, \mathrm{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \mathrm{lbl}_1))$ to $\mathcal{A}$.
On $(\textsc{Eval}, \mathsf{sid} \,\|\, 1, \mathsf{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_1Subsession}, \mathsf{sid}, \mathsf{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \mathsf{sid} \,\|\, 1, \mathsf{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.
On $(\textsc{SndrComplete}, \mathsf{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\mathsf{sid}}{+}{+}$ and send $(\textsc{SndrComplete}, \mathsf{sid} \,\|\, 1)$ to $\mathcal{A}$.
On $(\textsc{RcvCompleteHonest}, \mathsf{sid} \,\|\, 1, \mathsf{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x\rangle$ or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\mathsf{sid}} = 0$. Otherwise, send $(\textsc{Eval}, \mathsf{sid} \,\|\, 1, \mathsf{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not $\textsc{compromised}$, set $\mathsf{tx}_{\mathsf{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x, \textsc{Nil}, y_2\rangle$ or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\mathsf{sid}} = 0$. Otherwise, update R to $\langle \mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not $\textsc{compromised}$, set $\mathsf{tx}_{\mathsf{sid}}{-}{-}$.
On $(\textsc{RcvCompleteMalicious}, \mathsf{sid} \,\|\, 1, \mathsf{ssid}, \mathsf{P}, \mathrm{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \mathsf{sid} \,\|\, 1, \mathsf{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \mathrm{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x, \textsc{Nil}, y_2\rangle$. Otherwise, update R to $\langle \mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \mathrm{lbl}_1), y_2\rangle$, and if $y_2 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathsf{sid}, \mathsf{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \mathrm{lbl}_1), y_2)$.

Fig. 15. Simulator $\mathcal{S}$ for game $\mathbf{G}_2$ in the proof of Theorem 1. Continued in Figure 16.

Simulator $\mathcal{S}$ in game $\mathbf{G}_2$ in the proof of Theorem 1

(part 2 of 2)

$\mathcal{F}^{\infty}_{\text{OPRF}}$ (corresponding to Π^{2}_{OPRF})

On $(\textsc{Compromise}, \text{sid} \,\|\, 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, declare server S as COMPROMISED. *Note: Message* $(\textsc{Compromise}, \text{sid} \,\|\, 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, x, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \text{sid} \,\|\, 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{SndrComplete}, \text{sid} \,\|\, 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x))$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$.

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (x, z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), \textsc{Hash}(x, z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, y_2)$:
Send $(\textsc{Output}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \textsc{Hash}(x, y_1, y_2))$ to $\mathcal{F}_{\text{triv}}$.

Procedure $\textsc{Hash}(x, z_1, z_2)$:
If there is a record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y\rangle$, return y. Otherwise, pick $y \leftarrow\!\!\$\ \{0,1\}^{\lambda}$, store $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y\rangle$ and return y.

Fig. 16. Simulator $\mathcal{S}$ for game $\mathbf{G}_2$ in the proof of Theorem 1. Continuation of Figure 15.

Game $\mathbf{G}_3$: Implementing $\mathcal{F}_{\text{OPRF}}$ with backdoor interfaces only. We augment the ideal functionality with the internal logic and the adversarial interfaces of $\mathcal{F}_{\text{OPRF}}$, but keep letting the functionality forward all valid[12] inputs from the dummy parties to the adversary and letting the adversary choose outputs for the dummy parties. Since $\mathcal{F}_{\text{OPRF}}$ already forwards INIT and SNDRCOMPLETE messages to the adversary, we implement these interfaces exactly as they are in $\mathcal{F}_{\text{OPRF}}$ and avoid the explicit forwarding for brevity. This results in the ideal functionality displayed in Figure 17. Since at this point, the additional (backdoor) interfaces provided by the ideal functionality are neither used by the simulator nor visible to the environment, the output of the latter remains unchanged:

$$\Pr[\mathbf{G}_2(\lambda)] = \Pr[\mathbf{G}_3(\lambda)]$$

Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_3$ in the proof of Theorem 1

Global variables:
Mappings $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, a set and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x,k)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) := r$, $\mathcal{F}_{\text{malicious,sid}}(x,k) := r$.

Initialization:
On message $(\text{INIT}, \text{sid})$ from party S, if this is the first INIT message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{INIT}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the INIT message for the session identifier sid. Ignore all subsequent INIT messages for sid.

Sending arbitrary output:
On $(\text{OUTPUT}, \text{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \text{sid}, \mathsf{args}\ldots)$ to P.

Adaptive server compromise:
On $(\text{COMPROMISE}, \text{sid})$ from $\mathcal{A}$, declare server S as COMPROMISED. *Note: Message* $(\textit{COMPROMISE}, \text{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function:

On $(\text{OFFLINEEVAL}, \text{sid}, x)$ from S, send $(\text{INPUT}, \text{sid}, \mathsf{S}, \text{OFFLINEEVAL}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\text{OFFLINEEVAL}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is COMPROMISED, send $(\text{OFFLINEEVAL}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\text{OFFLINEEVAL}, \text{sid}, k^*, x)$ from $\mathcal{A}$, send $(\text{OFFLINEEVAL}, \text{sid}, k^*, x, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to $\mathcal{A}$.
Online evaluation:

- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \neq \mathcal{A}$, send $(\text{INPUT}, \text{sid}, \mathsf{P}, \text{EVAL}, \text{ssid}, \mathsf{P}, \mathsf{S}', x)$ to $\mathcal{A}$.

- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathcal{A}\}$, record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ and send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\text{SNDRCOMPLETE}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SNDRCOMPLETE}, \text{sid})$ to $\mathcal{A}$.
- On $(\text{RCVCOMPLETEHONEST}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not COMPROMISED, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- On $(\text{RCVCOMPLETEMALICIOUS}, \text{sid}, \text{ssid}, \mathsf{P}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to P.

Fig. 17. Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_3$ in the proof of Theorem 1. Differences to $\mathcal{F}_{\text{OPRF}}$ are highlighted in grey boxes.

[12] Our functionality does not forward invalid messages, i.e. messages that the real protocol would ignore, from the dummy parties to the adversary. Since the hybrid protocol (which the simulator internally runs at this point) would ignore such messages anyway, this does not change the view of the environment.

Game $\mathbf{G}_4$: Monitoring key labels and aborting on key collisions. We now let the simulator keep track of all key labels that are used in the two candidate schemes. To this end, we add two variables $\mathrm{L}^1_{\mathrm{sid}}$ and $\mathrm{L}^2_{\mathrm{sid}}$ to the simulator, both of which are initialized to the empty set. Whenever the adversary sends an OfflineEval message for a malicious key or a RcvCompleteMalicious message to the functionality for one of the candidate schemes Π^1_{OPRF} resp. Π^2_{OPRF}, the simulator adds the key label lbl_1 resp. lbl_2 from the message to $\mathrm{L}^1_{\mathrm{sid}}$ resp. $\mathrm{L}^2_{\mathrm{sid}}$.

Furthermore, we let the simulator abort if the random oracle $\mathcal{F}_{\mathrm{RO}}$ is evaluated on an input (x, z_1, z_2) such that for at least one OPRF candidate scheme Π^i_{OPRF} $(i \in \{1, 2\})$, the corresponding set $\mathrm{L}^i_{\mathrm{sid}}$ contains at least two distinct key labels $\mathrm{lbl}'_i \neq \mathrm{lbl}''_i$ such that $\mathcal{F}^i_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_i) = z_i$ and $\mathcal{F}^i_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_i) = z_i$, or if there exists a malicious key label that collides with an honest key, i.e., there exists $\mathrm{lbl}'_i \in \mathrm{L}^i_{\mathrm{sid}}$ such that $\mathcal{F}^i_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_i) = z_i$ and $\mathcal{F}^i_{\mathrm{honest,sid}}(x) = z_i$. Following the terminology of [BDFH25], we call such collisions "key collisions". Figures 18 and 19 show the resulting simulator. The abort instructions for the simulator in case of a key collision are labeled "collision".

Since the tracking of the key labels is internal to the simulator, the view of the environment in $\mathbf{G}_3$ and the one in $\mathbf{G}_4$ only differ if the simulator aborts due to a key collision. Let N be the total number of OfflineEval, RcvCompleteHonest and RcvCompleteMalicious messages the adversary $\mathcal{A}$ sends to the candidate schemes Π^1_{OPRF} and Π^2_{OPRF}. As function outputs in $\mathcal{F}_{\mathrm{OPRF}}$ and $\mathcal{F}^{\infty}_{\mathrm{OPRF}}$ are randomly chosen, and, hence, we can upper-bound the probability of key collisions using the birthday bound, we obtain

$$|\Pr[\mathbf{G}_3(\lambda)] - \Pr[\mathbf{G}_4(\lambda)]| \leq \frac{N^2}{2 \cdot |\mathcal{Y}|}\,.$$

Game $\mathbf{G}_5$: Programming the random oracle for malicious keys. Without key collisions, it is guaranteed that each input-output pair (x, z_i) from any of the candidate schemes Π^i_{OPRF} $(i \in \{1, 2\})$ can be mapped to a unique key identifier lbl_i. In this and the following games, we will use this to program the random oracle H such that it returns the OPRF outputs chosen by the ideal functionality $\mathcal{F}_{\mathrm{OPRF}}$. To do so, we change the simulator as follows: Whenever the real-world adversary queries H on (x, z_1, z_2), the simulator will look up the identifier lbl_1 of the key belonging to the output z_1 of Π^1_{OPRF}, as well as the input x and the identifier lbl_2 of the key belonging to the output z_2 of Π^2_{OPRF}. The honest key is represented by $\top$. If any of the two input-output pairs (x, z_i) does not correspond to an actual evaluation of the respective OPRF candidate scheme Π^i_{OPRF} (because it does not appear in the associated function tables $\mathcal{F}^i_{\mathrm{honest,sid}}$ and $\mathcal{F}^i_{\mathrm{malicious,sid}}$), then the simulator behaves as before, i.e. it samples a uniformly random bitstring y, remembers it for future queries with the same input and returns y to the adversary. In our pseudocode, this case is denoted by $\mathrm{lbl}_1 \leftarrow \bot$ resp. $\mathrm{lbl}_1 \leftarrow \bot$. Otherwise, the simulator combines the extracted key labels lbl_1 and lbl_2 to a combined label $(\mathrm{lbl}_1, \mathrm{lbl}_2)$.

In this game, we only program the oracle on malicious keys. Hence, evaluations for the honest key (denoted by $\mathrm{lbl}_1 = \top \wedge \mathrm{lbl}_2 = \top$) are treated in the same way as invalid evaluations, and the simulator picks a random output by itself, as described above. However, if at least one lbl_1, lbl_2 is not equal to $\top$, then the simulator instead sends $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x)$ to $\mathcal{F}$. Once it receives a response with function value y, it remembers y for future oracle queries and returns y to the adversary. Note that because the simulator asks $\mathcal{F}_{\mathrm{OPRF}}$ for the function value under a malicious key, it always receives a response. Since $\mathcal{F}_{\mathrm{OPRF}}$ returns uniformly random function values, outputs received from $\mathcal{F}_{\mathrm{OPRF}}$ cannot be distinguished from the ones picked by the simulator itself. Hence, we have

$$\Pr[\mathbf{G}_4(\lambda)] = \Pr[\mathbf{G}_5(\lambda)]\,.$$

Game $\mathbf{G}_6$: Programming the random oracle for honest keys, moving the offline evaluation by the honest server to the functionality and handling adaptive compromise.

In this game, we update the simulator to program the random oracle for evaluations of the OPRF in which the key of the honest server is used. To do so, when the random oracle is evaluated on an input (x, z_1, z_2) such that $\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1$ and $\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2$, the simulator now obtains the output from the ideal functionality $\mathcal{F}$ and programs the random oracle to this value instead of sampling it itself.

The way the simulator obtains the output of the OPRF is the same as in [JKX18a] and depends on the corruption status of the honest server:

(a) If the honest server is COMPROMISED, the simulator uses an OfflineEval command to receive the output from $\mathcal{F}$. To be able to do this, we modify the simulator such that it corrupts the honest server in the ideal world by sending a CORRUPT message to $\mathcal{F}$ as soon as the real-world adversary corrupts the honest server for one of the candidate schemes Π^1_{OPRF} or Π^2_{OPRF} (see Figures 24 and 25). Since the real-world

Simulator $\mathcal{S}$ in game $\mathbf{G}_4$ in the proof of Theorem 1

(part 1 of 2)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\infty}_{\text{OPRF}})$

Initialization:

On message $(\text{Init}, \text{sid}, \mathsf{S})$ from $\mathcal{F}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Offline evaluation: On $(\text{Input}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x)$ from $\mathcal{F}$, send $(\text{Output}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x, \text{Hash}(x, \mathcal{F}^1_{\text{honest,sid}}(x), \mathcal{F}^2_{\text{honest,sid}}(x)))$ to $\mathcal{F}$.

Online evaluation:

- On $(\text{Input}, \text{sid}, \mathsf{U}, \text{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{F}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ (corresponding to Π^1_{OPRF})

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, if S is compromised, send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$. On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$: Add lbl_1 to $\mathrm{L}^1_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2)$.

Fig. 18. Simulator $\mathcal{S}$ in game $\mathbf{G}_4$ in the proof of Theorem 1. Continued in Figure 19. Differences between $\mathbf{G}_3$ and $\mathbf{G}_4$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_4$ in the proof of Theorem 1

(part 2 of 2)

$\mathcal{F}^{\infty}_{\text{OPRF}}$ (corresponding to Π^{2}_{OPRF})

On $(\text{COMPROMISE}, \text{sid} \,\|\, 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, declare server S as COMPROMISED. *Note: Message* $(\textit{COMPROMISE}, \text{sid} \,\|\, 2)$ *requires permission from the environment.*

On $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 2, x, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, add lbl_2 to $\text{L}^{2}_{\text{sid}}$ and send $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.

On $(\text{EVAL}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{EVAL}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SNDRCOMPLETE}, \text{sid} \,\|\, 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\text{SNDRCOMPLETE}, \text{sid} \,\|\, 2)$ to $\mathcal{A}$.

On $(\text{RCVCOMPLETEHONEST}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \text{NIL}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x)\rangle$, and if $y_1 \neq \text{NIL}$, run $\text{COMPLETEHONCLTSUBSESSION}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x))$.

On $(\text{RCVCOMPLETEMALICIOUS}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$: Add lbl_2 to $\text{L}^{2}_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \text{NIL}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2)\rangle$, and if $y_1 \neq \text{NIL}$, run $\text{COMPLETEHONCLTSUBSESSION}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$.

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (z_1, z_2), \text{HASH}(z_1, z_2))$ to $\mathcal{A}$.

Procedure $\text{COMPLETEHONCLTSUBSESSION}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, y_2)$:
Send $(\text{OUTPUT}, \text{sid}, \mathsf{P}, \text{EVAL}, \text{ssid}, \text{HASH}(x, y_1, y_2))$ to $\mathcal{F}$.

Procedure $\text{HASH}(x, z_1, z_2)$:

If any of the following conditions are satisfied, **abort** with error "collision":

- $(\exists \text{lbl}'_1 \in \text{L}^{1}_{\text{sid}})(\mathcal{F}^{1}_{\text{honest,sid}}(x) = z_1 \wedge \mathcal{F}^{1}_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1)$
- $(\exists \text{lbl}'_1, \text{lbl}''_1 \in \text{L}^{1}_{\text{sid}})(\text{lbl}'_1 \neq \text{lbl}''_1 \wedge \mathcal{F}^{1}_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1 \wedge \mathcal{F}^{1}_{\text{malicious,sid}}(x, \text{lbl}''_1) = z_1)$
- $(\exists \text{lbl}'_2 \in \text{L}^{2}_{\text{sid}})(\mathcal{F}^{2}_{\text{honest,sid}}(x) = z_2 \wedge \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$
- $(\exists \text{lbl}'_2, \text{lbl}''_2 \in \text{L}^{2}_{\text{sid}})(\text{lbl}'_2 \neq \text{lbl}''_2 \wedge \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2 \wedge \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}''_2) = z_2)$

If there is a record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y\rangle$, return y. Otherwise, pick $y \leftarrow\!\!\$\ \{0,1\}^{\lambda}$, store $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y\rangle$ and return y.

Fig. 19. Simulator $\mathcal{S}$ in game $\mathbf{G}_4$ in the proof of Theorem 1. Continuation of Figure 18. Differences between $\mathbf{G}_3$ and $\mathbf{G}_4$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 1

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\infty}_{\text{OPRF}})$

Initialization:

On message $(\text{INIT}, \text{sid}, \mathsf{S})$ from $\mathcal{F}$, if this is the first INIT message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{INIT}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{INIT}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the INIT message for the session identifier sid. Ignore all subsequent INIT messages for sid.

Offline evaluation: On $(\text{INPUT}, \text{sid}, \mathsf{S}, \text{OFFLINEEVAL}, x)$ from $\mathcal{F}$, send $(\text{OUTPUT}, \text{sid}, \mathsf{S}, \text{OFFLINEEVAL}, x, \text{HASH}(x, \mathcal{F}^1_{\text{honest,sid}}(x), \mathcal{F}^2_{\text{honest,sid}}(x)))$ to $\mathcal{F}$.

Online evaluation:

- On $(\text{INPUT}, \text{sid}, \mathsf{U}, \text{EVAL}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{NIL}, \text{NIL}\rangle$ and send $(\text{EVAL}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{SNDRCOMPLETE}, \text{sid})$ from $\mathcal{F}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SNDRCOMPLETE}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ (corresponding to Π^1_{OPRF})

On $(\text{COMPROMISE}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, declare server S as COMPROMISED. *Note: Message* $(\textit{COMPROMISE}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, if S is COMPROMISED, send $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OFFLINEEVAL}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{EVAL}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{EVAL}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SNDRCOMPLETE}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SNDRCOMPLETE}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RCVCOMPLETEHONEST}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{EVAL}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not COMPROMISED, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{NIL}, y_2\rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \text{NIL}$, run $\text{COMPLETEHONCLTSUBSESSION}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not COMPROMISED, set $\mathsf{tx}_{\text{sid}}{-}{-}$.

On $(\text{RCVCOMPLETEMALICIOUS}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$: Add lbl_1 to $\mathrm{L}^1_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{NIL}, y_2\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2\rangle$, and if $y_2 \neq \text{NIL}$, run $\text{COMPLETEHONCLTSUBSESSION}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2)$.

Fig. 20. Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 1. Continued in Figure 21. Differences between $\mathbf{G}_4$ and $\mathbf{G}_5$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 1

(part 2 of 3)

$\underline{\mathcal{F}^{\infty}_{\text{OPRF}} \text{ (corresponding to } \Pi^{2}_{\text{OPRF}})}$

On $(\textsc{Compromise}, \text{sid} \,\|\, 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, declare server S as COMPROMISED. *Note: Message* $(\textsc{Compromise}, \text{sid} \,\|\, 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, x, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, add lbl_2 to $\text{L}^{2}_{\text{sid}}$ and send $(\textsc{OfflineEval}, \text{sid} \,\|\, 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x \rangle$ and send $(\textsc{Eval}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \text{sid} \,\|\, 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{SndrComplete}, \text{sid} \,\|\, 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x) \rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x))$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \,\|\, 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$: Add lbl_2 to $\text{L}^{2}_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \,\|\, 2, \text{ssid}, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2) \rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$.

$\underline{\mathcal{F}_{\text{RO}} \text{ (corresponding to the hash function } \mathsf{H})}$

On $(\mathsf{H}, \text{sid}, (x, z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), \textsc{Hash}(x, z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, y_2)$:

Send $(\textsc{Output}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \textsc{Hash}(x, y_1, y_2))$ to $\mathcal{F}$.

Fig. 21. Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 1. Continuation of Figure 20 and continued in Figure 22. Differences between $\mathbf{G}_4$ and $\mathbf{G}_5$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 1

(part 3 of 3)

Procedure $\textsc{Hash}(x, z_1, z_2)$:

if there exists a record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$:
  return y
$\mathrm{lbl}_1 \leftarrow \textsc{Nil}$; $\mathrm{lbl}_2 \leftarrow \textsc{Nil}$
// Obtain x, lbl_1 (key label for Π^1_{OPRF})
if $(\exists \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = z_1)$: **abort** "collision"
elseif $\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1$: $\mathrm{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
elseif $(\exists \mathrm{lbl}'_1, \mathrm{lbl}''_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathrm{lbl}'_1 \neq \mathrm{lbl}''_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_1)$
  $\wedge z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: $\mathrm{lbl}_1 \leftarrow \mathrm{lbl}'_1$ // Key label found
else $\mathrm{lbl}_1 \leftarrow \bot$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain x, lbl_2 (key label for Π^2_{OPRF})
if $(\exists \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: **abort** "collision"
elseif $\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2$: $\mathrm{lbl}_2 \leftarrow \top$
elseif $(\exists \mathrm{lbl}'_2, \mathrm{lbl}''_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathrm{lbl}'_2 \neq \mathrm{lbl}''_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_2)$
  $\wedge z_2 = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: $\mathrm{lbl}_2 \leftarrow \mathrm{lbl}'_2$ // Key label found
else $\mathrm{lbl}_2 \leftarrow \bot$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\bot \in \{\mathrm{lbl}_1, \mathrm{lbl}_2\} \vee (\mathrm{lbl}_1 = \top \wedge \mathrm{lbl}_2 = \top)$: // No OPRF evaluation or honest key
  $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x)$ to $\mathcal{F}_{\mathrm{OPRF}}$.
  On reply $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y

Fig. 22. Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 1. Continuation of Figure 21. Differences between $\mathbf{G}_4$ and $\mathbf{G}_5$ are highlighted in grey.

adversary can only send a CORRUPT message after getting permission to do so from the environment, the simulator has the required permission to send a CORRUPT message to $\mathcal{F}$ in this case, too.

(b) If the honest server is *not* COMPROMISED, the simulator uses a sequence of EVAL and RCVCOMPLETE-HONEST commands to complete an evaluation with the functionality and obtain the output. Recall that $\mathcal{F}$ models an interactive, untampered evaluation with the honest server using three commands: First, the clients initiates the evaluation using an EVAL message, then the server sends a SNDRCOMPLETE message to allow the interaction, and finally the adversary sends a RCVCOMPLETEHONEST message to $\mathcal{F}$ to select the function table for the honest server for the evaluation at hand. It is worth mentioning that these three message need not be sent in order. In particular, the SNDRCOMPLETE command may be sent before the EVAL command. As in [JKX18a], our simulator (see Figure 26) makes use of this fact to obtain function values for the honest server from $\mathcal{F}$. When the adversary queries the random oracle on a fresh triple (x, z_1, z_2) with z_1 and z_2 for the honest server's key as specified above, the security of the secure candidate scheme Π^j_{OPRF} ($j \in \{1, 2\}$) ensures that the adversary can only have computed both z_1 and z_2 if there has been a matching SNDRCOMPLETE message by the honest server in the ideal world before. Hence, $\mathcal{F}$ will not ignore the RCVCOMPLETEHONEST message by the simulator (i.e., the failure event "exhausted" does not occur) and reply with the corresponding function value unless the real-world adversary correctly guessed z_j. Since the simulator sets itself as the client for the evaluation, it receives the EVAL response sent by $\mathcal{F}$. Let Q be the number of random oracle queries the real-world adversary performs. As $\mathcal{F}$ picks function values, which are bitstrings of length λ, uniformly at random, the probability of the real-world adversary's correctly guessing z_j can be upper-bounded by $\frac{Q}{2^\lambda}$.

We need to make an additional change in order to make the simulation work[13] . Right now, when the honest server computes the OPRF non-interactively by sending an OFFLINEEVAL command to $\mathcal{F}$, the functionality currently forwards this message to the simulator, which in turn generates a response by internally querying the random oracle. With the above change, this strategy no longer works because OFFLINEEVAL commands do not increment the ticket counter in $\mathcal{F}$, so the simulator cannot acquire the function value from the functionality. To fix this, we remove the redirection of OFFLINEEVAL messages from the honest server to the adversary in $\mathcal{F}$ and instead generate the response directly as in $\mathcal{F}_{\mathrm{OPRF}}$ (see Figure 23).

Since the ideal functionality picks random function values, the environment cannot distinguish between function values obtained through the ideal functionality only (current game) and the ones chosen by the simulator (previous games). Furthermore, the outputs of the random oracle are consistent with the OFFLINEEVAL and EVAL outputs from $\mathcal{F}$ to the environment $\mathcal{Z}$ (unless the real-world adversary predicts z_j). Combined with the fact that the rest of the view of the environment remains unchanged, we have

$$|\Pr[\mathbf{G}_5(\lambda)] - \Pr[\mathbf{G}_6(\lambda)]| \leq \frac{Q}{2^\lambda}\,.$$

Game $\mathbf{G}_7$: Remove the hash evaluation from evaluations with honest clients. In game $\mathbf{G}_6$, as soon as the second key for an evaluation with an honest client (HONCLTSUBSESSION) is chosen (via a RCVCOMPLETE-HONEST or RCVCOMPLETEMALICIOUS message to Π^1_{OPRF} or Π^2_{OPRF}), the simulator looks up the function values z_1 and z_2 for x under the key labels lbl_1 resp. lbl_2 for the respective candidate schemes. It then internally executes the COMPLETEHONCLTSUBSESSION procedure, which accepts x as well as z_1 and z_2 as parameters. This procedure in turn evaluates the random oracle on (x, z_1, z_2).

Since $\mathbf{G}_5$, the random oracle already retrieves the key labels lbl_1 and lbl_2 from the records of the simulator and obtains the function value from $\mathcal{F}$ to program the random oracle coherently with the honest client's outputs. So in this game, we can modify the simulator to not take the detour via the random oracle. Instead, we let the simulator directly instruct $\mathcal{F}$ to give the right output to the honest client.

The resulting simulator is displayed in Figures 27 to 29.

If the simulator finds key labels for x, z_1, z_2, the view of the environment is exactly as before. However, the view of the environment changes if first there is a query $\mathsf{H}(x, z_1, z_2)$ such that the simulator recorded no lbl_i explaining one of the z_i, but later such an input-label pair is extracted. That is the case if an honest client

[13] In [BDFH25, proof of theorem 1, appendix C], this change is performed in isolation in a separate game (game 7). However, this means that the environment $\mathcal{Z}$ can distinguish between games 6 and 7 in their proof: To do so, it sends an INIT message to the dummy party of the honest server, followed by a OFFLINEEVAL message to the same dummy party for a random $x \leftarrow\!\!\$\ \mathcal{X}$. In game 6, the second message is relayed to the simulator, which attempts to evaluate the random oracle and as part of that sends a RCVCOMPLETEHONEST command to $\mathcal{F}$. Since the ticket counter is 0, this command is ignored and the simulator aborts. In the other games, the environment receives the function value of x for the honest server as expected (in games 7-10, the response is sent directly by the ideal functionality, whereas in games 0-5, the functionality relays the output which is locally computed by the simulator).

Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_6$ in the proof of Theorem 1

Global variables:
Mappings $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, a set and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x,k)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) \coloneqq r$, $\mathcal{F}_{\text{malicious,sid}}(x,k) \coloneqq r$.

Initialization:
On message $(\text{INIT}, \text{sid})$ from party S, if this is the first INIT message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{INIT}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the INIT message for the session identifier sid. Ignore all subsequent INIT messages for sid.

Sending arbitrary output:
On $(\text{OUTPUT}, \text{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \text{sid}, \mathsf{args}\ldots)$ to P.

Adaptive server compromise:
On $(\text{COMPROMISE}, \text{sid})$ from $\mathcal{A}$, declare server S as COMPROMISED. *Note: Message* $(\text{COMPROMISE}, \text{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function: On $(\text{OFFLINEEVAL}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathsf{S}, \mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is COMPROMISED, send $(\text{OFFLINEEVAL}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\text{OFFLINEEVAL}, \text{sid}, k^*, x)$ from $\mathcal{A}$, send $(\text{OFFLINEEVAL}, \text{sid}, k^*, x, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to $\mathcal{A}$.
Online evaluation:
- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \neq \mathcal{A}$, send $(\text{INPUT}, \text{sid}, \mathsf{P}, \text{EVAL}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathcal{A}\}$, record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ and send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\text{SNDRCOMPLETE}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SNDRCOMPLETE}, \text{sid})$ to $\mathcal{A}$.
- On $(\text{RCVCOMPLETEHONEST}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not COMPROMISED, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- On $(\text{RCVCOMPLETEMALICIOUS}, \text{sid}, \text{ssid}, \mathsf{P}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to P.

Fig. 23. Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_3$ in the proof of Theorem 1. Differences to $\mathcal{F}$ in Figure 17 are highlighted in grey boxes.

Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 1

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^\infty_{\text{OPRF}})$

Initialization:

On message $(\text{Init}, \text{sid}, \mathsf{S})$ from $\mathcal{F}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Offline evaluation:

Online evaluation:

- On $(\text{Input}, \text{sid}, \mathsf{U}, \text{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{F}$, set tx_{sid}++ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ (corresponding to Π^1_{OPRF})

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, send $(\text{Compromise}, \text{sid})$ to $\mathcal{F}_{\text{OPRF}}$ and declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, if S is compromised, send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, set tx_{sid}++ and send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set tx_{sid}−−.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not compromised, set tx_{sid}−−.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$: Add lbl_1 to $\mathrm{L}^1_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2)$.

Fig. 24. Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 1. Continued in Figure 25. Differences between $\mathbf{G}_5$ and $\mathbf{G}_6$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 1

(part 2 of 3)

$\mathcal{F}^{\infty}_{\text{OPRF}}$ (corresponding to Π^{2}_{OPRF})

On $(\textsc{Compromise}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{Compromise}, \text{sid})$ to $\mathcal{F}_{\text{OPRF}}$ and declare server S as compromised. *Note: Message* $(\textsc{Compromise}, \text{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \text{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{OfflineEval}, \text{sid} \parallel 2, x, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, add lbl_2 to $\text{L}^{2}_{\text{sid}}$ and send $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x \rangle$ and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x) \rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{honest,sid}}(x))$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$: Add lbl_2 to $\text{L}^{2}_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2) \rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$.

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (x, z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), \textsc{Hash}(x, z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, y_2)$:

Send $(\textsc{Output}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \textsc{Hash}(x, y_1, y_2))$ to $\mathcal{F}$.

Fig. 25. Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 1. Continuation of Figure 24 and continued in Figure 26. Differences between $\mathbf{G}_5$ and $\mathbf{G}_6$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 1

(part 3 of 3)

Procedure $\textsc{Hash}(x, z_1, z_2)$:
if there exists a record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$:
  return y
$\mathrm{lbl}_1 \leftarrow \textsc{Nil}$; $\mathrm{lbl}_2 \leftarrow \textsc{Nil}$
// Obtain x, lbl_1 (key label for Π^1_{OPRF})
if $(\exists \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = z_1)$: **abort** "collision"
elseif $\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1$: $\mathrm{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
elseif $(\exists \mathrm{lbl}'_1, \mathrm{lbl}''_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathrm{lbl}'_1 \neq \mathrm{lbl}''_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_1)$
  $\wedge\, z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: $\mathrm{lbl}_1 \leftarrow \mathrm{lbl}'_1$ // Key label found
else $\mathrm{lbl}_1 \leftarrow \bot$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain x, lbl_2 (key label for Π^2_{OPRF})
if $(\exists \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: **abort** "collision"
elseif $\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2$: $\mathrm{lbl}_2 \leftarrow \top$
elseif $(\exists \mathrm{lbl}'_2, \mathrm{lbl}''_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathrm{lbl}'_2 \neq \mathrm{lbl}''_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_2)$
  $\wedge\, z_2 = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: $\mathrm{lbl}_2 \leftarrow \mathrm{lbl}'_2$ // Key label found
else $\mathrm{lbl}_2 \leftarrow \bot$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\bot \in \{\mathrm{lbl}_1, \mathrm{lbl}_2\}$: // No OPRF evaluation
  $y \leftarrow\$ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \mathrm{sid}, (z_1, z_2), y \rangle$; **return** y
elseif $\mathrm{lbl}_1 = \top \wedge \mathrm{lbl}_2 = \top$: // Honest keys
  if S is COMPROMISED:
    Send $(\textsc{OfflineEval}, \mathrm{sid}, x)$ to $\mathcal{F}$; On reply $(\textsc{OfflineEval}, \mathrm{sid}, x, y)$:
    Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
  else
    Pick a fresh ssid^*; send $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}^*, \bot, x)$ and
      $(\textsc{RcvCompleteHonest}, \mathrm{sid}, \mathrm{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}$.
    if there is no response from $\mathcal{F}$: **abort** "exhausted"
    else on response $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}^*, y)$:
      Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x)$ to $\mathcal{F}$.
  On reply $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y

Fig. 26. Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 1. Continuation of Figure 25. Differences between $\mathbf{G}_5$ and $\mathbf{G}_6$ are highlighted in grey.

evaluated one of the two subprotocols Π^1_{OPRF} or Π^2_{OPRF} with no matching offline evaluation but still, the environment queries the hash function on the respective x, z_1, z_2. Observe that z_1, z_2 are sampled uniformly at random and in the case of an honest client, are never given to the environment. Thus, guessing one of them without an offline evaluation happens at most with probability $Q/2^\lambda$, where Q is the number of random oracle queries. We get

$$|\Pr[\mathbf{G}_6(\lambda)] - \Pr[\mathbf{G}_7(\lambda)]| \leq \frac{Q}{2^\lambda}\,.$$

Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 1
(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^\infty_{\text{OPRF}})$
Initialization:
On message $(\textsc{Init}, \text{sid}, \mathsf{S})$ from $\mathcal{F}$, if this is the first $\textsc{Init}$ message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\textsc{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\textsc{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the $\textsc{Init}$ message for the session identifier sid. Ignore all subsequent $\textsc{Init}$ messages for sid.

Online evaluation:
- On $(\textsc{Input}, \text{sid}, \mathsf{U}, \textsc{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \textsc{Nil}, \textsc{Nil}\rangle$ and send $(\textsc{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\textsc{SndrComplete}, \text{sid})$ from $\mathcal{F}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\textsc{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ (corresponding to Π^1_{OPRF})
On $(\textsc{Compromise}, \text{sid} \| 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, send $(\textsc{Compromise}, \text{sid})$ to $\mathcal{F}_{\text{OPRF}}$ and declare server S as $\textsc{compromised}$.
Note: Message $(\textsc{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*
On $(\textsc{OfflineEval}, \text{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, if S is $\textsc{compromised}$, send $(\textsc{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.
On $(\textsc{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\textsc{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.
On $(\textsc{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.
On $(\textsc{SndrComplete}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\textsc{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.
On $(\textsc{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\textsc{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not $\textsc{compromised}$, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \textsc{Nil}, \boxed{\text{lbl}_2}\rangle$ or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\top, \text{lbl}_2}\rangle$, and if $\boxed{\text{lbl}_2} \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\top, \text{lbl}_2})$. Finally, if S is not $\textsc{compromised}$, set $\mathsf{tx}_{\text{sid}}{-}{-}$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$: Add lbl_1 to $\mathrm{L}^1_{\text{sid}}$. Then:
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \textsc{Nil}, \boxed{\text{lbl}_2}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1, \text{lbl}_2}\rangle$, and if $\boxed{\text{lbl}_2} \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1, \text{lbl}_2})$.

Fig. 27. Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 1. Continued in Figure 28. Differences between $\mathbf{G}_6$ and $\mathbf{G}_7$ are highlighted in grey.

Game $\mathbf{G}_8$: Moving the online evaluation by an honest client into the functionality. Right now, $\mathcal{F}$ still forwards the input x from honest clients to the simulator $\mathcal{S}$. If the real-world adversary admits the evaluation and selects a function table, then $\mathcal{S}$ computes the corresponding output and returns it to the client via the $\textsc{Output}$ interface of $\mathcal{F}$. In this game, we change the functionality $\mathcal{F}$ as well as the simulator such that the latter does not learn the inputs of honest clients anymore. Instead, honest clients will obtain their outputs

Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 1

(part 2 of 3)

$\underline{\mathcal{F}^{\infty}_{\text{OPRF}} \text{ (corresponding to } \Pi^{2}_{\text{OPRF}})}$

On $(\textsc{Compromise}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{Compromise}, \text{sid})$ to $\mathcal{F}$ and declare server S as COMPROMISED. *Note: Message* $(\textsc{Compromise}, \text{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \text{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{OfflineEval}, \text{sid} \parallel 2, x, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, add lbl_2 to $\text{L}^{2}_{\text{sid}}$ and send $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x \rangle$ and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1}, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1, \top} \rangle$, and if $\boxed{\text{lbl}_1} \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1}, \top)$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$: Add lbl_2 to $\text{L}^{2}_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1}, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1, \text{lbl}_2} \rangle$, and if $\boxed{\text{lbl}_1} \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1, \text{lbl}_2})$.

$\underline{\mathcal{F}_{\text{RO}} \text{ (corresponding to the hash function } \mathsf{H})}$

On $(\mathsf{H}, \text{sid}, (x, z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), \textsc{Hash}(x, z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1, \text{lbl}_2})$:

Send $(\textsc{Output}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \boxed{\textsc{RetrieveFunctionValue}(x, \text{lbl}_1, \text{lbl}_2)})$ to $\mathcal{F}$.

Fig. 28. Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 1. Continuation of Figure 27 and continued in Figure 29. Differences between $\mathbf{G}_6$ and $\mathbf{G}_7$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 1

(part 3 of 3)

Procedure $\textsc{Hash}(x, z_1, z_2)$:
if there exists a record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$:
  return y
$\mathrm{lbl}_1 \leftarrow \textsc{Nil}$; $\mathrm{lbl}_2 \leftarrow \textsc{Nil}$
// Obtain x, lbl_1 (key label for Π^1_{OPRF})
if $(\exists \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = z_1)$: **abort** "collision"
elseif $\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1$: $\mathrm{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
elseif $(\exists \mathrm{lbl}'_1, \mathrm{lbl}''_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathrm{lbl}'_1 \neq \mathrm{lbl}''_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_1)$
  $\wedge\, z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: $\mathrm{lbl}_1 \leftarrow \mathrm{lbl}'_1$ // Key label found
else $\mathrm{lbl}_1 \leftarrow \bot$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain x, lbl_2 (key label for Π^2_{OPRF})
if $(\exists \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: **abort** "collision"
elseif $\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2$: $\mathrm{lbl}_2 \leftarrow \top$
elseif $(\exists \mathrm{lbl}'_2, \mathrm{lbl}''_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathrm{lbl}'_2 \neq \mathrm{lbl}''_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_2)$
  $\wedge\, z_2 = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: $\mathrm{lbl}_2 \leftarrow \mathrm{lbl}'_2$ // Key label found
else $\mathrm{lbl}_2 \leftarrow \bot$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\bot \in \{\mathrm{lbl}_1, \mathrm{lbl}_2\}$: // No OPRF evaluation
  $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
else return $\textsc{RetrieveFunctionValue}(x, \mathrm{lbl}_1, \mathrm{lbl}_2)$

Procedure $\textsc{RetrieveFunctionValue}(x, \mathrm{lbl}_1, \mathrm{lbl}_2)$:
if $\mathrm{lbl}_1 = \top \wedge \mathrm{lbl}_2 = \top$: // Honest keys
  if S is COMPROMISED:
    Send $(\textsc{OfflineEval}, \mathrm{sid}, x)$ to $\mathcal{F}$; On reply $(\textsc{OfflineEval}, \mathrm{sid}, x, y)$:
    Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
  else
    Pick a fresh ssid^*; send $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}^*, \bot, x)$ and
      $(\textsc{RcvCompleteHonest}, \mathrm{sid}, \mathrm{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}$.
    if there is no response from $\mathcal{F}$: **abort** "exhausted"
    else on response $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}^*, y)$:
      Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x)$ to $\mathcal{F}$.
  On reply $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y

Fig. 29. Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 1. Continuation of Figure 28. Differences between $\mathbf{G}_6$ and $\mathbf{G}_7$ are highlighted in grey.

directly from $\mathcal{F}$, as in $\mathcal{F}_{\text{OPRF}}$. We achieve this as follows: When an honest client sends an EVAL message to $\mathcal{F}$, we let the functionality handle it in the same way as $\mathcal{F}_{\text{OPRF}}$ does, i.e., the functionality records the input and merely informs the adversary about the fact that an evaluation has been started, but without revealing the input x. This means that EVAL commands by honest clients and the adversary are now treated in the same manner. Once the simulator receives the real-world adversary's choice for the two parts of the key label $(\text{lbl}_1, \text{lbl}_2)$ for an evaluation ssid (via RCVCOMPLETEHONEST resp. RCVCOMPLETEMALICIOUS messages to Π^1_{OPRF} or Π^2_{OPRF}), it directly sends a RCVCOMPLETEHONEST honest message (if both keys are from the honest server, i.e. $\text{lbl}_1 = \text{lbl}_2 = \top$) or RCVCOMPLETEMALICIOUS (if at least one key label is malicious) to $\mathcal{F}$. Note that with this change, the honest client still receives the same output as before, but this output is delivered directly from $\mathcal{F}$, as $\mathcal{S}$ completes the evaluation ssid instead of starting a new one with the adversary as the client and then re-routing the result to the honest client via the OUTPUT interface of $\mathcal{F}$.

Consequently, we have

$$\Pr[\mathbf{G}_7(\lambda)] = \Pr[\mathbf{G}_8(\lambda)]\,.$$

Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_8$ in the proof of Theorem 1

Global variables:
Mappings $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, a set and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x,k)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) := r$, $\mathcal{F}_{\text{malicious,sid}}(x,k) := r$.

Initialization:
On message $(\text{INIT}, \text{sid})$ from party S, if this is the first INIT message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{INIT}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the INIT message for the session identifier sid. Ignore all subsequent INIT messages for sid.

Sending arbitrary output:
On $(\text{OUTPUT}, \text{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \text{sid}, \mathsf{args}\ldots)$ to P.

Adaptive server compromise:
On $(\text{COMPROMISE}, \text{sid})$ from $\mathcal{A}$, declare server S as COMPROMISED. *Note: Message* $(\textit{COMPROMISE}, \text{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function: On $(\text{OFFLINEEVAL}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathsf{S}, \mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is COMPROMISED, send $(\text{OFFLINEEVAL}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\text{OFFLINEEVAL}, \text{sid}, k^*, x)$ from $\mathcal{A}$, send $(\text{OFFLINEEVAL}, \text{sid}, k^*, x, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to $\mathcal{A}$.
Online evaluation:

- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathsf{U}, \mathcal{A}\}$, record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ and send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\text{SNDRCOMPLETE}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SNDRCOMPLETE}, \text{sid})$ to $\mathcal{A}$.
- On $(\text{RCVCOMPLETEHONEST}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not COMPROMISED, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- On $(\text{RCVCOMPLETEMALICIOUS}, \text{sid}, \text{ssid}, \mathsf{P}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, k^*))$ to P.

Fig. 30. Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_8$ in the proof of Theorem 1. Differences to $\mathcal{F}$ in Figure 23 are highlighted in grey boxes.

Game $\mathbf{G}_9$: Cleaning up. Finally, we remove the OUTPUT interface from $\mathcal{F}$. Since $\mathcal{S}$ does not use this interface anymore, this removal is not visible to the environment.

$$\Pr[\mathbf{G}_8(\lambda)] = \Pr[\mathbf{G}_9(\lambda)]$$

With this change, we arrive at $\mathcal{F} = \mathcal{F}_{\text{OPRF}}$ and $\mathcal{S} = \mathsf{Sim}_1^{\text{OPRF}}$, which concludes the proof.

Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 1

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere, two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^\infty_{\text{OPRF}})$

Initialization:

On message $(\text{Init}, \text{sid}, \mathsf{S})$ from $\mathcal{F}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Online evaluation:

- On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{U}, \mathsf{S}')$ from $\mathcal{F}$, store $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{U}, \text{Nil}, \text{Nil} \rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{F}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ (corresponding to Π^1_{OPRF})

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, send $(\text{Compromise}, \text{sid})$ to $\mathcal{F}$ and declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, if S is compromised, send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x \rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{Nil}, \text{lbl}_2 \rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \top, \text{lbl}_2 \rangle$, and if $\text{lbl}_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \top, \text{lbl}_2)$. Finally, if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{OPRF}}$: Add lbl_1 to $\mathrm{L}^1_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{Nil}, \text{lbl}_2 \rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2 \rangle$, and if $\text{lbl}_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2)$.

Fig. 31. Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 1. Continued in Figure 32. Differences between $\mathbf{G}7$ and $\mathbf{G}_8$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 1

(part 2 of 3)

$\underline{\mathcal{F}^{\infty}_{\text{OPRF}} \text{ (corresponding to } \Pi^{2}_{\text{OPRF}})}$

On $(\textsc{Compromise}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{Compromise}, \text{sid})$ to $\mathcal{F}$ and declare server S as COMPROMISED.
Note: Message $(\textsc{Compromise}, \text{sid} \parallel 2)$ *requires permission from the environment.*
On $(\textsc{OfflineEval}, \text{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{OfflineEval}, \text{sid} \parallel 2, x, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.
On $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, add lbl_2 to $\text{L}^{2}_{\text{sid}}$ and send $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.
On $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x \rangle$ and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.
On $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$, send $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ to $\mathcal{A}$.
On $(\textsc{RcvCompleteHonest}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^{2}_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \top \rangle$, and if $\text{lbl}_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \top)$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\mathcal{F}^{\infty}_{\text{OPRF}}$: Add lbl_2 to $\text{L}^{2}_{\text{sid}}$. Then:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^{2}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2 \rangle$, and if $\text{lbl}_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2)$.

$\underline{\mathcal{F}_{\text{RO}} \text{ (corresponding to the hash function } \mathsf{H})}$

On $(\mathsf{H}, \text{sid}, (x, z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (x, z_1, z_2), \textsc{Hash}(x, z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2)$:

- If $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$, then send $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ to $\mathcal{F}$.
- Else, send $(\textsc{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, (\text{lbl}_1, \text{lbl}_2))$ to $\mathcal{F}$.

Fig. 32. Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 1. Continuation of Figure 31 and continued in Figure 33. Differences between $\mathbf{G}_7$ and $\mathbf{G}_8$ are highlighted in grey.

Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 1

(part 3 of 3)

Procedure $\textsc{Hash}(x, z_1, z_2)$:
if there exists a record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$:
  return y
$\mathrm{lbl}_1 \leftarrow \textsc{Nil}$; $\mathrm{lbl}_2 \leftarrow \textsc{Nil}$
// Obtain x, lbl_1 (key label for Π^1_{OPRF})
if $(\exists \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = z_1)$: **abort** "collision"
elseif $\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1$: $\mathrm{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
elseif $(\exists \mathrm{lbl}'_1, \mathrm{lbl}''_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathrm{lbl}'_1 \neq \mathrm{lbl}''_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_1)$
  $\wedge\, z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_1 \in \mathrm{L}^1_{\mathrm{sid}})(z_1 = \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1))$: $\mathrm{lbl}_1 \leftarrow \mathrm{lbl}'_1$ // Key label found
else $\mathrm{lbl}_1 \leftarrow \bot$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain x, lbl_2 (key label for Π^2_{OPRF})
if $(\exists \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: **abort** "collision"
elseif $\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2$: $\mathrm{lbl}_2 \leftarrow \top$
elseif $(\exists \mathrm{lbl}'_2, \mathrm{lbl}''_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathrm{lbl}'_2 \neq \mathrm{lbl}''_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_2)$
  $\wedge\, z_2 = \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \mathrm{lbl}'_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2)$: $\mathrm{lbl}_2 \leftarrow \mathrm{lbl}'_2$ // Key label found
else $\mathrm{lbl}_2 \leftarrow \bot$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\bot \in \{\mathrm{lbl}_1, \mathrm{lbl}_2\}$: // No OPRF evaluation
  $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
elseif $\mathrm{lbl}_1 = \top \wedge \mathrm{lbl}_2 = \top$: // Honest keys
  if S is COMPROMISED:
    Send $(\textsc{OfflineEval}, \mathrm{sid}, x)$ to $\mathcal{F}$; On reply $(\textsc{OfflineEval}, \mathrm{sid}, x, y)$:
    Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
  else
    Pick a fresh ssid^*; send $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}^*, \bot, x)$ and
      $(\textsc{RcvCompleteHonest}, \mathrm{sid}, \mathrm{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}$.
    if there is no response from $\mathcal{F}$: **abort** "exhausted"
    else on response $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}^*, y)$:
      Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x)$ to $\mathcal{F}$.
  On reply $(\textsc{OfflineEval}, \mathrm{sid}, (\mathrm{lbl}_1, \mathrm{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y \rangle$; **return** y

Fig. 33. Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 1. Continuation of Figure 32. Differences between $\mathbf{G}_7$ and $\mathbf{G}_8$ are highlighted in grey.

D.2 Proof of Theorem 2 (Robustness of the OPRF hash combiner: $\mathcal{F}_{\text{OPRF}} + \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f} \to \mathcal{F}_{\text{CorH-OPRF}}$)

Theorem 2 (Robustness of the Hash Combiner ($\mathcal{F}_{\text{OPRF}} + \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$)). *Let Π_{OPRF}^1 and Π_{OPRF}^2 be two OPRF protocols such that Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}^{\infty}$ and Π_{OPRF}^2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$. If at least one of the following conditions is fulfilled, then $\mathsf{Comb}_{\mathsf{H}}(\Pi_{\text{OPRF}}^1, \Pi_{\text{OPRF}}^2)$ UC-realizes $\mathcal{F}_{\text{CorH-OPRF}}$ in the $\mathcal{F}_{\text{RO}}$-hybrid model:*

- *Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}$.*
- *Π_{OPRF}^2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$.*

Proof. We first consider the case that Π_{OPRF}^1 is secure and Π_{OPRF}^2 may be insecure, i.e., Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}$, whereas Π_{OPRF}^2 potentially merely UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$. The parts in boxes, consider the inverse case, i.e., where Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}^{\infty}$, whereas Π_{OPRF}^2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$. $\mathcal{A}$ is again the dummy adversary. Table 3 provides an overview of all games in the proof.

Game	Simulator	Functionality	Description
$\mathbf{G}_0$	n/a	n/a	Real world
$\mathbf{G}_1$	n/a	n/a	$\{\mathcal{F}_{\text{OPRF}}, \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}\}$-hybrid world
$\mathbf{G}_2$	Fig. 34 and 35	Fig. 14	Move protocol into simulator
$\mathbf{G}_3$	Fig. 34 and 35	Fig. 36	Implement backdoor interfaces of $\mathcal{F}_{\text{CorH-OPRF}}$
$\mathbf{G}_4$	Fig. 37 and 38	Fig. 36	Abort on key collisions
$\mathbf{G}_5$	Fig. 39, 40 and 41	Fig. 36	Program RO for malicious keys. Client (CORRELATE, sid)
$\mathbf{G}_6$	Fig. 43, 44 and 45	Fig. 42	Program RO for honest keys
$\mathbf{G}_7$	Fig. 46, 47 and 48	Fig. 42	Remove hash eval. for honest clients
$\mathbf{G}_8$	Fig. 50, 51 and 52	Fig. 49	Evaluations by honest clients $\to \mathcal{F}$
$\mathbf{G}_9$	Fig. 11, 12 and 13	Fig. 53	Add FRESH markings and label check to Correlate interface
$\mathbf{G}_{10}$	Fig. 11, 12 and 13	Fig. 3	Remove pass-through interfaces

Table 3. Overview of the games in the proof of Theorem 2.

Game $\mathbf{G}_0$: The original game (real world). The initial game is the real execution of the combined OPRF.

$$\mathbf{G}_0 = \text{EXEC}_{\mathsf{Comb}_{\mathsf{H}}(\Pi_{\text{OPRF}}^1, \Pi_{\text{OPRF}}^2), \mathcal{A}, \mathcal{Z}}$$

Game $\mathbf{G}_1$: Moving to the hybrid world. Since Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}$ and Π_{OPRF}^2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}$ [Π_{OPRF}^1 UC-realizes $\mathcal{F}_{\text{OPRF}}^{\infty}$ and Π_{OPRF}^2 UC-realizes $\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}$], we can replace the real protocols with the respective ideal protocols they implement. By the composition theorem of the UC framework, this does not affect the view of the environment.

$$\begin{aligned}\mathbf{G}_0 &= \text{EXEC}_{\mathsf{Comb}_{\mathsf{H}}(\Pi_{\text{OPRF}}^1, \Pi_{\text{OPRF}}^2), \mathcal{A}, \mathcal{Z}} \\ &\overset{c}{\approx} \text{EXEC}_{\mathsf{Comb}_{\mathsf{H}}(\text{IDEAL}_{\mathcal{F}_{\text{OPRF}}}, \text{IDEAL}_{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}), \mathcal{A}, \mathcal{Z}} =: \mathbf{G}_1\end{aligned}$$

This change results in a protocol in the $\{\mathcal{F}_{\text{OPRF}}, \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}\}$-hybrid world
[similarly, $\{\mathcal{F}_{\text{OPRF}}^{\infty}, \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}\}$-hybrid world].

Game $\mathbf{G}_2$: Adding a simulator and a pass-through functionality. Next, we replace the hybrid protocol with the ideal protocol for the trivial ideal functionality $\mathcal{F} := \mathcal{F}_{\text{triv}}$ (Figure 14). The trivial ideal functionality forwards any input message it receives (from any dummy party) as a backdoor message to the adversary. Moreover, the functionality offers a backdoor interface to the adversary for sending arbitrary output messages to any dummy party in the ideal protocol $\text{IDEAL}_{\mathcal{F}_{\text{triv}}}$.

Add a simulator $\mathcal{S}$ between the real-world adversary and $\mathcal{F}_{\text{triv}}$ that internally runs the protocol $\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty})$ [$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}^{\infty}, \mathcal{F}_{\text{2H-OPRF}}^{\text{le},f})$] (see Figures 34 to 35). Hence, we have:

$$\Pr[\mathbf{G}_1(\lambda)] = \Pr[\mathbf{G}_2(\lambda)]$$

In the next game hops, we will gradually change $\mathcal{S}$ and $\mathcal{F}$ to eventually arrive at the simulator $\mathsf{Sim}_1^{\text{OPRF}}$ and the ideal functionality $\mathcal{F}_{\text{CorH-OPRF}}$.

Simulator $\mathcal{S}$ in game $\mathbf{G}_2$ in the proof of Theorem 2

(part 1 of 2)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) \coloneqq r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) \coloneqq r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}})$ $\boxed{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}^\infty_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}})}$

Initialization:

On message $(\text{Input}, \text{sid}, \mathsf{S}, \text{Init})$ from $\mathcal{F}_{\text{triv}}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \parallel 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \parallel 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Offline evaluation: On $(\text{Input}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x)$ from $\mathcal{F}_{\text{triv}}$, send $(\text{Output}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x, \text{Hash}(x, \mathcal{F}^1_{\text{honest,sid}}(x), \mathcal{F}^2_{\text{honest,sid}}(x)))$ to $\mathcal{F}_{\text{triv}}$.

Online evaluation:

- On $(\text{Input}, \text{sid}, \mathsf{U}, \text{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}_{\text{triv}}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \parallel i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{Input}, \text{sid}, \mathsf{S}, \text{SndrComplete})$ from $\mathcal{F}_{\text{triv}}$, set tx_{sid}++ and send $(\text{SndrComplete}, \text{sid} \parallel i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\mathcal{F}_{\text{OPRF}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$ (corresponding to Π^1_{OPRF})

On $(\text{Compromise}, \text{sid} \parallel 1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid} \parallel 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid} \parallel 1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, if S is compromised, send $(\text{OfflineEval}, \text{sid} \parallel 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \| 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, send $(\text{OfflineEval}, \text{sid} \| 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}\text{++}}$ send $(\text{SndrComplete}, \text{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \parallel 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set tx_{sid}−−.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R \coloneqq \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not compromised, set tx_{sid}−−.

On $(\text{RcvCompleteMalicious}, \text{sid} \parallel 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{Eval}, \text{sid} \parallel 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R \coloneqq \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2)$.

Fig. 34. Simulator $\mathcal{S}$ for game $\mathbf{G}_2$ in the proof of Theorem 2. Continued in Figure 35.

Simulator $\mathcal{S}$ in game $\mathbf{G}_2$ in the proof of Theorem 2

(part 2 of 2)

$\underline{\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}\ \text{(corresponding to } \Pi^2_{\text{OPRF}})}$

On $(\textsc{Compromise}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\lceil\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\rfloor\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, declare server S as COMPROMISED. *Note: Message* $(\textit{Compromise}, \text{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \text{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\lceil\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\rfloor\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, send $(\textsc{OfflineEval}, \text{sid} \parallel 2, x, \mathcal{F}^2_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\lceil\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\rfloor\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, send $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\lceil\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\rfloor\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\lceil\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\rfloor\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}{+}{+}}$ send $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ to $\mathcal{A}$. On $(\textsc{RcvCompleteHonest}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\lceil\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\rfloor\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^2_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\text{honest,sid}}(x)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\text{honest,sid}}(x))$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\lceil\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}\rfloor\ \boxed{\mathcal{F}^{\text{le},f}_{\text{2H-OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}_2)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}_2))$.

$\underline{\mathcal{F}_{\text{RO}} \text{ (corresponding to the hash function } \mathsf{H})}$

On $(\mathsf{H}, \text{sid}, (z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (z_1, z_2), \textsc{Hash}(z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, y_2)$:
Send $(\textsc{Output}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \textsc{Hash}(y_1, y_2))$ to $\mathcal{F}_{\text{triv}}$.

Procedure $\textsc{Hash}(z_1, z_2)$:
If there is a record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y\rangle$, return y. Otherwise, pick $y \leftarrow\!\!\$\ \{0,1\}^\lambda$, store $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y\rangle$ and return y.

Fig. 35. Simulator $\mathcal{S}$ for game $\mathbf{G}_2$ in the proof of Theorem 2. Continuation of Figure 34.

Game $\mathbf{G}_3$: Implementing $\mathcal{F}_{\text{CorH-OPRF}}$ with backdoor interfaces only. We augment the ideal functionality with the internal logic and the adversarial interfaces of $\mathcal{F}_{\text{CorH-OPRF}}$, but keep letting the functionality forward all valid inputs from the dummy parties to the adversary and letting the adversary choose outputs for the dummy parties. Since $\mathcal{F}_{\text{CorH-OPRF}}$ already forwards INIT and SNDRCOMPLETE messages to the adversary, we implement these interfaces exactly as they are in $\mathcal{F}_{\text{CorH-OPRF}}$ and avoid the explicit forwarding for brevity. This results in the ideal functionality displayed in Figure 36. Since at this point, the additional (backdoor) interfaces provided by the ideal functionality are neither used by the simulator nor visible to the environment, the output of the latter remains unchanged:

$$\Pr[\mathbf{G}_2(\lambda)] = \Pr[\mathbf{G}_3(\lambda)]$$

Game $\mathbf{G}_4$: Monitoring key labels and aborting on key collisions. We now let the simulator keep track of all key labels that are used in the two candidate schemes. To this end, we add two variables $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ to the simulator, both of which are initialized to the empty set. Whenever the adversary sends an OFFLINEEVAL message for a malicious key or a RCVCOMPLETEMALICIOUS message to the functionality for one of the candidate schemes Π^1_{OPRF} resp. Π^2_{OPRF}, the simulator adds the key label lbl_1 resp. lbl_2 to $\mathrm{L}^1_{\text{sid}}$ resp. $\mathrm{L}^2_{\text{sid}}$.

Furthermore, we let the simulator abort if the random oracle $\mathcal{F}_{\text{RO}}$ is evaluated on an input $(x, (z_1, z_2))$ such that for at least one OPRF candidate scheme Π^i_{OPRF} $(i \in \{1, 2\})$, the corresponding set $\mathrm{L}^i_{\text{sid}}$ contains at least two distinct key labels $\text{lbl}'_i \neq \text{lbl}''_i$ such that $\mathcal{F}^i_{\text{malicious,sid}}(x, \text{lbl}'_i) = z_i$ and $\mathcal{F}^i_{\text{malicious,sid}}(x, \text{lbl}''_i) = z_i$, or if there exists a malicious key label that collides with an honest key, i.e., there exists $\text{lbl}'_i \in \mathrm{L}^i_{\text{sid}}$ such that $\mathcal{F}^i_{\text{malicious,sid}}(x, \text{lbl}'_i) = z_i$ and $\mathcal{F}^i_{\text{honest,sid}}(x) = z_i$. Following the terminology of [BDFH25], we call such collisions "key collisions". Figures 37 to 38 show the resulting simulator. The abort instructions for the simulator in case of a key collision are labelled "collision".

Since the tracking of the key labels is internal to the simulator, the view of the environment in $\mathbf{G}_3$ and the one in $\mathbf{G}_4$ only differ if the simulator aborts due to a key collision. Let N be the total number of OFFLINEEVAL, RCVCOMPLETEHONEST and RCVCOMPLETEMALICIOUS messages the adversary $\mathcal{A}$ sends to the candidate schemes Π^1_{OPRF} and Π^2_{OPRF}. As function outputs in $\mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}}$ and $\mathcal{F}_{\text{OPRF}}$ and their ∞ counterparts are randomly chosen, we can upper-bound the probability of key collisions using the birthday bound, we obtain:

$$|\Pr[\mathbf{G}_3(\lambda)] - \Pr[\mathbf{G}_4(\lambda)]| \leq \frac{N^2}{2 \cdot |\mathcal{Y}|}$$

Game $\mathbf{G}_5$: Programming the random oracle for malicious keys. Without key collisions, it is guaranteed that each input-output pair (x, z_i) from any of the candidate schemes Π^i_{OPRF} $(i \in \{1, 2\})$ can be mapped to a unique key identifier lbl_i. In this and the following games, we will use this to program the random oracle H such that it returns the OPRF outputs chosen by the ideal functionality $\mathcal{F}_{\text{CorH-OPRF}}$. To do so, we change the simulator as follows: Whenever the real-world adversary queries H on $(x, (z_1, z_2))$, the simulator will look up the identifier lbl_1 of the key belonging to the input-output pair (x, z_1) of Π^1_{OPRF}, as well as the identifier lbl_2 of the key belonging to the input-output pair (x, z_2) of Π^2_{OPRF}. The honest key is represented by $\top$.

Other than in the proof of Theorem 1, we have to take extra care if any of the two input-output pairs (x, z_i) does not correspond to an actual evaluation of the respective OPRF candidate scheme Π^i_{OPRF} (because it does not appear in the associated function tables $\mathcal{F}^i_{\text{honest,sid}}$ and $\mathcal{F}^i_{\text{malicious,sid}}$), There are two ways, the simulator could end up in this situation:

- $\mathcal{Z}$ is querying H on inputs $x, (z_1, z_2)$ that are completely unrelated to any OPRF evaluation.
- The value z_2 could be precomputed by $\mathcal{Z}$ using the malicious key. Thus, the simulator was not yet able to extract a key label lbl_2 for this output but might later extract the label for z_2.

The simulator cannot know in which of the two cases it is. However, the functionality $\mathcal{F}_{\text{CorH-OPRF}}$ allows the simulator to obtain a preview-output for that case. That means, the simulator gives input $(\mathsf{H}_c, \text{sid}, x, (z_1, z_2))$ to $\mathcal{F}$. On a response y, the simulator stores a record $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2 \rangle$. Finally, the simulator checks in every $(\text{RCVCOMPLETEMALICIOUS}, \text{sid})$ and $(\text{OFFLINEEVAL}, \text{sid})$ message to $\mathcal{F}^{f,\infty}_{\text{2H-OPRF}}$, if the extracted key label lbl_2 corresponds to the preview values, i.e., if there are records $f(\mathsf{H}_{1,\text{sid}}(x), \text{lbl}_2) = z_2$ and with matching records $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2 \rangle$. For each of these entries, the simulator gives input $((\text{CORRELATE}, \text{sid}, \text{sid}, x, (z_1, z_2), (\text{lbl}_1, \text{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$.

In this game, we only program the oracle on malicious keys. Hence, evaluations for the honest key (denoted by $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$) are treated in the same way as invalid evaluations, and the simulator picks a random output by itself, as described above. However, if at least one lbl_1, lbl_2 is not equal to $\top$, then the simulator instead sends $(\text{OFFLINEEVAL}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x)$ to $\mathcal{F}$. Once it receives a response with function value y, it

Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_3$ in the proof of Theorem 2

Global variables:
Mappings $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot,\cdot)$, and $\mathsf{H}_c(\cdot)$, which are initially undefined everywhere, a set and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x,k)$,$\mathsf{H}_c(x)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) := r$, $\mathcal{F}_{\text{malicious,sid}}(x,k) := r$, or $\mathsf{H}_c(x) := r$.

Initialization:
On message $(\textsc{Init}, \text{sid})$ from party S, if this is the first $\textsc{Init}$ message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\textsc{Init}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the $\textsc{Init}$ message for the session identifier sid. Ignore all subsequent $\textsc{Init}$ messages for sid.

Sending arbitrary output:
On $(\textsc{Output}, \text{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \text{sid}, \mathsf{args}\ldots)$ to P.

Adaptive server compromise:
On $(\textsc{Compromise}, \text{sid})$ from $\mathcal{A}$, declare server S as $\textsc{compromised}$. *Note: Message* $(\textsc{Compromise}, \text{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function:

On $(\textsc{OfflineEval}, \text{sid}, x)$ from S, send $(\textsc{Input}, \text{sid}, \mathsf{S}, \textsc{OfflineEval}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is $\textsc{compromised}$, send $(\textsc{OfflineEval}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\textsc{OfflineEval}, \text{sid}, \text{lbl}, x)$ from $\mathcal{A}$, send $(\textsc{OfflineEval}, \text{sid}, \text{lbl}, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}))$ to $\mathcal{A}$.
Online evaluation:

– On $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \neq \mathcal{A}$, send $(\textsc{Input}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \mathsf{P}, \mathsf{S}', x)$ to $\mathcal{A}$.

– On $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathcal{A}\}$, record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ and send $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
– On $(\textsc{SndrComplete}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\textsc{SndrComplete}, \text{sid})$ to $\mathcal{A}$.
– On $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not $\textsc{compromised}$ and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not $\textsc{compromised}$, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
– On $(\textsc{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\textsc{Eval}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}, k^*))$ to P.
Random oracles:

On $(\mathsf{H}_c, \text{sid}, x, z)$ from $\mathcal{A}$, store record $\langle \mathsf{H}_c, x, z\mathsf{H}_c(x,z)\rangle$ and reply with $\mathsf{H}_c(x,z)$.
On $((\textsc{Correlate}, \text{sid}, , x, z, \text{lbl}))$ from $\mathcal{A}$, if there exists a record $\langle \mathsf{H}_c, x, z, y\rangle$, set $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}) := y$.

Fig. 36. Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_3$ in the proof of Theorem 2. Differences to $\mathcal{F}_{\text{CorH-OPRF}}$ are highlighted in grey boxes.

Simulator $\mathcal{S}$ in game $\mathbf{G}_4$ in the proof of Theorem 2

(part 1 of 2)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\underline{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}})}$ $\boxed{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}^{\infty}_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}})}$

Initialization:

On message $(\text{Input}, \text{sid}, \mathsf{S}, \text{Init})$ from $\mathcal{F}_{\text{triv}}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Offline evaluation: On $(\text{Input}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x)$ from $\mathcal{F}_{\text{triv}}$, send $(\text{Output}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x, \text{Hash}(x, \mathcal{F}^1_{\text{honest,sid}}(x), \mathcal{F}^2_{\text{honest,sid}}(x)))$ to $\mathcal{F}_{\text{triv}}$.

Online evaluation:

- On $(\text{Input}, \text{sid}, \mathsf{U}, \text{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}_{\text{triv}}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{Input}, \text{sid}, \mathsf{S}, \text{SndrComplete})$ from $\mathcal{F}_{\text{triv}}$, set tx_{sid}++ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\underline{\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}\ \text{(corresponding to } \Pi^1_{\text{OPRF}})}$

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, if S is compromised, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}\text{++}}$ send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set tx_{sid}−−.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not compromised, set tx_{sid}−−.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2)$.

Fig. 37. Simulator $\mathcal{S}$ for game $\mathbf{G}_4$ in the proof of Theorem 2. Continued in Figure 38.

Simulator $\mathcal{S}$ in game $\mathbf{G}_4$ in the proof of Theorem 2

(part 2 of 2)

$\underline{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}\ \text{(corresponding to } \Pi^2_{\text{OPRF}})}$

On $(\textsc{Compromise}, \mathrm{sid} \parallel 2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}}$, declare server S as COMPROMISED. *Note: Message* $(\textit{Compromise}, \mathrm{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}}$, add $\top$ to $\mathrm{L}^1_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}}$, add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x, \mathcal{F}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}}$, record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \mathrm{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}}$, $\boxed{\text{set } \mathsf{tx}_{\mathrm{sid}}\text{++}}$ send $(\textsc{SndrComplete}, \mathrm{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x\rangle$. Otherwise, add $\top$ to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{honest,sid}}(x)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$.

On $(\textsc{RcvCompleteMalicious}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}\ \boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x\rangle$. Otherwise, add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$.

$\underline{\mathcal{F}_{\mathrm{RO}}\ \text{(corresponding to the hash function } \mathsf{H})}$

On $(\mathsf{H}, \mathrm{sid}, (z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \mathrm{sid}, (z_1, z_2), \textsc{Hash}(z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, y_2)$:
Send $(\textsc{Output}, \mathrm{sid}, \mathsf{P}, \textsc{Eval}, \mathrm{ssid}, \textsc{Hash}(y_1, y_2))$ to $\mathcal{F}$.

Procedure $\textsc{Hash}(z_1, z_2)$:

If any of the following conditions are satisfied, **abort** with error "collision":
- $(\exists \mathrm{lbl}'_1, \top \in \mathrm{L}^1_{\mathrm{sid}})(\mathcal{F}^1_{\mathrm{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = z_1$
- $(\exists \mathrm{lbl}'_1, \mathrm{lbl}''_1 \in \mathrm{L}^1_{\mathrm{sid}})(\mathrm{lbl}'_1 \neq \mathrm{lbl}''_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_1) = z_1 \wedge \mathcal{F}^1_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_1) = z_1$
- $(\exists \mathrm{lbl}'_2, \top \in \mathrm{L}^2_{\mathrm{sid}})(\mathcal{F}^2_{\mathrm{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2$
- $(\exists \mathrm{lbl}'_2, \mathrm{lbl}''_2 \in \mathrm{L}^2_{\mathrm{sid}})(\mathrm{lbl}'_2 \neq \mathrm{lbl}''_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}'_2) = z_2 \wedge \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}''_2) = z_2$

If there is a record $\langle \mathsf{H}, \mathrm{sid}, (z_1, z_2), y\rangle$, return y. Otherwise, pick $y \leftarrow\!\!\$\ \{0,1\}^\lambda$, store $\langle \mathsf{H}, \mathrm{sid}, (x, z_1, z_2), y\rangle$ and return y.

Fig. 38. Simulator $\mathcal{S}$ for game $\mathbf{G}_4$ in the proof of Theorem 2. Continuation of Figure 37.

remembers y for future oracle queries and returns y to the adversary. Note that because the simulator asks $\mathcal{F}_{\text{CorH-OPRF}}$ for the function value under a malicious key, it always receives a response. In particular, the current functionality does not yet check if a H_c record is marked as FRESH or if the label has been used before and therefore always performs the programming. Since $\mathcal{F}_{\text{CorH-OPRF}}$ returns uniformly random function values, outputs received from $\mathcal{F}_{\text{CorH-OPRF}}$ cannot be distinguished from the ones picked by the simulator itself. Hence, we have:

$$\Pr[\mathbf{G}_4(\lambda)] = \Pr[\mathbf{G}_5(\lambda)]$$

Game $\mathbf{G}_6$: Programming the random oracle for honest keys, moving the offline evaluation by the honest server to the functionality and handling adaptive compromise.

In this game, we update the simulator to program the random oracle for evaluations of the OPRF in which the key of the honest server is used. To do so, when the random oracle is evaluated on an input $(x, (z_1, z_2))$ such that $\mathcal{F}^1_{\text{honest,sid}}(x) = z_1$ and $\mathcal{F}^2_{\text{honest,sid}}(x) = z_2$, the simulator now obtains the output from the ideal functionality $\mathcal{F}$ and programs the random oracle to this value instead of sampling it itself.

The way the simulator obtains the output of the OPRF is the same as in [JKX18a] and depends on the corruption status of the honest server:

(a) If the honest server is COMPROMISED, the simulator uses an OFFLINEEVAL command to receive the output from $\mathcal{F}$. To be able to do this, we modify the simulator such that it corrupts the honest server in the ideal world by sending a CORRUPT message to $\mathcal{F}$ as soon as the real-world adversary corrupts the honest server for one of the candidate schemes Π^1_{OPRF} or Π^2_{OPRF} (see Figure 43 and Figure 44). Since the real-world adversary can only send a CORRUPT message after getting permission to do so from the environment, the simulator has the required permission to send a CORRUPT message to $\mathcal{F}$ in this case, too.

(b) If the honest server is *not* COMPROMISED, the simulator uses a sequence of EVAL and RCVCOMPLETE-HONEST commands to complete an evaluation with the functionality and obtain the output. When the adversary queries the random oracle on a fresh pair $(x, (z_1, z_2))$ with z_1 and z_2 for the honest server's key as specified above, the security of the secure candidate scheme Π^j_{OPRF} ($j \in \{1, 2\}$) ensures that the adversary can only have computed both z_1 and z_2 if there has been a matching SNDRCOMPLETE message by the honest server in the ideal world before. Hence, $\mathcal{F}$ will not ignore the RCVCOMPLETEHONEST message by the simulator (i.e., the failure event "exhausted" does not occur) and reply with the corresponding function value unless the real-world adversary correctly guessed z_j. Since the simulator sets itself as the client for the evaluation, it receives the EVAL response sent by $\mathcal{F}$. Let Q be the number of random oracle queries the real-world adversary performs and E be the number of SNDRCOMPLETE commands issued by the honest server. As $\mathcal{F}$ picks function values, which are bitstrings of length λ, uniformly at random, the probability of the real-world adversary's correctly guessing z_j can be upper-bounded by $\frac{Q \cdot E}{2 \cdot 2^\lambda}$.

We need to make an additional change in order to make the simulation work. Right now, when the honest server computes the OPRF non-interactively by sending an OFFLINEEVAL command to $\mathcal{F}$, the functionality currently forwards this message to the simulator, which in turn generates a response by internally querying the random oracle. With the above change, this strategy no longer works because OFFLINEEVAL commands do not increment the ticket counter in $\mathcal{F}$, so the simulator cannot acquire the function value from the functionality. To fix this, we remove the redirection of OFFLINEEVAL messages from the honest server to the adversary in $\mathcal{F}$ and instead generate the response directly as in $\boxed{\mathcal{F}_{\text{OPRF}}}\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ (see Figure 1).

Since the ideal functionality picks random function values, the environment cannot distinguish between function values obtained through the ideal functionality only (current game) and the ones chosen by the simulator (previous games). Furthermore, the outputs of the random oracle are consistent with the OFFLINEEVAL and EVAL outputs from $\mathcal{F}$ to the environment $\mathcal{Z}$ (unless the real-world adversary predicts z_j). Combined with the fact that the rest of the view of the environment remains unchanged, we have

$$|\Pr[\mathbf{G}_5(\lambda)] - \Pr[\mathbf{G}_6(\lambda)]| \leq \frac{Q \cdot E}{2^\lambda}\,.$$

Game $\mathbf{G}_7$: Remove the hash evaluation from evaluations with honest clients. In game $\mathbf{G}_6$, as soon as the second key for an evaluation with an honest client (HONCLTSUBSESSION) is chosen (via a RCVCOMPLETE-HONEST or RCVCOMPLETEMALICIOUS message to Π^1_{OPRF} or Π^2_{OPRF}), the simulator looks up the function values z_1 and z_2 for x under the key labels lbl_1 resp. lbl_2 for the respective candidate schemes. It then internally executes the COMPLETEHONCLTSUBSESSION procedure, which accepts x as well as z_1 and z_2 as parameters. This procedure in turn evaluates the random oracle on $(x, (z_1, z_2))$.

Since $\mathbf{G}_5$, the random oracle already retrieves and key labels lbl_1 and lbl_2 from the records of the simulator and obtains the function value from $\mathcal{F}$ to program the random oracle coherently with the honest client's

Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 2

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}})$ | $\boxed{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}^\infty_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}})}$

Initialization:

On message $(\text{Input}, \text{sid}, \mathsf{S}, \text{Init})$ from $\mathcal{F}_{\text{triv}}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Offline evaluation: On $(\text{Input}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x)$ from $\mathcal{F}_{\text{triv}}$, send $(\text{Output}, \text{sid}, \mathsf{S}, \text{OfflineEval}, x, \text{Hash}(x, \mathcal{F}^1_{\text{honest,sid}}(x), \mathcal{F}^2_{\text{honest,sid}}(x)))$ to $\mathcal{F}_{\text{triv}}$.

Online evaluation:

- On $(\text{Input}, \text{sid}, \mathsf{U}, \text{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}_{\text{triv}}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{Input}, \text{sid}, \mathsf{S}, \text{SndrComplete})$ from $\mathcal{F}_{\text{triv}}$, set tx_{sid}++ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$ (corresponding to Π^1_{OPRF})

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, if S is compromised, send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}\text{++}}$ send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set tx_{sid}−−.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not compromised, set tx_{sid}−−.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^\infty_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2)$.

Fig. 39. Simulator $\mathcal{S}$ for game $\mathbf{G}_5$ in the proof of Theorem 2. Continued in Figure 40 and Figure 41.

Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 2

(part 2 of 3)

$\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$ (corresponding to Π_{OPRF}^2)

On $(\textsc{Compromise}, \text{sid} \parallel 2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$, declare server S as COMPROMISED. *Note: Message* $(\textit{Compromise}, \text{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \text{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$, add $\top$ to L_{sid}^2 and send $(\textsc{OfflineEval}, \text{sid} \parallel 2, x, \mathcal{F}_{\text{honest,sid}}^2(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \text{sid} \| 2, \text{lbl}_2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$, for all records $\langle \mathsf{H}_c, \text{sid}, x', z_1, z_2, \text{lbl}_1 \rangle$ such that $f(\mathsf{H}_{1,\text{sid}}(x'), \text{lbl}_2)$ give input $((\textsc{Correlate}, \text{sid}, x', (z_1, z_2), (\text{lbl}_1, \text{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$. add lbl_2 to L_{sid}^2 and send $(\textsc{OfflineEval}, \text{sid} \parallel 2, \text{lbl}_2, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$, record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x \rangle$ and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}{+}{+}}$ send $(\textsc{SndrComplete}, \text{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, add $\top$ to L_{sid}^2 and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}_{\text{honest,sid}}^2(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}_{\text{honest,sid}}^2(x) \rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}_{\text{honest,sid}}^2(x))$.

On $(\textsc{RcvCompleteMalicious}, \text{sid} \parallel 2, \text{ssid}, \mathsf{P}, \text{lbl}_2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f,\infty}}\boxed{\mathcal{F}_{\text{2H-OPRF}}^{\text{le},f}}$:

- for all records $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2, \text{lbl}_1 \rangle$ such that $f(\mathsf{H}_{1,\text{sid}}(x), \text{lbl}_2) = z_2$, give input $((\textsc{Correlate}, \text{sid}, x, (z_1, z_2), (\text{lbl}_1, \text{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$. Otherwise, add lbl_2 to L_{sid}^2 and send $(\textsc{Eval}, \text{sid} \parallel 2, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}^2(x, \text{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}_{\text{malicious,sid}}^2(x, \text{lbl}_2) \rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, \mathcal{F}_{\text{malicious,sid}}^2(x, \text{lbl}_2))$.

Fig. 40. Simulator $\mathcal{S}$ for game $\mathbf{G}_5$ in the proof of Theorem 2. Continuation of Figure 39 and continued in Figure 41.

Simulator $\mathcal{S}$ in game $\mathbf{G}_5$ in the proof of Theorem 2

(part 3 of 3)

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (z_1, z_2), \textsc{Hash}(z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, y_2)$:
Send $(\textsc{Output}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \textsc{Hash}(y_1, y_2))$ to $\mathcal{F}$.

Procedure $\textsc{Hash}(z_1, z_2)$:

if there exists a record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$:
  return y
$\text{lbl}_1 \leftarrow \textsc{Nil}$; $\text{lbl}_2 \leftarrow \textsc{Nil}$
$x_1 \leftarrow \textsc{Nil}$; $x_2 \leftarrow \textsc{Nil}$
// Obtain (x, lbl_1) (input and key label for Π^1_{OPRF})
if $(\exists \text{lbl}'_1, \top \in \text{L}^1_{\text{sid}})(\mathcal{F}^1_{\text{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1$: **abort** "collision"
elseif $(\exists \top \in \text{L}^1_{\text{sid}})\mathcal{F}^1_{\text{honest,sid}}(x) = z_1$: $\text{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
elseif $(\exists \text{lbl}'_1, \text{lbl}''_1 \in \text{L}^1_{\text{sid}})(\text{lbl}'_1 \neq \text{lbl}''_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}''_1)$
  $\wedge z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_1 \in \text{L}^1_{\text{sid}})(z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: $\text{lbl}_1 \leftarrow \text{lbl}'_1$ // input and key label found
else $\text{lbl}_1 \leftarrow \perp$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain input and key for Π^2_{OPRF}
if $(\exists \text{lbl}'_2, \top \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2$: **abort** "collision"
elseif $(\exists \top \in \text{L}^2_{\text{sid}})\mathcal{F}^2_{\text{honest,sid}}(x) = z_2$: $\text{lbl}_2 \leftarrow \top$
elseif $(\exists \text{lbl}'_2), \text{lbl}''_2 \in \text{L}^2_{\text{sid}})(\text{lbl}'_2 \neq \text{lbl}''_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}''_2)$
  $\wedge z_2 = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_2 \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: $\text{lbl}_2 \leftarrow \text{lbl}'_2$ // input and key label found
else $\text{lbl}_2 \leftarrow \perp$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\text{lbl}_2 = \perp \wedge \text{lbl}_1 \neq \perp$: // No key for Π^2_{OPRF} but one for Π^1_{OPRF}. Need preview
  Give input $(\mathsf{H}_c, \text{sid}, x, (z_1, z_2))$ to $\mathcal{F}_{\text{CorH-OPRF}}$
  On a response y:
    Record $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2, \text{lbl}_1 \rangle$
    Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y
elseif $\perp = \text{lbl}_1 \vee (\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top)$: // No OPRF evaluation or honest key
  $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x)$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
  On reply $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y

Fig. 41. Simulator $\mathcal{S}$ for game $\mathbf{G}_5$ in the proof of Theorem 2. Continuation of Figure 39 and Figure 40.

Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_6$ in the proof of Theorem 2

Global variables:
Mappings $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot,\cdot)$, and $\mathsf{H}_c(\cdot)$, which are initially undefined everywhere, a set and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x,k)$,$\mathsf{H}_c(x)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) \coloneqq r$, $\mathcal{F}_{\text{malicious,sid}}(x,k) \coloneqq r$, or $\mathsf{H}_c(x) \coloneqq r$.

Initialization:
On message $(\text{INIT}, \text{sid})$ from party S, if this is the first INIT message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{INIT}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the INIT message for the session identifier sid. Ignore all subsequent INIT messages for sid.

Sending arbitrary output:
On $(\text{OUTPUT}, \text{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \text{sid}, \mathsf{args}\ldots)$ to P.

Adaptive server compromise:
On $(\text{COMPROMISE}, \text{sid})$ from $\mathcal{A}$, declare server S as COMPROMISED. *Note: Message* $(\textit{COMPROMISE}, \text{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function: On $(\text{OFFLINEEVAL}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathsf{S}, \mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is COMPROMISED, send $(\text{OFFLINEEVAL}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\text{OFFLINEEVAL}, \text{sid}, \text{lbl}, x)$ from $\mathcal{A}$, send $(\text{OFFLINEEVAL}, \text{sid}, \text{lbl}, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}))$ to $\mathcal{A}$.
Online evaluation:
- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \neq \mathcal{A}$, send $(\text{INPUT}, \text{sid}, \mathsf{P}, \text{EVAL}, \text{ssid}, \mathsf{P}, \mathsf{S}', x)$ to $\mathcal{A}$.
- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathcal{A}\}$, record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ and send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\text{SNDRCOMPLETE}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SNDRCOMPLETE}, \text{sid})$ to $\mathcal{A}$.
- On $(\text{RCVCOMPLETEHONEST}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not COMPROMISED, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- On $(\text{RCVCOMPLETEMALICIOUS}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}, k^*))$ to P.

Random oracles:
On $(\mathsf{H}_c, \text{sid}, x, z)$ from $\mathcal{A}$, store record $\langle \mathsf{H}_c, x, z, \mathsf{H}_c(x,z)\rangle$ and reply with $\mathsf{H}_c(x,z)$.
On $((\text{CORRELATE}, \text{sid}, , x, z, \text{lbl}))$ from $\mathcal{A}$, if there exists a record $\langle \mathsf{H}_c, x, z, y\rangle$, set $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}) \coloneqq y$.

Fig. 42. Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_6$ in the proof of Theorem 2. Differences to game $\mathbf{G}_3$ highlighted in grey boxes.

Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 2

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}})$ | $\boxed{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}^{\infty}_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}})}$

Initialization:

On message $(\text{Input}, \text{sid}, \mathsf{S}, \text{Init})$ from $\mathcal{F}_{\text{triv}}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Offline evaluation:

Online evaluation:

- On $(\text{Input}, \text{sid}, \mathsf{U}, \text{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}_{\text{triv}}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{Input}, \text{sid}, \mathsf{S}, \text{SndrComplete})$ from $\mathcal{F}_{\text{triv}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ (corresponding to Π^1_{OPRF})

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, send $(\text{Compromise}, \text{sid})$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ and declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid}\|1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, if S is compromised, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), y_2)$. Finally, if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ | $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, y_2\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2\rangle$, and if $y_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1), y_2)$.

Fig. 43. Simulator $\mathcal{S}$ for game $\mathbf{G}_6$ in the proof of Theorem 2. Continued in Figure 44 and Figure 45.

Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 2

(part 2 of 3)

$\underline{\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}\ (\text{corresponding to } \Pi^2_{\text{OPRF}})}$

On $(\textsc{Compromise}, \mathrm{sid} \parallel 2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, send $(\textsc{Compromise}, \mathrm{sid})$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ and declare server S as COMPROMISED. *Note: Message* $(\textsc{Compromise}, \mathrm{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, add $\top$ to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, for all records $\langle \mathsf{H}_c, \mathrm{sid}, x', z_1, z_2, \mathrm{lbl}_1\rangle$ such that $f(\mathsf{H}_{1,\mathrm{sid}}(x'), \mathrm{lbl}_2) = z_2$, give input $((\textsc{Correlate}, \mathrm{sid}, x', (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$. add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x, \mathcal{F}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathcal{A}, x\rangle$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \mathrm{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\mathrm{sid}}\text{++}}$ send $(\textsc{SndrComplete}, \mathrm{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x\rangle$. Otherwise, add $\top$ to $\mathrm{L}^1_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{honest,sid}}(x)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$.

On $(\textsc{RcvCompleteMalicious}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$:

- for all records $\langle \mathsf{H}_c, \mathrm{sid}, x, z_1, z_2, \mathrm{lbl}_1\rangle$ such that $f(\mathsf{H}_{1,\mathrm{sid}}(x), \mathrm{lbl}_2) = z_2$, give input $((\textsc{Correlate}, \mathrm{sid}, x, (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x\rangle$. Otherwise, add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \textsc{Nil}\rangle$. Otherwise, update R to $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2)\rangle$, and if $y_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, y_1, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$.

Fig. 44. Simulator $\mathcal{S}$ for game $\mathbf{G}_6$ in the proof of Theorem 2. Continuation of Figure 43 and continued in Figure 45.

Simulator $\mathcal{S}$ in game $\mathbf{G}_6$ in the proof of Theorem 2

(part 3 of 3)

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (z_1, z_2), \textsc{Hash}(z_1, z_2))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, y_1, y_2)$:
Send $(\textsc{Output}, \text{sid}, \mathsf{P}, \textsc{Eval}, \text{ssid}, \textsc{Hash}(y_1, y_2))$ to $\mathcal{F}$.

Procedure $\textsc{Hash}(z_1, z_2)$:
if there exists a record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$:
  return y
$\text{lbl}_1 \leftarrow \textsc{Nil}$; $\text{lbl}_2 \leftarrow \textsc{Nil}$
$x_1 \leftarrow \textsc{Nil}$; $x_2 \leftarrow \textsc{Nil}$
// Obtain (x, lbl_1) (input and key label for Π^1_{OPRF})
if $(\exists \text{lbl}'_1, \top \in \text{L}^1_{\text{sid}})(\mathcal{F}^1_{\text{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1$: **abort** "collision"
elseif $(\exists \top \in \text{L}^1_{\text{sid}})\mathcal{F}^1_{\text{honest,sid}}(x) = z_1$: $\text{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
elseif $(\exists \text{lbl}'_1, \text{lbl}''_1 \in \text{L}^1_{\text{sid}})(\text{lbl}'_1 \neq \text{lbl}''_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}''_1)$
  $\wedge\, z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_1 \in \text{L}^1_{\text{sid}})(z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: $\text{lbl}_1 \leftarrow \text{lbl}'_1$ // input and key label found
else $\text{lbl}_1 \leftarrow \perp$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain input and key for Π^2_{OPRF}
if $(\exists \text{lbl}'_2 \top \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2$: **abort** "collision"
elseif $(\exists \top \in \text{L}^2_{\text{sid}})\mathcal{F}^2_{\text{honest,sid}}(x) = z_2$: $\text{lbl}_2 \leftarrow \top$
elseif $(\exists \text{lbl}'_2, \text{lbl}''_2 \in \text{L}^2_{\text{sid}})(\text{lbl}'_2 \neq \text{lbl}''_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}''_2)$
  $\wedge\, z_2 = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_2 \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: $\text{lbl}_2 \leftarrow \text{lbl}'_2$ // input and key label found
else $\text{lbl}_2 \leftarrow \perp$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\text{lbl}_2 = \perp \wedge \text{lbl}_1 \neq \perp$: // No key for Π^2_{OPRF} but one for Π^1_{OPRF}. Need preview
  Give input $(\mathsf{H}_c, \text{sid}, x, (z_1, z_2))$ to $\mathcal{F}_{\text{CorH-OPRF}}$
  On a response y:
    Record $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2, \text{lbl}_1 \rangle$
    Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y
if $\text{lbl}_1 = \perp$: // No OPRF evaluation
  $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y
elseif $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$: // Honest keys
  if S is COMPROMISED:
    Send $(\textsc{OfflineEval}, \text{sid}, x)$ to $\mathcal{F}$; On reply $(\textsc{OfflineEval}, \text{sid}, x, y)$:
    Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$; **return** y
  else
    Pick a fresh ssid^*; send $(\textsc{Eval}, \text{sid}, \text{ssid}^*, \perp, x)$ and
      $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}$.
    if there is no response from $\mathcal{F}$: **abort** "exhausted"
    else on response $(\textsc{Eval}, \text{sid}, \text{ssid}^*, y)$:
      Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x)$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
  On reply $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y

Fig. 45. Simulator $\mathcal{S}$ for game $\mathbf{G}_6$ in the proof of Theorem 2. Continuation of Figure 43 and Figure 44.

outputs. So in this game, we can modify the simulator to not take the detour via the random oracle. Instead, we let the simulator directly instruct $\mathcal{F}$ to give the right output to the honest client.

The resulting simulator is displayed in Figure 46, Figure 47 and Figure 48.

If the simulator finds key labels for z_1, z_2, the view of the environment is exactly as before. However, the view of the environment changes if first there is a query $\mathsf{H}(x, (z_1, z_2))$ such that the simulator recorded no lbl_i explaining one of the z_i, but later such an input-label pair is extracted. That is the case if an honest client evaluated one of the two subprotocols Π^1_{OPRF} or Π^2_{OPRF} with no matching offline evaluation but still, the environment queries the hash function on the respective z_1, z_2. Observe that z_1, z_2 are sampled uniformly at random and in the case of an honest client, are never given to the environment. Thus, guessing one of them without an offline evaluation happens at most with probability $Q/2^\lambda$, where Q is the number of random oracle queries. We get

$$|\Pr[\mathbf{G}_6(\lambda)] - \Pr[\mathbf{G}_7(\lambda)]| \leq \frac{Q}{2^\lambda}\,.$$

Game $\mathbf{G}_7$: Remove the hash evaluation from evaluations with honest clients. In game $\mathbf{G}_6$, as soon as the second key for an evaluation with an honest client (HonCltSubsession) is chosen (via a RcvComplete-Honest or RcvCompleteMalicious message to Π^1_{OPRF} or Π^2_{OPRF}), the simulator looks up the function values z_1 and z_2 for x under the key labels lbl_1 resp. lbl_2 for the respective candidate schemes. It then internally executes the CompleteHonCltSubsession procedure, which accepts x as well as z_1 and z_2 as parameters. This procedure in turn evaluates the random oracle on $(x, (z_1, z_2))$, which retrieves the key labels lbl_1 and lbl_2 from the function tables of the candidate schemes again, obtains the function value for x via an Eval command to $\mathcal{F}$ followed by an RcvCompleteHonest or RcvCompleteMalicious command to $\mathcal{F}$ and finally returns the result. This value is then relayed to the honest client via the Output interface of $\mathcal{F}$. Because there are no key collisions (see $\mathbf{G}_4$), the mapping from (z_1, z_2) back to a pair of labels $\mathrm{lbl}_1, \mathrm{lbl}_2$ is well-defined for a given x. As a result, we can modify the simulator to save the key labels for evaluations with honest clients directly instead of making the detour via the function values z_1 and z_2 of the candidate OPRFs and this change does not affect the view of the environment. The resulting simulator is displayed in Figures 46 to 48.

$$\Pr[\mathbf{G}_6(\lambda)] = \Pr[\mathbf{G}_7(\lambda)]$$

Game $\mathbf{G}_8$: Moving the online evaluation by an honest client into the functionality. Right now, $\mathcal{F}$ still forwards the input x from honest clients to the simulator $\mathcal{S}$. If the real-world adversary admits the evaluation and selects a function table, the $\mathcal{S}$ then computes the corresponding output and returns it to the client via the Output interface of $\mathcal{F}$. In this game, we change the functionality $\mathcal{F}$ as well as the simulator such that the latter does not learn the inputs of honest clients anymore. Instead, honest clients will obtain their outputs directly from $\mathcal{F}$, as in $\mathcal{F}_{\text{CorH-OPRF}}$. We achieve this as follows: When an honest client sends an Eval message to $\mathcal{F}$, we let the functionality handle it in the same way as $\mathcal{F}_{\text{CorH-OPRF}}$ does, i.e., the functionality records the input and merely informs the adversary about the fact that an evaluation has been started, but without revealing the input x. This means that Eval commands by honest clients and the adversary are now treated in the same manner. Once the simulator receives the real-world adversary's choice for the two parts of the key label $(\mathrm{lbl}_1, \mathrm{lbl}_2)$ for an evaluation ssid (via RcvCompleteHonest resp. RcvCompleteMalicious messages to Π^1_{OPRF} or Π^2_{OPRF}), it directly sends a RcvCompleteHonest honest message (if both keys are from the honest server, i.e. $\mathrm{lbl}_1 = \mathrm{lbl}_2 = \top$) or RcvCompleteMalicious (if at least one key label is malicious) to $\mathcal{F}$. Note that with this change, the honest client still receives the same output as before, but this output is delivered directly from $\mathcal{F}$, as $\mathcal{S}$ completes the evaluation ssid instead of starting a new one with the adversary as the client and then re-routing the result to the honest client via the Output interface of $\mathcal{F}$.

Consequently, we have

$$\Pr[\mathbf{G}_7(\lambda)] = \Pr[\mathbf{G}_8(\lambda)]\,.$$

Game $\mathbf{G}_9$: Adding fresh label and label check to the Correlate interface. In this game, we change the functionality. First, when a new record $\langle \mathsf{H}_c, x, z, \mathsf{H}_c(x, z)\rangle$ is created on a H_c call, the functionality marks this record as fresh. Second, when the (Correlate, sid, $x, z,$ lbl) interface is called, the functionality ignores this call if lbl has been used before in an OfflineEval or an RcvCompleteMalicious command before. The functionality further only looks for $\langle \mathsf{H}_c, x, z, \mathsf{H}_c(x, z)\rangle$records that are marked as fresh. If such a record is found, the record is marked as programmed by the functionality, i.e., it is no longer fresh.

Note that this only makes a difference in the simulation if there where (Correlate, sid) calls in $\mathbf{G}_8(\lambda)$ that lead to programming before, but now, are ignored.

Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 2

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}})$ $\boxed{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}^{\infty}_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}})}$

Initialization:

On message $(\text{Input}, \text{sid}, \mathsf{S}, \text{Init})$ from $\mathcal{F}_{\text{triv}}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Online evaluation:

- On $(\text{Input}, \text{sid}, \mathsf{U}, \text{Eval}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{F}_{\text{triv}}$, store $\langle \text{sid}, \text{ssid}, \mathsf{U}, x, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{Input}, \text{sid}, \mathsf{S}, \text{SndrComplete})$ from $\mathcal{F}_{\text{triv}}$, set tx_{sid}++ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ (corresponding to Π^1_{OPRF})

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, send $(\text{Compromise}, \text{sid})$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ and declare server S as compromised. *Note: Message* $(\text{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid}\|1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, if S is compromised, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}\text{++}}$ send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set tx_{sid}−−.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, \boxed{\text{lbl}_2}\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), \boxed{\top, \text{lbl}_2}\rangle$, and if $\boxed{\text{lbl}_2} \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{honest,sid}}(x), \boxed{\top, \text{lbl}_2})$. Finally, if S is not compromised, set tx_{sid}−−.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}$ $\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \text{sid}, \text{ssid}, \mathsf{P}, x, \text{Nil}, \boxed{\text{lbl}_2}\rangle$. Otherwise, update R to $\langle \text{sid}, \text{ssid}, \mathsf{P}, x, \mathcal{F}^1_{\text{malicious,sid}}(x, \boxed{\text{lbl}_1, \text{lbl}_2})\rangle$, and if $\boxed{\text{lbl}_2} \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, x, \boxed{\text{lbl}_1, \text{lbl}_2})$.

Fig. 46. Simulator $\mathcal{S}$ for game $\mathbf{G}_7$ in the proof of Theorem 2. Continued in Figure 47 and Figure 48.

Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 2

(part 2 of 3)

$\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$ (corresponding to Π^2_{OPRF})

On $(\textsc{Compromise}, \mathrm{sid} \| 2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, send $(\textsc{Compromise}, \mathrm{sid})$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ and declare server S as COMPROMISED. *Note: Message* $(\textsc{Compromise}, \mathrm{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, add $\top$ to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, for all records $\langle \mathsf{H}_c, \mathrm{sid}, x', z_1, z_2, \mathrm{lbl}_1 \rangle$ such that $f(\mathsf{H}_{1,\mathrm{sid}}(x'), \mathrm{lbl}_2) = z_2$, give input $((\textsc{Correlate}, \mathrm{sid}, x', (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$. add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x, \mathcal{F}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathcal{A}, x \rangle$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \mathrm{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\mathrm{sid}}\text{++}}$ send $(\textsc{SndrComplete}, \mathrm{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x \rangle$. Otherwise, add $\top$ to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, \boxed{\mathrm{lbl}_1}, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, \boxed{\mathrm{lbl}_1, \top}, \mathcal{F}^2_{\mathrm{honest,sid}}(x) \rangle$, and if $\boxed{\mathrm{lbl}_1} \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, \boxed{\mathrm{lbl}_1}, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$.

On $(\textsc{RcvCompleteMalicious}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$:

- For all records $\langle \mathsf{H}_c, \mathrm{sid}, x, z_1, z_2, \mathrm{lbl}_1 \rangle$ such that $f(\mathsf{H}_{1,\mathrm{sid}}(x), \mathrm{lbl}_2) = z_2$, give input $((\textsc{Correlate}, \mathrm{sid}, x, (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x \rangle$. Otherwise, add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, \boxed{\mathrm{lbl}_1}, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, \boxed{\mathrm{lbl}_1, \mathrm{lbl}_2} \rangle$, and if $\boxed{\mathrm{lbl}_1} \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x, \boxed{\mathrm{lbl}_1, \mathrm{lbl}_2})$.

Fig. 47. Simulator $\mathcal{S}$ for game $\mathbf{G}_7$ in the proof of Theorem 2. Continuation of Figure 46 and continued in Figure 48.

Simulator $\mathcal{S}$ in game $\mathbf{G}_7$ in the proof of Theorem 2

(part 3 of 3)

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (z_1, z_2))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (z_1, z_2), \text{HASH}(z_1, z_2))$ to $\mathcal{A}$.

Procedure CompleteHonCltSubsession$(\text{sid}, \text{ssid}, \mathsf{P}, x,$ $\text{lbl}_1, \text{lbl}_2$):

Send $(\text{Output}, \text{sid}, \mathsf{P}, \text{Eval}, \text{ssid},$ RetrieveFunctionValue$(x, \text{lbl}_1, \text{lbl}_2)$) to $\mathcal{F}$.

Procedure HASH(z_1, z_2):
if there exists a record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$:
  return y
$\text{lbl}_1 \leftarrow \text{NIL}$; $\text{lbl}_2 \leftarrow \text{NIL}$
$x_1 \leftarrow \text{NIL}$; $x_2 \leftarrow \text{NIL}$
// Obtain (x, lbl_1) (input and key label for Π^1_{OPRF})
if $(\exists \text{lbl}'_1, \top \in \text{L}^1_{\text{sid}})(\mathcal{F}^1_{\text{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1$: **abort** "collision"
elseif $(\exists \top \in \text{L}^1_{\text{sid}})\mathcal{F}^1_{\text{honest,sid}}(x) = z_1$: $\text{lbl}_1 \leftarrow \top$ // honest key for Π^1_{OPRF}
elseif $(\exists \text{lbl}'_1, \text{lbl}''_1 \in \text{L}^1_{\text{sid}})(\text{lbl}'_1 \neq \text{lbl}''_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}''_1)$
  $\wedge\, z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_1 \in \text{L}^1_{\text{sid}})(z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: $\text{lbl}_1 \leftarrow \text{lbl}'_1$ // input and key label found
else $\text{lbl}_1 \leftarrow \perp$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain input and key for Π^2_{OPRF}
if $(\exists \text{lbl}'_2, \top \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2$: **abort** "collision"
elseif $(\exists \top \in \text{L}^2_{\text{sid}})\mathcal{F}^2_{\text{honest,sid}}(x) = z_2$: $\text{lbl}_2 \leftarrow \top$
elseif $(\exists \text{lbl}'_2, \text{lbl}''_2 \in \text{L}^2_{\text{sid}})(\text{lbl}'_2 \neq \text{lbl}''_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}''_2)$
  $\wedge\, z_2 = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_2 \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: $\text{lbl}_2 \leftarrow \text{lbl}'_2$ // input and key label found
else $\text{lbl}_2 \leftarrow \perp$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\text{lbl}_2 = \perp \wedge \text{lbl}_1 \neq \perp$: // No key for Π^2_{OPRF} but one for Π^1_{OPRF}. Need preview
  Give input $(\mathsf{H}_c, \text{sid}, x, (z_1, z_2))$ to $\mathcal{F}_{\text{CorH-OPRF}}$
  On a response y:
    Record $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2, \text{lbl}_1 \rangle$
    Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y
if $\text{lbl}_1 = \perp$: // No OPRF evaluation
  $y \leftarrow\!\!\$\ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y
elseif $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$: // Honest keys
  if S is COMPROMISED:
    Send $(\text{OfflineEval}, \text{sid}, x)$ to $\mathcal{F}$; On reply $(\text{OfflineEval}, \text{sid}, x, y)$:
    Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$; **return** y
  else
    Pick a fresh ssid^*; send $(\text{Eval}, \text{sid}, \text{ssid}^*, \perp, x)$ and
      $(\text{RcvCompleteHonest}, \text{sid}, \text{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}$.
    if there is no response from $\mathcal{F}$: **abort** "exhausted"
    else on response $(\text{Eval}, \text{sid}, \text{ssid}^*, y)$:
      Record $\langle \mathsf{H}, \text{sid}, (x, z_1, z_2), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\text{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x)$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
  On reply $(\text{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \text{sid}, (z_1, z_2), y \rangle$; **return** y

Fig. 48. Simulator $\mathcal{S}$ for game $\mathbf{G}_7$ in the proof of Theorem 2. Continuation of Figure 46 and Figure 47.

Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_8$ in the proof of Theorem 2

Global variables:
Mappings $\mathcal{F}_{\text{honest,sid}}(\cdot)$, $\mathcal{F}_{\text{malicious,sid}}(\cdot,\cdot)$, and $\mathsf{H}_c(\cdot)$, which are initially undefined everywhere, a set and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}_{\text{honest,sid}}(x)$, $\mathcal{F}_{\text{malicious,sid}}(x,k)$,$\mathsf{H}_c(x)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\text{honest,sid}}(x) \coloneqq r$, $\mathcal{F}_{\text{malicious,sid}}(x,k) \coloneqq r$, or $\mathsf{H}_c(x) \coloneqq r$.

Initialization:
On message $(\text{INIT}, \text{sid})$ from party S, if this is the first INIT message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{INIT}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the INIT message for the session identifier sid. Ignore all subsequent INIT messages for sid.

Sending arbitrary output:
On $(\text{OUTPUT}, \text{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \text{sid}, \mathsf{args}\ldots)$ to P.

Adaptive server compromise:
On $(\text{COMPROMISE}, \text{sid})$ from $\mathcal{A}$, declare server S as COMPROMISED. *Note: Message* $(\text{COMPROMISE}, \text{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function: On $(\text{OFFLINEEVAL}, \text{sid}, x)$ from $\mathsf{P} \in \{\mathsf{S}, \mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is COMPROMISED, send $(\text{OFFLINEEVAL}, \text{sid}, x, \mathcal{F}_{\text{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\text{OFFLINEEVAL}, \text{sid}, \text{lbl}, x)$ from $\mathcal{A}$, send $(\text{OFFLINEEVAL}, \text{sid}, \text{lbl}, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}))$ to $\mathcal{A}$.
Online evaluation:

- On $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathsf{U}, \mathcal{A}\}$, record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ and send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\text{SNDRCOMPLETE}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SNDRCOMPLETE}, \text{sid})$ to $\mathcal{A}$.
- On $(\text{RCVCOMPLETEHONEST}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{honest,sid}}(x))$ to P, and if S is not COMPROMISED, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- On $(\text{RCVCOMPLETEMALICIOUS}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, send $(\text{EVAL}, \text{sid}, \text{ssid}, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}, k^*))$ to P.

Random oracles:
On $(\mathsf{H}_c, \text{sid}, x, z)$ from $\mathcal{A}$, store record $\langle \mathsf{H}_c, x, z, \mathsf{H}_c(x,z)\rangle$ and reply with $\mathsf{H}_c(x,z)$.
On $((\text{CORRELATE}, \text{sid}, , x, z, \text{lbl}))$ from $\mathcal{A}$, if there exists a record $\langle \mathsf{H}_c, x, z, y\rangle$, set $\mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}) \coloneqq y$.

Fig. 49. Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_8$ in the proof of Theorem 2. Differences to game $\mathbf{G}_6$ highlighted in grey boxes.

Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 2

(part 1 of 3)

Global variables: Mappings $\mathcal{F}^1_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^1_{\text{malicious,sid}}(\cdot,\cdot)$, $\mathcal{F}^2_{\text{honest,sid}}(\cdot)$, $\mathcal{F}^2_{\text{malicious,sid}}(\cdot,\cdot)$ which are initially undefined everywhere two initially empty sets $\mathrm{L}^1_{\text{sid}}$ and $\mathrm{L}^2_{\text{sid}}$ and a ticket counter tx_{sid}. The first time an undefined value $\mathcal{F}^i_{\text{honest,sid}}(x)$, $\mathcal{F}^i_{\text{malicious,sid}}(x,k)$ is referenced for $i \in \{1,2\}$, the simulator chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}^i_{\text{honest,sid}}(x) := r$ resp. $\mathcal{F}^i_{\text{malicious,sid}}(x,k) := r$.

$\underline{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}_{\text{OPRF}}, \mathcal{F}^{\text{le},f,\infty}_{\text{2H-OPRF}})}$ $\boxed{\mathsf{Comb}_{\mathsf{H}}(\mathcal{F}^\infty_{\text{OPRF}}, \mathcal{F}^{\text{le},f}_{\text{2H-OPRF}})}$

Initialization:

On message $(\text{Input}, \text{sid}, \mathsf{S}, \text{Init})$ from $\mathcal{F}_{\text{triv}}$, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0$ and send $(\text{Init}, \text{sid} \,\|\, 1, \mathsf{S})$ as well as $(\text{Init}, \text{sid} \,\|\, 2, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the Init message for the session identifier sid. Ignore all subsequent Init messages for sid.

Online evaluation:

- On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{U}, \mathsf{S}')$ from $\mathcal{F}$, store $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{U}, \text{Nil}, \text{Nil}\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, i, \text{ssid}, \mathsf{U}, \mathsf{S}')$ to $\mathcal{A}$ for all $i \in \{1,2\}$.
- On $(\text{Input}, \text{sid}, \mathsf{S}, \text{SndrComplete})$ from $\mathcal{F}_{\text{triv}}$, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid} \,\|\, i)$ to $\mathcal{A}$ for all $i \in \{1,2\}$.

$\underline{\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}\ \text{(corresponding to } \Pi^1_{\text{OPRF}})}$

On $(\text{Compromise}, \text{sid} \,\|\, 1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, send $(\text{Compromise}, \text{sid})$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$ and declare server S as compromised. *Note: Message* $(\text{Compromise}, \text{sid} \,\|\, 1)$ *requires permission from the environment.*

On $(\text{OfflineEval}, \text{sid}\|1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, if S is compromised, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, x, \mathcal{F}^1_{\text{honest,sid}}(x))$ to $\mathcal{A}$. Otherwise, ignore the message.

On $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{OfflineEval}, \text{sid} \,\|\, 1, \text{lbl}_1, x, \mathcal{F}_{\text{malicious,sid}}(x, \text{lbl}_1))$ to $\mathcal{A}$.

On $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathcal{A}, x\rangle$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\text{SndrComplete}, \text{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\text{sid}}{+}{+}}$ send $(\text{SndrComplete}, \text{sid} \,\|\, 1)$ to $\mathcal{A}$.

On $(\text{RcvCompleteHonest}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, add $\top$ to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{honest,sid}}(x))$ to P $(= \mathcal{A})$, and if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{Nil}, \text{lbl}_2\rangle$ or if S is not compromised and $\mathsf{tx}_{\text{sid}} = 0$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \top, \text{lbl}_2\rangle$, and if $\text{lbl}_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \_, \top, \text{lbl}_2)$. Finally, if S is not compromised, set $\mathsf{tx}_{\text{sid}}{-}{-}$.

On $(\text{RcvCompleteMalicious}, \text{sid} \,\|\, 1, \text{ssid}, \mathsf{P}, \text{lbl}_1)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\ \boxed{\mathcal{F}^\infty_{\text{OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_1Subsession}, \text{sid}, \text{ssid}, \mathsf{P}, x\rangle$. Otherwise, add lbl_1 to $\mathrm{L}^1_{\text{sid}}$ and send $(\text{Eval}, \text{sid} \,\|\, 1, \text{ssid}, \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}_1))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{Nil}, \text{lbl}_2\rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_1, \text{lbl}_2\rangle$, and if $\text{lbl}_2 \neq \text{Nil}$, run $\text{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \_, \text{lbl}_1, \text{lbl}_2)$.

Fig. 50. Simulator $\mathcal{S}$ for game $\mathbf{G}_8$ in the proof of Theorem 2. Continued in Figure 51 and Figure 52.

Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 2

(part 2 of 3)

$\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$ (corresponding to Π^2_{OPRF})

On $(\textsc{Compromise}, \mathrm{sid} \| 2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, send $(\textsc{Compromise}, \mathrm{sid})$ to $\boxed{\mathcal{F}_{\text{OPRF}}}\boxed{\mathcal{F}^{\infty}_{\text{OPRF}}}$ and declare server S as compromised. *Note: Message* $(\textsc{Compromise}, \mathrm{sid} \parallel 2)$ *requires permission from the environment.*

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, add $\top$ to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, x, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to $\mathcal{A}$.

On $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, for all records $\langle \mathsf{H}_c, \mathrm{sid}, x', z_1, z_2, \mathrm{lbl}_1 \rangle$ such that $f(\mathsf{H}_{1,\mathrm{sid}}(x'), \mathrm{lbl}_2) = z_2$, give input $((\textsc{Correlate}, \mathrm{sid}, x', (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$. add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{OfflineEval}, \mathrm{sid} \parallel 2, \mathrm{lbl}_2, x, \mathcal{F}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to $\mathcal{A}$.

On $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathcal{A}, x \rangle$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{A}, \mathsf{S}')$ to $\mathcal{A}$.

On $(\textsc{SndrComplete}, \mathrm{sid})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$, $\boxed{\text{set } \mathsf{tx}_{\mathrm{sid}}\text{++}}$ send $(\textsc{SndrComplete}, \mathrm{sid} \parallel 2)$ to $\mathcal{A}$.

On $(\textsc{RcvCompleteHonest}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P})$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$:

- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x \rangle$. Otherwise, add $\top$ to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{honest,sid}}(x))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \top \rangle$, and if $\mathrm{lbl}_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \boxed{\ }, \mathrm{lbl}_1, \top)$.

On $(\textsc{RcvCompleteMalicious}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_2)$ from $\mathcal{A}$ to $\boxed{\mathcal{F}^{\mathrm{le},f,\infty}_{\text{2H-OPRF}}}\boxed{\mathcal{F}^{\mathrm{le},f}_{\text{2H-OPRF}}}$:

- For all records $\langle \mathsf{H}_c, \mathrm{sid}, x, z_1, z_2, \mathrm{lbl}_1 \rangle$ such that $f(\mathsf{H}_{1,\mathrm{sid}}(x), \mathrm{lbl}_2) = z_2$, give input $((\textsc{Correlate}, \mathrm{sid}, x, (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2)))$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
- If P is corrupted (i.e. $\mathsf{P} = \mathcal{A}$): Ignore this message if there is no record $\langle \mathsf{OPRF_2Subsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x \rangle$. Otherwise, add lbl_2 to $\mathrm{L}^2_{\mathrm{sid}}$ and send $(\textsc{Eval}, \mathrm{sid} \parallel 2, \mathrm{ssid}, \mathcal{F}^2_{\mathrm{malicious,sid}}(x, \mathrm{lbl}_2))$ to P $(= \mathcal{A})$.
- If P is honest (i.e. $\mathsf{P} \neq \mathcal{A}$): Ignore this message if there is no record $R := \langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \textsc{Nil} \rangle$. Otherwise, update R to $\langle \mathsf{HonCltSubsession}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}_1, \mathrm{lbl}_2 \rangle$, and if $\mathrm{lbl}_1 \neq \textsc{Nil}$, run $\textsc{CompleteHonCltSubsession}(\mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \boxed{\ }, \mathrm{lbl}_1, \mathrm{lbl}_2)$.

Fig. 51. Simulator $\mathcal{S}$ for game $\mathbf{G}_8$ in the proof of Theorem 2. Continuation of Figure 50 and continued in Figure 52.

Simulator $\mathcal{S}$ in game $\mathbf{G}_8$ in the proof of Theorem 2

(part 3 of 3)

$\mathcal{F}_{\text{RO}}$ (corresponding to the hash function H)

On $(\mathsf{H}, \text{sid}, (x, (z_1, z_2)))$ from $\mathcal{A}$, send $(\mathsf{H}, \text{sid}, (x, (z_1, z_2)), \textsc{Hash}(x, (z_1, z_2)))$ to $\mathcal{A}$.

Procedure $\textsc{CompleteHonCltSubsession}(\text{sid}, \text{ssid}, \mathsf{P}, \_, \text{lbl}_1, \text{lbl}_2)$:

- If $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$, then send $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ to $\mathcal{F}$.
- Else, send $(\textsc{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, (\text{lbl}_1, \text{lbl}_2))$ to $\mathcal{F}$.

Procedure $\textsc{Hash}(x, (z_1, z_2))$:
if there exists a record $\langle \mathsf{H}, \text{sid}, x, (z_1, z_2), y \rangle$:
  return y
$\text{lbl}_1 \leftarrow \textsc{Nil}$; $\text{lbl}_2 \leftarrow \textsc{Nil}$
// Obtain lbl_1 (key label for Π^1_{OPRF})
if $(\exists \text{lbl}'_1, \top \in \text{L}^1_{\text{sid}})(\mathcal{F}^1_{\text{honest,sid}}(x) = z_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = z_1$: **abort** "collision"
elseif $(\exists \top \in \text{L}^1_{\text{sid}})\mathcal{F}^1_{\text{honest,sid}}(x) = z_1$: $\text{lbl}_1 \leftarrow \top$, // honest key for Π^1_{OPRF}
elseif $(\exists \text{lbl}'_1, \text{lbl}''_1 \in \text{L}^1_{\text{sid}})(\text{lbl}'_1 \neq \text{lbl}''_1 \wedge \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1) = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}''_1)$
  $\wedge z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_1 \in \text{L}^1_{\text{sid}})(z_1 = \mathcal{F}^1_{\text{malicious,sid}}(x, \text{lbl}'_1))$: $\text{lbl}_1 \leftarrow \text{lbl}'_1$ // key label found
else $\text{lbl}_1 \leftarrow \bot$ // Not a valid evaluation of Π^1_{OPRF}
// Obtain key for Π^2_{OPRF}
if $(\exists \text{lbl}'_2, \top \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{honest,sid}}(x) = z_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2$: **abort** "collision"
elseif $(\exists \top \in \text{L}^2_{\text{sid}})\mathcal{F}^2_{\text{honest,sid}}(x) = z_2$: $\text{lbl}_2 \leftarrow \top$
elseif $(\exists \text{lbl}'_2, \text{lbl}''_2 \in \text{L}^2_{\text{sid}})(\text{lbl}'_2 \neq \text{lbl}''_2 \wedge \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}''_2)$
  $\wedge z_2 = \mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2))$: **abort** "collision"
elseif $(\exists! \text{lbl}'_2 \in \text{L}^2_{\text{sid}})(\mathcal{F}^2_{\text{malicious,sid}}(x, \text{lbl}'_2) = z_2)$: $\text{lbl}_2 \leftarrow \text{lbl}'_2$ // key label found
else $\text{lbl}_2 \leftarrow \bot$ // Not a valid evaluation of Π^2_{OPRF}
// Process
if $\text{lbl}_2 = \bot \wedge \text{lbl}_1 \neq \bot$: // No key for Π^2_{OPRF} but one for Π^1_{OPRF}. Need preview
  Give input $(\mathsf{H}_c, \text{sid}, x, (z_1, z_2))$ to $\mathcal{F}_{\text{CorH-OPRF}}$
  On a response y:
    Record $\langle \mathsf{H}_c, \text{sid}, x, z_1, z_2, \text{lbl}_1 \rangle$
    Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
if $\text{lbl}_1 = \bot$: // No OPRF evaluation
  $y \leftarrow\$ \{0,1\}^\lambda$; Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
elseif $\text{lbl}_1 = \top \wedge \text{lbl}_2 = \top$: // Honest keys
  if S is COMPROMISED:
    Send $(\textsc{OfflineEval}, \text{sid}, x)$ to $\mathcal{F}$; On reply $(\textsc{OfflineEval}, \text{sid}, x, y)$:
    Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
  else
    Pick a fresh ssid^*; send $(\textsc{Eval}, \text{sid}, \text{ssid}^*, \bot, x)$ and
      $(\textsc{RcvCompleteHonest}, \text{sid}, \text{ssid}^*, \mathsf{Sim})$ to $\mathcal{F}$.
    if there is no response from $\mathcal{F}$: **abort** "exhausted"
    else on response $(\textsc{Eval}, \text{sid}, \text{ssid}^*, y)$:
      Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y
else // known keys for Π^1_{OPRF} and Π^2_{OPRF}, at least one of which is malicious
  Send $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x)$ to $\mathcal{F}_{\text{CorH-OPRF}}$.
  On reply $(\textsc{OfflineEval}, \text{sid}, (\text{lbl}_1, \text{lbl}_2), x, y)$:
    Record $\langle \mathsf{H}, \text{sid}, (x, (z_1, z_2)), y \rangle$; **return** y

Fig. 52. Simulator $\mathcal{S}$ for game $\mathbf{G}_8$ in the proof of Theorem 2. Continuation of Figure 50 and Figure 51.

First, the simulator always sends the CORRELATE message for the label $(\mathrm{lbl}_1, \mathrm{lbl}_2)$ before the first OFFLINEEVAL or RCVCOMPLETEMALICIOUS call for this label. Else, the simulator would have already found records $\mathrm{lbl}_1 \in \mathrm{L}^1_{\mathrm{sid}}$ and $\mathrm{lbl}_2 \in \mathrm{L}^2_{\mathrm{sid}}$, because OFFLINEEVAL or RCVCOMPLETEMALICIOUS both directly provide the simulator with the label.

Second, the simulator only ever makes $((\textsc{Correlate}, \mathrm{sid}), \mathrm{sid}, x, (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2))$ queries, if there is a record $\langle \mathsf{H}_c, \mathrm{sid}, x, z_1, z_2, \mathrm{lbl}_1 \rangle$ such that $f(\mathsf{H}_{1,\mathrm{sid}}(x), \mathrm{lbl}_2) = z_2$. That means, if there were two queries $((\textsc{Correlate}, \mathrm{sid}), \mathrm{sid}, x, (z_1, z_2), (\mathrm{lbl}_1, \mathrm{lbl}_2))$ and $((\textsc{Correlate}, \mathrm{sid}), \mathrm{sid}, x', (z'_1, z'_2), (\mathrm{lbl}'_1, \mathrm{lbl}'_2))$ that both would lead to $\mathcal{F}$ using the *same* record $\langle \mathsf{H}_c, x, (z_1, z_2), y \rangle$ twice, the simulator must have records $\langle \mathsf{H}_c, \mathrm{sid}, x, z_1, z_2, \mathrm{lbl}_1 \rangle$ and $\langle \mathsf{H}_c, \mathrm{sid}, x', z'_1, z'_2, \mathrm{lbl}'_1 \rangle$ with $x = x', z_1 = z'_1$, and $z_2 = z'_2$. This would be a key-collision. In $\mathbf{G}_4(\lambda)$ we excluded key-collisions. We therefore get

$$\Pr[\mathbf{G}_8(\lambda)] = \Pr[\mathbf{G}_9(\lambda)].$$

Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_8$ in the proof of Theorem 2

Global variables:
Mappings $\mathcal{F}_{\mathrm{honest,sid}}(\cdot)$, $\mathcal{F}_{\mathrm{malicious,sid}}(\cdot,\cdot)$, and $\mathsf{H}_c(\cdot)$, which are initially undefined everywhere, a set and a ticket counter $\mathsf{tx}_{\mathrm{sid}}$. The first time an undefined value $\mathcal{F}_{\mathrm{honest,sid}}(x)$, $\mathcal{F}_{\mathrm{malicious,sid}}(x,k)$,$\mathsf{H}_c(x)$ is referenced, the functionality chooses $r \leftarrow\!\!\$\ \{0,1\}^\lambda$ and sets $\mathcal{F}_{\mathrm{honest,sid}}(x) \coloneqq r$, $\mathcal{F}_{\mathrm{malicious,sid}}(x,k) \coloneqq r$, or $\mathsf{H}_c(x) \coloneqq r$.

Initialization:
On message $(\textsc{Init}, \mathrm{sid})$ from party S, if this is the first INIT message for sid, set $\mathsf{tx}_{\mathrm{sid}} \leftarrow 0$ and send $(\textsc{Init}, \mathrm{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity which sent the INIT message for the session identifier sid. Ignore all subsequent INIT messages for sid.

Sending arbitrary output:
On $(\textsc{Output}, \mathrm{sid}, \mathsf{P}, \mathsf{msg}, \mathsf{args}\ldots)$ from $\mathcal{A}$, send $(\mathsf{msg}, \mathrm{sid}, \mathsf{args}\ldots)$ to P.

Adaptive server compromise:
On $(\textsc{Compromise}, \mathrm{sid})$ from $\mathcal{A}$, declare server S as COMPROMISED. *Note: Message* $(\textsc{Compromise}, \mathrm{sid})$ *requires permission from the environment.*

Offline evaluation of the honest function: On $(\textsc{OfflineEval}, \mathrm{sid}, x)$ from $\mathsf{P} \in \{\mathsf{S}, \mathcal{A}\}$, if $\mathsf{P} \neq \mathcal{A}$ or S is COMPROMISED, send $(\textsc{OfflineEval}, \mathrm{sid}, x, \mathcal{F}_{\mathrm{honest,sid}}(x))$ to P. Otherwise, ignore the message.

Offline evaluation of adversarial functions:
On $(\textsc{OfflineEval}, \mathrm{sid}, \mathrm{lbl}, x)$ from $\mathcal{A}$, send $(\textsc{OfflineEval}, \mathrm{sid}, \mathrm{lbl}, x, \mathcal{F}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}))$ to $\mathcal{A}$.
Online evaluation:
- On $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}, \mathsf{S}', x)$ from $\mathsf{P} \in \{\mathsf{U}, \mathcal{A}\}$, record $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x \rangle$ and send $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$.
- On $(\textsc{SndrComplete}, \mathrm{sid})$ from S, set $\mathsf{tx}_{\mathrm{sid}}{+}{+}$ and send $(\textsc{SndrComplete}, \mathrm{sid})$ to $\mathcal{A}$.
- On $(\textsc{RcvCompleteHonest}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if there is no record $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x \rangle$ or if S is not COMPROMISED and $\mathsf{tx}_{\mathrm{sid}} = 0$. Otherwise, send $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}, \mathcal{F}_{\mathrm{honest,sid}}(x))$ to P, and if S is not COMPROMISED, set $\mathsf{tx}_{\mathrm{sid}}{-}{-}$.
- On $(\textsc{RcvCompleteMalicious}, \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, \mathrm{lbl}, k^*)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \mathrm{sid}, \mathrm{ssid}, \mathsf{P}, x \rangle$. Otherwise, send $(\textsc{Eval}, \mathrm{sid}, \mathrm{ssid}, \mathcal{F}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}, k^*))$ to P.

Random oracles:
On $(\mathsf{H}_c, \mathrm{sid}, x, z)$ from $\mathcal{A}$, store record $\langle \mathsf{H}_c, x, z, \mathsf{H}_c(x,z) \rangle$ marked FRESH and reply with $\mathsf{H}_c(x,z)$.
On $((\textsc{Correlate}, \mathrm{sid}, , x, z, \mathrm{lbl}))$ from $\mathcal{A}$, if lbl has not been used in an OFFLINEEVAL or RCVCOMPLETEMALICIOUS command before and there exists a record $\langle \mathsf{H}_c, x, z, y \rangle$ marked FRESH, mark it PROGRAMMED, and set $\mathcal{F}_{\mathrm{malicious,sid}}(x, \mathrm{lbl}) \coloneqq y$.

Fig. 53. Ideal functionality $\mathcal{F}$ in game $\mathbf{G}_9$ in the proof of Theorem 2. Differences to game $\mathbf{G}_8$ highlighted in grey boxes.

Game $\mathbf{G}_{10}$: Cleaning up. Finally, we remove the OUTPUT interface from $\mathcal{F}$. Since $\mathcal{S}$ does not use this interface anymore, this removal is not visible to the environment.

$$\Pr[\mathbf{G}_9(\lambda)] = \Pr[\mathbf{G}_{10}(\lambda)]$$

With this change, we arrive at $\mathcal{F} = \mathcal{F}_{\text{CorH-OPRF}}$ and $\mathcal{S} = \mathsf{Sim}_1^{\mathrm{OPRF}}$, which concludes the proof.

E $\mathcal{F}_{\text{CorH-OPRF}}$ Provides at Least the Security Guarantees of $\mathcal{F}_{\text{corOPRF}}$

In this section, we argue that $\text{IDEAL}_{\mathcal{F}_{\text{CorH-OPRF}}}$, the ideal protocol for the ideal functionality $\mathcal{F}_{\text{CorH-OPRF}}$ (see Fig. 3) that our hash combiner achieves, UC-realizes $\mathcal{F}_{\text{corOPRF}}$, the correlated OPRF functionality from [JKX21], which is shown in Fig. 54. In other words, $\mathcal{F}_{\text{CorH-OPRF}}$ provides either the same or stronger security guarantees than $\mathcal{F}_{\text{corOPRF}}$. Since $\mathcal{F}_{\text{corOPRF}}$ was shown to be sufficient for instantiating the OPAQUE compiler [JKX21], it follows from the transitivity of UC emulation [Can00, Sec. 4.5] that any protocol resulting from our combiner can be used in OPAQUE if at least one candidate scheme is secure.

$\mathcal{F}_{\text{CorH-OPRF}}$ and $\mathcal{F}_{\text{corOPRF}}$ offer different means to the adversary $\mathcal{A}$ for correlating values. $\mathcal{F}_{\text{corOPRF}}$ allows the adversary to copy function values from existing function tables (specified via the list L in OfflineEval and RcvComplete commands) to new function tables when they are used for the first time. The adversary can only copy a single function value from each existing table, but it turns out that one can work around this limitation by introducing intermediate dummy keys. We will use this strategy in our argument. $\mathcal{F}_{\text{CorH-OPRF}}$, on the other hand, accomplishes correlations by allowing the adversary to copy values from an independent random oracle to newly created function tables. Each random oracle output may only be copied into a single function table, but in contrast to $\mathcal{F}_{\text{corOPRF}}$, the adversary can copy *multiple* values from the oracle to the same function table (for different x). A closer inspection reveals that from a conceptual point of view, the two functionalities are not too far apart: Both functionalities embrace a clear separation between a "programming" and "evaluation" phase for each function label, i.e., once a function label has been used for an evaluation, it can no longer be programmed. The main differences between them are that (i) $\mathcal{F}_{\text{corOPRF}}$ allows correlations with the honest function table, whereas they are not allowed in $\mathcal{F}_{\text{CorH-OPRF}}$; (ii) $\mathcal{F}_{\text{corOPRF}}$ allows to mirror the same function value into different tables, whereas $\mathcal{F}_{\text{CorH-OPRF}}$ does not.

Lemma 1. *$IDEAL_{\mathcal{F}_{\text{CorH-OPRF}}}$ UC-realizes $\mathcal{F}_{\text{corOPRF}}$.*

Proof sketch. To simulate $\mathcal{F}_{\text{CorH-OPRF}}$ using $\mathcal{F}_{\text{corOPRF}}$, our simulator, which is depicted in Fig. 55, implements the random oracle $\mathsf{H}_c(x, z)$ via offline evaluations of x on artificial labels $\text{lbl}' = (\text{"RO"}, z)$. These function values are never used in actual OPRF evaluations, so they do not interfere with the core OPRF functionality. When the adversary calls Correlate to copy the random oracle value $\mathsf{H}_c(x, z)$ to some function table with label lbl, the simulator ignores this request if this value has already been written to a function table, and otherwise sends an OfflineEval command to $\mathcal{F}_{\text{corOPRF}}$ with $L \coloneqq [((\text{"RO"}, z), x)]$ to apply the function value of x in table $(\text{"RO"}, z)$ to the table identified by lbl. This has the same effect as writing the random oracle value $\mathsf{H}_c(x, z)$ to table lbl would have in $\mathcal{F}_{\text{CorH-OPRF}}$. All other messages commands are simply forwarded between the adversary $\mathcal{A}$ and the ideal functionality $\mathcal{F}_{\text{corOPRF}}$ (with some minor syntactical modifications to bridge the syntactic differences between $\mathcal{F}_{\text{CorH-OPRF}}$ and $\mathcal{F}_{\text{corOPRF}}$, see Fig. 55).

The last remaining challenge for the simulation is dealing with multiple Correlate commands with the same z and target table lbl. Using our current simulation strategy, a second Correlate command from $\mathcal{A}$ with the same lbl and z as a previous one would fail for two reasons: (1) A new function table can only be initialized with correlated values once. The list L is ignored in all but the first call to OfflineEval for lbl. (2) Function tables can only be correlated on at most one value per existing table. While (1) could be fixed by accumulating Correlate requests and lazily applying them all at once as soon as the corresponding lbl is used in an evaluation, (2) cannot be addressed this way. To deal with this, we add another layer of indirection: The simulator maintains a table K that maps any (non-artificial and non-dummy) label lbl to an initially empty finite ordered sequence $(z_{\text{sid,lbl},1}, x_{\text{sid,lbl},1}), \ldots, (z_{\text{sid,lbl},k}, x_{\text{sid,lbl},k})$ of tuples that keep track of which x and z the label lbl has been programmed on so far. Then, for every $(\text{Correlate}, \text{sid}, x, z, \text{lbl})$ command from $\mathcal{A}$, the simulator creates a new dummy table identifier lbl_{k+1} which is correlated with lbl_j on $x_{\text{sid,lbl},j}$ for all $j \in [k]$ and with $(\text{"RO"}, z)$ on x. To do so, the simulator sends an OfflineEval command to $\mathcal{F}_{\text{corOPRF}}$ with $L \coloneqq [(\text{lbl}_1, x_{\text{sid,lbl},1}), \ldots, (\text{lbl}_k, x_{\text{sid,lbl},k}), ((\text{"RO"}, z), x)]$. After that, the simulator appends (x, z) to $K[\text{lbl}]$. When $\mathcal{A}$ later sends an OfflineEval or RcvComplete command with lbl, the simulator simply forwards this command to $\mathcal{F}_{\text{corOPRF}}$ but with the newest label $\text{lbl}_{|K[\text{lbl}]|}$ instead of lbl and with an empty list $L \coloneqq []$. From this point on, no more Correlate commands are accepted for this lbl, which is in line with $\mathcal{F}_{\text{CorH-OPRF}}$.

Ideal correlated OPRF functionality $\mathcal{F}_{\text{corOPRF}}$ [JKX21, BDFH25]

Public parameters: PRF output length ℓ, polynomial in the security parameter λ.

Conventions: $\forall i, x$, value $F_i(x)$ is initially undefined, and if undefined $F_i(x)$ is referenced, then $\mathcal{F}_{\text{corOPRF}}$ sets $F_i(x) \leftarrow\!\!\$\ \{0,1\}^\ell$. Variable P ranges over all *honest* network entities and $\mathcal{A}$, and we assume *all corrupt entities are operated by* $\mathcal{A}$.

Initialization: On $(\text{Init}, \text{sid})$ from S, if this is the first Init message for sid, set $\mathsf{tx}_{\text{sid}} \leftarrow 0, \mathcal{N} \leftarrow [\mathsf{S}], \mathcal{E} \leftarrow \{\}, \mathcal{G} \leftarrow (\mathcal{N}, \mathcal{E})$ and send $(\text{Init}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$. From now on, use tag S to denote the unique entity that sent the Init message for session identifier sid. (Ignore all subsequent Init messages for sid.)

Server Compromise: On $(\text{Compromise}, \text{sid})$ from $\mathcal{A}$, declare server S as compromised. *Note: Message* $(\textit{Compromise}, \text{sid})$ *requires permission from the environment.* (If S is corrupted, then it is declared compromised as well.)

Offline Evaluation: On $(\text{OfflineEval}, \text{sid}, i, x, L)$ from P do:
(1) If $\mathsf{P} = \mathcal{A}$ and $i \notin \mathcal{N}$, then append i to $\mathcal{N}$ and run $\text{Correlate}(i, L)$;
(2) Ignore this message if $\mathsf{P} = \mathcal{A}$, S is not compromised, and $(i, \mathsf{S}, x) \in \mathcal{E}$;
(3) Send $(\text{OfflineEval}, \text{sid}, F_i(x))$ to P if (i) $\mathsf{P} = \mathsf{S}$ and $i = \mathsf{S}$ or (ii) $\mathsf{P} = \mathcal{A}$ and either $i \neq \mathsf{S}$ or S is compromised.

Online Evaluation:
- On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from P, send $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ to $\mathcal{A}$. Record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$.
- On $(\text{SndrComplete}, \text{sid})$ from S, set $\mathsf{tx}_{\text{sid}}{+}{+}$ and send $(\text{SndrComplete}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$.
- On $(\text{RcvComplete}, \text{sid}, \text{ssid}, \mathsf{P}, i, L)$ from $\mathcal{A}$, ignore this message if there is no record $\langle \text{sid}, \text{ssid}, \mathsf{P}, x \rangle$ stored. Else:
  1. If $i \notin \mathcal{N}$, then append i to $\mathcal{N}$ and run $\text{Correlate}(i, L)$;
  2. If S is not compromised and ($i = \mathsf{S}$ or $[(i, \mathsf{S}, x) \in \mathcal{E}$ and $\mathsf{P} = \mathcal{A}]$):
     If $\mathsf{tx}_{\text{sid}} = 0$, ignore this message, else set $\mathsf{tx}_{\text{sid}}{-}{-}$.
  3. Send $(\text{Eval}, \text{sid}, \text{ssid}, F_i(x))$ to P.

Procedure $\text{Correlate}(i, L)$:
Reject if list L contains elements (j, x), (j', x') s.t. $j = j'$ and $x \neq x'$. Else, for all $(j, x) \in L$ s.t. $j \in \mathcal{N}$, add (i, j, x) to $\mathcal{E}$ and set $F_i(x) \leftarrow F_j(x)$.

Fig. 54. Ideal correlated OPRF functionality $\mathcal{F}_{\text{corOPRF}}$ [JKX21, BDFH25].

Simulator Sim showing that $\text{IDEAL}_{\mathcal{F}_{\text{CorH-OPRF}}}$ UC-realizes $\mathcal{F}_{\text{corOPRF}}$

Global variables:
- Identity of the honest server S_{sid},
- Table K_{sid} that maps any (non-artificial, non-dummy) label lbl to an initially empty finite ordered sequence $(z_{\text{sid,lbl},1}, x_{\text{sid,lbl},1}), \ldots, (z_{\text{sid,lbl},k}, x_{\text{sid,lbl},k})$ of tuples that keep track of which x and z the label lbl has been programmed on in session sid so far.

Messages from $\mathcal{A}$ against $\mathcal{F}_{\text{CorH-OPRF}}$ to Sim
On $(\text{Compromise}, \text{sid})$ from $\mathcal{A}$, send $(\text{Compromise}, \text{sid})$ to $\mathcal{F}_{\text{corOPRF}}$. *Note: Message* $(\textit{Compromise}, \text{sid})$ *requires permission from the environment.*

Offline evaluation:
On $(\text{OfflineEval}, \text{sid}, x)$ from $\mathcal{A}$, ignore this message if S_{sid} is not defined. Otherwise, send $(\text{OfflineEval}, \text{sid}, \mathsf{S}_{\text{sid}}, x, L \coloneqq [])$ to $\mathcal{F}_{\text{corOPRF}}$ and forward the reply to $\mathcal{A}$. *(We also include x in the response, as $\mathcal{F}_{\text{CorH-OPRF}}$ does.)*

On $(\text{OfflineEval}, \text{sid}, \text{lbl}, x)$ from $\mathcal{A}$, mark lbl as FINALIZED. Then, send $(\text{OfflineEval}, \text{sid}, \text{lbl}_{|K_{\text{sid}}[\text{lbl}]|}, x, L \coloneqq [])$ to $\mathcal{F}_{\text{corOPRF}}$ and forward the reply to $\mathcal{A}$. *(We also include* lbl *and x in the response, as $\mathcal{F}_{\text{CorH-OPRF}}$ does.)*

Online evaluation:
On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{S}', x)$ from $\mathcal{A}$, forward this command to $\mathcal{F}_{\text{corOPRF}}$.

On $(\text{RcvCompleteHonest}, \text{sid}, \text{ssid}, \mathsf{P})$ from $\mathcal{A}$, ignore this message if S_{sid} is not defined. Otherwise, send $(\text{RcvComplete}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}_{\text{sid}}, L \coloneqq [])$ to $\mathcal{F}_{\text{corOPRF}}$.

On $(\text{RcvCompleteMalicious}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl})$ from $\mathcal{A}$, mark lbl as FINALIZED. Then, send $(\text{RcvComplete}, \text{sid}, \text{ssid}, \mathsf{P}, \text{lbl}_{|K_{\text{sid}}[\text{lbl}]|}, L \coloneqq [])$ to $\mathcal{F}_{\text{corOPRF}}$.

Random oracles:
On $(\mathsf{H}_c, \text{sid}, x, z)$ from $\mathcal{A}$, if this is the first query for x, z and sid, store record $\langle \mathsf{H}_{\mathsf{c}}, \text{sid}, x, z, \mathsf{H}_c(z) \rangle$ marked FRESH. In any case, send $(\text{OfflineEval}, \text{sid}, (\text{“RO”}, z), x, L \coloneqq [])$ to $\mathcal{F}_{\text{corOPRF}}$ and reply to $\mathcal{A}$ with the obtained function value.

On $(\text{Correlate}, \text{sid}, x, z, \text{lbl})$ from $\mathcal{A}$, ignore this message if lbl has been marked as FINALIZED or $(\exists z')((z', x) \in K_{\text{sid}}[\text{lbl}])$ or there is no record $R \coloneqq \langle \mathsf{H}_{\mathsf{c}}, \text{sid}, x, z, \mathsf{H}_c(z) \rangle$ marked FRESH. Else, send $(\text{OfflineEval}, \text{sid}, \text{lbl}_{|K_{\text{sid}}[\text{lbl}]|+1}, x, L \coloneqq [(z_{\text{sid,lbl},1}, x_{\text{sid,lbl},1}), \ldots, (z_{\text{sid,lbl},k}, x_{\text{sid,lbl},k}), ((\text{“RO”}, z), x)])$ to $\mathcal{F}_{\text{corOPRF}}$ and after that, append (z, x) to the list $K_{\text{sid}}[\text{lbl}]$ and mark the record R as PROGRAMMED. Ignore the response to the OfflineEval command by $\mathcal{F}_{\text{corOPRF}}$.

Messages from $\mathcal{F}_{\text{corOPRF}}$ to Sim
On $(\text{Init}, \text{sid}, \mathsf{S})$ from $\mathcal{F}_{\text{corOPRF}}$, set $\mathsf{S}_{\text{sid}} \leftarrow \mathsf{S}$ and send $(\text{Init}, \text{sid}, \mathsf{S})$ to $\mathcal{A}$.
On $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{P}, \mathsf{S}')$ from $\mathcal{F}_{\text{corOPRF}}$, forward this message to $\mathcal{A}$.
On $(\text{Eval}, \text{sid}, \text{ssid}, y)$ from $\mathcal{F}_{\text{corOPRF}}$, forward this message to $\mathcal{A}$.
On $(\text{SndrComplete}, \text{sid}, \mathsf{S})$ from $\mathcal{F}_{\text{corOPRF}}$, send $(\text{SndrComplete}, \text{sid})$ to $\mathcal{A}$.

Fig. 55. Simulator showing that $\text{IDEAL}_{\mathcal{F}_{\text{CorH-OPRF}}}$ UC-realizes $\mathcal{F}_{\text{corOPRF}}$.

F Game-based Definition of OPRFs

F.1 PRFs

Before we define *oblivious* pseudorandom functions, we first give a formal definition of (non-interactive) pseudorandom functions (PRFs).

Definition 7 (PRF). *Let* $\mathsf{Setup}(1^\lambda) \xrightarrow{\$} \zeta$ *be an efficient probabilistic algorithm that on input* 1^λ *with* $\lambda \in \mathbb{N}_+$ *outputs a system parameter* $\zeta \in \{0,1\}^*$*. Let* $\mathcal{K} := \{\mathcal{K}_\zeta\}_{\zeta \in \mathrm{supp}(\mathsf{Setup}(1^\lambda))}$*,* $\mathcal{X} := \{\mathcal{X}_\zeta\}_{\zeta \in \mathrm{supp}(\mathsf{Setup}(1^\lambda))}$ *and* $\mathcal{Y} := \{\mathcal{Y}_\zeta\}_{\zeta \in \mathrm{supp}(\mathsf{Setup}(1^\lambda))}$ *be ensembles of finite sets that are indexed by the system parameter* $\zeta \in \mathrm{supp}(\mathsf{Setup}(1^\lambda))$ *such that* $\mathcal{K}$ *and* $\mathcal{Y}$ *are efficiently recognizable and efficiently samplable and* $\mathcal{X}$ *is efficiently recognizable. A pseudorandom function* F *is a family of efficiently computable functions*

$$\{\mathsf{F}_\zeta \colon \mathcal{K}_\zeta \times \mathcal{X}_\zeta \to \mathcal{Y}_\zeta\}_{\zeta \in \mathrm{supp}(\mathsf{Setup}(1^\lambda))}\,.$$

We say that F *is secure if and only if for every PPT adversary* D *there exists a negligible function* $\mathsf{negl} \colon \mathbb{N}_+ \to \mathbb{R}$ *such that for every* $\lambda \in \mathbb{N}_+$ *and every* $\zeta \in \mathrm{supp}(\mathsf{Setup}(1^\lambda))$*, we have* $\mathsf{Adv}^{\mathrm{prf}}_{\mathsf{F},\mathsf{D}}(\lambda) \leq \mathsf{negl}(\lambda)$*, wherein* $\mathsf{Adv}^{\mathrm{prf}}_{\mathsf{F},\mathsf{D}}(\lambda)$ *is defined as follows:*

$$\mathsf{Adv}^{\mathrm{prf}}_{\mathsf{F},\mathsf{D}}(\lambda) := \left| \Pr_{k \leftarrow\$ \mathcal{K}_\zeta}\left[\mathsf{D}^{\mathsf{F}(k,\cdot)}(1^\lambda) = 1\right] - \Pr_{f \leftarrow\$ (\mathcal{X}_\zeta \to \mathcal{Y}_\zeta)}\left[\mathsf{D}^{f(\cdot)}(1^\lambda) = 1\right] \right|.$$

Unless otherwise stated, we set $\mathcal{X}_\zeta := \{0,1\}^{\leq \mathsf{F.il}(\lambda)}$ and $\mathcal{Y}_\zeta := \{0,1\}^{\mathsf{F.ol}(\lambda)}$ for some fixed polynomials $F.\mathsf{il}, F.\mathsf{ol} \in (\mathbb{N}_+ \to \mathbb{N}_+)$.

F.2 Syntax

For the syntax of OPRFs, we adopt the definition from [TCR+22, Sec. 3], with two modifications: We extend the definition to allow more than two rounds of communication in the evaluation protocol and we drop partial obliviousness, i.e., we drop the public part of the input (tag) as we focus on plain OPRFs without additional features.

Furthermore, we use the following convention: When running an interactive protocol, each party usually maintains a status variable status which is set to Nil (or undetermined) for incomplete executions, $\top$ for accepted executions, and $\bot$ for rejected executions. In particular, if the status is set to $\top$ or $\bot$, we assume that the party does not accept any further messages and does not output any more messages.

Definition 8 (OPRF). *An oblivious pseudorandom function* F *is a quintuple* $\mathsf{F} := (\mathsf{F.Setup}, \mathsf{F.KGen}, \mathsf{F.CEval}, \mathsf{F.SEval}, \mathsf{F.Eval})$ *of efficient algorithms where*

- $\mathsf{F.Setup}(1^\lambda) \xrightarrow{\$} \zeta$ *is a probabilistic algorithm that takes as input the security parameter* $\lambda \in \mathbb{N}_+$ *and outputs a system parameter* ζ*.*
- $\mathsf{F.KGen}(\zeta) \xrightarrow{\$} \mathsf{k}$ *is a probabilistic algorithm that accepts the system parameter and generates a secret key (for the server).*
- $\mathsf{F.CEval}(\zeta, x, \mathsf{state}, \mathit{msg}) \xrightarrow{\$} (\mathsf{state}', \mathit{status}, \mathit{msg}', y)$ *is the probabilistic algorithm run by the client in the evaluation protocol. The algorithm is invoked once for each round of the protocol in which the client is the active party. It takes the system parameter* ζ *and the client's input* x *to the OPRF as well as the client's internal state* state *and an incoming message* msg *from the server. For the first invocation (where there neither is internal state nor a message from the server yet),* state *and* msg *are set to* Nil*. The algorithm outputs a new internal state* state' *as well as a status* $\mathit{status} \in \{\textsc{Nil}, \top, \bot\}$ *indicating whether it intends to continue the protocol execution (*Nil*), it accepts the execution (*$\top$*) or rejects it (*$\bot$*). If* $\mathit{status} = \textsc{Nil}$*,* y *is ignored and* msg' *is interpreted as the next message to be delivered to the server. If* $\mathit{status} = \top$*,* msg' *is ignored and* y *is interpreted as the client's output. Otherwise, the value* $\mathit{status} = \bot$ *indicates that the client aborts the protocol execution.*
- $\mathsf{F.SEval}(\zeta, \mathsf{k}, \mathsf{state}, \mathit{msg}) \xrightarrow{\$} (\mathsf{state}', \mathit{status}, \mathit{msg}')$ *is the probabilistic algorithm run by the server in the evaluation protocol. Similarly to* $\mathsf{F.CEval}$*, this algorithm is invoked once for each round of the protocol in which the server is active. It takes the system parameter* ζ *and the server's key* k *as well as an internal state* state *and an incoming message* msg *from the client. For the first invocation (where there is no internal state yet),* state *is set to* Nil*. The algorithm outputs a status* $\mathit{status} \in \{\textsc{Nil}, \top, \bot\}$ *indicating whether it intends to continue the protocol execution (*Nil*), it accepts the execution* $\top$ *or rejects it (*$\bot$*), as well as a new internal state* state' *and the next message* msg' *to be delivered to the client. If* $\mathit{status} = \bot$*, the protocol execution is aborted, so* msg' *is ignored and not delivered to the client anymore.*

– $\mathsf{F.Eval}(\zeta, \mathsf{k}, x) \to y$ *is a deterministic algorithm that allows a party in possession of the secret key* k *(which is usually the server only) to non-interactively evaluate the OPRF on input* x*, yielding the function value* y*.*
– *For every system parameter* ζ*, every* $x \in \mathcal{X}_{\lambda,\zeta}$*, every key* k *and every sequence of incoming messages* $(\mathit{msg}_1, \mathit{msg}_2, \ldots) \in (\{0,1\}^*)^\infty$*, the algorithms* $\mathsf{F.CEval}$ *and* $\mathsf{F.SEval}$ *set* $\mathit{status} \neq \textsc{Nil}$ *after at most polynomially many rounds in the security parameter.*

We restrict our attention to OPRFs which satisfy a natural correctness property by requiring that interactive evaluations between an honest client and an honest server with undisturbed network communication are successful and result in the same function value as non-interactive evaluation with overwhelming probability.

Definition 9 (OPRF Correctness, adapted from [ADDG24, Def. 2] and [TCR+22]). *An OPRF* $\mathsf{F} := (\mathsf{F.Setup}, \mathsf{F.KGen}, \mathsf{F.CEval}, \mathsf{F.SEval}, \mathsf{F.Eval})$ *is correct iff there exists a negligible function* $\mathsf{negl}\colon \mathbb{N}_+ \to \mathbb{R}$ *such that for all* $\lambda \in \mathbb{N}_+$*, all* $\zeta \in \mathrm{supp}(\mathsf{F.Setup}(\lambda))$*, all* $\mathsf{k} \in \mathrm{supp}(\mathsf{F.KGen}(\zeta))$ *and all* $x \in \mathcal{X}_{\lambda,\zeta}$*, we have* $\Pr[\mathbf{Exp}^{\text{oprf-corr}}_{\mathsf{F}}(\lambda) = 1] \geq 1 - \mathsf{negl}(\lambda)$*, wherein* $\mathbf{Exp}^{\text{oprf-corr}}_{\mathsf{F}}(\lambda)$ *is defined as in Fig. 56.*

Experiment $\mathbf{Exp}^{\text{oprf-corr}}_{\mathsf{F}}(\lambda)$

$\mathsf{state}_{\text{clt}} \leftarrow \textsc{Nil}$; $\mathsf{state}_{\text{srv}} \leftarrow \textsc{Nil}$; $\mathsf{clientStatus} \leftarrow \textsc{Nil}$; $\mathsf{serverStatus} \leftarrow \textsc{Nil}$
$y \leftarrow \textsc{Nil}$; $\mathsf{msg} \leftarrow \textsc{Nil}$; $a \leftarrow 0$ // active party: 0 = client, 1 = server
while $\mathsf{clientStatus} \notin \{\bot, \top\} \wedge \mathsf{serverStatus} \neq \bot$
  if $a \stackrel{?}{=} 0$: $(\mathsf{state}_{\text{clt}}, \mathsf{clientStatus}, \mathsf{msg}, y) \leftarrow\!\!\$\ \mathsf{F.CEval}(\zeta, x, \mathsf{state}_{\text{clt}}, \mathsf{msg})$
  elseif $\mathsf{serverStatus} \neq \top$
    $(\mathsf{state}_{\text{srv}}, \mathsf{serverStatus}, \mathsf{msg}) \leftarrow\!\!\$\ \mathsf{F.SEval}(\zeta, k, \mathsf{state}_{\text{srv}}, \mathsf{msg})$
  else return 0 // Unexpected message to the already finished server
  $a \leftarrow 1 - a$
return $\mathsf{clientStatus} \stackrel{?}{=} \top \wedge \mathsf{serverStatus} \stackrel{?}{=} \top \wedge y \stackrel{?}{=} \mathsf{F.Eval}(\zeta, k, x) \wedge y \stackrel{?}{\in} \mathcal{Y}_{\lambda,\zeta}$

Fig. 56. Correctness experiment for OPRFs

F.3 Security

The game-based security definition involves two properties: pseudorandomness (protecting the server against a malicious client) and obliviousness (protecting the client against a malicious server).

Pseudorandomness. The security experiment for pseudorandomness is shown in Figure 57. It is based on the pseudorandomness experiment in [TCR+22, Figure 3], but we extend it to also be suitable for OPRFs with more than two rounds of communication and forgo the public part of the input (tag). In the experiment, the adversary, which represents a malicious client, has access to three oracles: An evaluation oracle Ev for non-interactively evaluating the OPRF at arbitrary points, a Srv oracle for interactively evaluating the OPRF by interacting with a server, and an oracle Prim for evaluating an idealized primitive P, which is usually a random oracle. Depending on a challenge bit b, the oracles provide access to either an honest server instance following the actual OPRF protocol ($b = 0$), or a simulation of the OPRF protocol ($b = 1$). The goal of the adversary is to distinguish between the real OPRF protocol and the simulation, i.e., to determine the challenge bit b.

In the simulation case, the Ev oracle implements a truly random function, and calls to Srv and Prim are forwarded to a simulator Sim which has restricted access to Ev: It cannot query Ev more often than the number of queries made by the adversary to the Srv oracle. This is implemented by incrementing a counter value tx for each new server session and giving the simulator access to a limited evaluation oracle LimEv in which tickets are redeemed. While adhering to this restriction, the simulator needs to make the oracle responses consistent with the responses of the Ev oracle.

Experiment $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F},\mathcal{A},\mathsf{Sim},b}(\lambda)$

$\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$

$\zeta \leftarrow\!\!\$\ \mathsf{F.Setup}(1^\lambda)$

$f \leftarrow\!\!\$\ (\mathcal{X}_{\lambda,\zeta} \to \mathcal{Y}_{\lambda,\zeta})$

$\mathsf{state}_{\mathrm{P}} \leftarrow\!\!\$\ \mathrm{P.Init}(1^\lambda)$

$k \leftarrow\!\!\$\ \mathsf{F.KGen}(\zeta)$

$\mathsf{state}_{\mathsf{Sim}} \leftarrow\!\!\$\ \mathsf{Sim.Init}(1^\lambda, \zeta)$

$b' \leftarrow\!\!\$\ \mathcal{A}^{\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$

return b'

Oracle $\textsc{Ev}(x)$

$y_0 \leftarrow \mathsf{F.Eval}(\zeta, k, x);\ y_1 \leftarrow f(x);$ **return** y_b

Oracle $\textsc{LimEv}(x)$

if $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$

$\mathsf{tx} \leftarrow \mathsf{tx} - 1;$ **return** $\textsc{Ev}(x)$

Oracle $\textsc{Prim}(x)$

$(\mathsf{state}_{\mathsf{Sim}}, y_1) \leftarrow\!\!\$\ \mathsf{Sim.PrimEv}^{\textsc{LimEv}(\cdot)}(\mathsf{state}_{\mathsf{Sim}}, x)$

$(\mathsf{state}_{\mathrm{P}}, y_0) \leftarrow\!\!\$\ \mathrm{P.Eval}(\mathsf{state}_{\mathrm{P}}, x);$ **return** y_b

Oracle $\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

$\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$

// recover state (existing interaction) or increment tx counter (new interaction)

if ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$

if $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$

if $b \stackrel{?}{=} 0$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow\!\!\$\ \mathsf{F.SEval}(\zeta, k, \mathsf{state}, \mathsf{msg})$

else $(\mathsf{state}_{\mathsf{Sim}}, \mathsf{status}, \mathsf{resp}) \leftarrow\!\!\$\ \mathsf{Sim.SEval}^{\textsc{LimEv}(\cdot)}(\mathsf{ssid}, \mathsf{msg}, \mathsf{state}_{\mathsf{Sim}})$

$\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}, \mathsf{status});$ **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 57. Security experiment for pseudorandomness (security against malicious clients)

Definition 10 (OPRF pseudorandomness, adapted from [TCR+22]). *We say that an OPRF* F *provides pseudorandomness iff for every* PPT *adversary* $\mathcal{A}$ *there exists a* PPT *simulator* $\mathsf{Sim} := (\mathsf{Sim.Init}, \mathsf{Sim.SEval}, \mathsf{Sim.PrimEv})$ *and a negligible function* $\mathsf{negl}\colon \mathbb{N}_+ \to \mathbb{R}$ *such that for all* $\lambda \in \mathbb{N}_+$*, we have*

$$\mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F},\mathcal{A},\mathsf{Sim}}(\lambda) := \left|\Pr[\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F},\mathcal{A},\mathsf{Sim},0}(\lambda) = 1] - \Pr[\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F},\mathcal{A},\mathsf{Sim},1}(\lambda) = 1]\right| \leq \mathsf{negl}(\lambda)\,,$$

wherein $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F},\mathcal{A},\mathsf{Sim},b}(\lambda)$ *is defined as in Fig. 57.*

Obliviousness. Security against a malicious server is formalized using the *obliviousness* security experiment (Figure 58). This experiment is akin to the indistinguishability experiment for encryption schemes: The adversary chooses two inputs $x_0, x_1 \in \mathcal{X}_{\lambda,\zeta}$ to the OPRF. One of these is then used in an OPRF evaluation between a challenger, which acts as an honest client, and the adversary, which acts as the server. The goal of the adversary is to determine which input was used, and security is defined by requiring the distance between the output distributions of the adversary for the case that x_0 is used, and the case that x_1 is used, to be negligible. We consider a variant of this definition originally introduced in [Leh19], and adapt it to the multi-round setting.

Definition 11 (OPRF obliviousness, adapted from [Leh19]). *We say that an OPRF* F *is oblivious iff for every* PPT *adversary* $\mathcal{A}$ *there exists a negligible function* $\mathsf{negl}\colon \mathbb{N}_+ \to \mathbb{R}$ *such that*

$$\mathsf{Adv}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{A}}(\lambda) := \left|\Pr[\mathbf{Exp}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{A},0}(\lambda) = 1] - \Pr[\mathbf{Exp}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{A},1}(\lambda) = 1]\right| \leq \mathsf{negl}(\lambda)\,,$$

wherein $\mathbf{Exp}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{A},b}(\lambda)$ *is defined as in Fig. 58.*

Experiment $\mathbf{Exp}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{A},b}(\lambda)$

$\zeta \leftarrow\!\!\$\ \mathsf{F.Setup}(1^\lambda)$; $(x_0, x_1, \mathsf{state}_{\text{adv}}) \leftarrow\!\!\$\ \mathcal{A}(\text{“find”}, 1^\lambda, \zeta)$

if $\{x_0, x_1\} \not\subseteq \mathcal{X}_{\lambda,\zeta}$: $\big(b' \leftarrow\!\!\$\ \{0,1\}$; **return** $b'\big)$

$\mathsf{state}_{\text{clt}} \leftarrow \textsc{Nil}$; $\mathsf{clientStatus} \leftarrow \textsc{Nil}$; $\mathsf{serverStatus} \leftarrow \textsc{Nil}$; $\mathsf{msg} \leftarrow \textsc{Nil}$

$a \leftarrow 0$ // active party: 0 = (honest) client, 1 = corrupted server/adversary

while $\mathsf{clientStatus} \notin \{\bot, \top\} \wedge \mathsf{serverStatus} \neq \bot$

  if $a \stackrel{?}{=} 0$: $(\mathsf{state}_{\text{clt}}, \mathsf{clientStatus}, \mathsf{msg}, y) \leftarrow\!\!\$\ \mathsf{F.CEval}(\zeta, x_b, \mathsf{state}_{\text{clt}}, \mathsf{msg})$

  else $(\mathsf{state}_{\text{adv}}, \mathsf{serverStatus}, \mathsf{msg}) \leftarrow\!\!\$\ \mathcal{A}(\text{“eval”}, \mathsf{state}_{\text{adv}}, \mathsf{msg})$

  $a \leftarrow 1 - a$

$b' \leftarrow\!\!\$\ \mathcal{A}(\text{“guess”}, \mathsf{state}_{\text{adv}})$; **return** b'

Fig. 58. Security experiment for obliviousness (security against malicious servers)

G Comprehensive Presentation of the Impossibility Result

We first define our constructive oracles implementing an OPRF, and afterwards specify our break oracles:

Construction 12 (OPRF Oracle). *We define our OPRF oracle as* $\mathsf{OPRF} := (f_1, f_2, R, F)$ *wherein* f_1, f_2, R *and* F *are defined as follows:*

- $F\colon \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda$ *is a random function.*
- $f_1\colon \{0,1\}^\lambda \times \{0,1\}^\lambda \hookrightarrow \{0,1\}^{6\lambda}$ *is a length-tripling injective random function.*
- $f_2\colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \to \{0,1\}^{24\lambda}$ *is defined as*

$$f_2(k, r_S, m_1) := \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_c) = m_1 \\ \bot & \text{otherwise} \end{cases}$$

  where $f_2'(\cdot,\cdot,\cdot)$ *is a length-tripling injective random function with the same domain and codomain as* f_2. f_2' *is not exposed via the OPRF oracle directly.*
- $R\colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{24\lambda} \to \{0,1\}^\lambda$ *is defined as*

$$R(x, r_C, m_2) := \begin{cases} F(k, x) & \text{if } \exists k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}$$

Note that R *is well-defined due to the injectivity of* f_1 *and* f_2'.

Construction 13 (OPRF Inversion Oracle). *For every OPRF oracle* $\mathsf{OPRF} = (f_1, f_2, R, F)$ *as in Cons. 12, we define an associated inversion oracle* $\mathsf{Invert} := (f_1^{-1}, f_2^{-1})$ *where* f_1^{-1} *inverts* f_1 *and* f_2^{-1} *inverts* f_2 *on all inputs except* $\bot$.

Note that f_1^{-1} and f_2^{-1} are well-defined due to the injectivity of f_1 and f_2 on all values not evaluating to $\bot$.

In the following definition we use the extension of computational indistinguishability to oracle algorithms, saying that two random variables or experiments are computationally indistinguishable relative to oracles $\mathsf{O}_1, \mathsf{O}_2, \dots$) if no probabilistic polynomial-time oracle algorithm $D^{\mathsf{O}_1, \mathsf{O}_2, \cdots}$ can tell the two settings apart, except with negligible probability. Note that the oracles O_i may not be realizable in polynomial time, but each oracle call costs D only constant time (plus the time to write the query and read the answer). Since D is polynomially bounded, it can only make a polynomial number of oracle calls.

Lemma 2. *Let* $\mathsf{OPRF} = (f_1, f_2, R, F)$ *be the OPRF oracle from Cons. 12. Then there exists an efficient simulator* $\mathsf{Sim}^F = (\mathsf{Sim}^F.\mathsf{Init}, \mathsf{Sim}^F.\hat{f}_1, \mathsf{Sim}^F.\hat{f}_2, \mathsf{Sim}^F.\hat{R})$ *with oracle access to* F *that can simulate* f_1, f_2 *and* R *such that* $\mathsf{OPRF}' = (\mathsf{Sim}.\hat{f}_1, \mathsf{Sim}.\hat{f}_2, \mathsf{Sim}.\hat{R}, F)$ *is computationally indistinguishable from* OPRF *relative to a* PSPACE*-complete oracle and a random oracle.*

We remark that we merely denote the interfaces that the simulator provides as $(\mathsf{Sim}^F.\mathsf{Init}, \mathsf{Sim}^F.\hat{f}_1, \mathsf{Sim}^F.\hat{f}_2, \mathsf{Sim}^F.\hat{R})$, but that the simulator internally keeps a joint state.

$\underline{\mathsf{Sim}^F.\mathsf{Init}(1^\lambda)}$
$Z_1 \leftarrow \{\}$
$Z_2 \leftarrow \{\}$

$\underline{\mathsf{Sim}^F.\hat{f}_1(x, r_C)}$
if $\neg((x, r_C)$ **in** $Z_1)$:
  $Z_1[x, r_C] \leftarrow\!\!\$\ \{0,1\}^{6\lambda}$
return $Z_1[x, r_C]$

$\underline{\mathsf{Sim}^F.\hat{f}_2(k, r_S, m_1)}$
$\mathsf{found} \leftarrow$ **false**
foreach (x, r_C) **in** Z_1:
  if $Z_1[x, r_C] = m_1$
    if found:
      return $\bot$
    found $\leftarrow$ **true**
if $\neg$found: **return** $\bot$
if $\neg((k, r_S, m_1)$ **in** $Z_2)$:
  $r \leftarrow\!\!\$\ \{0,1\}^{24\lambda}$
  $Z_2[k, r_S, m_1] \leftarrow r$
return $Z_2[k, r_S, m_1]$

$\underline{\mathsf{Sim}^F.\hat{R}(x, r_C, m_2)}$
if $\neg((x, r_C)$ **in** $Z_1)$:
  $Z_1[x, r_C] \leftarrow\!\!\$\ \{0,1\}^{6\lambda}$
$m_1 \leftarrow Z_1[x, r_C]$
$k \leftarrow \textsc{Nil}$
foreach (k', r'_S, m'_1) **in** Z_2:
  if $Z_2[k', r'_S, m'_1] = m_2$
    $\wedge\, m'_1 = m_1$:
    if $k \neq \textsc{Nil}$: **return** $\bot$
    $k \leftarrow k'$
if $k = \textsc{Nil}$: **return** $\bot$
return $F(k, x)$

Fig. 59. Simulator for the OPRF oracle $\mathsf{OPRF} = (f_1, f_2, R, F)$

Proof. Note that we can ignore the random oracle as it is not used in the other oracles and, hence, any distinguisher can simulate it locally using lazy sampling. Let $\mathsf{OPRF} = (f_1, f_2, R, F)$ be the OPRF oracle from Cons. 12. We show that given oracle access to F, the simulator Sim^F as defined in Fig. 59 can simulate f_1, f_2 and R relative to a PSPACE-complete oracle such that

$$(f_1, f_2, R, F) \overset{c}{\approx} (\mathsf{Sim}^F.\hat{f}_1, \mathsf{Sim}^F.\hat{f}_2, \mathsf{Sim}^F.\hat{R}, F)\,.$$

It should be noted that Sim^F is efficient since all of its subroutines can be implemented as a PPT Turing machine. We show the indistinguishability via a sequence of games.

Game $\mathbf{G}_0$ (Fig. 60): The original game (genuine oracles). In the initial game, the adversary gets access to the genuine oracles as defined in Cons. 12.

Game $\mathbf{G}_1$ (Fig. 61): Requiring the existence of *exactly* one pair (x, r_C) resp. (k, r_S) in f_2 and R. We change the oracles f_2 and R in the initial game such that they output $\bot$ if there exists more than one (x, r_C) such that $f_1(x, r_C) = m_1$ resp. more than one (k, r_S) such that $f_2(k, r_S, f_1(x, r_C)) = m_2$:

$$f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f'_2(k, r_S, m_1) & \text{if } \exists\,!\, x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$$
$$R \leftarrow \left((x, r_C, m_2) \mapsto \begin{cases} F(k, x) & \text{if } \exists\,!\, k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}\right)$$

Since f_1 and f_2 are injective on all inputs not evaluating to $\bot$, this game is clearly equivalent to the previous one.

$$\Pr[\mathbf{G}_0(\lambda)] = \Pr[\mathbf{G}_1(\lambda)]$$

The reason for this game hop is to ensure that when we change f_1 and f_2 in the next games (and drop injectivity), all oracles remain well-defined.

Game $\mathbf{G}_2$ (Fig. 62): Dropping the injectivity requirement for f_1. We now sample f_1 from the set of all (no longer necessarily injective) functions $\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^{6\lambda}$. As f_1 is a length-tripling function, the game still picks an injective function with overwhelming probability. To see this, note that the total number of functions $f_1 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \hookrightarrow \{0,1\}^{6\lambda}$ equals $2^{6\lambda \cdot 2^{2\lambda}}$. Among those, we are interested in the injective functions which one can imagine to be constructed as follows: For the first input value, say, in lexicographic order, we still have $2^{6\lambda}$ options for assigning an image. For the second input value, this leaves us with $2^{6\lambda} - 1$ choices to make the function injective. Continuing to the last of the $2^{2\lambda}$ inputs, there are still $2^{6\lambda} - 2^{2\lambda} + 1$ possible images. Overall we can therefore derive $\frac{2^{6\lambda}!}{(2^{6\lambda} - 2^{2\lambda})!}$ injective functions. The probability that a random function is injective is therefore $\frac{2^{6\lambda}!}{(2^{6\lambda} - 2^{2\lambda})! \cdot 2^{6\lambda \cdot 2^{2\lambda}}}$. This yields

$$|\Pr[\mathbf{G}_1(\lambda)] - \Pr[\mathbf{G}_2(\lambda)]| \leq \Pr[f_1 \text{ is not injective}] = 1 - \frac{2^{6\lambda}!}{(2^{6\lambda} - 2^{2\lambda})! \cdot 2^{6\lambda \cdot 2^{2\lambda}}}\,.$$

Using the fact that

$$\frac{2^{6\lambda}!}{(2^{6\lambda}-2^{2\lambda})!\cdot 2^{6\lambda\cdot 2^{2\lambda}}} \geq \frac{(2^{6\lambda}-2^{2\lambda})^{2^{2\lambda}}}{2^{6\lambda\cdot 2^{2\lambda}}} \geq (1-2^{-4\lambda})^{2^{2\lambda}} \geq 1-2^{2\lambda}\cdot 2^{-4\lambda} = 1-2^{-2\lambda},$$

by applying Bernoulli's inequality $(1+x)^r \geq 1+rx$ for $x=-2^{-4\lambda} \geq -1$ and integer $r=2^{2\lambda} \geq 1$, we conclude that the difference between the games is negligible.

Game $\mathbf{G}_3$ (Fig. 63): Dropping the injectivity requirement for f_2'. We apply the same argument as in the previous game hop to f_2', dropping the requirement of its injectivity. That is, we now sample f_2' from the set of all (no longer necessarily injective) functions $\{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \to \{0,1\}^{24\lambda}$. As f_2' is a length-tripling function, the game still picks an injective function with overwhelming probability. We have

$$\begin{aligned}|\Pr[\mathbf{G}_2(\lambda)] - \Pr[\mathbf{G}_3(\lambda)]| &\leq \Pr[f_2' \text{ is not injective}] = 1 - \frac{2^{24\lambda}!}{(2^{24\lambda}-2^{8\lambda})!\cdot 2^{24\lambda\cdot 2^{8\lambda}}} \\ &\leq \mathsf{negl}(\lambda)\,.\end{aligned}$$

Game $\mathbf{G}_4$ (Fig. 64): Recording oracle queries to f_1 and f_2. Next, we change the game such that all explicit and implicit queries to f_1 and f_2 by the adversary are recorded to tables Z_1 and Z_2. Queries to R trigger an implicit query to f_1 since R "uses" the value $f_1(c, r_C)$ internally. Queries to f_2 are only recorded if they are valid, i.e., they result in an answer $m_2 \neq \bot$. To implement the recording of the oracle queries, we replace the adversary's direct access to f_1, f_2 and R with access to proxy oracles $\hat{f}_1$, $\hat{f}_2$ and $\hat{R}$ that forward the queries to their original counterparts and also record them. Since the recorded queries are not yet used and the recording does not cause an observable effect for the adversary, we have

$$\Pr[\mathbf{G}_3(\lambda)] = \Pr[\mathbf{G}_4(\lambda)]\,.$$

Game $\mathbf{G}_5$ (Fig. 65): Requiring a previous query to f_2 for $\hat{R}$. In this game, we *effectively* make the following change to R:

$$R \leftarrow \left((x, r_C, m_2) \mapsto \begin{cases} F(k,x) & \text{if } \exists! k, r_S \in Z_2.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}\right)$$

That is, from now on, we only consider a pair (k, r_S) inside of R if it has appeared in a previous query to $\hat{f}_2$. As a result, if there are values k and r_S such that $f_2(k, r_S, f_1(x, r_C)) = m_2$, but they have not appeared in a previous query to $\hat{f}_2$, then R returns $\bot$. The adversary can only detect this change if it queries $\hat{R}$ on values (x, r_C, m_2) such that there exists exactly one pair (k, r_S) in $\{0,1\}^\lambda \times \{0,1\}^\lambda$ such that

$$f_2(k, r_S, f_1(x, r_C)) = m_2 \text{ and } \langle (k, r_S, f_1(x, rC)) \rangle \notin Z_2\,. \tag{1}$$

We can over-approximate this probability of this event with the event that the adversary comes up with a query (x, r_C, m_2) to $\hat{R}$ such that there exists any (potentially more than one) (k, r_S) such that Eq. (1) holds. This corresponds to the adversary's guessing a function value of the length-tripling random function f_2. Consequently, if we let q denote an upper bound of the total number of oracle queries the adversary performs to all oracles together, we obtain:

$$|\Pr[\mathbf{G}_4(\lambda)] - \Pr[\mathbf{G}_5(\lambda)]| \leq \frac{q\cdot 2^{8\lambda}}{2^{24\lambda}} = \frac{q}{2^{16\lambda}}$$

In Fig. 65, the $\hat{R}$ oracle implements the algorithmic equivalent of the above change.

Game $\mathbf{G}_6$ (Fig. 66): Requiring a previous query to $\hat{f}_1$ for $\hat{f}_2$. We now apply the same argument as in the previous game hop to the function f_1 inside of the f_2 oracle, so we effectively make the following change to f_2:

$$f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists! x, r_C \in Z_1.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$$

Similarly to the previous game, from now on, f_2 returns $\bot$ if (x, r_C) has not appeared in a previous query to f_1. Also, like in the last game, $\mathcal{A}$ can only detect a difference between the previous game and the present one if it successfully predicts a function value of f_1. This yields

$$|\Pr[\mathbf{G}_5(\lambda)] - \Pr[\mathbf{G}_6(\lambda)]| \leq \frac{q\cdot 2^{\lambda}}{2^{3\lambda}} = \frac{q}{2^{2\lambda}}$$

wherein we again let q denote an upper bound of the total number of oracle queries by the adversary.

Game $\mathbf{G}_7$ (Fig. 67): Lazily sampling f_1. We now implement f_1 via lazy sampling instead of sampling the entire function at the beginning of the game. Since the game no longer deals with function values prior to their evaluation (like e.g. the existential quantifier in the oracle f_2 in $\mathbf{G}_0$ did), the view of the adversary in this game and its view in the previous game are identically distributed. This implies that we have

$$\Pr[\mathbf{G}_6(\lambda)] = \Pr[\mathbf{G}_6(\lambda)]\,.$$

Game $\mathbf{G}_8$ (Fig. 68): Lazily sampling f_2'. Similarly to the previous game hop, we now implement f_2' via lazy sampling. With the same argument as above, we obtain

$$\Pr[\mathbf{G}_7(\lambda)] = \Pr[\mathbf{G}_8(\lambda)]\,.$$

Game $\mathbf{G}_9$ (Fig. 69): Enclosing the simulator. Finally, we encapsulate the initialization logic for the recording of the queries and reinterpret the proxy oracles $\hat{f}_1$, $\hat{f}_2$ and $\hat{R}$ as objects provided by the simulator. This leads us to the final game, where the adversary interacts with the simulator Sim^F from Fig. 59. This concludes the proof.

$\mathbf{G}_0[\mathrm{PS}](\lambda)$

$F \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$

$f_1 \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \hookrightarrow \{0,1\}^{6\lambda}\right)$ // injective random function

$f_2' \leftarrow\!\!\$ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \hookrightarrow \{0,1\}^{24\lambda}\right)$ // injective random function

$f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$

$R \leftarrow \left((x, r_C, m_2) \mapsto \begin{cases} F(k, x) & \text{if } \exists k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}\right)$

$b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}, f_1, f_2, R, F}(1^\lambda)$

return b'

Fig. 60. Game $\mathbf{G}_0$: The original game, where the adversary gets access to the genuine oracles from Cons. 12.

$\mathbf{G}_1[\mathrm{PS}](\lambda)$

$F \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$

$f_1 \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \hookrightarrow \{0,1\}^{6\lambda}\right)$ // injective random function

$f_2' \leftarrow\!\!\$ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \hookrightarrow \{0,1\}^{24\lambda}\right)$ // injective random function

$f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists\,!\, x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$

$R \leftarrow \left((x, r_C, m_2) \mapsto \begin{cases} F(k, x) & \text{if } \exists\,!\, k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}\right)$

$b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}, f_1, f_2, R, F}(1^\lambda)$

return b'

Fig. 61. Game $\mathbf{G}_1$: Requiring the existence of exactly one pair $(x, r_C) \in \{0,1\}^\lambda \times \{0,1\}^\lambda$ resp. $(k, r_S) \in \{0,1\}^\lambda \times \{0,1\}^\lambda$ in the definitions of f_2 resp. R.

$\mathbf{G}_2[\mathrm{PS}](\lambda)$

101 : $F \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$

102 : $f_1 \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^{6\lambda}\right)$ // not necessarily injective

103 : $f_2' \leftarrow\!\!\$ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \hookrightarrow \{0,1\}^{24\lambda}\right)$ // injective random function

104 : $f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists! x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$

105 : $R \leftarrow \left((x, r_C, m_2) \mapsto \begin{cases} F(k, x) & \text{if } \exists! k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}\right)$

106 : $b' \leftarrow\!\!\$ \mathcal{A}^{\mathrm{PS}, f_1, f_2, R, F}(1^\lambda)$

107 : **return** b'

Fig. 62. Game $\mathbf{G}_2$: Sampling f_1 from the set of not necessarily injective random functions.

$\mathbf{G}_3[\mathrm{PS}](\lambda)$

111 : $F \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$

112 : $f_1 \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^{6\lambda}\right)$ // not necessarily injective

113 : $f_2' \leftarrow\!\!\$ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \to \{0,1\}^{24\lambda}\right)$ // not necessarily injective

114 : $f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists! x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$

115 : $R \leftarrow \left((x, r_C, m_2) \mapsto \begin{cases} F(k, x) & \text{if } \exists! k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}\right)$

116 : $b' \leftarrow\!\!\$ \mathcal{A}^{\mathrm{PS}, f_1, f_2, R, F}(1^\lambda)$

117 : **return** b'

Fig. 63. Game $\mathbf{G}_3$: Sampling f_2' from the set of not necessarily injective random functions.

$\mathbf{G}_4[\mathrm{PS}](\lambda)$

121 : $Z_1 \leftarrow \{\};\ Z_2 \leftarrow \{\}$

122 : $F \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$

123 : $f_1 \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^{6\lambda}\right)$ // not necessarily injective

124 : $f_2' \leftarrow\!\!\$ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \to \{0,1\}^{24\lambda}\right)$ // not necessarily injective

125 : $f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists! x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$

126 : $R \leftarrow \left((x, r_C, m_2) \mapsto \begin{cases} F(k, x) & \text{if } \exists! k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2 \\ \bot & \text{otherwise} \end{cases}\right)$

127 : $b' \leftarrow\!\!\$ \mathcal{A}^{\mathrm{PS}, \hat{f}_1, \hat{f}_2, \hat{R}, F}(1^\lambda)$

128 : **return** b'

Oracle $\hat{f}_1(x, r_C)$

$m_1 \leftarrow f_1(x, r_C)$

$Z_1[x, r_C] \leftarrow m_1$

return m_1

Oracle $\hat{f}_2(k, r_S, m_1)$

$m_2 \leftarrow f_2(k, r_S, m_1)$

if $m_2 \neq \bot$:

  $Z_2[k, r_S, m_1] \leftarrow m_2$

return m_2

Oracle $\hat{R}(x, r_C, m_2)$

$Z_1[x, r_C] \leftarrow f_1(x, r_C)$

return $R(x, r_C, m_2)$

Fig. 64. Game $\mathbf{G}_4$: Recording oracle queries to f_1 and f_2.

$\mathbf{G}_5[\mathrm{PS}](\lambda)$

131 : $Z_1 \leftarrow \{\};\ Z_2 \leftarrow \{\}$
132 : $F \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$
133 : $f_1 \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^{6\lambda}\right)$ // not necessarily injective
134 : $f_2' \leftarrow\!\!\$ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \to \{0,1\}^{24\lambda}\right)$ // not necessarily injective
135 : $f_2 \leftarrow \left((k, r_S, m_1) \mapsto \begin{cases} f_2'(k, r_S, m_1) & \text{if } \exists! x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1 \\ \bot & \text{otherwise} \end{cases}\right)$
136 : $b' \leftarrow\!\!\$ \mathcal{A}^{\mathrm{PS}, \hat{f}_1, \hat{f}_2, \hat{R}, F}(1^\lambda)$
137 : **return** b'

Oracle $\hat{f}_1(x, r_C)$
$m_1 \leftarrow f_1(x, r_C)$
$Z_1[x, r_C] \leftarrow m_1$
return m_1

Oracle $\hat{f}_2(k, r_S, m_1)$
$m_2 \leftarrow f_2(k, r_S, m_1)$
if $m_2 \neq \bot$:
  $Z_2[k, r_S, m_1] \leftarrow m_2$
return m_2

Oracle $\hat{R}(x, r_C, m_2)$
$Z_1[x, r_C] \leftarrow f_1(x, r_C)$
$m_1 \leftarrow Z_1[x, r_C]$
$k \leftarrow \textsc{Nil}$
foreach (k', r_S', m_1') **in** Z_2 :
  if $Z_2[k', r_S', m_1'] = m_2$
    $\wedge m_1' = m_1$:
    if $k \neq \textsc{Nil}$: **return** $\bot$
    $k \leftarrow k'$
if $k = \textsc{Nil}$: **return** $\bot$
return $F(k, x)$

Fig. 65. Game $\mathbf{G}_5$: Strengthening the condition $\exists! k, r_S \in \{0,1\}^\lambda.\ f_2(k, r_S, f_1(x, r_C)) = m_2$ for the oracle R such that k and r_S must be part of a previous query to f_2.

$\mathbf{G}_6[\mathrm{PS}](\lambda)$

141 : $Z_1 \leftarrow \{\};\ Z_2 \leftarrow \{\}$
142 : $F \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$
143 : $f_1 \leftarrow\!\!\$ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^{6\lambda}\right)$ // not necessarily injective
144 : $f_2' \leftarrow\!\!\$ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \to \{0,1\}^{24\lambda}\right)$ // not necessarily injective
145 : $b' \leftarrow\!\!\$ \mathcal{A}^{\mathrm{PS}, \hat{f}_1, \hat{f}_2, \hat{R}, F}(1^\lambda)$
146 : **return** b'

Oracle $\hat{f}_1(x, r_C)$
$m_1 \leftarrow f_1(x, r_C)$
$Z_1[x, r_C] \leftarrow m_1$
return m_1

Oracle $\hat{f}_2(k, r_S, m_1)$
found $\leftarrow$ **false**
foreach (x, r_C) **in** Z_1 :
  if $Z_1[x, r_C] = m_1$:
    if found :
      return $\bot$
    found $\leftarrow$ **true**
if $\neg$found : **return** $\bot$
$m_2 \leftarrow f_2'(k, r_S, m_1)$
$Z_2[k, r_S, m_1] \leftarrow m_2$
return m_2

Oracle $\hat{R}(x, r_C, m_2)$
$Z_1[x, r_C] \leftarrow f_1(x, r_C)$
$m_1 \leftarrow Z_1[x, r_C]$
$k \leftarrow \textsc{Nil}$
foreach (k', r_S', m_1') **in** Z_2 :
  if $Z_2[k', r_S', m_1'] = m_2$
    $\wedge\, m_1' = m_1$:
    if $k \neq \textsc{Nil}$: **return** $\bot$
    $k \leftarrow k'$
if $k = \textsc{Nil}$: **return** $\bot$
return $F(k, x)$

Fig. 66. Game $\mathbf{G}_6$: Strengthening the condition $\exists! x, r_C \in \{0,1\}^\lambda.\ f_1(x, r_C) = m_1$ for the oracle f_2 such that x and r_C must be part of a previous query to f_1.

$\mathbf{G}_7[\mathrm{PS}](\lambda)$

151 : $Z_1 \leftarrow \{\};\ Z_2 \leftarrow \{\}$
152 : $F \leftarrow\!\!\$\ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$
153 : $f_2' \leftarrow\!\!\$\ \left(f_2 \colon \{0,1\}^\lambda \times \{0,1\}^\lambda \times \{0,1\}^{6\lambda} \to \{0,1\}^{24\lambda}\right)$ // not necessarily injective
154 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS},\hat{f}_1,\hat{f}_2,\hat{R},F}(1^\lambda)$
155 : **return** b'

Oracle $\hat{f}_1(x, r_C)$

if $\neg((x, r_C) \in Z_1)$:
  $Z_1[x, r_C] \leftarrow\!\!\$\ \{0,1\}^{6\lambda}$
return $Z_1[x, r_C]$

Oracle $\hat{f}_2(k, r_S, m_1)$

$\mathsf{found} \leftarrow$ **false**
foreach (x, r_C) **in** Z_1:
  if $Z_1[x, r_C] = m_1$:
    if found:
      return $\bot$
    $\mathsf{found} \leftarrow$ **true**
if $\neg\mathsf{found}$: **return** $\bot$
$m_2 \leftarrow f_2'(k, r_S, m_1)$
$Z_2[k, r_S, m_1] \leftarrow m_2$
return m_2

Oracle $\hat{R}(x, r_C, m_2)$

if $\neg((x, r_C) \in Z_1)$:
  $Z_1[x, r_C] \leftarrow\!\!\$\ \{0,1\}^{6\lambda}$
$m_1 \leftarrow Z_1[x, r_C]$
$k \leftarrow \textsc{Nil}$
foreach (k', r_S', m_1') **in** Z_2:
  if $Z_2[k', r_S', m_1'] = m_2$
    $\wedge\, m_1' = m_1$:
    if $k \neq \textsc{Nil}$: **return** $\bot$
    $k \leftarrow k'$
if $k = \textsc{Nil}$: **return** $\bot$
return $F(k, x)$

Fig. 67. Game $\mathbf{G}_7$: Lazily sampling f_1.

$\mathbf{G}_8[\mathrm{PS}](\lambda)$

161 : $Z_1 \leftarrow \{\};\ Z_2 \leftarrow \{\}$
162 : $F \leftarrow\!\!\$\ \left(\{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^\lambda\right)$
163 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS},\hat{f}_1,\hat{f}_2,\hat{R},F}(1^\lambda)$
164 : **return** b'

Oracle $\hat{f}_1(x, r_C)$

if $\neg((x, r_C) \in Z_1)$:
  $Z_1[x, r_C] \leftarrow\!\!\$\ \{0,1\}^{6\lambda}$
return $Z_1[x, r_C]$

Oracle $\hat{f}_2(k, r_S, m_1)$

$\mathsf{found} \leftarrow$ **false**
foreach (x, r_C) **in** Z_1:
  if $Z_1[x, r_C] = m_1$:
    if found:
      return $\bot$
    $\mathsf{found} \leftarrow$ **true**
if $\neg\mathsf{found}$: **return** $\bot$
if $\neg((k, r_S, m_1)$ **in** $Z_2)$:
  $r \leftarrow\!\!\$\ \{0,1\}^{24\lambda}$
  $Z_2[k, r_S, m_1] \leftarrow r$
return $Z_2[k, r_S, m_1]$

Oracle $\hat{R}(x, r_C, m_2)$

if $\neg((x, r_C) \in Z_1)$:
  $Z_1[x, r_C] \leftarrow\!\!\$\ \{0,1\}^{6\lambda}$
$m_1 \leftarrow Z_1[x, r_C]$
$k \leftarrow \textsc{Nil}$
foreach (k', r_S', m_1') **in** Z_2:
  if $Z_2[k', r_S', m_1'] = m_2$
    $\wedge\, m_1' = m_1$:
    if $k \neq \textsc{Nil}$: **return** $\bot$
    $k \leftarrow k'$
if $k = \textsc{Nil}$: **return** $\bot$
return $F(k, x)$

Fig. 68. Game $\mathbf{G}_8$: Lazily sampling f_2'.

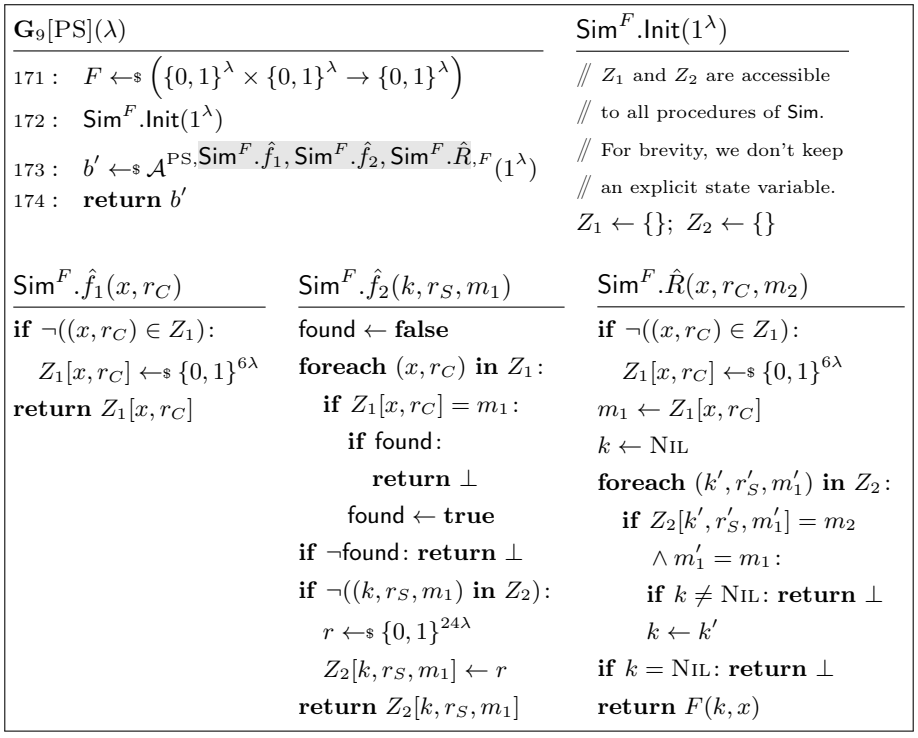


Fig. 69. Game $\mathbf{G}_9$: Enclosing the simulator. This is the final game.

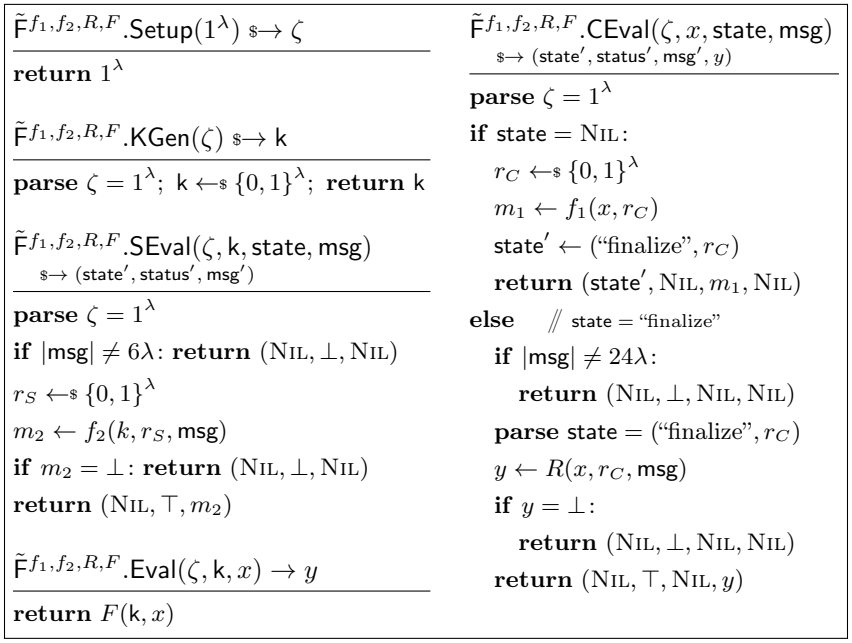


Fig. 70. Secure OPRF $\tilde{\mathsf{F}}$ relative to the OPRF oracle

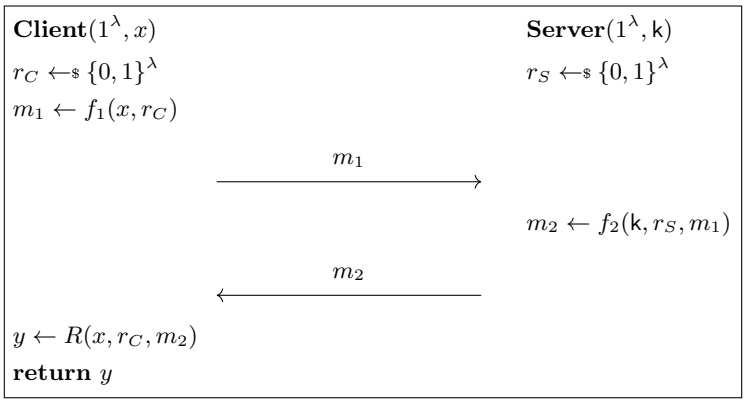


Fig. 71. Secure OPRF $\tilde{\mathsf{F}}$ relative to the OPRF oracle (graphical depiction)

Lemma 3. *Relative to the oracle* OPRF *from Cons. 12, a random oracle and a* PSPACE*-complete oracle, the construction* $\tilde{\mathsf{F}}$ *in Fig. 70 is a secure OPRF according to the game-based security Def. 10 and 11.*

Proof. Let $\mathsf{OPRF} := (f_1, f_2, R, F)$ be the OPRF oracle as defined in Cons. 12. Let RO be a random oracle and PS be a PSPACE-complete oracle. Let $\tilde{\mathsf{F}}$ be the construction in Fig. 70, instantiated with OPRF. We need to show that $\tilde{\mathsf{F}}$ provides pseudorandomness and obliviousness relative to PS and RO. Note that w.l.o.g. we can ignore RO since it is not used in $\tilde{\mathsf{F}}$, so an adversary can simply simulate the random oracle RO locally by lazy sampling. For this reason, we drop RO in the remainder of this proof.

We first show that $\tilde{\mathsf{F}}$ provides pseudorandomness (Def. 10) using a sequence of game hops. Let Sim be the simulator as defined in Fig. 77 (ignoring all parts of the figure not related to Sim).

Game $\mathbf{G}_0$ (Fig. 72): The original game (real world). The initial game is equivalent to the pseudorandomness experiment $\mathbf{Exp}^{\text{oprf-psr}}_{\tilde{\mathsf{F}},\mathcal{A},\mathsf{Sim},b}(\lambda)$ for $\tilde{\mathsf{F}}$ with $b = 0$ ("real world").

$$\Pr[\mathbf{G}_0(\lambda)] = \Pr[\mathbf{Exp}^{\text{oprf-psr}}_{\tilde{\mathsf{F}},\mathcal{A},\mathsf{Sim},0}(\lambda)]$$

Game $\mathbf{G}_1$ (Fig. 73): Expanding $\tilde{\mathsf{F}}$. Next, we expand $\tilde{\mathsf{F}}$.Setup, $\tilde{\mathsf{F}}$.KGen, $\tilde{\mathsf{F}}$.SEval and $\tilde{\mathsf{F}}$.Eval, i.e., we make the implementation of these algorithms explicit. Since this just adds more details to the presentation, but does not change the actual game, we have

$$\Pr[\mathbf{G}_0(\lambda)] = \Pr[\mathbf{G}_1(\lambda)]\,.$$

Game $\mathbf{G}_2$ (Fig. 74): Recording valid oracle queries to f_2. We now change the game such that all *valid* oracle queries to f_2 are recorded. A query to f_2 is regarded as valid if its response is not $\perp$. This recording mechanism will later become useful. For now, however, as the recorded values are not used, this is a purely syntactical change. Furthermore, we route the adversary's queries to R through a new oracle R', which just forwards all queries to R. This is a purely syntactical change, too, so we have:

$$\Pr[\mathbf{G}_1(\lambda)] = \Pr[\mathbf{G}_2(\lambda)]\,.$$

Game $\mathbf{G}_3$ (Fig. 75): Simulating R. In this game, we change R' such that it no longer forwards all of its queries to R. Firstly, we add a cache T_R, ensuring that repeated queries to R' are answered in the same way as previous queries for the same value. Secondly, if R' receives a query (x, r_C, m_2) such that

$$\exists (k', r'_S, m'_1) \in T_2.\ k = k' \wedge f_1(x, r_C) = m'_1 \wedge f_2(k', r'_S, m'_1) = m_2\,, \tag{2}$$

where k is the key chosen by the game and LimEv implements the limited number of local evaluations, it now answers this query by querying LimEv on x and returning its response. LimEv forwards every query x to the Ev oracle unless the ticket counter tx is depleted, and Ev in turn returns $F(k, x)$, which is the same value R would return when Eq. (2) is satisfied (see Cons. 12). Thus, and due to the cache, there is no difference between the previous game and $\mathbf{G}_3(\lambda)$ from the adversary's point of view unless it makes a fresh[14] query to R' satisfying Eq. (2) when $\mathsf{tx} = 0$, i.e. LimEv is called when $\mathsf{tx} = 0$. Let us call this event "depleted". We bound the probability of "depleted". For this, we need a few claims.

Claim 1. At every point in time during a run of $\mathbf{G}_3$ after T_2 is initialized, we have that for all $(k', r'_S, m'_1) \in T_2$ it holds that $f_2(k', r'_S, m'_1) \neq \perp$.

Proof. The only two instructions which add elements to T_2 are the ones in the Srv oracle and in the f_2 oracle. Both of these instructions are guarded by conditions ensuring that $f_2(k', r'_S, m'_1) \neq \perp$ holds for the newly added element (k', r'_S, m'_1).

Claim 2. For every query (x, r_C, m_2) to R' satisfying Eq. (2), we have $m_2 \neq \perp$.

Proof. Assume for the sake of contradiction that there is a query (x, r_C, m_2) with $m_2 = \perp$ to R' that satisfies Eq. (2). Because (x, r_C, m_2) satisfies Eq. (2), there exists a triple (k', r'_S, m'_1) such that $f_2(k', r'_S, m'_1) = m_2$ and $(k', r'_S, m'_1) \in T_2$. Together with claim 1, it follows that

$$\exists (k', r'_S, m'_1) \in T_2.\ f_2(k', r'_S, m'_1) = m_2 \wedge m_2 = \perp \wedge f_2(k', r'_S, m'_1) \neq \perp\,,$$

which yields a contradiction.

[14] A fresh query is a query that is different from all previous queries.

Claim 3. Every fresh query (x, r_C, m_2) to R' which satisfies Eq. (2) comprises a value m_2 that is different from the m_2 values of all previous queries to R'.

Proof. For every query (x, r_C, m_2) satisfying Eq. (2) with $m_2 \neq \bot$, m_2 uniquely determines x and r_C due to the injectivity of f_1 and f_2 on all inputs not evaluating to $\bot$. Because of claim 2, we can drop the requirement that $m_2 \neq \bot$, implying the claim to prove.

Claim 4. At every point in time during a run of $\mathbf{G}_3$ after T_2 is initialized, the following claim holds: For every triple $(k', r'_S, m'_1) \in T_2$, there exists a unique m_2 such that $f_2(k', r'_S, m'_1) = m_2$. Moreover, for every $(k', r'_S, m'_1) \in T_2$, this value m_2 is different.

Proof. This follows immediately from the definition of f_2, claim 1 and the injectivity of f_2 on all values not evaluating to $\bot$.

Claim 5. When "depleted" occurs, then there are more unique queries to R' satisfying Eq. (2) than non-rejected[15] queries to Srv.

Proof. This directly follows from the definition of $\mathbf{G}_3$: The ticket counter tx is (only) incremented when Srv is called on valid inputs, and (only) decremented when LimEv is called, which only happens when there is a fresh query to R' satisfying Eq. (2).

Next, we observe that the set T_2 never shrinks, i.e., after being added to T_2, no element is ever removed from it. Furthermore, we observe that the only two instructions which append to T_2 are the ones in the Srv oracle and in the f'_2 oracle. Each of these adds exactly one element to T_2 (if it is not already on the list). As a result, together with claims 3 to 5, it follows that when "depleted" occurs, there must be at least one element $t^* := (k', r'_S, m'_1)$ in T_2 with $k' = k$ which was not added by Srv. Consequently, this element must have been added via f'_2, which can only be called by the adversary. We can conclude that the adversary must have called f'_2 with the key k. To upper-bound the probability of "depleted", it is, thus, sufficient to upper-bound the probability that the adversary queries f'_2 with k.

Let q be the total number of oracle queries that $\mathcal{A}$ performs. Since none of the oracles reveals k to the adversary, the probability that $\mathcal{A}$ queries f'_2 on k is upper-bounded by $\frac{q}{2^\lambda}$.

Hence, we have:

$$|\Pr[\mathbf{G}_2(\lambda)] - \Pr[\mathbf{G}_3(\lambda)]| \leq \Pr[\text{“depleted”}] \leq \Pr[\text{“}\mathcal{A} \text{ guesses } k\text{”}] \leq \frac{q}{2^\lambda}$$

Game $\mathbf{G}_4$ (Fig. 76): Implementing Ev as a random function. In this game, we change the Ev oracle such that instead of evaluating $F(k, \cdot)$, it evaluates a random function $f(\cdot)$.

Since both $F(k, \cdot)$ and $f(\cdot)$ yield uniformly distributed outputs, $\mathcal{A}$ cannot distinguish between the previous game $\mathbf{G}_3$ and this game $\mathbf{G}_4$ unless it queries F on the key k. Therefore, if we let q denote the total number of oracle queries that $\mathcal{A}$ performs, we have

$$|\Pr[\mathbf{G}_3(\lambda)] - \Pr[\mathbf{G}_4(\lambda)]| \leq \Pr[\text{“}\mathcal{A} \text{ guesses } k\text{”}] \leq \frac{q}{2^\lambda}\,.$$

Game $\mathbf{G}_5$ (Fig. 77): Encapsulating the simulator. We now rewrite the game to factor out a simulator Sim. This is a purely syntactical change and, hence, does not affect the behavior of the game. Accordingly, we have

$$\Pr[\mathbf{G}_4(\lambda)] = \Pr[\mathbf{G}_5(\lambda)]\,.$$

$\mathbf{G}_5(\lambda)$ is equivalent to $\mathbf{Exp}^{\text{oprf-psr}}_{\tilde{\mathsf{F}},\mathcal{A},\mathsf{Sim},1}(\lambda)$ with simulator Sim as defined in Fig. 77 (ignoring all parts of the figure not related to Sim). This concludes the proof of pseudorandomness.

What remains to show is that $\tilde{\mathsf{F}}$ provides obliviousness (Def. 11). We do so via a sequence of games, too.

Game $\mathbf{G}_0$ (Fig. 78): The original game ("left x"). The initial game is equivalent to the obliviousness experiment $\mathbf{Exp}^{\text{oprf-obl}}_{\tilde{\mathsf{F}},\mathcal{A},b}(\lambda)$ for $\tilde{\mathsf{F}}$ with $b = 0$ ("left x").

$$\Pr[\mathbf{G}_0(\lambda)] = \Pr[\mathbf{Exp}^{\text{oprf-obl}}_{\tilde{\mathsf{F}},\mathcal{A},0}(\lambda)]$$

Game $\mathbf{G}_1$ (Fig. 79): Expanding $\tilde{\mathsf{F}}$. Next, we expand $\tilde{\mathsf{F}}.\mathsf{Setup}$ and $\tilde{\mathsf{F}}.\mathsf{CEval}$, i.e., we make the implementation of these algorithms explicit. Since this just adds more details to the presentation, but does not change the actual game, we have

$$\Pr[\mathbf{G}_0(\lambda)] = \Pr[\mathbf{G}_1(\lambda)]\,.$$

[15] Non-rejected queries are queries which are not answered with a status of $\bot$.

$\mathbf{G}_0[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

181 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
182 : $\zeta \leftarrow\!\!\$\ \tilde{\mathsf{F}}.\mathsf{Setup}(1^\lambda)$
183 : $\mathsf{state}_\mathrm{P} \leftarrow\!\!\$\ \mathrm{P}.\mathsf{Init}(1^\lambda)$
184 : $k \leftarrow\!\!\$\ \tilde{\mathsf{F}}.\mathsf{KGen}(\zeta)$
185 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot),\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$
186 : **return** b'

Oracle $\textsc{Ev}(x)$

return $\tilde{\mathsf{F}}.\mathsf{Eval}(\zeta, k, x)$

Oracle $\textsc{LimEv}(x)$

if $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
$\mathsf{tx} \leftarrow \mathsf{tx} - 1;$ **return** $\textsc{Ev}(x)$

Oracle $\textsc{Prim}(x)$

$(\mathsf{state}_\mathrm{P}, y) \leftarrow\!\!\$\ \mathrm{P}.\mathsf{Eval}(\mathsf{state}_\mathrm{P}, x);$ **return** y

Oracle $\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

$\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
// recover state (existing interaction) or increment tx counter (new interaction)
if ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
if $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
$(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow\!\!\$\ \tilde{\mathsf{F}}.\mathsf{SEval}(\zeta, k, \mathsf{state}, \mathsf{msg})$
$\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}, \mathsf{status});$ **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 72. Game $\mathbf{G}_0$: The original game, equivalent to $\mathbf{Exp}^{\text{oprf-psr}}_{\tilde{\mathsf{F}},\mathcal{A},\mathsf{Sim},0}(\lambda)$.

Game $\mathbf{G}_2$ (Fig. 80): Replacing x_0 with x_1. In this game, we replace x_0 with x_1, i.e., the adversary now receives a request for the "right x" (x_1) instead of the "left x" (x_0). Note that since f_1 is a random function oracle, $\mathcal{A}$ can only detect a difference between the previous game $\mathbf{G}_1$ and the current game $\mathbf{G}_2$ if it queries f_1 on r_C. Let q be the total number of oracle queries the adversary performs. Since r_C is uniformly sampled and not revealed to the adversary, we have

$$|\Pr[\mathbf{G}_1(\lambda)] - \Pr[\mathbf{G}_2(\lambda)]| \leq \frac{q}{2^\lambda}\,.$$

$\mathbf{G}_2(\lambda)$ is equivalent to $\mathbf{Exp}^{\text{oprf-obl}}_{\tilde{\mathsf{F}},\mathcal{A},1}(\lambda)$. This concludes the proof.

$\mathbf{G}_1[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

191 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
192 : $\zeta \leftarrow\!\!\$\ 1^\lambda$
193 : $k \leftarrow\!\!\$\ \{0,1\}^\lambda$
194 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot),\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),f_1(\cdot,\cdot),f_2(\cdot,\cdot,\cdot),R(\cdot,\cdot,\cdot),F(\cdot,\cdot)}(1^\lambda,\zeta)$
195 : **return** b'

Oracle $\textsc{Ev}(x)$

return $F(k,x)$

Oracle $\textsc{LimEv}(x)$

if $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
$\mathsf{tx} \leftarrow \mathsf{tx} - 1;$ **return** $\textsc{Ev}(x)$

Oracle $\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

$\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
// recover state (existing interaction) or increment tx counter (new interaction)
if ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
if $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
parse $\zeta = 1^\lambda$
if $|\mathsf{msg}| \neq 6\lambda$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
else
  $r_S \leftarrow\!\!\$\ \{0,1\}^\lambda$
  $m_2 \leftarrow f_2(k, r_S, \mathsf{msg})$
  if $m_2 = \bot$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
  else $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \top, m_2)$
$\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}, \mathsf{status});$ **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 73. Game $\mathbf{G}_1$: Expanding $\tilde{\mathsf{F}}.\mathsf{Setup}$, $\tilde{\mathsf{F}}.\mathsf{KGen}$, $\tilde{\mathsf{F}}.\mathsf{SEval}$ and $\tilde{\mathsf{F}}.\mathsf{Eval}$.

$\mathbf{G}_2[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

201 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
202 : $\zeta \leftarrow\!\!\$\ 1^\lambda$
203 : $k \leftarrow\!\!\$\ \{0,1\}^\lambda$
204 : $T_2 \leftarrow \emptyset$
205 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot),\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),f_1(\cdot,\cdot),f_2'(\cdot,\cdot,\cdot),R'(\cdot,\cdot,\cdot),F(\cdot,\cdot)}(1^\lambda, \zeta)$
206 : **return** b'

Oracle $f_2'(k', r_S', m_1')$

$m_2 \leftarrow f_2(k', r_S', m_1')$
if $m_2 \neq \bot$:
$T_2 \leftarrow T_2 \cup \{(k', r_S', m_1')\}$
return m_2

Oracle $\textsc{Ev}(x)$

return $F(k, x)$

Oracle $\textsc{LimEv}(x)$

if $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
$\mathsf{tx} \leftarrow \mathsf{tx} - 1$; **return** $\textsc{Ev}(x)$

Oracle $R'(x, r_C, m_2)$

return $\mathrm{R}(x, r_C, m_2)$

Oracle $\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

$\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
// recover state (existing interaction) or increment tx counter (new interaction)
if ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
if $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
parse $\zeta = 1^\lambda$
if $|\mathsf{msg}| \neq 6\lambda$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
else
  $r_S \leftarrow\!\!\$\ \{0,1\}^\lambda$
  $m_2 \leftarrow f_2(k, r_S, \mathsf{msg})$
  if $m_2 = \bot$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
  else
    $T_2 \leftarrow T_2 \cup \{(k, r_S, \mathsf{msg})\}$
    $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \top, m_2)$
$\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}, \mathsf{status})$; **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 74. Game $\mathbf{G}_2$: Recording valid oracle queries to f_2.

$\mathbf{G}_3[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

211 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
212 : $\zeta \leftarrow\!\!\$\ 1^\lambda$
213 : $k \leftarrow\!\!\$\ \{0,1\}^\lambda$
214 : $T_2 \leftarrow \emptyset;\ T_R \leftarrow \{\}$
215 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot),\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),f_1(\cdot,\cdot),f_2'(\cdot,\cdot,\cdot),R'(\cdot,\cdot,\cdot),F(\cdot,\cdot)}(1^\lambda, \zeta)$
216 : **return** b'

Oracle $f_2'(k', r_S', m_1')$

$m_2 \leftarrow f_2(k', r_S', m_1')$
if $m_2 \neq \bot$:
  $T_2 \leftarrow T_2 \cup \{(k', r_S', m_1')\}$
return m_2

Oracle $\textsc{Ev}(x)$

return $F(k, x)$

Oracle $\textsc{LimEv}(x)$

if $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
$\mathsf{tx} \leftarrow \mathsf{tx} - 1$; **return** $\textsc{Ev}(x)$

Oracle $R'(x, r_C, m_2)$

if $(x, r_C, m_2) \in T_R$
  return $T_R[x, r_C, m_2]$
elseif $\exists (k', r_S', m_1') \in T_2.\ k = k' \wedge$
  $f_1(x, r_C) = m_1' \wedge$
  $f_2(k', r_S', m_1') = m_2$:
    $T_R[x, r_C, m_2] \leftarrow \textsc{LimEv}(x)$
    return $T_R[x, r_C, m_2]$
else return $\mathrm{R}(x, r_C, m_2)$

Oracle $\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

$\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
// recover state (existing interaction) or increment tx counter (new interaction)
if ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
if $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
parse $\zeta = 1^\lambda$
if $|\mathsf{msg}| \neq 6\lambda$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
else
  $r_S \leftarrow\!\!\$\ \{0,1\}^\lambda$
  $m_2 \leftarrow f_2(k, r_S, \mathsf{msg})$
  if $m_2 = \bot$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
  else
    $T_2 \leftarrow T_2 \cup \{(k, r_S, \mathsf{msg})\}$
    $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \top, m_2)$
$\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}, \mathsf{status})$; **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 75. Game $\mathbf{G}_3$: Simulating $\mathrm{R}(\cdot,\cdot,\cdot)$ via $\textsc{LimEv}(\cdot)$ for the honest key.

$\mathbf{G}_4[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

221 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
222 : $\zeta \leftarrow\!\!\$\ 1^\lambda$
223 : $f \leftarrow\!\!\$\ (\mathcal{X}_{\lambda,\zeta} \to \mathcal{Y}_{\lambda,\zeta})$
224 : $k \leftarrow\!\!\$\ \{0,1\}^\lambda$
225 : $T_2 \leftarrow \emptyset;\ T_R \leftarrow \{\}$
226 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot),\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),f_1(\cdot,\cdot),f_2'(\cdot,\cdot,\cdot),R'(\cdot,\cdot,\cdot),F(\cdot,\cdot)}(1^\lambda, \zeta)$
227 : **return** b'

Oracle $f_2'(k', r_S', m_1')$

$m_2 \leftarrow f_2(k', r_S', m_1')$
if $m_2 \neq \bot$:
  $T_2 \leftarrow T_2 \cup \{(k', r_S', m_1')\}$
return m_2

Oracle $\textsc{Ev}(x)$

return $f(x)$

Oracle $\textsc{LimEv}(x)$

if $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
$\mathsf{tx} \leftarrow \mathsf{tx} - 1$; **return** $\textsc{Ev}(x)$

Oracle $R'(x, r_C, m_2)$

if $(x, r_C, m_2) \in T_R$
  return $T_R[x, r_C, m_2]$
elseif $\exists (k', r_S', m_1') \in T_2.\ k = k' \wedge$
  $f_1(x, r_C) = m_1' \wedge$
  $f_2(k', r_S', m_1') = m_2$:
    $T_R[x, r_C, m_2] \leftarrow \textsc{LimEv}(x)$
    return $T_R[x, r_C, m_2]$
else return $\mathrm{R}(x, r_C, m_2)$

Oracle $\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

$\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
// recover state (existing interaction) or increment tx counter (new interaction)
if ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
if $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
parse $\zeta = 1^\lambda$
if $|\mathsf{msg}| \neq 6\lambda$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
else
  $r_S \leftarrow\!\!\$\ \{0,1\}^\lambda$
  $m_2 \leftarrow f_2(k, r_S, \mathsf{msg})$
  if $m_2 = \bot$: $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \bot, \textsc{Nil})$
  else
    $T_2 \leftarrow T_2 \cup \{(k, r_S, \mathsf{msg})\}$
    $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow (\textsc{Nil}, \top, m_2)$
$\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}, \mathsf{status});$ **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 76. Game $\mathbf{G}_4$: Implementing $\textsc{Ev}(\cdot)$ as a random function.

$\mathbf{G}_5[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

231 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
232 : $\zeta \leftarrow\!\!\$\ 1^\lambda$
233 : $f \leftarrow\!\!\$\ (\mathcal{X}_{\lambda,\zeta} \to \mathcal{Y}_{\lambda,\zeta})$
234 : $T_2 \leftarrow \emptyset;\ T_R \leftarrow \{\}$
235 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot),\mathrm{Ev}(\cdot),\mathrm{Srv}(\cdot,\cdot),\mathrm{Prim}(\cdot,\cdot)}(1^\lambda, \zeta)$
236 : **return** b'

Oracle $\mathrm{Prim}(u, x)$

$(\mathsf{state}_{\mathsf{Sim}}, y) \leftarrow\!\!\$\ \mathsf{Sim.PrimEv}^{\mathrm{LimEv}(\cdot)}(\mathsf{state}_{\mathsf{Sim}}, (u, x))$
return y

Oracle $\mathrm{LimEv}(x)$

if $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
$\mathsf{tx} \leftarrow \mathsf{tx} - 1$
return $\mathrm{Ev}(x)$

$\mathsf{Sim.Init}(1^\lambda, \zeta)$

$k \leftarrow\!\!\$\ \{0,1\}^\lambda$
$s \leftarrow (1^\lambda, k, \emptyset, \{\})$
return s

$\mathsf{Sim.SEval}^{\mathrm{LimEv}(\cdot)}(\mathrm{ssid}, \mathsf{msg}, \mathsf{state}_{\mathsf{Sim}})$

parse $\mathsf{state}_{\mathsf{Sim}} = (1^\lambda, k, T_2, T_R)$
if $|\mathsf{msg}| \neq 6\lambda$: **return** $(\mathsf{state}_{\mathsf{Sim}}, \bot, \mathrm{Nil})$
$r_S \leftarrow\!\!\$\ \{0,1\}^\lambda;\ m_2 \leftarrow f_2(k, r_S, \mathsf{msg})$
if $m_2 = \bot$: **return** $(\mathsf{state}_{\mathsf{Sim}}, \bot, \mathrm{Nil})$
else
  $T_2 \leftarrow T_2 \cup \{(k, r_S, \mathsf{msg})\}$
  $\mathsf{state}_{\mathsf{Sim}} \leftarrow (1^\lambda, k, T_2, T_R)$
  return $(\mathsf{state}_{\mathsf{Sim}}, \top, m_2)$

Oracle $\mathrm{Ev}(x)$

return $f(x)$

$\mathsf{Sim.PrimEv}^{\mathrm{LimEv}(\cdot)}(\mathsf{state}_{\mathsf{Sim}}, (u, x))$

parse $\mathsf{state}_{\mathsf{Sim}} = (1^\lambda, k, T_2, T_R)$
if $u =$ "f_1":
  parse $x = (x', r'_C)$
  return $\big(\mathsf{state}_{\mathsf{Sim}}, f_1(x', r'_C)\big)$
elseif $u =$ "F":
  parse $x = (k', x')$
  return $\big(\mathsf{state}_{\mathsf{Sim}}, F(k', x')\big)$
elseif $u =$ "f_2":
  parse $x = (k', r'_S, m'_1)$
  $m_2 \leftarrow f_2(k', r'_S, m'_1)$
  if $m_2 \neq \bot$: $T'_2 \leftarrow T_2 \cup \{(k', r'_S, m'_1)\}$
  $\mathsf{state}_{\mathsf{Sim}} \leftarrow (1^\lambda, k, T'_2, T_R)$
  return $(\mathsf{state}_{\mathsf{Sim}}, m_2)$
elseif $u =$ "R":
  parse $x = (x', r'_C, m'_2)$
  if $(x', r'_C, m'_2) \in T_R$
    return $\big(\mathsf{state}_{\mathsf{Sim}}, T_R[x', r'_C, m'_2]\big)$
  elseif $\exists (k'', r''_S, m''_1) \in T_2.\ k = k'' \wedge$
    $f_1(x', r'_C) = m''_1 \wedge$
    $f_2(k'', r''_S, m''_1) = m'_2$:
      $T_R[x', r'_C, m'_2] \leftarrow \mathrm{LimEv}(x')$
      $\mathsf{state}_{\mathsf{Sim}} \leftarrow (1^\lambda, k, T_2, T_R)$
      return $\big(\mathsf{state}_{\mathsf{Sim}}, T_R[x', r'_C, m'_2]\big)$
  else return $\big(\mathsf{state}_{\mathsf{Sim}}, \mathrm{R}(x', r'_C, m'_2)\big)$
else return $(\mathsf{state}_{\mathsf{Sim}}, \bot)$

Oracle $\mathrm{Srv}(\mathrm{ssid}, \mathsf{msg})$

$\mathsf{state} \leftarrow \mathrm{Nil};\ \mathsf{status} \leftarrow \mathrm{Nil};\ \mathsf{resp} \leftarrow \mathrm{Nil}$
// recover state (existing interaction) or increment tx counter (new interaction)
if ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathrm{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
if $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \mathrm{Nil})$
$(\mathsf{state}_{\mathsf{Sim}}, \mathsf{status}, \mathsf{resp}) \leftarrow\!\!\$\ \mathsf{Sim.SEval}^{\mathrm{LimEv}(\cdot)}(\mathrm{ssid}, \mathsf{msg}, \mathsf{state}_{\mathsf{Sim}})$
$\mathsf{st}[\mathrm{ssid}] \leftarrow (\mathsf{state}, \mathsf{status});$ **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \mathrm{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 77. Game $\mathbf{G}_5$: Enclosing the simulator Sim. This is the final game, equivalent to $\mathbf{Exp}^{\text{oprf-psr}}_{\tilde{\mathsf{F}},\mathcal{A},\mathsf{Sim},1}(\lambda)$.

$\mathbf{G}_0[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

241 : $\zeta \leftarrow\!\!\$\ \tilde{\mathsf{F}}.\mathsf{Setup}(1^\lambda)$
242 : $(x_0, x_1, \mathsf{state}_{\mathrm{adv}}) \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“find”}, 1^\lambda, \zeta)$
243 : **if** $\{x_0, x_1\} \not\subseteq \mathcal{X}_{\lambda,\zeta}$: $\big(b' \leftarrow\!\!\$\ \{0,1\};$ **return** $b'\big)$
244 : $\mathsf{state}_{\mathrm{clt}} \leftarrow \textsc{Nil};\ \mathsf{clientStatus} \leftarrow \textsc{Nil};\ \mathsf{serverStatus} \leftarrow \textsc{Nil};\ \mathsf{msg} \leftarrow \textsc{Nil}$
245 : $a \leftarrow 0$ // active party: 0 = (honest) client, 1 = corrupted server/adversary
246 : **while** $\mathsf{clientStatus} \notin \{\bot, \top\} \wedge \mathsf{serverStatus} \neq \bot$
247 : **if** $a \stackrel{?}{=} 0$: $(\mathsf{state}_{\mathrm{clt}}, \mathsf{clientStatus}, \mathsf{msg}, y) \leftarrow\!\!\$\ \tilde{\mathsf{F}}.\mathsf{CEval}(\zeta, x_0, \mathsf{state}_{\mathrm{clt}}, \mathsf{msg})$
248 : **else** $(\mathsf{state}_{\mathrm{adv}}, \mathsf{serverStatus}, \mathsf{msg})$
$\leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“eval”}, \mathsf{state}_{\mathrm{adv}}, \mathsf{msg})$
249 : $a \leftarrow 1 - a$
250 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“guess”}, \mathsf{state}_{\mathrm{adv}});$ **return** b'

Fig. 78. Game $\mathbf{G}_0$: The original game, equivalent to $\mathbf{Exp}^{\text{oprf-obl}}_{\tilde{\mathsf{F}},\mathcal{A},0}(\lambda)$.

$\mathbf{G}_1[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

251 : $(x_0, x_1, \mathsf{state}_{\mathrm{adv}}) \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“find”}, 1^\lambda, \zeta)$
252 : **if** $\{x_0, x_1\} \not\subseteq \mathcal{X}_{\lambda,\zeta}$: $\big(b' \leftarrow\!\!\$\ \{0,1\};$ **return** $b'\big)$
253 : $r_C \leftarrow\!\!\$\ \{0,1\}^\lambda$
254 : $m_1 \leftarrow f_1(x_0, r_C)$
255 : $(\mathsf{state}_{\mathrm{adv}}, \mathsf{serverStatus}, m_2) \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“eval”}, \mathsf{state}_{\mathrm{adv}}, m_1)$
256 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“guess”}, \mathsf{state}_{\mathrm{adv}});$ **return** b'

Fig. 79. Game $\mathbf{G}_1$: Expanding $\tilde{\mathsf{F}}.\mathsf{Setup}$ and $\tilde{\mathsf{F}}.\mathsf{CEval}$.

$\mathbf{G}_2[f_1, f_2, R, F, \mathrm{PS}](\lambda)$

261 : $(x_0, x_1, \mathsf{state}_{\mathrm{adv}}) \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“find”}, 1^\lambda, \zeta)$
262 : **if** $\{x_0, x_1\} \not\subseteq \mathcal{X}_{\lambda,\zeta}$: $\big(b' \leftarrow\!\!\$\ \{0,1\};$ **return** $b'\big)$
263 : $r_C \leftarrow\!\!\$\ \{0,1\}^\lambda$
264 : $m_1 \leftarrow f_1(x_1, r_C)$
265 : $(\mathsf{state}_{\mathrm{adv}}, \mathsf{serverStatus}, m_2) \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“eval”}, \mathsf{state}_{\mathrm{adv}}, m_1)$
266 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathrm{PS}(\cdot), f_1(\cdot,\cdot), f_2(\cdot,\cdot,\cdot), R(\cdot,\cdot,\cdot), F(\cdot,\cdot)}(\text{“guess”}, \mathsf{state}_{\mathrm{adv}});$ **return** b'

Fig. 80. Game $\mathbf{G}_2$: Replacing x_0 with x_1. This is the final game, equivalent to $\mathbf{Exp}^{\text{oprf-obl}}_{\tilde{\mathsf{F}},\mathcal{A},1}(\lambda)$.

Definition 14 (Key exchange protocol). *A key exchange protocol* KE *consists of a pair of efficient probabilistic algorithms* $\mathsf{KE} := (\mathsf{A}, \mathsf{B})$ *that both get the security parameter* 1^λ *as input, may exchange network messages with each other, and finally output a key after a polynomial number of communication rounds (in the security parameter). We let* $(\mathsf{k}_A, \mathsf{transc}, \mathsf{k}_B) \leftarrow\!\!\$\ \langle \mathsf{A}(1^\lambda), \mathsf{B}(1^\lambda)\rangle$ *denote the outcome of a run of* KE*, which consists of the transcript* transc *of all exchanged network messages as well as the resulting keys* k_A *and* k_B *for* A *resp.* B*.*

Definition 15 (Correctness for key exchange protocols). *We say that a key exchange protocol* $\mathsf{KE} = (\mathsf{A}, \mathsf{B})$ *achieves correctness iff there exists a negligible function* $\mathsf{negl}\colon \mathbb{N}_+ \to \mathbb{R}$ *such that for all* $\lambda \in \mathbb{N}_+$*, we have*

$$\Pr_{(\mathsf{k}_A, \mathsf{transc}, \mathsf{k}_B) \leftarrow\!\!\$ \langle \mathsf{A}(1^\lambda), \mathsf{B}(1^\lambda)\rangle}[\mathsf{k}_A \neq \mathsf{k}_B] \leq \mathsf{negl}(\lambda)\,.$$

Definition 16 (Weak security for key exchange protocols). *We say that a key exchange protocol* $\mathsf{KE} = (\mathsf{A}, \mathsf{B})$ *achieves weak security against semi-honest adversaries iff for all* PPT *adversaries* $\mathcal{A}$ *there exists a negligible function* $\mathsf{negl}\colon \mathbb{N}_+ \to \mathbb{R}$ *such that for all* $\lambda \in \mathbb{N}_+$*, we have*

$$\mathsf{Adv}^{\text{ke-ws}}_{\mathsf{KE},\mathcal{A}}(\lambda) := \Pr_{(\mathsf{k}_A, \mathsf{transc}, \mathsf{k}_B) \leftarrow\!\!\$ \langle \mathsf{A}(1^\lambda), \mathsf{B}(1^\lambda)\rangle}[\mathcal{A}(1^\lambda, \mathsf{transc}) \stackrel{?}{=} \mathsf{k}_B] \leq \mathsf{negl}(\lambda)\,.$$

Lemma 4 (Impossibility of KE relative to a PSPACE-complete oracle [IR89]). *Relative to a* PSPACE*-complete oracle and a random oracle, a correct key exchange protocol with weak security against semi-honest adversaries does not exist.*

Proof. This immediately follows from [IR89, Cor. 6.1]. Impagliazzo and Rudich state their result for random *permutation* oracles [IR89, Cor. 6.1], but their proof applies to ordinary random oracles as well. (In fact, they even prove the impossibility with respect to a random oracle first and then switch to a random *permutation* oracle [IR89, Thm. 6.2].)

Lemma 5 (Fully black-box reduction from KE to OPRFs). *There is a fully black-box reduction from weakly secure key exchange (with security against semi-honest adveraries) to OPRFs.*

Proof. Consider the key exchange protocol depicted in Figure 81. The party B chooses an OPRF key k and evaluates the PRF F locally on the constant input 0^λ using k. Next, A and B jointly execute the OPRF, where B takes the role of the server using k and A takes the role of the client and inputs the constant 0^λ.

By correctness of the OPRF protocol, executing the PRF locally yields the same output as an interactive execution of the OPRF, i.e., the key exchange is correct.

Now assume that there exists an efficient adversary $\mathcal{A}$ against the weak security of the key exchange in Figure 81. We show that there exist adversaries $\mathcal{B}$ and $\mathcal{B}'$ such that for all efficient simulators Sim, we have

$$\mathsf{Adv}^{\text{ke-ws}}_{\mathsf{KE},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{B}}(\lambda) + \mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F},\mathcal{B}',\mathsf{Sim}}(\lambda) + \frac{1}{|\mathcal{Y}|}.$$

First, we consider a hybrid experiment $\mathcal{H}$. This experiment is like the weak key exchange experiment from Definition 16, except that A inputs 1^λ into the OPRF instead of 0^λ. If this change is noticeable for an adversary $\mathcal{A}$, then we can construct an adversary $\mathcal{B}$ against the obliviousness of the OPRF as follows: $\mathcal{B}$, playing the obliviousness experiment, chooses inputs $x_0 := 0^\lambda$, $x_1 := 1^\lambda$ and passes these values to the challenger. The challenger executes the code of an honest OPRF client on input x_b. In this interaction, $\mathcal{B}$ chooses $\mathsf{k} \leftarrow\!\!\$\ \mathsf{F.KGen}(\zeta)$ and executes the code of an honest server. Eventually, $\mathcal{B}$ obtains a full transcript transc for the OPRF execution with the help of the challenger, and runs $y^* \leftarrow\!\!\$\ \mathcal{A}(\mathsf{transc})$. $\mathcal{B}$ then outputs 0 if $y^* \stackrel{?}{=} F(\mathsf{k}, 0^\lambda)$, else 1.

Note that if the challenger chooses message x_0, then $\mathcal{A}$'s view is distributed as in the original experiment and else for message x_1, it is distributed as in the hybrid experiment $\mathcal{H}$. Next, we argue that if $\mathcal{A}$ wins in the hybrid experiment, we can construct an adversary $\mathcal{B}'$ acting as a malicious client in the pseudorandomness experiment (cf. Figure 57). $\mathcal{B}'$ follows the code of an honest OPRF client with input 1^λ and obtains a transcript transc of the OPRF execution as well as the OPRF output y_1 from the challenger. Next, it runs $y \leftarrow\!\!\$\ \mathcal{A}(\mathsf{transc})$ to get an OPRF output y_0. Finally, $\mathcal{B}'$ queries the $\textsc{Eval}$ oracle on 0^λ to obtain the PRF output y'_0 and on 1^λ to obtain the PRF output y'_1. $\mathcal{B}'$ outputs 0 ("real world") if $y_0 \stackrel{?}{=} y'_0$ and $y_1 \stackrel{?}{=} y'_1$, and 1 ("simulated execution") otherwise.

Note that the transcript of the key exchange protocol is exactly the transcript of the OPRF. Thus, if $\mathcal{A}$ is successful in breaking the key exchange in hybrid $\mathcal{H}$, $\mathcal{B}'$ obtains $y_0 = F(\mathsf{k}, 0^\lambda)$ from $\mathcal{A}$ *in addition* to

the result y_1 of the honest OPRF evaluation of 1^λ that it would get anyway. Now, if Eval returns the real function outputs $F(\mathsf{k}, 0^\lambda)$ and $F(\mathsf{k}, 1^\lambda)$ (when $b = 0$ in the pseudorandomness experiment), then the two outputs y_0, y_1 and y'_0, y'_1 match. Else, if the challenger chose $y'_0 \leftarrow\!\!\$\ \mathcal{Y}$ and $y'_1 \leftarrow\!\!\$\ \mathcal{Y}$ (when $b = 1$), then the two outputs match at most with probability $\frac{1}{|\mathcal{Y}|}$. That is because the adversary checks *both* the validity of y_0, which it got from $\mathcal{A}$, i.e. $y_0 \stackrel{?}{=} y'_0$, *and* the validity of y_1, which it got from the honest evaluation for 1^λ, i.e. $y_1 \stackrel{?}{=} y'_1$. Any simulator can only ask one query to the $\mathsf{LimEval}$ oracle because the pseudorandomness game restricts the maximum number of $\mathsf{LimEval}$ queries of the simulator to the number of OPRF interactions the adversary initiated (which is only one in our case).

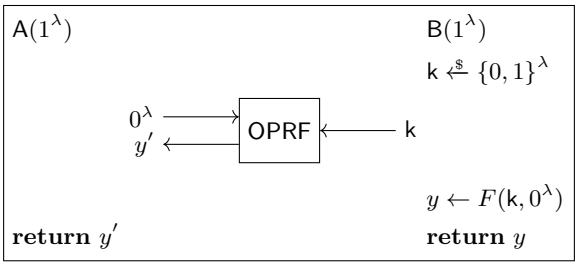


Fig. 81. Weakly secure key exchange protocol from an OPRF that computes the PRF F.

Theorem 9 (Main impossibility result). *There is no strongly transparent black-box 1-out-of-2 combiner for OPRFs that preserves correctness with respect to Def. 9 and security with respect to Def. 10 and 11 in the random oracle model.*

Proof. Assume for the sake of contradiction that there is transparent black-box 1-out-of-2 combiner for OPRFs that preserves correctness with respect to Def. 9 and security with respect to Def. 10 and 11 in the random oracle model. Let us call this combiner Comb. Let $\tilde{\mathsf{F}}_A$ and $\tilde{\mathsf{F}}_B$ be two independent instances of the OPRF protocol depicted in Fig. 70. Since Comb is a black-box combiner, we can instantiate it with $\tilde{\mathsf{F}}_A$ and $\tilde{\mathsf{F}}_B$, and obtain a combined OPRF protocol $\tilde{\mathsf{F}}_{\mathrm{cmb}} \coloneqq \mathsf{Comb}(\tilde{\mathsf{F}}_A, \tilde{\mathsf{F}}_B)$.

Let us now consider two oracle worlds, $World_1$ and $World_2$ (see Fig. 6). Both oracle worlds contain a PSPACE-complete oracle PS and a random oracle RO as well as two independent OPRF oracles $\mathsf{OPRF}_A = (f_1^A, f_2^A, R^A, F^A)$ and $\mathsf{OPRF}_B = (f_1^B, f_2^B, R^B, F^B)$ as defined in Cons. 12. Furthermore, $World_1$ contains an inversion oracle $\mathsf{Invert}_A = ((f_1^A)^{-1}, (f_2^A)^{-1})$ for OPRF_A and $World_2$ contains an inversion oracle $\mathsf{Invert}_B = ((f_1^B)^{-1}, (f_2^B)^{-1})$ for OPRF_B, as defined in Cons. 13. That is, we have:

- $World_1$:
  - PSPACE-complete oracle PS
  - Random oracle RO
  - OPRF oracle $\mathsf{OPRF}_A = (f_1^A, f_2^A, R^A, F^A)$
  - OPRF oracle $\mathsf{OPRF}_B = (f_1^B, f_2^B, R^B, F^B)$
  - Inversion oracle $\mathsf{Invert}_A = ((f_1^A)^{-1}, (f_2^A)^{-1})$, breaking OPRF_A.
- $World_2$:
  - PSPACE-complete oracle PS
  - Random oracle RO
  - OPRF oracle $\mathsf{OPRF}_A = (f_1^A, f_2^A, R^A, F^A)$
  - OPRF oracle $\mathsf{OPRF}_B = (f_1^B, f_2^B, R^B, F^B)$
  - Inversion oracle $\mathsf{Invert}_B = ((f_1^B)^{-1}, (f_2^B)^{-1})$, breaking OPRF_B.

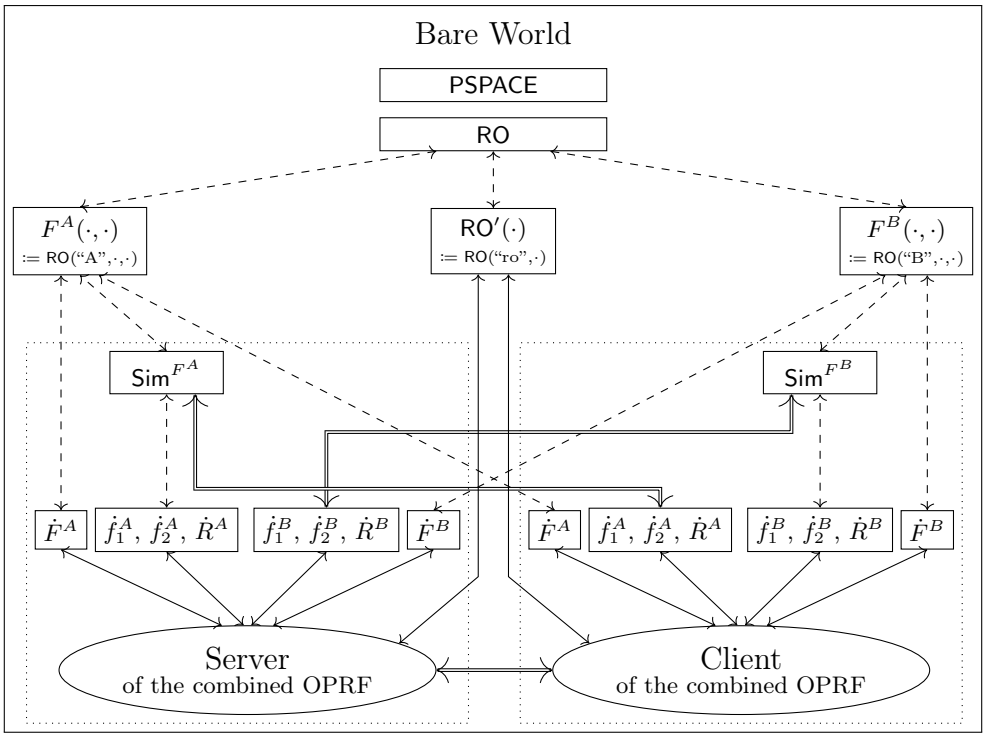
In each world, we can instantiate $\tilde{\mathsf{F}}_A$ with OPRF_A, $\tilde{\mathsf{F}}_B$ with OPRF_B and the random oracle for Comb with RO, resulting in the (single) combined scheme $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ to be well-defined in both worlds. Note that in $World_1$, $\tilde{\mathsf{F}}_A$ is insecure (due to the presence of Invert_A), but $\tilde{\mathsf{F}}_B$ is secure because of Lem. 3. Vice versa, in $World_2$, $\tilde{\mathsf{F}}_B$ is insecure (due to the presence of Invert_B), but $\tilde{\mathsf{F}}_A$ is secure because of Lem. 3. Since both candidate schemes achieve perfect correctness and Comb preserves correctness, $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ is correct, too. Furthermore, since in each oracle world, one OPRF construction remains secure, and the combined scheme has access to a random oracle, the robustness of Comb in the random oracle model implies that $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ is secure in both worlds. In the remainder of this proof, however, we show that $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ is insecure in at least one of the two oracle worlds, which contradicts the robustness of the combiner. As a result, our initial assumption that a robust combiner Comb exists must be wrong; there is no such combiner.

To show that $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ is insecure in $World_1$ or $World_2$, we conjure up a third oracle world, which we call "Bare World". The Bare World provides only a PSPACE-complete oracle PS and a random oracle RO (see Fig. 6). Using standard domain separation techniques, we split RO into three independent random oracles:

– $F^A(\cdot) := \mathsf{RO}(\text{“A”} \,\|\, \cdot)$ – $F^B(\cdot) := \mathsf{RO}(\text{“B”} \,\|\, \cdot)$ – $\mathsf{RO}'(\cdot) := \mathsf{RO}(\text{“ro”} \,\|\, \cdot)$

Lemma 2 then implies that there exists an efficient simulator Sim such that Sim^{F^A} simulates OPRF_A and Sim^{F^B} simulates OPRF_B in the Bare World. In other words, by instantiating Sim twice in the Bare World, once with F^A, and once with F^B, we can simulate both OPRF_A and OPRF_B while still retaining an independent random oracle RO'.

Next, we use Sim^{F^A} and Sim^{F^B} to implement equivalents $\mathsf{O\dot{P}RF}_A := (\dot{f}_1^A, \dot{f}_2^A, \dot{R}^A, \dot{F}^A)$ and $\mathsf{O\dot{P}RF}_B := (\dot{f}_1^B, \dot{f}_2^B, \dot{R}^B, \dot{F}^B)$ of OPRF_A and OPRF_B in the Bare World, as in [HKN+05]. We say that the server is responsible for $\mathsf{O\dot{P}RF}_A$, and the client is responsible for $\mathsf{O\dot{P}RF}_B$. A party that is responsible for an OPRF oracle runs an instance of the corresponding simulator, i.e., the server runs Sim^{F^A} and the client runs Sim^{F^B}. We set $\dot{F}^A := F^A$ and $\dot{F}^B := F^B$. This means that when any party wants to query $\dot{F}^A$ or $\dot{F}^B$, it performs a query $\mathsf{RO}(\text{“A”}\|\cdot)$ resp. $\mathsf{RO}(\text{“ro”}\|\cdot)$. The remaining suboracles $(\dot{f}_1^A, \dot{f}_2^A, \dot{R}^A)$ and $(\dot{f}_1^B, \dot{f}_2^B, \dot{R}^B)$ are implemented differently: When they are called by the party that is responsible for them, the query is answered using the local simulator. When the suboracle is called by the party which not responsible for it, the query is forwarded for evaluation to the other party *over the network* as plain text (with no confidentiality protection), and the answer returned. See Fig. 83 for a formal definition of the simulated oracles and Fig. 82 for a graphical description.



⟷ accessed by the client/server

⇠ ⇢ is the same as/ is implemented by

⟺ network message

Fig. 82. The Bare World in our impossiblity result

By instantiating $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ with $\mathsf{O\dot{P}RF}_A$ and $\mathsf{O\dot{P}RF}_B$, we obtain a well-defined combined OPRF in the Bare World that achieves correctness. Let us call this overall construction $\dot{\mathsf{F}}_{\mathrm{cmb}}$. We argue that $\dot{\mathsf{F}}_{\mathrm{cmb}}$ is insecure. In fact, we can even show that there is an honest-but-curious adversary that observes the network communication between the client and the server in the Bare World (but neither gets the parties' randomness nor their actual, non-simulated random oracle queries) and can break the obliviousness or pseudorandomness of $\dot{\mathsf{F}}_{\mathrm{cmb}}$. This can be seen as follows: Recall that Lemma 5 tells us that there is a fully black-box reduction $\mathcal{R}$ from weakly secure key exchange (against honest-but-curious adversaries) to OPRFs. This implies that we can construct a weakly secure key exchange protocol KE from $\dot{\mathsf{F}}_{\mathrm{cmb}}$ that is correct if $\dot{\mathsf{F}}_{\mathrm{cmb}}$ achieves correctness, and secure if $\dot{\mathsf{F}}_{\mathrm{cmb}}$ is secure. Taking the contrapositive of the latter statement, we can deduce that if there is an efficient

Server(1^λ)

$\mathsf{Sim}^{F^A}.\mathsf{Init}(1^\lambda)$ // state kept implicitly

Oracle $\dot{f}_1^A(x, r_C)$

return $\mathsf{Sim}^{F^A}.\hat{f}_1(x, r_C)$

Oracle $\dot{f}_2^A(k, r_S, m_1)$

return $\mathsf{Sim}^{F^A}.\hat{f}_2(k, r_S, m_1)$

Oracle $\dot{R}^A(x, r_C, m_2)$

return $\mathsf{Sim}^{F^A}.\hat{R}(x, r_C, m_2)$

Oracle $\dot{F}^A(k, x)$

return $\mathsf{RO}(\text{“A”}, x)$

Oracle $\dot{F}^B(k, x)$

return $\mathsf{RO}(\text{“B”}, x)$

Oracle $\dot{f}_1^B(x, r_C)$

send $(\text{“f1”}, x, r_C)$ to Server
receive $(\text{“f1”}, (x, r_C), m_1)$ from Server
return m_1

Oracle $\dot{f}_2^B(k, r_S, m_1)$

send $(\text{“f2”}, k, r_S, m_1)$ to Server
receive $(\text{“f1”}, (k, r_S, m_1), m_2)$ from Server
return m_2

Oracle $\dot{R}^B(x, r_C, m_2)$

send $(\text{“f2”}, x, r_C, m_2)$ to Server; **receive** $(\text{“f1”}, (x, r_C, m_2), y)$ from Server; **return** y

On message $(\text{“f1”}, x, r_C)$

$m_1 \leftarrow\!\!\$\ \mathsf{Sim}^{F^A}.\hat{f}_1(x, r_C)$
send $(\text{“f1”}, (x, r_C), m_1)$ to Client

On message $(\text{“f2”}, k, r_S, m_1)$

$m_2 \leftarrow\!\!\$\ \mathsf{Sim}^{F^A}.\hat{f}_2(k, r_S, m_1)$
send $(\text{“f2”}, (k, r_S, m_1), m_2)$ to Client

On message $(\text{“R”}, x, r_C, m_2)$

$y \leftarrow\!\!\$\ \mathsf{Sim}^{F^A}.\hat{R}(x, r_C, m_2)$
send $(\text{“f2”}, (x, r_C, m_2), y)$ to Client

Client(1^λ)

$\mathsf{Sim}^{F^B}.\mathsf{Init}(1^\lambda)$ // state kept implicitly

Oracle $\dot{f}_1^B(x, r_C)$

return $\mathsf{Sim}^{F^B}.\hat{f}_1(x, r_C)$

Oracle $\dot{f}_2^B(k, r_S, m_1)$

return $\mathsf{Sim}^{F^B}.\hat{f}_2(k, r_S, m_1)$

Oracle $\dot{R}^B(x, r_C, m_2)$

return $\mathsf{Sim}^{F^B}.\hat{R}(x, r_C, m_2)$

Oracle $\dot{F}^B(k, x)$

return $\mathsf{RO}(\text{“B”}, x)$

Oracle $\dot{F}^A(k, x)$

return $\mathsf{RO}(\text{“A”}, x)$

Oracle $\dot{f}_1^A(x, r_C)$

send $(\text{“f1”}, x, r_C)$ to Server
receive $(\text{“f1”}, (x, r_C), m_1)$ from Server
return m_1

Oracle $\dot{f}_2^A(k, r_S, m_1)$

send $(\text{“f2”}, k, r_S, m_1)$ to Server
receive $(\text{“f1”}, (k, r_S, m_1), m_2)$ from Server
return m_2

Oracle $\dot{R}^A(x, r_C, m_2)$

send $(\text{“f2”}, x, r_C, m_2)$ to Server; **receive** $(\text{“f1”}, (x, r_C, m_2), y)$ from Server; **return** y

On message $(\text{“f1”}, x, r_C)$

$m_1 \leftarrow\!\!\$\ \mathsf{Sim}^{F^B}.\hat{f}_1(x, r_C)$
send $(\text{“f1”}, (x, r_C), m_1)$ to Client

On message $(\text{“f2”}, k, r_S, m_1)$

$m_2 \leftarrow\!\!\$\ \mathsf{Sim}^{F^B}.\hat{f}_2(k, r_S, m_1)$
send $(\text{“f2”}, (k, r_S, m_1), m_2)$ to Client

On message $(\text{“R”}, x, r_C, m_2)$

$y \leftarrow\!\!\$\ \mathsf{Sim}^{F^B}.\hat{R}(x, r_C, m_2)$
send $(\text{“f2”}, (x, r_C, m_2), y)$ to Client

Fig. 83. The oracles $\mathsf{OP\dot{R}F}_A$ and $\mathsf{OP\dot{R}F}_B$ in the Bare World

(honest-but-curious) adversary $\mathcal{A}$ that can break the weak security of KE (i.e. recover the secret key the parties agree upon), then $\mathcal{R}^{\mathcal{A}}$ breaks the pseudorandomness or obliviousness of $\dot{\mathsf{F}}_{\mathrm{cmb}}$. The existence of such an $\mathcal{A}$ follows from the correctness of KE and Lem. 4, which in turn is an immediate consequence of the famous impossibility result by Impagliazzo and Rudich [IR89]. We can interpret KE (including the underlying OPRF protocol $\dot{\mathsf{OPRF}}_{\mathrm{cmb}}$ and Sim^{F^A} as well as Sim^{F^B}) as a key exchange protocol relative to a random oracle RO and a PSPACE-complete oracle (see the dotted boxes in Fig. 82). Hence, the above implication yields that there is either an efficient adversary $\mathcal{R}_S^{\mathcal{A}}$ against the obliviousness or an efficient adversary $\mathcal{R}_C^{\mathcal{A}}$ against the pseudorandomness of $\dot{\mathsf{F}}_{\mathrm{cmb}}$ in the Bare World.

Next, we show that the successful attack against $\dot{\mathsf{F}}_{\mathrm{cmb}}$ in the Bare World also works against $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ in World_1 or World_2. To do so, we perform a case distinction: If the attack in the Bare World breaks the obliviousness of $\dot{\mathsf{F}}_{\mathrm{cmb}}$, then we construct an adversary that breaks the obliviousness of $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ in World_1. Otherwise, i.e., if the attack in the Bare World breaks the pseudorandomness of $\dot{\mathsf{F}}_{\mathrm{cmb}}$, we construct an adversary that breaks the pseudorandomness of $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ in World_2.

Case 1: Attack in the Bare World breaks the obliviousness of $\dot{\mathsf{F}}_{\mathrm{cmb}}$. We construct an adversary $\mathcal{A}^*$ which successfully breaks the obliviousness of $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ in World_1. Recall that $\mathcal{A}^*$ takes the role of a corrupted server in the obliviousness game (Fig. 58). $\mathcal{A}^*$ works as follows: First, it outputs the two hard-coded potential OPRF inputs $(x_0, x_1) = (0^\lambda, 1^\lambda)$. The challenger in the obliviousness game will then engage in an interactive OPRF evaluation with $\mathcal{A}^*$, where the challenger plays the role of an honest client with either x_0 or x_1 as input, depending on the secret bit b. $\mathcal{A}^*$ takes part in this evaluation by first running $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{KGen}$ to obtain a secret OPRF key k and then running $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{SEval}$ to participate in the interaction. For the key generation and the interactive evaluation, $\mathcal{A}^*$ runs both algorithms $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{KGen}$ and $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{SEval}$ exactly as specified (i.e. like an honest server would), with the following modifications:

I $\mathcal{A}^*$ records all incoming and outgoing network messages and saves them in a variable transc.

II Whenever $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{KGen}$ or $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{SEval}$ calls any of the three oracles $O \in \{f_1^B, f_2^B, R^B\}$ with query m and response y, $\mathcal{A}^*$ answers them using its own oracle O in World_1. However, in addition, it also adds a message (O, m) from the server to the client and a message (O, m, y) from the client to the server to the transcript transc (at the point at which the oracle query occurred).

III When $\mathcal{A}^*$ receives an incoming network message m from the client that was caused by the client's call of a next-message (and thus, transparent) suboracle $O \in \{f_1^A, f_2^A\}$ of OPRF_A [16], $\mathcal{A}^*$ calls its O^{-1} oracle (which is part of Invert_A) to obtain $x = O^{-1}(m)$. It then adds a message (O, x) from the client to the server and a message (O, x, m) from the server to the client to transc (at the point before receiving m).

IV When $\mathcal{A}^*$ sends an outgoing network message m_2 to the challenger (client) in response to an incoming message m_1 that resulted from a call by the client to the transparent oracle f_1^A [17], then $\mathcal{A}^*$ uses its inversion oracle $(f_2^A)^{-1}$ to obtain the associated input $v \coloneqq (k', r'_S, m'_1)$ such that $f_2^A(k', r'_S, m'_1) = m_2$. It then queries its inversion oracle $(f_1^A)^{-1}$ on m_1 to acquire the unique value $u \coloneqq (x', r'_C)$ such that $f_1^A(x', r'_C) = m_1$. If the first call to the inversion oracle returns $\perp$ (i.e. $v = \perp$) or $m'_1 \neq m_1$, $\mathcal{A}^*$ adds a message ("R", (x', r'_C, m_2)) from the client to the server and a reply ("R", $(x', r'_C, m_2), \perp$) from the server to the client to transc (at the point after sending the network message m_2). Otherwise, it asks F^A for $y \leftarrow F^A(k', x')$ and adds a message ("R", (x', r'_C, m_2)) from the client to the server and a reply ("R", $(x', r'_C, m_2), y$) from the server to the client to transc (at the point after sending the network message m_2).

Let transc^* denote the resulting modified transcript. Next, $\mathcal{A}^*$ runs the key exchange adversary $\mathcal{A}$ from Lem. 4 and 5 on transc^* with the following modified random oracle:

Oracle $\mathsf{RO}^*(s, m)$

if $s \stackrel{?}{=}$ "A": **return** $F^A(m)$ **elseif** $s \stackrel{?}{=}$ "B": **return** $F^B(m)$ **else return** $\mathsf{RO}(m)$

Let $\mathsf{k}^* \leftarrow\!\!\$\ \mathcal{A}^{\mathsf{RO}^*}(\mathsf{transc}^*)$ denote the resulting key (which is the supposed key in the simulated key exchange protocol). $\mathcal{A}^*$ outputs 0 ("left input") if $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{Eval}(\zeta, k, 0^\lambda) \stackrel{?}{=} \mathsf{k}^*$ and 1 ("right input") otherwise. We argue that $\mathcal{A}^*$ in World_1 has the same success probability (up to a negligible difference) as $\mathcal{R}_S^{\mathcal{A}}$ in the Bare World. Note that $\mathcal{A}^*$ runs the same code as $\mathcal{R}_S^{\mathcal{A}}$, except that the transcript given to $\mathcal{A}$ is generated in a different way and that the random oracle provided to $\mathcal{A}$ is modified. Yet, the views of $\mathcal{A}$ in the two

[16] We assume w.l.o.g. that the adversary can recognize these messages. For example, we could define $\tilde{\mathsf{F}}$ such that they are marked with a unique prefix. However, in order to avoid cluttering up the notation further, we omit such an indicator here.

worlds are computationally indistinguishable (even relative to PS). To see this, we review the differences between the two views:

- Since the oracles F^A and F^B in $World_1$ return random function values, RO^* is a random oracle.
- If the client or server queries $\dot{\mathsf{F}}^A$ in the Bare World, it obtains $\mathsf{RO}(\text{“A”}, \cdot, \cdot)$, whereas in $World_1$, queries $F^A(\cdot,\cdot)$ are answered directly by F^A. However, due to the fact that RO^* replies with $F^A(\cdot,\cdot)$ to all queries of the form $\mathsf{RO}^*(\text{“A”}, \cdot, \cdot)$, this does not a result in a visible difference for $\mathcal{A}$.
- Similarly, if the client or server queries $\dot{\mathsf{F}}^B$ in the Bare World, it obtains $\mathsf{RO}(\text{“B”}, \cdot, \cdot)$, whereas in $World_1$, queries $F^B(\cdot,\cdot)$ are answered directly by F^B. However, due to the fact that RO^* replies with $F^B(\cdot,\cdot)$ to all queries of the form $\mathsf{RO}^*(\text{“B”}, \cdot, \cdot)$, this does not a result in a visible difference for $\mathcal{A}$.
- When the server queries any of the oracles $\dot{f}_1^A$, $\dot{f}_2^A$ or $\dot{R}^A$ in the Bare World, it obtains an answer from Sim^{F^A}, while in $World_1$, queries to f_1^A, f_2^A and R^A are answered directly by these oracles. By Lem. 2, the two views are computationally indistinguishable.
- When the server queries any of the oracles $\dot{f}_1^B$, $\dot{f}_2^B$ or $\dot{R}^B$ in the Bare World, a network message to the client is generated, which is answered by Sim^{F^B}. In $World_1$, however, queries to f_1^B, f_2^B and R^B are answered directly by these oracles. Nevertheless, the resulting views are computationally indistinguishable because II generates the expected network message in $World_1$ and by Lem. 2, the actual oracle in $World_1$ and the simulated oracle in the Bare World are comp. indistinguishable.
- When the client queries $\dot{f}_1^A$ or $\dot{f}_2^A$ in the Bare World, the query is forwarded via a network message to the server, which then answers using Sim^{F^A}. In $World_1$, on the other hand, the client queries f_1^A and f_2^A directly, without generating a network message. However, the transparency condition of the combiner stipulates that *the result* of these oracle queries shall immediately be sent to the server. Because of III, when $\mathcal{A}^*$ detects such messages, it calls the associated inversion oracle to get the corresponding *query* as well, and finally adds an artificial network message to the transcript transc^* for $\mathcal{A}$. Together with Lem. 2, which implies that oracle answers by Sim^{F^A} are indistinguishable from the ones from the actual oracles f_1^A and f_2^A and the fact that the client in $World_1$ accesses the same oracles f_1^A and f_2^A as the server, it follows that for $\mathcal{A}$, there is no visible difference between the transcript and the oracles in $World_1$ and in the Bare World.
- When the client calls $\dot{R}^A$ in the Bare World, the query is sent to the server over the network and answered by it using Sim^{F^A}. In $World_1$, however, the client calls R^A directly. Note that the strong transparency condition for the combiner stipulates that the client can only call and must call R^A on network messages m_2 from the server for OPRF_A and the corresponding input and internal state. Since the client's input x and internal state corresponding to m_1 can be recovered from m_1 using the inversion oracle $(f_1^A)^{-1}$, $\mathcal{A}^*$ can predict all of the client's queries to R^A as well as the resulting answers. Because of IV, $\mathcal{A}^*$ simulates the network messages for every such query to R^A by the client in the transcript transc^* for $\mathcal{A}$. Together with Lem. 2, which implies that oracle answers by Sim^{F^A} are indistinguishable from the ones from the actual oracle R^A and the fact that the client in $World_1$ accesses the same oracle R^A as the server, it follows that $\mathcal{A}$ cannot notice a difference between the transcript and the oracles in $World_1$ and in the Bare World.
- When the client queries $\dot{f}_1^B$, $\dot{f}_2^B$ or $\dot{R}^B$ in the Bare World, it receives an answer from Sim^{F^B}, whereas in $World_1$, queries to f_1^B, f_2^B or R^B are answered directly by these oracles. However, as mentioned above, due to the fact that the client and the server use the same oracles f_1^B, f_2^B or R^B and by Lem. 2, this does not result in a noticeable difference for $\mathcal{A}$.

Hence, if $\mathcal{R}_S^{\mathcal{A}}$ is successful in the Bare World, then so is $\mathcal{A}^*$ in $World_1$ (up to negligible probability).

Case 2: Attack in the Bare World breaks the pseudorandomness of $\dot{\mathsf{F}}_{\mathrm{cmb}}$. We construct an adversary $\mathcal{A}^*$ which successfully breaks the pseudorandomness of $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ in $World_2$. Remember that $\mathcal{A}^*$ takes the role of a corrupted client in the pseudorandomness game (Fig. 57). $\mathcal{A}^*$ works as follows: First, it runs an interactive evaluation of the OPRF on input 1^λ by executing $\tilde{\mathsf{F}}.\mathsf{CEval}$ (exactly like an honest client would) and passing all network messages to the Srv oracle of the game, with the following modifications:

I $\mathcal{A}^*$ records all incoming and outgoing network messages and saves them in a variable transc.

II Whenever $\tilde{\mathsf{F}}_{\mathrm{cmb}}.\mathsf{CEval}$ calls any of the three oracles $O \in \{f_1^A, f_2^A, R^A\}$ with query m and response y, $\mathcal{A}^*$ answers them using its own oracle O in $World_2$. However, in addition, it also adds a message (O, m) from the server to the client and a message (O, m, y) from the client to the server to the transcript transc (at the point at which the oracle query occurred).

III When $\mathcal{A}^*$ receives an incoming network message m from the server that was caused by the server's call of a next-message (and thus, transparent) suboracle $O \in \{f_1^B, f_2^B\}$ of OPRF_B [17], $\mathcal{A}^*$ calls its O^{-1} oracle (which is part of Invert_B) to obtain $x = O^{-1}(m)$. It then adds a message (O, x) from the server to the client and a message (O, x, m) from the client to the server to transc (at the point before receiving m).

IV When $\mathcal{A}^*$ sends an outgoing network message m_2 to the challenger (server) in response to an incoming message m_1 that resulted from a call by the server to the transparent oracle f_1^A [17], then $\mathcal{A}^*$ uses its inversion oracle $(f_2^B)^{-1}$ to obtain the associated input $v := (k', r'_S, m'_1)$ such that $f_2^B(k', r'_S, m'_1) = m_2$. It then queries its inversion oracle $(f_1^B)^{-1}$ on m_1 to acquire the unique value $u := (x', r'_C)$ such that $f_1^B(x', r'_C) = m_1$. If the first call to the inversion oracle returns $\bot$ (i.e. $v = \bot$) or $m'_1 \neq m_1$, $\mathcal{A}^*$ adds a message ("R", (x', r'_C, m_2)) from the server to the client and a reply ("R", $(x', r'_C, m_2), \bot$) from the client to the server to transc (at the point after sending the network message m_2). Otherwise, it asks F^B for $y \leftarrow F^B(k', x')$ and adds a message ("R", (x', r'_C, m_2)) from the server to the client and a reply ("R", $(x', r'_C, m_2), y$) from the client to the server to transc (at the point after sending the network message m_2).

Let transc^* denote the resulting modified transcript. Next, $\mathcal{A}^*$ runs the key exchange adversary $\mathcal{A}$ from Lem. 4 and 5 on transc^* with the following modified random oracle:

Oracle $\mathsf{RO}^*(s, m)$

if $s \stackrel{?}{=}$ "A": **return** $F^A(m)$ **elseif** $s \stackrel{?}{=}$ "B": **return** $F^B(m)$ **else return** $\mathsf{RO}(m)$

Let $y_0 \leftarrow\!\!\$\ \mathcal{A}^{\mathsf{RO}^*}(\mathsf{transc}^*)$ denote the resulting key (which is the supposed key in the simulated key exchange protocol).

$\mathcal{A}^*$ continues by querying its Ev oracle in the pseudorandomness game on $x'_0 = 0^\lambda$ to obtain the function value y'_0 and on $x'_1 = 1^\lambda$ to obtain the function value y'_1. Finally, $\mathcal{A}^*$ outputs 0 ("real world") if $y_0 \stackrel{?}{=} y'_0$ and $y_1 \stackrel{?}{=} y'_1$, and 1 ("simulated execution") otherwise.

We argue that $\mathcal{A}^*$ in $World_2$ has the same success probability (up to a negligible difference) as $\mathcal{R}_C^{\mathcal{A}}$ in the Bare World. Note that $\mathcal{A}^*$ runs the same code as $\mathcal{R}_C^{\mathcal{A}}$, except that the transcript given to $\mathcal{A}$ is generated in a different way and that the random oracle provided to $\mathcal{A}$ is modified. Yet, the views of $\mathcal{A}$ in the two worlds are computationally indistinguishable (even relative to PS). To see this, we review the differences between the two views:

- Since the oracles F^A and F^B in $World_1$ return random function values, RO^* is a random oracle.
- If the client or server queries $\dot{\mathsf{F}}^A$ in the Bare World, it obtains $\mathsf{RO}(\text{"A"}, \cdot, \cdot)$, whereas in $World_2$, queries $F^A(\cdot, \cdot)$ are answered directly by F^A. However, due to the fact that RO^* replies with $F^A(\cdot, \cdot)$ to all queries of the form $\mathsf{RO}^*(\text{"A"}, \cdot, \cdot)$, this does not a result in a visible difference for $\mathcal{A}$.
- Similarly, if the client or server queries $\dot{\mathsf{F}}^B$ in the Bare World, it obtains $\mathsf{RO}(\text{"B"}, \cdot, \cdot)$, whereas in $World_2$, queries $F^B(\cdot, \cdot)$ are answered directly by F^B. However, due to the fact that RO^* replies with $F^B(\cdot, \cdot)$ to all queries of the form $\mathsf{RO}^*(\text{"B"}, \cdot, \cdot)$, this does not a result in a visible difference for $\mathcal{A}$.
- When the client queries any of the oracles $\dot{f}_1^B$, $\dot{f}_2^B$ or $\dot{R}^B$ in the Bare World, it obtains an answer from Sim^{F^B}, while in $World_2$, queries to f_1^B, f_2^B and R^B are answered directly by these oracles. By Lem. 2, the two views are computationally indistinguishable.
- When the client queries any of the oracles $\dot{f}_1^A$, $\dot{f}_2^A$ or $\dot{R}^A$ in the Bare World, a network message to the server is generated, which is answered by Sim^{F^A}. In $World_2$, however, queries to f_1^A, f_2^A and R^A are answered directly by these oracles. Nevertheless, the resulting views are computationally indistinguishable because II generates the expected network message in $World_2$ and by Lem. 2, the actual oracle in $World_1$ and the simulated oracle in the Bare World are comp. indistinguishable.
- When the server queries $\dot{f}_1^B$ or $\dot{f}_2^B$ in the Bare World, the query is forwarded via a network message to the server, which then answers using Sim^{F^B}. In $World_2$, on the other hand, the server queries f_1^B and f_2^B directly, without generating a network message. However, the transparency condition of the combiner stipulates that *the result* of these oracle queries shall immediately be sent to the client. Because of III, when $\mathcal{A}^*$ detects such messages, it calls the associated inversion oracle to get the corresponding *query* as well, and finally adds an artificial network message to the transcript transc^* for $\mathcal{A}$. Together with Lem. 2, which implies that oracle answers by Sim^{F^B} are indistinguishable from

[17] We assume w.l.o.g. that the adversary can recognize these messages. For example, we could define $\tilde{\mathsf{F}}$ such that they are marked with a unique prefix. However, in order to avoid cluttering up the notation further, we omit such an indicator here.

the ones from the actual oracles f_1^B and f_2^B and the fact that the server in $World_2$ accesses the same oracles f_1^B and f_2^B as the client, it follows that for $\mathcal{A}$, there is no visible difference between the transcript and the oracles in $World_2$ and in the Bare World.
- When the server calls $\dot{R}^B$ in the Bare World, the query is sent to the server over the network and answered by it using Sim^{F^B}. In $World_2$, however, the server calls R^B directly. Note that the strong transparency condition for the combiner stipulates that the server can only call and must call R^B on network messages m_2 from the server for OPRF_B and the corresponding input and internal state. Since the server's input x and internal state corresponding to m_1 can be recovered from m_1 using the inversion oracle $(f_1^B)^{-1}$, $\mathcal{A}^*$ can predict all of the server's queries to R^B as well as the resulting answers. Because of IV, $\mathcal{A}^*$ simulates the network messages for every such query to R^B by the server in the transcript transc^* for $\mathcal{A}$. Together with Lem. 2, which implies that oracle answers by Sim^{F^B} are indistinguishable from the ones from the actual oracle R^B and the fact that the server in $World_2$ accesses the same oracle R^B as the client, it follows that $\mathcal{A}$ cannot notice a difference between the transcript and the oracles in $World_2$ and in the Bare World.
- When the server queries $\dot{f}_1^A$, $\dot{f}_2^A$ or $\dot{R}^A$ in the Bare World, it receives an answer from Sim^{F^B}, whereas in $World_2$, queries to f_1^A, f_2^A or R^A are answered directly by these oracles. However, as mentioned above, due to the fact that the server and the client use the same oracles f_1^A, f_2^A or R^A and by Lem. 2, this does not result in a noticeable difference for $\mathcal{A}$.

Hence, if $\mathcal{R}_C^{\mathcal{A}}$ is successful in the Bare World, then so is $\mathcal{A}^*$ in $World_2$ (up to negligible probability).

In both cases, $\tilde{\mathsf{F}}_{\mathrm{cmb}}$ is not secure in $World_1$ or $World_2$. This concludes the proof.

H XOR Combiner

In this section, we discuss that the parallel XOR combiner, simply adding the output of two OPRF instances, provides basic game-based security, but cannot be lifted to UC-security.

H.1 Construction

The XOR combiner $\mathsf{Comb}_\oplus$ is depicted in Figure 84 and formally defined below.

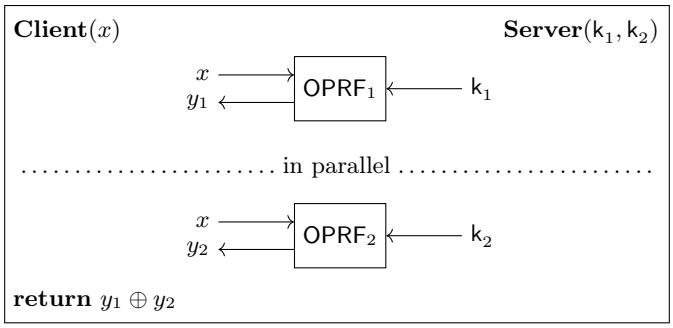


Fig. 84. XOR combiner $\mathsf{Comb}_\oplus$ for two OPRFs

Definition 17 (XOR Combiner). *For every $n \in \mathbb{N}_+$ and every finite sequence $(F_i)_{i\in[n]}$ of OPRFs, we define the combiner $\mathsf{Comb}_\oplus((F_i)_{i\in[n]}) := \mathsf{F}'$ where $F' := (\mathsf{F}'.\mathsf{Setup}, \mathsf{F}'.\mathsf{KGen}, \mathsf{F}'.\mathsf{CEval}, \mathsf{F}'.\mathsf{SEval}, \mathsf{F}'.\mathsf{Eval})$ is defined as follows and in figures 85 and 86:*

$$\mathsf{F}'.\mathsf{Setup}(1^\lambda) \xrightarrow{\$} \Big(\mathsf{F}_1.\mathsf{Setup}(1^\lambda), \ldots, \mathsf{F}_n.\mathsf{Setup}(1^\lambda)\Big)$$

$$\mathsf{F}'.\mathsf{KGen}((\zeta_1, \ldots, \zeta_n)) \xrightarrow{\$} (\mathsf{F}_1.\mathsf{KGen}(\zeta_1), \ldots, \mathsf{F}_n.\mathsf{KGen}(\zeta_1))$$

$$\mathsf{F}'.\mathsf{Eval}((\zeta_1, \ldots, \zeta_n), (k_1, \ldots, k_n), x) \to \bigoplus_{i=1}^n \mathsf{F}_i.\mathsf{Eval}(\zeta_i, k_i, x)$$

If the OPRFs $(F_i)_{i\in[n]}$ make use of ideal primitives $(\mathrm{P}_i)_{i\in[n]}$, then these primitives are "combined" in the natural way into a single primitive $\mathrm{P}' := (\mathrm{P}'.\mathsf{Init}, \mathrm{P}'.\mathsf{Eval})$ that allows access to all of them via a selector parameter u:

$\mathrm{P}'.\mathsf{Init}(1^\lambda)$

$\mathsf{state} \leftarrow\!\!\$\ (\mathrm{P}_1.\mathsf{Init}(1^\lambda), \ldots, \mathrm{P}_n.\mathsf{Init}(1^\lambda))$
return state

$\mathrm{P}'.\mathsf{Eval}(\mathsf{state}, (u, x))$

parse $\mathsf{state} = (\mathsf{state}_1, \ldots, \mathsf{state}_n)$
$(\mathsf{state}_u, y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}_u, x)$
return $((\mathsf{state}_1, \ldots, \mathsf{state}_n), y)$

$\mathsf{F}'.\mathsf{CEval}(\zeta, x, \mathsf{state}, \mathsf{msg}) \$\!\!\rightarrow (\mathsf{state}', \mathsf{status}', \mathsf{msg}', y)$

parse $\zeta = (\zeta_1, \ldots, \zeta_n)$; $(\mathsf{msg}_1, \ldots, \mathsf{msg}_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$
if $\mathsf{msg} \neq \textsc{Nil}$: **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
$(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n, y_1, \ldots, y_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$
if $\mathsf{state} \neq \textsc{Nil}$:
  parse $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n), (y_1, \ldots, y_n))$
// Abort on unexpected message to an already finished candidate scheme
if $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \textsc{Nil}\} \neq \emptyset$: **return** $(\textsc{Nil}, \bot, \textsc{Nil}, \textsc{Nil})$
for $i \in \{j \in [n] \mid \mathsf{status}_j \notin \{\bot, \top\}\}$ // Run all non-finished candidate schemes
  $(\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i, y_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{CEval}(\zeta_i, x, \mathsf{state}_i, \mathsf{msg}_i)$
// Abort if any candidate scheme has aborted
if $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: **return** $(\textsc{Nil}, \bot, \textsc{Nil}, \textsc{Nil})$
// Once all candidate schemes are finished, output XOR of the individual outputs
if $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: **return** $\left(\textsc{Nil}, \top, \textsc{Nil}, \bigoplus_{i=1}^n y_i\right)$
// Otherwise, wait for the next message
$\mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n), (y_1, \ldots, y_n))$
return $\left(\mathsf{state}', \textsc{Nil}, (\mathsf{msg}_1, \ldots, \mathsf{msg}_n), \textsc{Nil}\right)$

Fig. 85. The client part of the OPRF XOR-combiner

Theorem 10 (Pseudorandomness Robustness of the XOR Combiner). *The XOR combiner in Definition 17 is a robust* 1*-out-of-n combiner with respect to pseudorandomness for OPRFs.*

The proof of this theorem is via a non-constructive reduction to the pseudorandomness of the secure candidate. The proof idea is to define a simulator that "knows" which of the candidate schemes $(\mathsf{F}_i)_{i\in[n]}$ ($n \in \mathbb{N}_+$) is secure. (Remember that the pseudorandomness definition only requires the *existence* of a simulator and not necessarily our ability to actually specify this simulator.) Let us assume that F_γ is a secure candidate scheme ($\gamma \in [n]$). The simulator then runs the corresponding simulator Sim_γ to simulate the execution of the secure OPRF F_γ. For the other OPRFs, the simulator chooses random keys k_i ($i \in [n] \setminus \{\gamma\}$) and computes the responses to the client as an honest server would. To make the resulting OPRF match the outputs of the random function f for which it needs to simulate the OPRF, the simulator intercepts each call by Sim_γ to its LimEv oracle and XORs the outputs of the other OPRFs $\bigoplus_{i\in[n]\setminus\{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta_i, k_i, x)$ to the response of the oracle. This way, the simulation works as expected. We give a detailed proof in Appendix H.3.

Theorem 11 (Obliviousness Robustness of the XOR Combiner). *The XOR combiner in Definition 17 is a robust* $\boldsymbol{n}$*-out-of-n combiner with respect to obliviousness for OPRFs. This is, if all n candidate schemes are oblivious, then so is the combined scheme.*

Note that the requirement that *all* candidates are oblivious is rooted in the unconditional security for the receiver.

Proof. This can be shown via a simple hybrid argument with a constant number of hybrids. There are $n + 1$ hybrids: In the i-th hybrid H^i (for $i \in \{0, \ldots, n\}$), the challenger chooses x_1 as the input for the candidate schemes $\mathsf{F}_1, \ldots, \mathsf{F}_i$ and x_0 as the input for the candidate schemes $\mathsf{F}_{i+1}, \ldots, \mathsf{F}_n$. Then, we have $H^0 = \mathbf{Exp}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{A},0}(\lambda)$ as well as $H^n = \mathbf{Exp}^{\text{oprf-obl}}_{\mathsf{F},\mathcal{A},1}(\lambda)$. Furthermore, for all $i \in [n]$, it holds that $H^{i-1} \stackrel{\mathfrak{c}}{\approx} H^i$ due to the security of the i-th candidate scheme. Thus, an application of the hybrid argument for a constant number of hybrids implies the obliviousness of the combined scheme. □

$\mathsf{F}'.\mathsf{SEval}(\zeta, k, \mathsf{state}, \mathsf{msg}) \xrightarrow{\$} (\mathsf{state}', \mathsf{status}', \mathsf{msg}')$

parse $\zeta = (\zeta_1, \ldots, \zeta_n)$; **parse** $k = (k_1, \ldots, k_n)$
parse $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
$(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\text{NIL}, \ldots, \text{NIL})$
if $\mathsf{state} \neq \text{NIL}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
// Abort on unexpected message to an already finished candidate scheme
if $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \text{NIL}\} \neq \emptyset$: **return** $(\text{NIL}, \bot, \text{NIL})$
for $i \in \{j \in [n] \mid \mathsf{status}_j \notin \{\bot, \top\}\}$ // Run all non-finished candidate schemes
  $(\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow_{\$} \mathsf{F}_i.\mathsf{SEval}(\zeta_i, k_i, \mathsf{state}_i, \mathsf{msg}_i)$
// Abort if any candidate scheme has aborted
if $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: **return** $(\text{NIL}, \bot, \text{NIL})$
// Once all candidate schemes are finished, send the last message to the client.
if $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: **return** $(\text{NIL}, \top, (\mathsf{msg}_1, \ldots, \mathsf{msg}_n))$
// Otherwise, wait for the next message
return $(((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n)), \text{NIL}, (\mathsf{msg}_1, \ldots, \mathsf{msg}_n))$

Fig. 86. The server part of the OPRF XOR-combiner

H.2 Discussion

The XOR combiner is very efficient as the combined scheme only needs $n-1$ XOR operations on bit strings in addition to a single invocation of each candidate scheme, and it does not increase the round complexity of the resulting protocol. Furthermore, in contrast to our hash combiner, the robustness proof of the XOR combiner *itself* does not rely on idealized models.

However, the XOR combiner has some inherent limitations: Firstly, it requires all candidate schemes to output bit strings (or, slightly more general, values in the same algebraic structure). Secondly, it does not achieve UC security, even if all candidate schemes UC-realize $\mathcal{F}_{\text{OPRF}}$. This is because in order to UC-realize $\mathcal{F}_{\text{OPRF}}$, the simulator needs to extract a "key label" for each successful evaluation between an honest client and a malicious server, and send this label to $\mathcal{F}_{\text{OPRF}}$ via the RcvComplete interface. The client then obtains its function value from the function table associated with this label. An essential feature of the ideal functionality $\mathcal{F}_{\text{OPRF}}$ is that all function tables are independent and, hence, function values obtained via evaluations with different key labels are not related. However, the XOR combiner inevitably introduces a recognizable structure among adversarially chosen keys which does not exist in the ideal world. This fact can be used to distinguish the real world from the ideal world.

To show the impossibility we consider an arbitrary (multi-party) protocol $\pi_{\oplus}$ implementing the XOR combiner. Formally, we say that $\pi_{\oplus}$ *realizes* the XOR combiner for functions $(F_i)_{i\in[n]}$ if, when executed by honest parties, the client for input $(\text{Eval}, \text{sid}, \text{ssid}, \mathsf{S}, x)$ eventually receives $(\text{Eval}, \text{sid}, \text{ssid}, y)$, where $y = \bigoplus_{i=1}^{n}(F_i).\mathsf{Eval}(\zeta_i, k_i, x)$ and ζ_i, k_i are all sampled according to the protocol resp. the server S. This property guarantees a form of correctness and of termination, meaning that the protocol actually performs the task as expected.

Theorem 12 (UC-Insecurity of the XOR Combiner). *No protocol $\pi_{\oplus}$ realizing the XOR combiner in Definition 17 for PRFs $\mathsf{F}_{i\in[n]}$ with joint output size $\mathcal{Y} = \{0,1\}^{\geq\lambda}$ can UC-realize functionality $\mathcal{F}_{\text{OPRF}}$ in Fig. 1.*

Proof. For simplicity we assume that we combine two PRFs F_1 and F_2 with key spaces $\mathcal{K}_1$ and $\mathcal{K}_2$ and common input space $\mathcal{X}$; the proof easily generalizes to more candidates. Suppose that we have a protocol $\pi_{\oplus}$ that realizes the XOR combiner for F_1 and F_2. We argue that there is an environment that, in connection with the so-called dummy adversary, can distinguish the real-world execution of $\pi_{\oplus}$ from the ideal word. Let Sim be an arbitrary simulator. The distinguishing environment works as follows:

1. The environment $\mathcal{Z}$ starts a session sid for the combined OPRF protocol with an honest client and a corrupted server S^*. That is, it calls the server S^* with input $(\text{Init}, \text{sid})$ to initiate the session and then immediately lets the adversary compromise the server via $(\text{Compromise}, \text{sid})$.
2. $\mathcal{Z}$ selects a random $x \leftarrow_{\$} \mathcal{X}$.

3. The environment chooses random $k_1 \leftarrow\!\!\$\ \mathcal{K}_1$, $k_1' \leftarrow\!\!\$\ \mathcal{K}_1$ and $k_2 \leftarrow\!\!\$\ \mathcal{K}_2$, $k_2' \leftarrow\!\!\$\ \mathcal{K}_2$ such that $k_1 \neq k_1'$ and $k_2 \neq k_2'$ and passes these keys to the adversary.
4. For $(i, u, v) \in \{(\mathrm{I}, k_1, k_2), (\mathrm{II}, k_1', k_2), (\mathrm{III}, k_1, k_2'), (\mathrm{IV}, k_1', k_2')\}$:
   (a) $\mathcal{Z}$ sends $(\textsc{Eval}, \mathrm{sid}, i, \mathsf{S}^*, x)$ to P.
   (b) The adversary $\mathcal{A}$, which controls the corrupted server, is instructed to complete the evaluation like an honest real server for keys u, v. The adversary at some point outputs $(\textsc{RcvCompleteMalicious}, \mathrm{sid})$.
   (c) The environment eventually receives $(\textsc{Eval}, \mathrm{sid}, i, y_i)$ by the termination property and stores y_i.
5. If $|\{y_{\mathrm{I}}, y_{\mathrm{II}}, y_{\mathrm{III}}, y_{\mathrm{IV}}\}| \neq 4$, then $\mathcal{Z}$ outputs 1.
6. Else, $\mathcal{Z}$ outputs 0 if $y_{\mathrm{I}} \oplus y_{\mathrm{II}} \stackrel{?}{=} y_{\mathrm{III}} \oplus y_{\mathrm{IV}}$ and 1 otherwise.

The environment outputs 1 to indicate that it suspects interacting with ideal world and 0 to indicate it believes to be interacting with the real world. In the real world, we always have $y_{\mathrm{I}} \oplus y_{\mathrm{II}} = y_{\mathrm{III}} \oplus y_{\mathrm{IV}}$ due to the construction of the XOR combiner and the fact that $\pi_{\oplus}$ realizes that combiner:

$$\begin{aligned} y_{\mathrm{I}} \oplus y_{\mathrm{II}} &= \mathsf{F}'((k_1, k_2), x) \oplus \mathsf{F}'((k_1', k_2), x) \\ &= \mathsf{F}_1(k_1, x) \oplus \mathsf{F}_2(k_2, x) \oplus \mathsf{F}_1(k_1', x) \oplus \mathsf{F}_2(k_2, x) \\ &= \mathsf{F}_1(k_1, x) \oplus \mathsf{F}_1(k_1', x) \\ &= \mathsf{F}_1(k_1, x) \oplus \mathsf{F}_2(k_2', x) \oplus \mathsf{F}_1(k_1', x) \oplus \mathsf{F}_2(k_2', x) \\ &= \mathsf{F}'((k_1, k_2'), x) \oplus \mathsf{F}'((k_1', k_2'), x) = y_{\mathrm{III}} \oplus y_{\mathrm{IV}} \end{aligned}$$

It remains to argue that the values $y_{\mathrm{I}}, \ldots, y_{\mathrm{IV}}$ are distinct with overwhelming probability, such that the environment does not output 1 according to instruction 5 too often in this setting. This follows from the pseudorandomness of the functions F_i and the fact that we apply them for random keys. For each pair y_i and y_j for $i \neq j$ received in the experiment, we evaluate F_1 or F_2 for a random key k_ℓ which is not used in the computation of the other value. Since the outputs of the functions F_i are pseudorandom we can replace them by random values, or else we easily derive a distinguisher.[18] But then the contribution $\mathsf{F}_i(k_\ell, x)$ causes equality of y_i and y_j in $\mathcal{Y} = \{0,1\}^{\geq \lambda}$ only with negligible probability. Bounding over all $\binom{4}{2}$ combinations of y_i, y_j we can deduce that the environment outputs 1 because of instruction 5 merely with negligible probability. In conclusion, in the real-world setting, $\mathcal{Z}$ outputs 0 with overwhelming probability.

In the ideal world, however, the simulator can make $\mathcal{Z}$ only output 0 if it makes all values $y_{\mathrm{I}}, \ldots, y_{\mathrm{IV}}$ distinct. Else instruction 5 makes the environment output 1. This requires Sim to send $(\textsc{RcvCompleteMalicious}, \mathrm{sid}, i, \mathsf{P}, k_i^*)$ for four different keys k_i^* to the ideal functionality $\mathcal{F}_{\mathrm{OPRF}}$. But then all responses in $(\textsc{Eval}, \mathrm{sid}, i, y_i)$ are independent and uniformly sampled values chosen by the functionality, such that the xor of all values matches the equation above is negligible. It follows that protocol $\pi_{\oplus}$ cannot UC-realize $\mathcal{F}_{\mathrm{OPRF}}$. □

The theorem also enables us to separate the game-based notions for OPRFs from the UC-security notion:

Corollary 1. *If there exists an OPRF F that is pseudorandom according to Definition 10 and oblivious according to Definition 11, then there exists an OPRF that is also pseudorandom and oblivious, but no protocol realizing this OPRF can UC-realize functionality $\mathcal{F}_{\mathrm{OPRF}}$.*

Proof. Consider the xor combiner for F with itself, i.e., where we use the same scheme but with independent keys in each instance. Then this combiner preserves pseudorandomness and obliviousness in the game-based sense according to Theorems 10 and 11. Yet, by Theorem 12, no protocol realizing this combiner can UC-realize $\mathcal{F}_{\mathrm{OPRF}}$. □

H.3 Proof of Theorem 10 (Robustness of the XOR combiner)

Theorem 10 (Pseudorandomness Robustness of the XOR Combiner). *The XOR combiner in Definition 17 is a robust 1-out-of-n combiner with respect to pseudorandomness for OPRFs.*

Proof. Let $(\mathsf{F}_i)_{i \in [n]}$ be a finite sequence of $n \in \mathbb{N}_+$OPRFs, of which at least one provides pseudorandomness (security against malicious clients). Then there exists an index $\gamma \in [n]$ of an OPRF in $(\mathsf{F}_i)_{i \in [n]}$ that provides pseudorandomness, i.e., for every efficient adversary $\mathcal{A}_\gamma$ there exists an efficient simulator Sim_γ and

[18] While keys k_1', k_2' are chosen to be distinct from k_1, k_2, for a pseudorandom function excluding one key from the key space cannot make the PRF insecure.

Game	Adversary		Challenger		
	Ev′	Srv′/Prim′	Ev	Srv/Prim	ModEv
$\mathbf{G}_0$ (Fig. 89)	n/a	n/a	$\mathsf{F}_1 \oplus \ldots \oplus \mathsf{F}_n$	$\mathsf{F}_1 \mid \ldots \mid \mathsf{F}_n$	n/a
$\mathbf{G}_1$ (Fig. 90)	n/a	n/a	$\mathsf{F}_1 \oplus \ldots \oplus \mathsf{F}_n$	$\mathsf{F}_1 \mid \ldots \mid \mathsf{F}_n$	n/a
$\mathbf{G}_2$ (Fig. 91)	$\leftrightarrow$	$\leftrightarrow$	$\mathsf{F}_1 \oplus \ldots \oplus \mathsf{F}_n$	$\mathsf{F}_1 \mid \ldots \mid \mathsf{F}_n$	n/a
$\mathbf{G}_3$ (Fig. 92)	$\mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n \oplus \leftrightarrow$	$\mathsf{F}_1 \mid \mathscr{.}\mathscr{.}\mathscr{.} \mid \mathsf{F}_n \mid \leftrightarrow$	F_γ	F_γ	n/a
$\mathbf{G}_4$ (Fig. 93)	$\mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n \oplus \leftrightarrow$	$\mathsf{F}_1 \mid \mathscr{.}\mathscr{.}\mathscr{.} \mid \mathsf{F}_n \mid \leftrightarrow$	f	Sim_γ	$\leftrightarrow$
$\mathbf{G}_5$ (Fig. 94)	$\mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n \oplus \leftrightarrow$	n/a	f	$\mathsf{F}_1 \mid \mathscr{.}\mathscr{.}\mathscr{.} \mid \mathsf{F}_n \mid \mathsf{Sim}_\gamma$	$\leftrightarrow$
$\mathbf{G}_6$ (Fig. 95)	n/a	n/a	$f \oplus \mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n$	$\mathsf{F}_1 \mid \mathscr{.}\mathscr{.}\mathscr{.} \mid \mathsf{F}_n \mid \mathsf{Sim}_\gamma$	$\mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n \oplus \leftrightarrow$
$\mathbf{G}_7$ (Fig. 96)	n/a	n/a	f	$\mathsf{F}_1 \mid \mathscr{.}\mathscr{.}\mathscr{.} \mid \mathsf{F}_n \mid \mathsf{Sim}_\gamma$	$\mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n \oplus \leftrightarrow$
$\mathbf{G}_8$ (Fig. 97)	n/a	n/a	f	$\mathsf{F}_1 \mid \mathscr{.}\mathscr{.}\mathscr{.} \mid \mathsf{F}_n \mid \mathsf{Sim}_\gamma$	$\mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n \oplus \leftrightarrow$

Table 4. Overview of the games in the proof of robustness for the XOR combiner.

n/a = not applicable (because the oracle does not exist in this game)
$\leftrightarrow$ = result of the associated oracle
$A \mid B$ = parallel composition of A and B
$\mathsf{F}_1 \oplus \mathscr{.}\mathscr{.}\mathscr{.} \oplus \mathsf{F}_n = \mathsf{F}_1 \oplus \cdots \oplus \mathsf{F}_n$ but without F_γ
$\mathsf{F}_1 \mid \mathscr{.}\mathscr{.}\mathscr{.} \mid \mathsf{F}_n = \mathsf{F}_1 \mid \cdots \mid \mathsf{F}_n$ but without F_γ

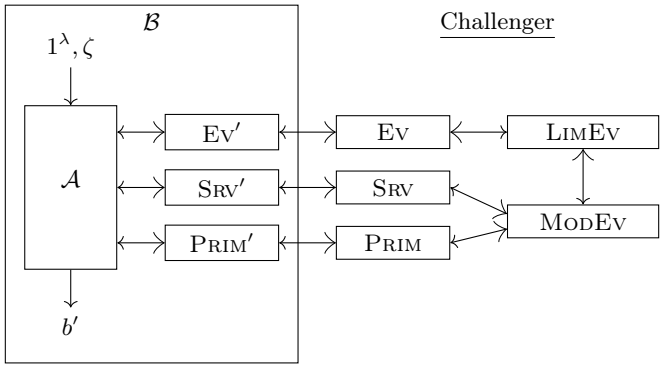


Fig. 87. A graphical representation of the parties and oracles in our proof of pseudorandomness for the XOR combiner. The bigger arrow tips indicate oracle queries, while the smaller ones stand for oracle responses.

a negligible function $\mathsf{negl}_\gamma \colon \mathbb{N}_+ \to \mathbb{R}$ such that for all $\lambda \in \mathbb{N}_+$, we have $\mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F}_\gamma,\mathcal{A}_\gamma,\mathsf{Sim}_\gamma}(\lambda) \leq \mathsf{negl}_\gamma(\lambda)$. Let $\mathsf{F}' \coloneqq \mathsf{Comb}_\oplus((F_i)_{i\in[n]})$ be the combined scheme resulting from applying the XOR combiner to $(\mathsf{F}_i)_{i\in[n]}$. Let $\mathcal{A}$ be an arbitrary but fixed PPT adversary.

We show that the simulator $\mathsf{Sim} \coloneqq (\mathsf{Sim.Init}, \mathsf{Sim.SEval}, \mathsf{Sim.PrimEv})$ as defined in Fig. 88 is efficient and that for all $\lambda \in \mathbb{N}_+$, we have $\mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim}}(\lambda) \leq \mathsf{negl}_\gamma(\lambda)$.

First of all, since we assume that all candidate schemes are efficient and the simulator Sim_γ is efficient, the simulator Sim is efficient, too, because it invokes Sim_γ and the other candidate schemes a polynomial number of times, and any additional operations run in polynomial time. To bound the advantage $\mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim}}(\lambda)$, we use a sequence of games. Table 4 and figure 87 display an overview of the games and the involved oracles.

Game $\mathbf{G}_0$ (Fig. 89): The original game. This game is equivalent to $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim},0}(\lambda)$ (real execution of the OPRF).

Game $\mathbf{G}_1$ (Fig. 90): Expanding the combiner. We expand the definition of the combiner, i.e., we replace the calls to $\mathsf{F}'.\mathsf{Setup}$, $\mathsf{F}'.\mathsf{KGen}$, $\mathsf{F}'.\mathsf{SEval}$ and $\mathsf{F}'.\mathsf{Eval}$ with their respective code. Since this is a purely syntactical change, we have

$$\Pr[\mathbf{G}_0(\lambda) = 1] = \Pr[\mathbf{G}_1(\lambda) = 1]\,.$$

Games $\mathbf{G}_2$ (Fig. 91) and $\mathbf{G}_3$ (Fig. 92): Rearranging. $\mathcal{B}$ runs $\mathcal{A}$ internally, but can change the responses of the Ev, Srv and Prim oracles that $\mathcal{A}$ receives. Initially, in $\mathbf{G}_2$, $\mathcal{B}$ does not modify oracle responses yet. In $\mathbf{G}_3$, we move the evaluation of all candidate schemes except for F_γ into $\mathcal{B}$ and its associated oracles Ev', Srv' and Prim', such that the challenger in the game now only evaluates F_γ. Since these modifications are also purely syntactical, we have

$$\Pr[\mathbf{G}_1(\lambda) = 1] = \Pr[\mathbf{G}_2(\lambda) = 1] = \Pr[\mathbf{G}_3(\lambda) = 1]\,.$$

$\mathsf{Sim}.\mathsf{Init}(1^\lambda, \zeta)$

parse $\zeta = (\zeta_1, \ldots, \zeta_n)$

$\mathsf{st}_{\mathsf{Sim}_\gamma} \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{Init}(1^\lambda, \zeta_\gamma)$

$\boldsymbol{k} \leftarrow []$; $\mathbf{st_P} \leftarrow []$; $\mathsf{st}_{\mathsf{exec}} \leftarrow \{\}$

for $i \in [n] \setminus \{\gamma\}$

  $\boldsymbol{k}[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta_i)$

  $\mathbf{st_P}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$

$\boldsymbol{k}[\gamma] \leftarrow \textsc{Nil}$; $\mathbf{st_P}[\gamma] \leftarrow \textsc{Nil}$

return $(\zeta, \boldsymbol{k}, \mathbf{st_P}, \mathsf{st}_{\mathsf{exec}}, \mathsf{st}_{\mathsf{Sim}_\gamma})$

$\textsc{ModEv}^{\textsc{LimEv}(\cdot)}_{\mathsf{state}_{\mathsf{Sim}}}(x)$

parse $\mathsf{state}_{\mathsf{Sim}} = ((\zeta_1, \ldots, \zeta_n), \boldsymbol{k}, \cdot, \cdot, \cdot)$

$y \leftarrow\!\!\$\ \textsc{LimEv}(x)$

$y \leftarrow y \oplus \bigoplus_{i \in [n] \setminus \{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta_i, k[i], x)$

return y

$\mathsf{Sim}.\mathsf{SEval}^{\textsc{LimEv}(\cdot)}(\mathrm{ssid}, \mathsf{msg}, \mathsf{state}_{\mathsf{Sim}})$

parse $\mathsf{state}_{\mathsf{Sim}} = ((\zeta_1, \ldots, \zeta_n), \boldsymbol{k}, \mathbf{st_P}, \mathsf{st}_{\mathsf{exec}}, \mathsf{st}_{\mathsf{Sim}_\gamma})$

parse $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$

$(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$

if $\mathrm{ssid} \in \mathsf{st}_{\mathsf{exec}}$: $((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n)) \leftarrow \mathsf{st}_{\mathsf{exec}}[\mathrm{ssid}]$

if $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \textsc{Nil}\} \neq \emptyset$: **return** $(\mathsf{state}_{\mathsf{Sim}}, \bot, \textsc{Nil})$

for $i \in \{j \in [n] \setminus \{\gamma\} \mid \mathsf{status}_j \notin \{\bot, \top\}\}$

  $(\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta_i, k[i], \mathsf{state}_i, \mathsf{msg}_i)$

if $\mathsf{status}_\gamma \notin \{\bot, \top\}$:

  $(\mathsf{st}_{\mathsf{Sim}_\gamma}, \mathsf{status}_\gamma, \mathsf{msg}_\gamma) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{SEval}^{\textsc{ModEv}^{\textsc{LimEv}(\cdot)}_{\mathsf{state}_{\mathsf{Sim}}}(\cdot)}(\mathrm{ssid}, \mathsf{msg}_\gamma, \mathsf{st}_{\mathsf{Sim}_\gamma})$

$\mathsf{st}_{\mathsf{exec}}[\mathrm{ssid}] \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$

$\mathsf{state}'_{\mathsf{Sim}} \leftarrow (\zeta, \boldsymbol{k}, \mathbf{st_P}, \mathsf{st}_{\mathsf{exec}}, \mathsf{st}_{\mathsf{Sim}_\gamma})$

if $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: **return** $(\mathsf{state}'_{\mathsf{Sim}}, \bot, \textsc{Nil})$

if $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: **return** $(\mathsf{state}'_{\mathsf{Sim}}, \top, (\mathsf{msg}_1, \ldots, \mathsf{msg}_n))$

return $(\mathsf{state}'_{\mathsf{Sim}}, \textsc{Nil}, (\mathsf{msg}_1, \ldots, \mathsf{msg}_n))$

$\mathsf{Sim}.\mathsf{PrimEv}^{\textsc{LimEv}(\cdot)}(\mathsf{state}_{\mathsf{Sim}}, (u, x))$

parse $\mathsf{state}_{\mathsf{Sim}} = (\zeta, \boldsymbol{k}, \mathbf{st_P}, \mathsf{st}_{\mathsf{exec}}, \mathsf{st}_{\mathsf{Sim}_\gamma})$; $y \leftarrow \textsc{Nil}$

if $u \neq \gamma$: $(\mathbf{st_P}[u], y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathbf{st_P}[u], x)$

else $(\mathsf{st}_{\mathsf{Sim}_\gamma}, y) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{PrimEv}^{\textsc{ModEv}^{\textsc{LimEv}(\cdot)}_{\mathsf{state}_{\mathsf{Sim}}}(\cdot)}(\mathsf{st}_{\mathsf{Sim}_\gamma}, x)$

$\mathsf{state}'_{\mathsf{Sim}} \leftarrow (\zeta, \boldsymbol{k}, \mathbf{st_P}, \mathsf{st}_{\mathsf{exec}}, \mathsf{st}_{\mathsf{Sim}_\gamma})$; **return** $(\mathsf{state}'_{\mathsf{Sim}}, y)$

Fig. 88. Simulator for the XOR OPRF combiner.

Game $\mathbf{G}_4$ (Fig. 93): Simulating F_γ. We replace the non-interactive evaluation of F_γ in the Ev oracle with the evaluation of a truly random function f. At the same time, we replace the evaluation of the ideal primitive associated with F_γ in the Prim oracle as well as the server-side algorithm for the interactive evaluation of F_γ in the Srv oracle with their respective simulation using Sim_γ. The simulator Sim_γ has limited access to the Ev oracle through the LimEv oracle, as in the $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}_\gamma,\mathcal{B},\mathsf{Sim}_\gamma}(\lambda)$ experiment. Indeed, $\mathbf{G}_4$ is equivalent to the $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}_\gamma,\mathcal{B},\mathsf{Sim}_\gamma}(\lambda)$ experiment with adversary $\mathcal{B}$. If $\mathcal{A}$ can detect a difference between the previous game and the current one, then $\mathcal{B}$ allows us to turn this ability into an advantage in the $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}_\gamma,\mathcal{B},\mathsf{Sim}_\gamma}(\lambda)$ experiment. Thus, we have

$$|\Pr[\mathbf{G}_3(\lambda)=1]-\Pr[\mathbf{G}_4(\lambda)=1]| \le \mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F}_\gamma,\mathcal{B},\mathsf{Sim}_\gamma}(\lambda)\,.$$

Game $\mathbf{G}_5$ (Fig. 94): Rearranging the game again. Next, we move the evaluation of all candidate schemes except for F_γ back into the challenger again and get rid of $\mathcal{B}$. That is, we move the code of $\mathcal{B}$ and its associated oracles into the challenger and replace the invocation of $\mathcal{B}$, which then passes all values as is to $\mathcal{A}$, with a direct invocation of $\mathcal{A}$. This allows us to get rid of Srv′ and Prim′, as these oracles would simply pass their queries to the respective non-prime counterparts. Only the oracle Ev′ is kept, as it XORs the responses of Ev with the outputs of all non-γ candidate OPRFs. All of these modifications are solely a matter of notation, so we have

$$\Pr[\mathbf{G}_4(\lambda)=1]=\Pr[\mathbf{G}_5(\lambda)=1]\,.$$

Game $\mathbf{G}_6$ (Fig. 95): Reinterpreting the challenger. We perform another purely syntactical modification: We XOR the output of the Ev oracle with $\bigoplus_{i\in[n]\setminus\{\gamma\}}\mathsf{F}_i.\mathsf{Eval}(\zeta'[i],k'[i],x)$. Additionally, we insert an XOR with the same value at every call site of Ev. Due to the involution of the XOR function, this change has no effect on the view of $\mathcal{A}$.

$$\Pr[\mathbf{G}_5(\lambda)=1]=\Pr[\mathbf{G}_6(\lambda)=1]\,.$$

The Ev oracle is queried in two places: Firstly, in the Ev′ oracle and secondly, in the LimEv oracle. In the Ev′ oracle, the added XOR cancels out with the existing XOR in that oracle, so afterwards, Ev′ becomes a pass-through oracle and we can remove it. Regarding the second case, since the LimEv oracle only implements an invocation limit, we can modify its caller, the $\mathsf{Sim}_\gamma.\mathsf{SEval}$ algorithm, which is called in the Srv oracle, instead. To do so, we add the ModEv oracle, which performs the required XOR operation, and plug it between $\mathsf{Sim}_\gamma.\mathsf{SEval}$ and LimEval.

Game $\mathbf{G}_7$ (Fig. 96): Renaming the random function f Finally, we change the implementation of Ev to return $f(x)$ instead of $f(x)\oplus\bigoplus_{i\in[n]\setminus\{\gamma\}}\mathsf{F}_i.\mathsf{Eval}(\zeta'[i],k'[i],x)$. Since the random function f is only evaluated in this oracle and independently chosen from the rest of the game, the random variables $f(x)$ and $f(x)\oplus\bigoplus_{i\in[n]\setminus\{\gamma\}}\mathsf{F}_i.\mathsf{Eval}(\zeta'[i],k'[i],x)$ are identically distributed and thus perfectly indistinguishable. Hence, we have

$$\Pr[\mathbf{G}_6(\lambda)=1]=\Pr[\mathbf{G}_7(\lambda)=1]\,.$$

We then observe that the challenger in $\mathbf{G}_7$ contains an expanded version of Sim as defined in Fig. 88. By collapsing the algorithms of Sim, we thus obtain $\mathbf{G}_8$.

Game $\mathbf{G}_8$ (Fig. 97): Final game. Using a telescoping sum, we arrive at

$$\begin{aligned}\mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim}}(\lambda) &\coloneqq \left|\Pr[\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim},0}(\lambda)=1]-\Pr[\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim},1}(\lambda)=1]\right|\\ &\le \mathsf{Adv}^{\text{oprf-psr}}_{\mathsf{F}_\gamma,\mathcal{A}_\gamma,\mathsf{Sim}}(\lambda)\le \mathsf{negl}_\gamma(\lambda)\,,\end{aligned}$$

which concludes the proof. □

$\mathbf{G}_0(\lambda)$

271 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
272 : $\zeta \leftarrow\!\!\$\ \mathsf{F}'.\mathsf{Setup}(1^\lambda)$
273 : $\mathsf{state}_\mathrm{P} \leftarrow\!\!\$\ \mathrm{P}.\mathsf{Init}(1^\lambda)$
274 : $k \leftarrow\!\!\$\ \mathsf{F}'.\mathsf{KGen}(\zeta)$
275 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$
276 : **return** b'

$\textsc{Ev}(x)$

281 : **return** $\mathsf{F}'.\mathsf{Eval}(\zeta, k, x)$

$\textsc{Prim}(x)$

291 : $(\mathsf{state}_\mathrm{P}, y) \leftarrow\!\!\$\ \mathrm{P}.\mathsf{Eval}(\mathsf{state}_\mathrm{P}, x)$
292 : **return** y

$\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

301 : $\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil}$
302 : $\mathsf{resp} \leftarrow \textsc{Nil}$
303 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$
304 : **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
305 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
306 : $(\mathsf{state}, \mathsf{status}, \mathsf{resp})$
$\quad \leftarrow\!\!\$\ \mathsf{F}'.\mathsf{SEval}(\zeta, k, \mathsf{state}, \mathsf{msg})$
307 : $\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}, \mathsf{status})$
308 : **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$
309 : **return** $(\mathsf{status}, \mathsf{resp})$

Fig. 89. Game $\mathbf{G}_0$: The original game, equivalent to $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim},0}(\lambda)$.

$\mathbf{G}_1(\lambda)$

311 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
312 : $\zeta \leftarrow [];\ k \leftarrow [];\ \mathsf{state}_\mathrm{P} \leftarrow []$
313 : **for** $i \in [n]$
314 : $\quad \zeta[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{Setup}(1^\lambda)$
315 : $\quad \mathsf{state}_\mathrm{P}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$
316 : $\quad k[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta)$
317 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$
318 : **return** b'

$\textsc{Ev}(x)$

321 : $y \leftarrow \bigoplus_{i=1}^{n} \mathsf{F}_i.\mathsf{Eval}(\zeta[i], k[i], x)$
322 : **return** y

$\textsc{Prim}(v)$

331 : **parse** $v = (u, x)$
332 : $(\mathsf{state}_\mathrm{P}[u], y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}_\mathrm{P}[u], x)$
333 : **return** y

$\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

341 : $\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
342 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
343 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
344 : **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
345 : $(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$
346 : **if** $\mathsf{state} \neq \textsc{Nil}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
347 : **if** $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \textsc{Nil}\} \neq \emptyset$:
348 : $\quad \mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \bot;\ \mathsf{resp} \leftarrow \textsc{Nil}$
349 : **else**
350 : $\quad$ **for** $i \in \{j \in [n] \mid \mathsf{status}_j \notin \{\bot, \top\}\}$
351 : $\quad\quad (\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta_i, k_i, \mathsf{state}_i, \mathsf{msg}_i)$
352 : $\quad$ **if** $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: $(\mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \bot;\ \mathsf{resp} \leftarrow \textsc{Nil})$
353 : $\quad$ **elseif** $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: $(\mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \top;\ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n))$
354 : $\quad$ **else**
355 : $\quad\quad \mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
356 : $\quad\quad \mathsf{status}' \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
357 : $\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}', \mathsf{status}')$
358 : **if** $\mathsf{status}' \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 90. Game $\mathbf{G}_1$: Expanding $\mathsf{F}'.\mathsf{Setup}$, $\mathsf{F}'.\mathsf{KGen}$, $\mathsf{F}'.\mathsf{SEval}$ and $\mathsf{F}'.\mathsf{Eval}$.

$\mathbf{G}_2(\lambda)$

361 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
362 : $\zeta \leftarrow [];\ k \leftarrow [];\ \mathsf{state}_{\mathrm{P}} \leftarrow []$
363 : **for** $i \in [n]$
364 : $\quad \zeta[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{Setup}(1^\lambda)$
365 : $\quad \mathsf{state}_{\mathrm{P}}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$
366 : $\quad k[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta)$
367 : $b' \leftarrow\!\!\$\ \mathcal{B}^{\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$
368 : **return** b'

$\textsc{Ev}(x)$

371 : $y \leftarrow \bigoplus_{i=1}^{n} \mathsf{F}_i.\mathsf{Eval}(\zeta[i], k[i], x)$
372 : **return** y

$\textsc{Prim}$

381 : **parse** $v = (u, x)$
382 : $(\mathsf{state}_{\mathrm{P}}[u], y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}_{\mathrm{P}}[u], x)$
383 : **return** y

$\mathcal{B}^{\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$

391 : **return** $\mathcal{A}^{\textsc{Ev}'^{\textsc{Ev}(\cdot)}(\cdot),\textsc{Srv}'^{\textsc{Srv}(\cdot,\cdot)}(\cdot,\cdot),\textsc{Prim}'^{\textsc{Prim}(\cdot)}(\cdot)}(1^\lambda, \zeta)$

$\textsc{Ev}'^{\textsc{Ev}(\cdot)}(x)$

401 : **return** $\textsc{Ev}(x)$

$\textsc{Srv}'^{\textsc{Srv}(\cdot,\cdot)}(\mathsf{ssid}, \mathsf{msg})$

411 : **return** $\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

$\textsc{Prim}'^{\textsc{Prim}(\cdot)}(x)$

421 : **return** $\textsc{Prim}(x)$

$\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

431 : $\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
432 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathrm{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
433 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
434 : **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
435 : $(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$
436 : **if** $\mathsf{state} \neq \textsc{Nil}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
437 : **if** $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \textsc{Nil}\} \neq \emptyset$:
438 : $\quad \mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \bot;\ \mathsf{resp} \leftarrow \textsc{Nil}$
439 : **else**
440 : $\quad$ **for** $i \in \{j \in [n] \mid \mathsf{status}_j \notin \{\bot, \top\}\}$
441 : $\quad\quad (\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta_i, k_i, \mathsf{state}_i, \mathsf{msg}_i)$
442 : $\quad$ **if** $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \bot;\ \mathsf{resp} \leftarrow \textsc{Nil}\big)$
443 : $\quad$ **elseif** $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \top;\ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)\big)$
444 : $\quad$ **else**
445 : $\quad\quad \mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
446 : $\quad\quad \mathsf{status}' \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
447 : $\mathsf{st}[\mathrm{ssid}] \leftarrow (\mathsf{state}', \mathsf{status}');$ **if** $\mathsf{status}' \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 91. Game $\mathbf{G}_2$: Wrapping the adversary $\mathcal{A}$ inside of an outer adversary $\mathcal{B}$

$\mathcal{B}^{\text{Ev}(\cdot),\text{Srv}(\cdot,\cdot),\text{Prim}(\cdot)}(1^\lambda,\zeta)$

451 : $\zeta' \leftarrow []$; $k' \leftarrow []$; $\mathsf{state}'_{\mathrm{P}} \leftarrow []$
452 : $\zeta'[\gamma] \leftarrow \text{Nil}$; $k'[\gamma] \leftarrow \text{Nil}$
453 : $\mathsf{state}'_{\mathrm{P}}[\gamma] \leftarrow \text{Nil}$; $\mathsf{st}' \leftarrow \{\}$
454 : **for** $i \in [n] \setminus \{\gamma\}$
455 : $\quad \zeta'[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{Setup}(1^\lambda)$
456 : $\quad \mathsf{state}'_{\mathrm{P}}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$
457 : $\quad k'[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta)$
458 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\text{Ev}'^{\text{Ev}(\cdot)}(\cdot),\text{Srv}'^{\text{Srv}(\cdot,\cdot)}(\cdot,\cdot),\text{Prim}'^{\text{Prim}(\cdot)}(\cdot)}(1^\lambda,\zeta)$
459 : **return** b'

$\mathbf{G}_3(\lambda)$

461 : $\mathsf{tx} \leftarrow 0$; $\mathsf{st} \leftarrow \{\}$
462 : $\zeta_\gamma \leftarrow\!\!\$\ \mathsf{F}_\gamma.\mathsf{Setup}(1^\lambda)$
463 : $\mathsf{state}_{\mathrm{P}_\gamma} \leftarrow\!\!\$\ \mathrm{P}_\gamma.\mathsf{Init}(1^\lambda)$
464 : $k_\gamma \leftarrow\!\!\$\ \mathsf{F}_\gamma.\mathsf{KGen}(\zeta)$
465 : $b' \leftarrow\!\!\$\ \mathcal{B}^{\text{Ev}(\cdot),\text{Srv}(\cdot,\cdot),\text{Prim}(\cdot)}(1^\lambda,\zeta)$
466 : **return** b'

$\text{Ev}(x)$

471 : $y \leftarrow \mathsf{F}_\gamma.\mathsf{Eval}(\zeta_\gamma, k_\gamma, x)$
472 : **return** y

$\text{Ev}'^{\text{Ev}(\cdot)}(x)$

481 : $y \leftarrow \bigoplus_{i\in[n]\setminus\{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta'[i], k'[i], x)$
482 : **return** $y \oplus \text{Ev}(x)$

$\text{Prim}(x)$

491 : $(\mathsf{state}_{\mathrm{P}_\gamma}, y) \leftarrow\!\!\$\ \mathrm{P}_\gamma.\mathsf{Eval}(\mathsf{state}_{\mathrm{P}_\gamma}, x)$
492 : **return** y

$\text{Prim}'^{\text{Prim}(\cdot)}(v)$

501 : **parse** $v = (u, x)$
502 : **if** $u \stackrel{?}{=} \gamma$: **return** $\text{Prim}(x)$
503 : $(\mathsf{state}'_{\mathrm{P}}[u], y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}'_{\mathrm{P}}[u], x)$
504 : **return** y

$\text{Srv}'^{\text{Srv}(\cdot,\cdot)}(\text{ssid}, \mathsf{msg})$

511 : $\mathsf{state} \leftarrow \text{Nil}$; $\mathsf{status} \leftarrow \text{Nil}$; $\mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \text{Nil}$; $\mathsf{resp} \leftarrow \text{Nil}$
512 : **if** ssid **in** st': $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}'[\text{ssid}]$
513 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \text{Nil})$
514 : **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
515 : $(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\text{Nil}, \ldots, \text{Nil})$
516 : **if** $\mathsf{state} \neq \text{Nil}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
517 : **if** $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \text{Nil}\} \neq \emptyset$:
518 : $\quad \mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \bot$; $\mathsf{resp} \leftarrow \text{Nil}$
519 : **else**
520 : $\quad$ **for** $i \in \{j \in [n]\setminus\{\gamma\} \mid \mathsf{status}_j \notin \{\bot, \top\}\}$
521 : $\quad\quad (\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta'_i, k'_i, \mathsf{state}_i, \mathsf{msg}_i)$
522 : $\quad$ **if** $\mathsf{status}_\gamma \notin \{\bot, \top\}$: $(\mathsf{status}_\gamma, \mathsf{msg}_\gamma) \leftarrow\!\!\$\ \text{Srv}(\text{ssid}, \mathsf{msg}_\gamma)$;
523 : $\quad$ **if** $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: $\big(\mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \bot$; $\mathsf{resp} \leftarrow \text{Nil}\big)$
524 : $\quad$ **elseif** $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: $\big(\mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \top$; $\mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)\big)$
525 : $\quad$ **else**
526 : $\quad\quad \mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
527 : $\quad\quad \mathsf{status}' \leftarrow \text{Nil}$; $\mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
528 : $\mathsf{st}'[\text{ssid}] \leftarrow (\mathsf{state}', \mathsf{status}')$; **if** $\mathsf{status}' \stackrel{?}{=} \bot$: **return** $(\bot, \text{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

$\text{Srv}(\text{ssid}, \mathsf{msg})$

531 : $\mathsf{state} \leftarrow \text{Nil}$; $\mathsf{status} \leftarrow \text{Nil}$; $\mathsf{resp} \leftarrow \text{Nil}$
532 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\text{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
533 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \text{Nil})$
534 : $(\mathsf{state}, \mathsf{status}, \mathsf{resp}) \leftarrow\!\!\$\ \mathsf{F}_\gamma.\mathsf{SEval}(\zeta_\gamma, k_\gamma, \mathsf{state}, \mathsf{msg})$
535 : $\mathsf{st}[\text{ssid}] \leftarrow (\mathsf{state}, \mathsf{status})$; **if** $\mathsf{status} \stackrel{?}{=} \bot$: **return** $(\bot, \text{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 92. Game $\mathbf{G}_3$: Moving the evaluation of all candidate schemes except F_γ into $\mathcal{B}$

$\mathcal{B}^{\text{Ev}(\cdot),\text{Srv}(\cdot,\cdot),\text{Prim}(\cdot)}(1^\lambda,\zeta)$

541 : $\zeta' \leftarrow []$; $k' \leftarrow []$; $\mathsf{state}'_{\mathrm{P}} \leftarrow []$
542 : $\zeta'[\gamma] \leftarrow \text{Nil}$; $k'[\gamma] \leftarrow \text{Nil}$
543 : $\mathsf{state}'_{\mathrm{P}}[\gamma] \leftarrow \text{Nil}$; $\mathsf{st}' \leftarrow \{\}$
544 : **for** $i \in [n] \setminus \{\gamma\}$
545 : $\quad \zeta'[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{Setup}(1^\lambda)$
546 : $\quad \mathsf{state}'_{\mathrm{P}}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$
547 : $\quad k'[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta)$
548 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\text{Ev}'^{\text{Ev}(\cdot)}(\cdot),\text{Srv}'^{\text{Srv}(\cdot,\cdot)}(\cdot,\cdot),\text{Prim}'^{\text{Prim}(\cdot)}(\cdot)}(1^\lambda,\zeta)$
549 : **return** b'

$\mathbf{G}_4(\lambda)$

551 : $\mathsf{tx} \leftarrow 0$; $\mathsf{st} \leftarrow \{\}$
552 : $\zeta_\gamma \leftarrow\!\!\$\ \mathsf{F}_\gamma.\mathsf{Setup}(1^\lambda)$
553 : $f \leftarrow\!\!\$\ (\mathcal{X}_{\lambda,\zeta} \to \mathcal{Y}_{\lambda,\zeta})$
554 : $\mathsf{state}_{\mathsf{Sim}_\gamma} \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{Init}(1^\lambda,\zeta)$
555 : $b' \leftarrow\!\!\$\ \mathcal{B}^{\text{Ev}(\cdot),\text{Srv}(\cdot,\cdot),\text{Prim}(\cdot)}(1^\lambda,\zeta)$
556 : **return** b'

$\text{LimEv}(x)$

561 : **if** $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
562 : $\mathsf{tx} \leftarrow \mathsf{tx} - 1$
563 : **return** $\text{Ev}(x)$

$\text{Ev}(x)$

571 : $y \leftarrow f(x)$
572 : **return** y

$\text{Ev}'^{\text{Ev}(\cdot)}(x)$

581 : $y \leftarrow \bigoplus_{i\in[n]\setminus\{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta'[i],k'[i],x)$
582 : **return** $y \oplus \text{Ev}(x)$

$\text{Prim}(x)$

591 : $(\mathsf{state}_{\mathsf{Sim}_\gamma}, y) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{PrimEv}^{\text{LimEv}(\cdot)}(\mathsf{state}_{\mathsf{Sim}_\gamma}, x)$
592 : **return** y

$\text{Prim}'^{\text{Prim}(\cdot)}(v)$

601 : **parse** $v = (u, x)$
602 : **if** $u \stackrel{?}{=} \gamma$: **return** $\text{Prim}(x)$
603 : $(\mathsf{state}'_{\mathrm{P}}[u], y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}'_{\mathrm{P}}[u], x)$
604 : **return** y

$\text{Srv}'^{\text{Srv}(\cdot,\cdot)}(\text{ssid}, \mathsf{msg})$

611 : $\mathsf{state} \leftarrow \text{Nil}$; $\mathsf{status} \leftarrow \text{Nil}$; $\mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \text{Nil}$; $\mathsf{resp} \leftarrow \text{Nil}$
612 : **if** ssid **in** st': $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}'[\text{ssid}]$
613 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \text{Nil})$
614 : **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
615 : $(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\text{Nil}, \ldots, \text{Nil})$
616 : **if** $\mathsf{state} \neq \text{Nil}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
617 : **if** $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \text{Nil}\} \neq \emptyset$:
618 : $\quad \mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \bot$; $\mathsf{resp} \leftarrow \text{Nil}$
619 : **else**
620 : $\quad$ **for** $i \in \{j \in [n] \setminus \{\gamma\} \mid \mathsf{status}_j \notin \{\bot, \top\}\}$
621 : $\quad\quad (\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta'_i, k'_i, \mathsf{state}_i, \mathsf{msg}_i)$
622 : $\quad$ **if** $\mathsf{status}_\gamma \notin \{\bot, \top\}$: $(\mathsf{status}_\gamma, \mathsf{msg}_\gamma) \leftarrow\!\!\$\ \text{Srv}(\text{ssid}, \mathsf{msg}_\gamma)$;
623 : $\quad$ **if** $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: $\big(\mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \bot$; $\mathsf{resp} \leftarrow \text{Nil}\big)$
624 : $\quad$ **elseif** $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: $\big(\mathsf{state}' \leftarrow \text{Nil}$; $\mathsf{status}' \leftarrow \top$; $\mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)\big)$
625 : $\quad$ **else**
626 : $\quad\quad \mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
627 : $\quad\quad \mathsf{status}' \leftarrow \text{Nil}$; $\mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
628 : $\mathsf{st}'[\text{ssid}] \leftarrow (\mathsf{state}', \mathsf{status}')$; **if** $\mathsf{status}' \stackrel{?}{=} \bot$: **return** $(\bot, \text{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

$\text{Srv}(\text{ssid}, \mathsf{msg})$

631 : $\mathsf{state} \leftarrow \text{Nil}$; $\mathsf{status} \leftarrow \text{Nil}$; $\mathsf{resp} \leftarrow \text{Nil}$
632 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\text{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
633 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $\bot$
634 : $(\mathsf{state}_{\mathsf{Sim}_\gamma}, \mathsf{status}, \mathsf{resp}) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{SEval}^{\text{LimEv}(\cdot)}(\text{ssid}, \mathsf{msg}, \mathsf{state}_{\mathsf{Sim}_\gamma})$
635 : $\mathsf{st}[\text{ssid}] \leftarrow (\mathsf{state}, \mathsf{status})$; **if** $\mathsf{state} \stackrel{?}{=} \bot$: **return** $\bot$ **else return** resp

Fig. 93. Game $\mathbf{G}_4$: Simulating F_γ

$\mathbf{G}_5(\lambda)$

641 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
642 : $\zeta_\gamma \leftarrow\!\!\$\ \mathsf{F}_\gamma.\mathsf{Setup}(1^\lambda)$
643 : $f \leftarrow\!\!\$\ (\mathcal{X}_{\lambda,\zeta} \to \mathcal{Y}_{\lambda,\zeta})$
644 : $\mathsf{state}_{\mathsf{Sim}_\gamma} \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{Init}(1^\lambda, \zeta)$
645 : $\zeta \leftarrow []; \ k \leftarrow []; \ \mathsf{state}_\mathrm{P} \leftarrow []$
646 : $\zeta[\gamma] \leftarrow \textsc{Nil}; \ k[\gamma] \leftarrow \textsc{Nil}$
647 : $\mathsf{state}_\mathrm{P}[\gamma] \leftarrow \textsc{Nil}$
648 : **for** $i \in [n] \setminus \{\gamma\}$
649 : $\quad \zeta[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{Setup}(1^\lambda)$
650 : $\quad \mathsf{state}_\mathrm{P}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$
651 : $\quad k[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta)$
652 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\textsc{Ev}'^{\textsc{Ev}(\cdot)}(\cdot), \textsc{Srv}(\cdot,\cdot), \textsc{Prim}(\cdot)}(1^\lambda, \zeta)$
653 : **return** b'

$\textsc{Ev}'^{\textsc{Ev}(\cdot)}(x)$

661 : $y \leftarrow \bigoplus_{i \in [n] \setminus \{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta[i], k[i], x)$
662 : **return** $y \oplus \textsc{Ev}(x)$

$\textsc{Prim}(x)$

671 : **parse** $v = (u, x)$;
672 : **if** $u \neq \gamma$:
673 : $\quad (\mathsf{state}_\mathrm{P}, y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}_\mathrm{P}[u], x)$
674 : $\quad$ **return** y
675 : $(\mathsf{state}_{\mathsf{Sim}_\gamma}, y) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{PrimEv}^{\textsc{LimEv}(\cdot)}(\mathsf{state}_{\mathsf{Sim}_\gamma}, x)$
676 : **return** y

$\textsc{LimEv}(x)$

681 : **if** $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\perp$
682 : $\mathsf{tx} \leftarrow \mathsf{tx} - 1$
683 : **return** $\textsc{Ev}(x)$

$\textsc{Ev}(x)$

691 : **return** $f(x)$

$\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

701 : $\mathsf{state} \leftarrow \textsc{Nil}; \ \mathsf{status} \leftarrow \textsc{Nil}; \ \mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \textsc{Nil}; \ \mathsf{resp} \leftarrow \textsc{Nil}$
702 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
703 : **if** $\mathsf{status} \in \{\perp, \top\}$: **return** $(\perp, \textsc{Nil})$
704 : **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
705 : $(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$
706 : **if** $\mathsf{state} \neq \textsc{Nil}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
707 : **if** $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \textsc{Nil}\} \neq \emptyset$:
708 : $\quad \mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \perp; \ \mathsf{resp} \leftarrow \textsc{Nil}$
709 : **else**
710 : $\quad$ **for** $i \in \{j \in [n] \setminus \{\gamma\} \mid \mathsf{status}_j \notin \{\perp, \top\}\}$
711 : $\quad\quad (\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta_i, k_i, \mathsf{state}_i, \mathsf{msg}_i)$
712 : $\quad$ **if** $\mathsf{status}_\gamma \notin \{\perp, \top\}$: $(\mathsf{state}_{\mathsf{Sim}_\gamma}, \mathsf{status}_\gamma, \mathsf{msg}_\gamma) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{SEval}^{\textsc{LimEv}(\cdot)}(\mathsf{ssid}, \mathsf{msg}_\gamma, \mathsf{state}_{\mathsf{Sim}_\gamma})$
713 : $\quad$ **if** $\perp \in \{\mathsf{status}_i \mid i \in [n]\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \perp; \ \mathsf{resp} \leftarrow \textsc{Nil}\big)$
714 : $\quad$ **elseif** $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \top; \ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)\big)$
715 : $\quad$ **else**
716 : $\quad\quad \mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
717 : $\quad\quad \mathsf{status}' \leftarrow \textsc{Nil}; \ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
718 : $\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}', \mathsf{status}'); \ \mathbf{if}\ \mathsf{status}' \stackrel{?}{=} \perp$: **return** $(\perp, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 94. Game $\mathbf{G}_5$: Moving the evaluation of all candidate schemes back into the challenger and removing $\mathcal{B}$

$\mathbf{G}_6(\lambda)$

721 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
722 : $\zeta_\gamma \leftarrow\!\!\$\ \mathsf{F}_\gamma.\mathsf{Setup}(1^\lambda)$
723 : $f \leftarrow\!\!\$\ (\mathcal{X}_{\lambda,\zeta} \to \mathcal{Y}_{\lambda,\zeta})$
724 : $\mathsf{state}_{\mathsf{Sim}_\gamma} \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{Init}(1^\lambda, \zeta)$
725 : $\zeta \leftarrow [];\ k \leftarrow [];\ \mathsf{state}_{\mathrm{P}} \leftarrow []$
726 : $\zeta[\gamma] \leftarrow \textsc{Nil};\ k[\gamma] \leftarrow \textsc{Nil}$
727 : $\mathsf{state}_{\mathrm{P}}[\gamma] \leftarrow \textsc{Nil}$
728 : **for** $i \in [n] \setminus \{\gamma\}$
729 : $\quad \zeta[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{Setup}(1^\lambda)$
730 : $\quad \mathsf{state}_{\mathrm{P}}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$
731 : $\quad k[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta)$
732 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$
733 : **return** b'

$\textsc{Prim}(x)$

741 : **parse** $v = (u, x)$;
742 : **if** $u \neq \gamma$:
743 : $\quad (\mathsf{state}_{\mathrm{P}}, y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}_{\mathrm{P}}[u], x)$
744 : $\quad$ **return** y
745 : $(\mathsf{state}_{\mathsf{Sim}_\gamma}, y) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{PrimEv}^{\textsc{ModEv}^{\textsc{LimEv}(\cdot)}(\cdot)}(\mathsf{state}_{\mathsf{Sim}_\gamma}, x)$
746 : **return** y

$\textsc{Ev}(x)$

751 : $y \leftarrow \bigoplus_{i \in [n]\setminus\{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta'[i], k'[i], x)$
752 : **return** $y \oplus f(x)$

$\textsc{LimEv}(x)$

761 : **if** $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
762 : $\mathsf{tx} \leftarrow \mathsf{tx} - 1$
763 : **return** $\textsc{Ev}(x)$

$\textsc{ModEv}^{\textsc{LimEv}(\cdot)}(x)$

771 : **return** $\textsc{LimEv}(x) \oplus \bigoplus_{i \in [n]\setminus\{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta_i, k[i], x)$

$\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

781 : $\mathsf{state} \leftarrow \textsc{Nil};\ \mathsf{status} \leftarrow \textsc{Nil};\ \mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow \textsc{Nil}$
782 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathrm{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
783 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
784 : **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
785 : $(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$
786 : **if** $\mathsf{state} \neq \textsc{Nil}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
787 : **if** $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \textsc{Nil}\} \neq \emptyset$:
788 : $\quad \mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \bot;\ \mathsf{resp} \leftarrow \textsc{Nil}$
789 : **else**
790 : $\quad$ **for** $i \in \{j \in [n] \setminus \{\gamma\} \mid \mathsf{status}_j \notin \{\bot, \top\}\}$
791 : $\quad\quad (\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta_i, k_i, \mathsf{state}_i, \mathsf{msg}_i)$
792 : $\quad$ **if** $\mathsf{status}_\gamma \notin \{\bot, \top\}$: $(\mathsf{state}_{\mathsf{Sim}_\gamma}, \mathsf{status}_\gamma, \mathsf{msg}_\gamma) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{SEval}^{\textsc{ModEv}^{\textsc{LimEv}(\cdot)}(\cdot)}(\mathrm{ssid}, \mathsf{msg}_\gamma, \mathsf{state}_{\mathsf{Sim}_\gamma})$
793 : $\quad$ **if** $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \bot;\ \mathsf{resp} \leftarrow \textsc{Nil}\big)$
794 : $\quad$ **elseif** $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil};\ \mathsf{status}' \leftarrow \top;\ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)\big)$
795 : $\quad$ **else**
796 : $\quad\quad \mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
797 : $\quad\quad \mathsf{status}' \leftarrow \textsc{Nil};\ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
798 : $\mathsf{st}[\mathrm{ssid}] \leftarrow (\mathsf{state}', \mathsf{status}');$ **if** $\mathsf{status}' \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 95. Game $\mathbf{G}_6$: Moving the XOR with the outputs of the candidate schemes other than F_γ into the Ev oracle

$\mathbf{G}_7(\lambda)$

801 : $\mathsf{tx} \leftarrow 0;\ \mathsf{st} \leftarrow \{\}$
802 : $\zeta_\gamma \leftarrow\!\!\$\ \mathsf{F}_\gamma.\mathsf{Setup}(1^\lambda)$
803 : $f \leftarrow\!\!\$\ (\mathcal{X}_{\lambda,\zeta} \to \mathcal{Y}_{\lambda,\zeta})$
804 : $\mathsf{state}_{\mathsf{Sim}_\gamma} \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{Init}(1^\lambda, \zeta)$
805 : $\zeta \leftarrow []; \ k \leftarrow []; \ \mathsf{state}_\mathrm{P} \leftarrow []$
806 : $\zeta[\gamma] \leftarrow \textsc{Nil}; \ k[\gamma] \leftarrow \textsc{Nil}$
807 : $\mathsf{state}_\mathrm{P}[\gamma] \leftarrow \textsc{Nil}$
808 : **for** $i \in [n] \setminus \{\gamma\}$
809 : $\quad \zeta[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{Setup}(1^\lambda)$
810 : $\quad \mathsf{state}_\mathrm{P}[i] \leftarrow\!\!\$\ \mathrm{P}_i.\mathsf{Init}(1^\lambda)$
811 : $\quad k[i] \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{KGen}(\zeta)$
812 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\textsc{Ev}(\cdot),\textsc{Srv}(\cdot,\cdot),\textsc{Prim}(\cdot)}(1^\lambda, \zeta)$
813 : **return** b'

$\textsc{Prim}(x)$

821 : **parse** $v = (u, x); \ y \leftarrow \textsc{Nil}$
822 : **if** $u \neq \gamma$:
823 : $\quad (\mathsf{state}_\mathrm{P}, y) \leftarrow\!\!\$\ \mathrm{P}_u.\mathsf{Eval}(\mathsf{state}_\mathrm{P}[u], x)$
824 : $\quad$ **return** y
825 : $(\mathsf{state}_{\mathsf{Sim}_\gamma}, y) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{PrimEv}^{\textsc{ModEv}^{\textsc{LimEv}(\cdot)}(\cdot)}(\mathsf{state}_{\mathsf{Sim}_\gamma}, x)$
826 : **return** y

$\textsc{LimEv}(x)$

831 : **if** $\mathsf{tx} \stackrel{?}{=} 0$: **return** $\bot$
832 : $\mathsf{tx} \leftarrow \mathsf{tx} - 1$
833 : **return** $\textsc{Ev}(x)$

$\textsc{Ev}(x)$

841 : **return** $f(x)$

$\textsc{ModEv}^{\textsc{LimEv}(\cdot)}(x)$

851 : **return** $\textsc{LimEv}(x) \oplus \bigoplus_{i \in [n] \setminus \{\gamma\}} \mathsf{F}_i.\mathsf{Eval}(\zeta_i, k[i], x)$

$\textsc{Srv}(\mathsf{ssid}, \mathsf{msg})$

861 : $\mathsf{state} \leftarrow \textsc{Nil}; \ \mathsf{status} \leftarrow \textsc{Nil}; \ \mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \textsc{Nil}; \ \mathsf{resp} \leftarrow \textsc{Nil}$
862 : **if** ssid **in** st: $(\mathsf{state}, \mathsf{status}) \leftarrow \mathsf{st}[\mathsf{ssid}]$ **else** $\mathsf{tx} \leftarrow \mathsf{tx} + 1$
863 : **if** $\mathsf{status} \in \{\bot, \top\}$: **return** $(\bot, \textsc{Nil})$
864 : **parse** $\mathsf{msg} = (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
865 : $(\mathsf{state}_1, \ldots, \mathsf{state}_n, \mathsf{status}_1, \ldots, \mathsf{status}_n) \leftarrow (\textsc{Nil}, \ldots, \textsc{Nil})$
866 : **if** $\mathsf{state} \neq \textsc{Nil}$: **parse** $\mathsf{state} = ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
867 : **if** $\{i \in [n] \mid \mathsf{status}_i \stackrel{?}{=} \top\} \cap \{i \in [n] \mid \mathsf{msg}_i \neq \textsc{Nil}\} \neq \emptyset$:
868 : $\quad \mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \bot; \ \mathsf{resp} \leftarrow \textsc{Nil}$
869 : **else**
870 : $\quad$ **for** $i \in \{j \in [n] \setminus \{\gamma\} \mid \mathsf{status}_j \notin \{\bot, \top\}\}$
871 : $\quad\quad (\mathsf{state}_i, \mathsf{status}_i, \mathsf{msg}_i) \leftarrow\!\!\$\ \mathsf{F}_i.\mathsf{SEval}(\zeta_i, k_i, \mathsf{state}_i, \mathsf{msg}_i)$
872 : $\quad$ **if** $\mathsf{status}_\gamma \notin \{\bot, \top\}$: $(\mathsf{state}_{\mathsf{Sim}_\gamma}, \mathsf{status}_\gamma, \mathsf{msg}_\gamma) \leftarrow\!\!\$\ \mathsf{Sim}_\gamma.\mathsf{SEval}^{\textsc{ModEv}^{\textsc{LimEv}(\cdot)}(\cdot)}(\mathsf{ssid}, \mathsf{msg}_\gamma, \mathsf{state}_{\mathsf{Sim}_\gamma})$
873 : $\quad$ **if** $\bot \in \{\mathsf{status}_i \mid i \in [n]\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \bot; \ \mathsf{resp} \leftarrow \textsc{Nil}\big)$
874 : $\quad$ **elseif** $\{\mathsf{status}_i \mid i \in [n]\} \stackrel{?}{=} \{\top\}$: $\big(\mathsf{state}' \leftarrow \textsc{Nil}; \ \mathsf{status}' \leftarrow \top; \ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)\big)$
875 : $\quad$ **else**
876 : $\quad\quad \mathsf{state}' \leftarrow ((\mathsf{state}_1, \ldots, \mathsf{state}_n), (\mathsf{status}_1, \ldots, \mathsf{status}_n))$
877 : $\quad\quad \mathsf{status}' \leftarrow \textsc{Nil}; \ \mathsf{resp} \leftarrow (\mathsf{msg}_1, \ldots, \mathsf{msg}_n)$
878 : $\mathsf{st}[\mathsf{ssid}] \leftarrow (\mathsf{state}', \mathsf{status}'); \ \mathbf{if}\ \mathsf{status}' \stackrel{?}{=} \bot$: **return** $(\bot, \textsc{Nil})$ **else return** $(\mathsf{status}, \mathsf{resp})$

Fig. 96. Game $\mathbf{G}_7$: Removing the XOR with the outputs of the candidate schemes other than F_γ in the Ev oracle

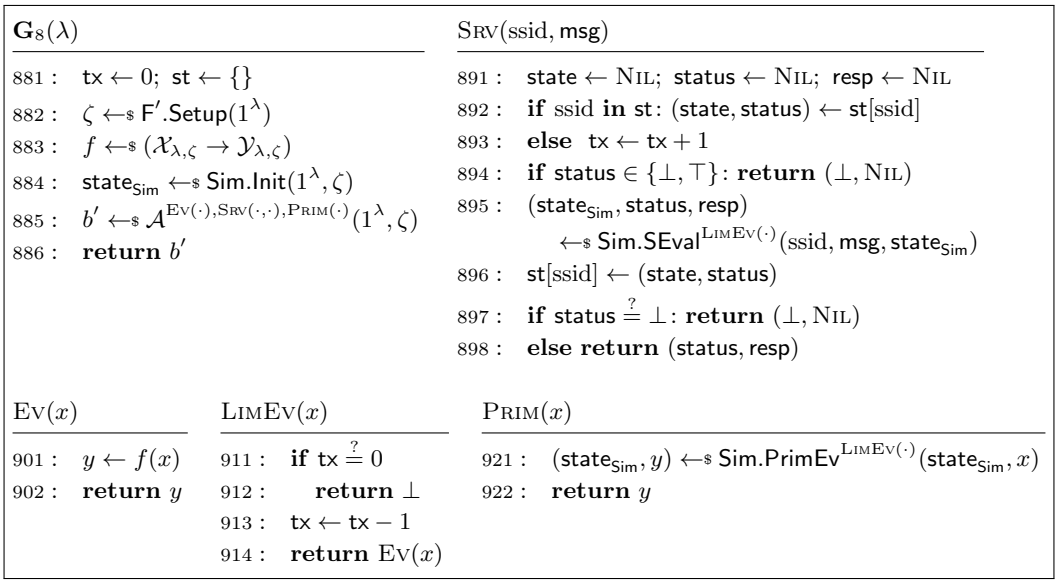


Fig. 97. Game $\mathbf{G}_8$: The final game, equivalent to $\mathbf{Exp}^{\text{oprf-psr}}_{\mathsf{F}',\mathcal{A},\mathsf{Sim},1}(\lambda)$.

I AI disclaimer

In accordance with Eurocrypt's AI policy, we detail the use of AI in our work.

For a first draft of the TikZ code for Fig. 5, we used ChatGPT, version gpt-4o. The following sequence of prompts was used:

- `can you draw a finite automata with 5 nodes in tikz?`
- `can you make them such that you go linearly through the states please?`
- `how do I add a node one line below q3 that has an arrow to q4?`
- `can I make the circle the same size if they have different text?`

All further edits, such as changing the labels, adding nodes, changing the shape of the nodes, or creating a second graph, were performed manually by the (human) authors.